

Database Management Systems

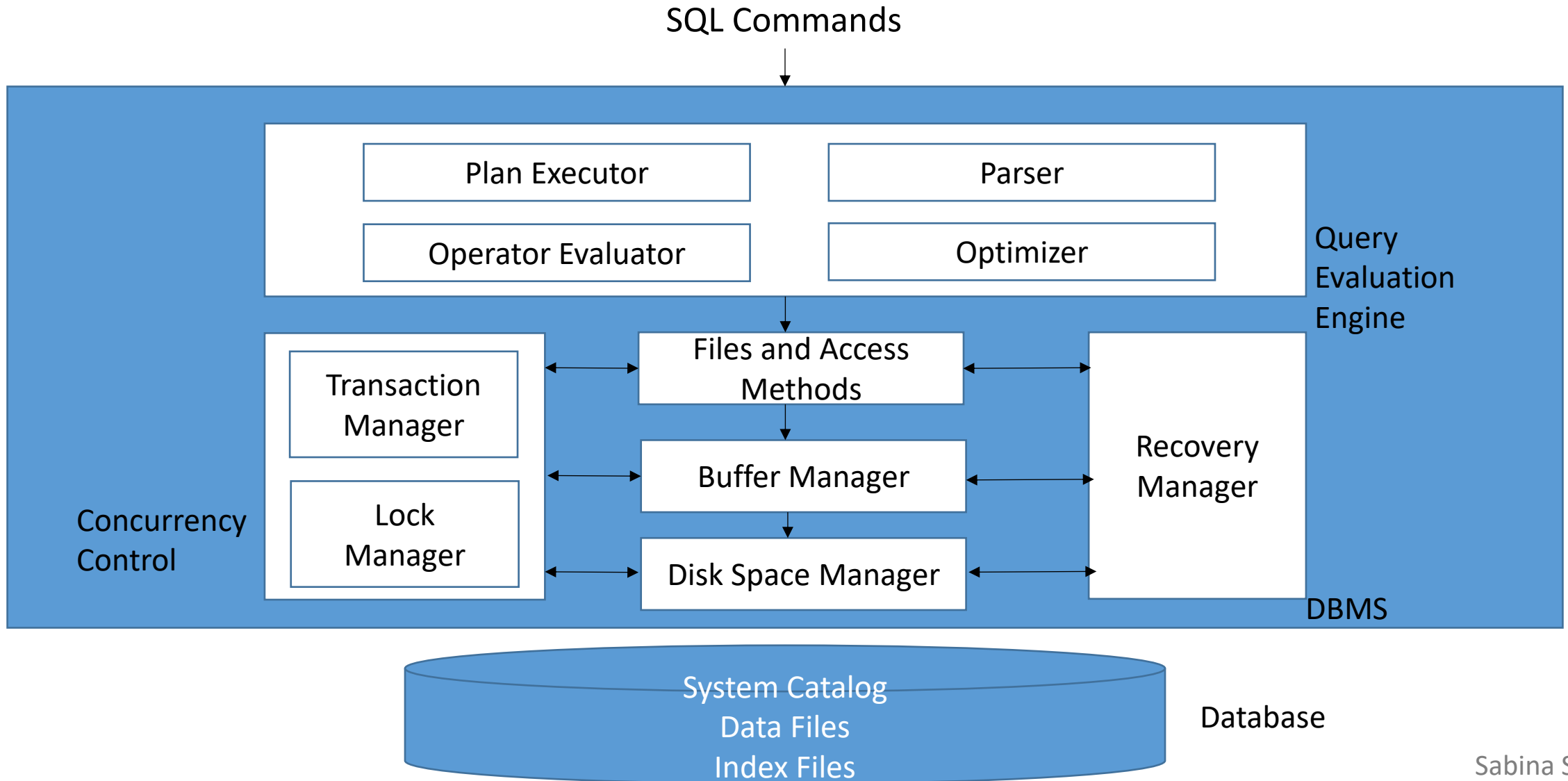
Lecture 1

Introduction - Transactions

DBMSs

- Lecture2 + Seminar1 + Laboratory1
- grading
 - written exam (W) - 50%
 - practical exam (P) - 25%
 - labs (L) - 25%
 - W, P ≥ 5
- to attend the exam, a student must have at least 6 laboratory attendances and at least 5 seminar attendances, according to the Computer Science Department's decision:
 - <https://www.cs.ubbcluj.ro/wp-content/uploads/Hotarare-CDI-29.04.2020.pdf>
- <https://www.cs.ubbcluj.ro/~sabina>
- sabina@cs.ubbcluj.ro

DBMS Architecture



Concurrency in a DBMS

- scenarios

1. airline reservation system

- travel agent 1 (TA1) processes customer 1's (C1) request for a seat on flight 10
 - system retrieves from the DB tuple *<flight: 10, available seats: 1>* and displays it to TA1
 - C1 takes time to decide whether or not to make the reservation
- travel agent 2 (TA2) processes customer 2's (C2) request for a seat on flight 10
 - system displays the same tuple to TA2: *<flight: 10, available seats: 1>*
 - C2 immediately reserves the seat:
 - system updates the tuple retrieved by TA2: *<flight: 10, available seats: 0>*

Concurrency in a DBMS

- scenarios

1. airline reservation system

- system updates the database, i.e., replaces flight 10's tuple in the DB with the new tuple *<flight: 10, available seats: 0>*
 - C1 also decides to reserve the seat
 - however, TA1 is still working on the previously retrieved tuple, not knowing another user has changed the data
 - system updates the tuple retrieved by TA1: *<flight: 10, available seats: 0>*
 - system updates the database, i.e., replaces flight 10's tuple in the DB with the new tuple *<flight: 10, available seats: 0>*
- => the last remaining seat on flight 10 has been sold 2 times!

Concurrency in a DBMS

- scenarios

2. bank system

- 1000 accounts: $A_1, A_2, \dots, A_{1000}$
- program P1 is computing the total balance: $\sum_{i=1}^{1000} \text{Balance}(A_i)$
- program P2 transfers 500 lei from A_1 to A_{1000}
- P2's transfer occurs after P1 has “seen” A_1 , but before it has “seen” $A_{1000} \Rightarrow$ the total balance will be incorrect: $\sum_{i=1}^{1000} \text{Balance}(A_i) + 500$
- objective: prevent such anomalies without disallowing concurrent access!

Concurrency in a DBMS

- what happens when concurrent access is forbidden?

- consider again scenario 1

- regard the activity of TA_i as invoking a copy of the program below:

```
S ← seatsAvailable(flight 10)
```

```
if (S > 0) then
```

```
    if (customer decides to purchase ticket)
```

```
        S--
```

```
        seatsAvailable(flight 10) ← S
```

```
    endif
```

```
else
```

```
    tell customer there are no more seats on flight 10
```

```
endif
```

Concurrency in a DBMS

- what happens when concurrent access is forbidden?
 - if the scheduling policy doesn't allow interleaved execution, TA2's copy of the previous program can begin execution only after TA1's invocation of the program has completed, i.e., after C1 has made the decision and the DB has been updated
- => if C1 decided to buy the ticket, TA2 sees tuple *<flight: 10, available seats: 0>* => C2 will be correctly informed that flight 10 is sold out

Concurrency in a DBMS

- what happens when concurrent access is forbidden?
- disallowing concurrent execution can lead to unacceptable delays
 - e.g., while C1 decides whether to make the reservation, travel agent 3 (TA3) receives a request from customer 3 (C3) for a seat on flight 11
 - since C1 and C3 are interested in different flights, their requests can be concurrently processed
 - if concurrent access is not allowed, C3's request can be processed only after C1 has made the decision and TA1's invocation of the program has finished

Concurrency in a DBMS

- what happens when concurrent access is forbidden?
- obs. when programs are executed serially (i.e., one after the other), results are correct
- however, the DBMS interleaves the operations (reads / writes of database objects) of different user programs, as concurrent execution is fundamental for the performance of the application
- the disk is frequently accessed (and access times are relatively slow)
- overlap I/O and CPU activity
- if concurrent execution is allowed, one needs to tackle the effects of interleaving user programs

Transactions

- database partitioned into items (e.g., *tuple*, *attribute*)
- transaction
 - any *one execution* of a program in a DBMS
 - sequence of one or more operations on a database: *read* / *write* / *commit* / *abort*
 - final operation:
 - commit or abort

Transactions

- I – an item in a DB
- fundamental operations in a transaction T (T can be omitted when clear from context):
 - $\text{read}(T, I)$
 - $\text{write}(T, I)$
 - $\text{commit}(T)$
 - successful completion: T 's changes are to become permanent
 - $\text{abort}(T)$
 - T is *rolled back*: its changes are undone

Transaction Properties – ACID

- properties a DBMS must ensure for a transaction in order to maintain data in the presence of concurrent execution / system failures
- atomicity
 - a transaction is atomic
 - either all the operations in the transaction are executed, or none are (*all-or-nothing*)
- consistency
 - a transaction must preserve the consistency of the database after execution, i.e., a transaction is a correct program
- isolation
 - a transaction is protected from the effects of concurrently scheduling other transactions

Transaction Properties – ACID

- durability
 - the effects of a successfully completed transaction should persist even if a system crash occurs before all the changes have been written to disk

Transaction Properties – Atomicity

- a transaction can commit / abort:
 - *commit* – after finalizing all its operations (successful completion)
 - *abort* – after executing some of its operations; the DBMS can itself abort a transaction
- user's perspective:
 - either all operations in a transaction are executed, or none are
 - the transaction is treated as an indivisible unit of execution
- the DBMS *logs* all transactions' writes, so it can *undo* them if necessary (i.e., undo the actions of incomplete transactions)
- *crash recovery*
 - guarantees atomicity in the presence of system crashes

Transaction Properties – Atomicity

- e.g., consider the following transaction transferring money from AccountA to AccountB:

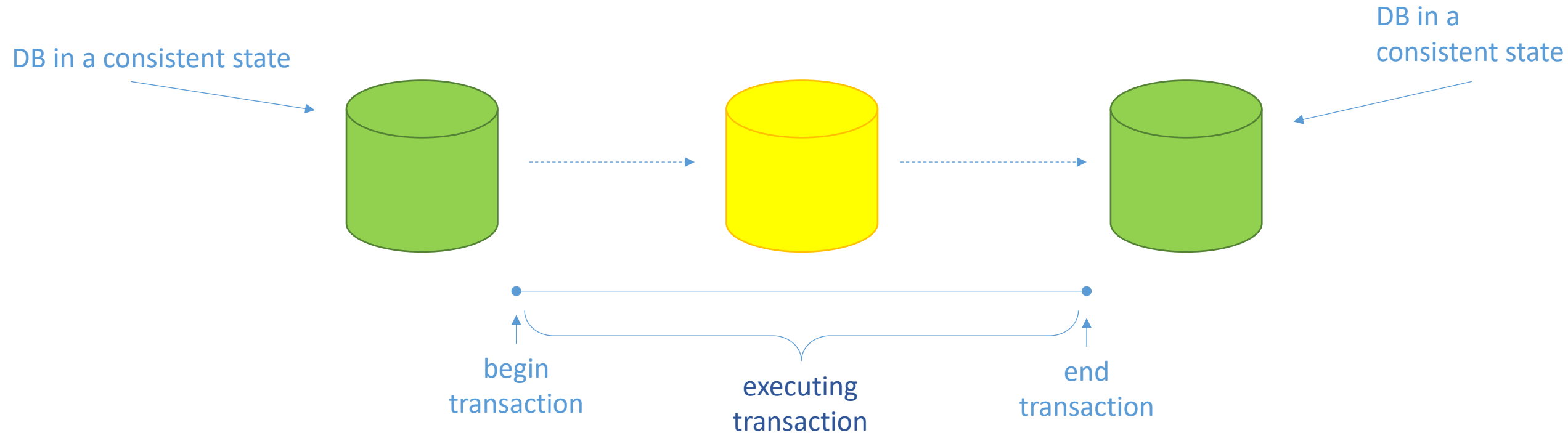
AccountA $\leftarrow$ AccountA - 100

AccountB $\leftarrow$ AccountB + 100

- if the transaction fails in the middle, 100 lei must be restored to AccountA, i.e., partial effects are undone

Transaction Properties – Consistency

- in the absence of other transactions, a transaction that executes on a consistent database state leaves the database in a consistent state



Transaction Properties – Consistency

- transactions do not violate the integrity constraints (ICs) specified on the database (e.g., restrictions declared via CREATE TABLE statements) and enforced by the DBMS
- transactions are correct programs
- e.g.: consider again the money transfer transaction:

$\text{AccountA} \leftarrow \text{AccountA} - 100$

$\text{AccountB} \leftarrow \text{AccountB} + 100$

- consistency constraint: the transaction preserves the total amount of money in the two accounts
- it's the responsibility of the user to write a correct transaction that meets the above constraint

Transaction Properties – Consistency

- the DBMS does not understand the semantics of the data, of user actions (e.g., how is the interest computed, the fact that AccountB must be credited with the amount of money debited from AccountA, etc.)

Transaction Properties – Isolation

- transactions are protected from the effects of concurrently scheduling other transactions
- users concurrently submit transactions to the DBMS, but can think of their transactions as if they were executing in isolation, independently of other users' transactions
- i.e., users must not deal with arbitrary effects produced by other concurrently running transactions; a transaction is seen as if it were in single-user mode

Transaction Properties – Durability

- the system must guarantee the effects of a successfully completed transaction won't be lost, even if failures subsequently occur
- i.e., once the DBMS informs the user a transaction has completed successfully, its effects should persist, even if a system crash occurs before all changes have been saved on the disk
 - the *Write-Ahead Log* property
 - changes are written to the log (on the disk) before being reflected in the database
- the log ensures atomicity, durability

Interleaved Executions - Example

- consider the two transactions below:

T1: BEGIN $A \leftarrow A - 100$ $B \leftarrow B + 100$ END

T2: BEGIN $A \leftarrow 1.05 * A$ $B \leftarrow 1.05 * B$ END

- T1 is transferring 100 lei from account A to account B
- T2 is adding 5% interest to both accounts
- T1 and T2 are submitted together to the DBMS
- serial execution: T1 can execute before or after T2 on a database DB, resulting in one of the following database states:
 - State((T1T2), DB) or
 - State((T2T1), DB)
- interleaved execution: the net effect of executing T1 and T2 *must* be identical to State((T1 T2), DB) or State((T2 T1), DB)

Interleaved Executions - Example

- consider the following interleaving of operations (schedule):

T1	T2
$A \leftarrow A - 100$	
	$A \leftarrow 1.05 * A$
$B \leftarrow B + 100$	
	$B \leftarrow 1.05 * B$



Interleaved Executions - Example

- but what about the following schedule:



Interleaved Executions - Example

- the DBMS's view of the second schedule:

T1	T2
R(A)	
W(A)	
	R(A)
	W(A)
	R(B)
	W(B)
R(B)	
W(B)	

Scheduling Transactions

- *schedule*

- a list of operations (Read / Write / Commit / Abort) of a set of transactions with the property that the order of the operations in each individual transaction is preserved

Scheduling Transactions

T1

read(V)
read(sum)

read(V)
sum := sum + V
write(sum)
commit

T2

read(V)
V := V + 50
write(V)
commit

Schedule

read1(V)
read1(sum)
read2(V)

write2(V)
commit2
read1(V)

write1(sum)
commit1

Serial and Non-Serial Schedules

- serial schedule

- a schedule in which the actions of different transactions are not interleaved

T1

```
read(V)
read(sum)
read(V)
sum := sum + V
write(sum)
commit
```

T2

```
read(V)
V := V + 50
write(V)
commit
```

Serial and Non-Serial Schedules

- *non-serial schedule*

- a schedule in which the actions of different transactions are interleaved

read1(V)

read1(sum)

read2(V)

write2(V)

commit2

read1(V)

write1(sum)

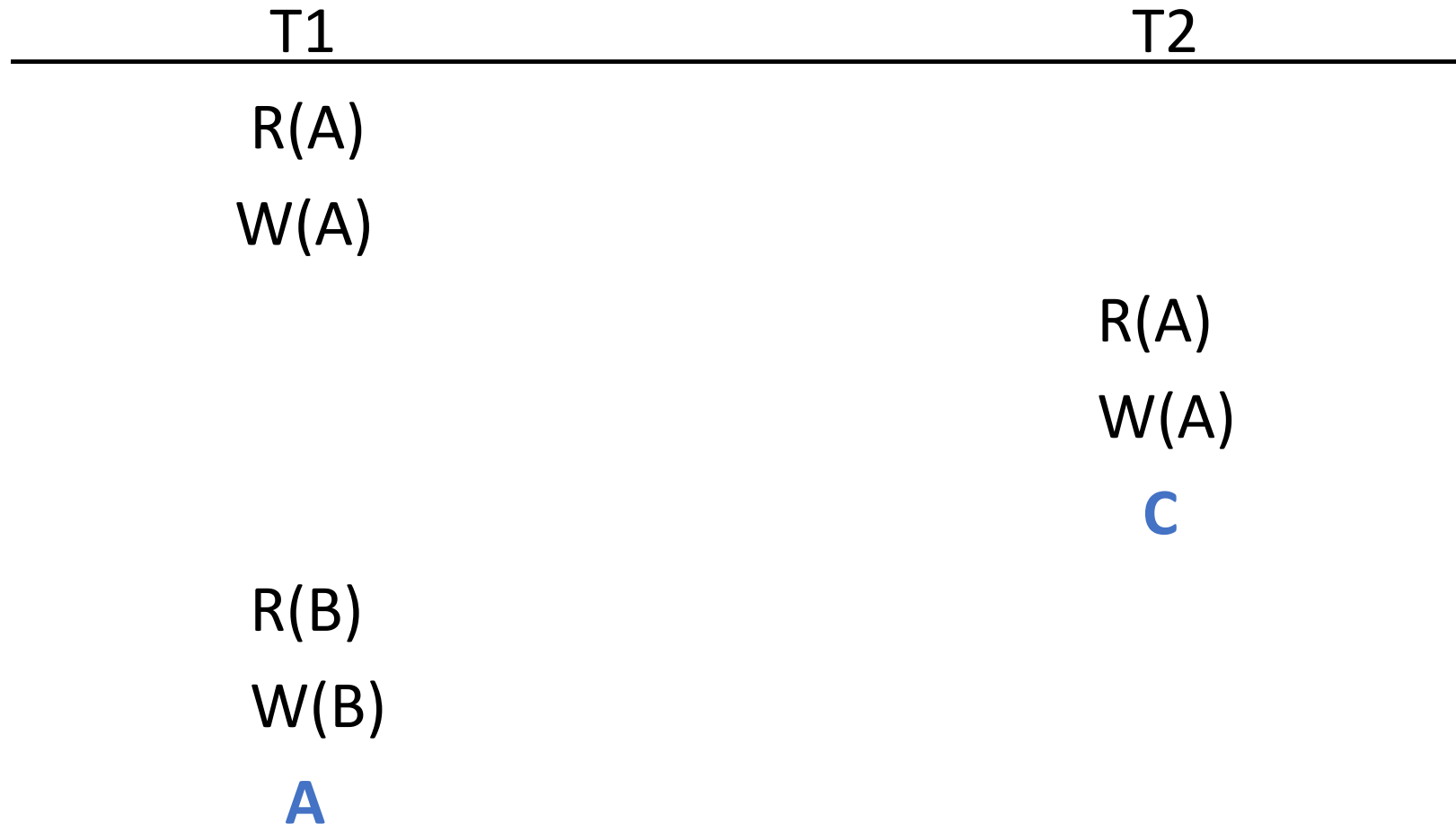
commit1

Interleaved Executions - Anomalies

- two transactions are only reading a data object => no conflict, order of execution not important
- two transactions are reading and / or writing completely separate data objects => no conflict, order of execution not important
- two transactions are operating on the same data object, and at least one of them performs a write operation => order of execution is important
 - WR conflict
 - T2 is reading a data object previously written by T1
 - RW conflict
 - T2 is writing a data object previously read by T1
 - WW conflict
 - T2 is writing a data object previously written by T1

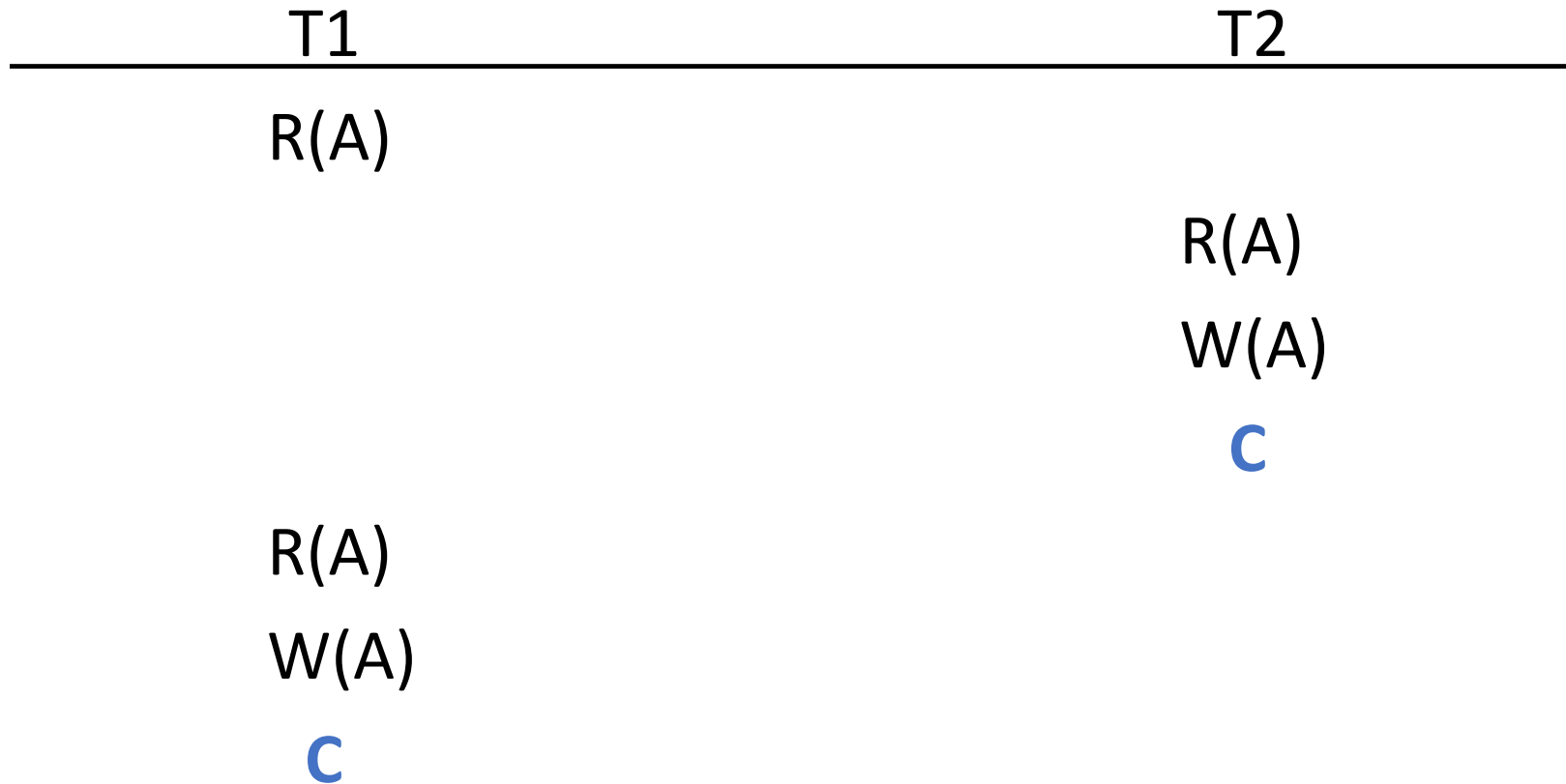
Interleaved Executions - Anomalies

- reading uncommitted data (WR conflicts, *dirty reads*)



Interleaved Executions - Anomalies

- unrepeatable reads (RW conflicts)



Interleaved Executions - Anomalies

- overwriting uncommitted data (WW conflicts):



Serializability

- C – set of transactions
- $Sch(C)$ – the set of schedules for C
- serializable schedule
 - a schedule $S \in Sch(C)$ is serializable iff the effect of S on any consistent database instance is identical to the effect of some serial schedule $S_0 \in Sch(C)$

Serializability

- serializability - correctness criterion for an interleaved schedule
 - consider the serial schedule $(T_1, T_2, \dots, T_n), T_i \in C$
 - assume the database instance is in a correct state prior to executing T_1
 - each transaction must preserve the consistency of the database
 - $\Rightarrow$ the database is in a correct state after T_n completes execution
 - $\Rightarrow$ if a serializable schedule is executed on a correct database instance, it produces a correct database instance (since it is equivalent to some serial schedule)
- ensuring serializability prevents inconsistencies that may be generated by concurrent transactions interfering with one another
- objective - finding interleaved schedules enabling transactions to execute concurrently without interfering with each other (such schedules result in a correct database state)

References

- [Ra02] RAMAKRISHNAN, R., GEHRKE, J., Database Management Systems (3rd Edition), McGraw-Hill, 2002
- [Le99] LEVENE, M., LOIZOU, G., A Guided Tour of Relational Databases and Beyond, Springer, 1999
- [Ga09] GARCIA-MOLINA, H., ULLMAN, J., WIDOM, J., Database Systems: The Complete Book (2nd Edition), Pearson Education, 2009
- [Ra02S] RAMAKRISHNAN, R., GEHRKE, J., Database Management Systems, Slides for the 3rd Edition,
<http://pages.cs.wisc.edu/~dbbook/openAccess/thirdEdition/slides/slides3ed.html>
- [Si19] SILBERSCHATZ, A., KORTH, H., SUDARSHAN, S., Database System Concepts (7th Edition), McGraw-Hill, 2019
- [Si19S] SILBERSCHATZ, A., KORTH, H., SUDARSHAN, S., Database System Concepts, Slides for the 7th Edition, <http://codex.cs.yale.edu/avi/db-book/>
- [Ul11] ULLMAN, J., WIDOM, J., A First Course in Database Systems,
<http://infolab.stanford.edu/~ullman/fcdb.html>

Database Management Systems

Lecture 2

Transactions. Concurrency Control

Conflict Serializability

- C – set of transactions
- $Sch(C)$ – the set of schedules for C
- $Op(C)$ – set of operations of the transactions in C
- consider schedule $S \in Sch(C)$
- the *conflict relation* of S is defined as:
 - $conflict(S) = \{(op_1, op_2) \mid op_1, op_2 \in Op(C), op_1 \text{ occurs before } op_2 \text{ in } S, op_1 \text{ and } op_2 \text{ are in conflict}\}$

Conflict Serializability

- C – set of transactions
- $Sch(C)$ – the set of schedules for C
- two schedules S_1 and $S_2 \in Sch(C)$ are conflict equivalent, written $S_1 \equiv_c S_2$, iff $conflict(S_1) = conflict(S_2)$, i.e.:
 - S_1 and S_2 contain the same operations of the same transactions and
 - every pair of conflicting operations is ordered in the same manner in S_1 and S_2

Conflict Serializability

S1

T1	T2
Read(A)	
A := A – 100	
Write(A)	
Read(B)	
B := B + 200	
Write(B)	
	Read(A)
	A := A * 0.2
	Write(A)
	Read(B)
	B := B + 300
	Write(B)

S2

T1	T2
Read(A)	
A := A – 100	
Write(A)	
	Read(A)
	A := A * 0.2
	Write(A)
Read(B)	
B := B + 200	
Write(B)	
	Read(B)
	B := B + 300
	Write(B)

$$\text{conflict}(S1) = \text{conflict}(S2)$$

$$\Rightarrow S1 \equiv_c S2$$

$$\text{conflict}(S1) = \{(\text{Read}(T1, A), \text{Write}(T2, A)), (\text{Write}(T1, A), \text{Read}(T2, A)), (\text{Write}(T1, A), \text{Write}(T2, A)), (\text{Read}(T1, B), \text{Write}(T2, B)), (\text{Write}(T1, B), \text{Read}(T2, B)), (\text{Write}(T1, B), \text{Write}(T2, B))\}$$

Conflict Serializability

S1

T1	T2
Read(A)	
A := A - 100	
Write(A)	
Read(B)	
B := B + 200	
Write(B)	
	Read(A)
	A := A * 0.2
	Write(A)
	Read(B)
	B := B + 300
	Write(B)

S3

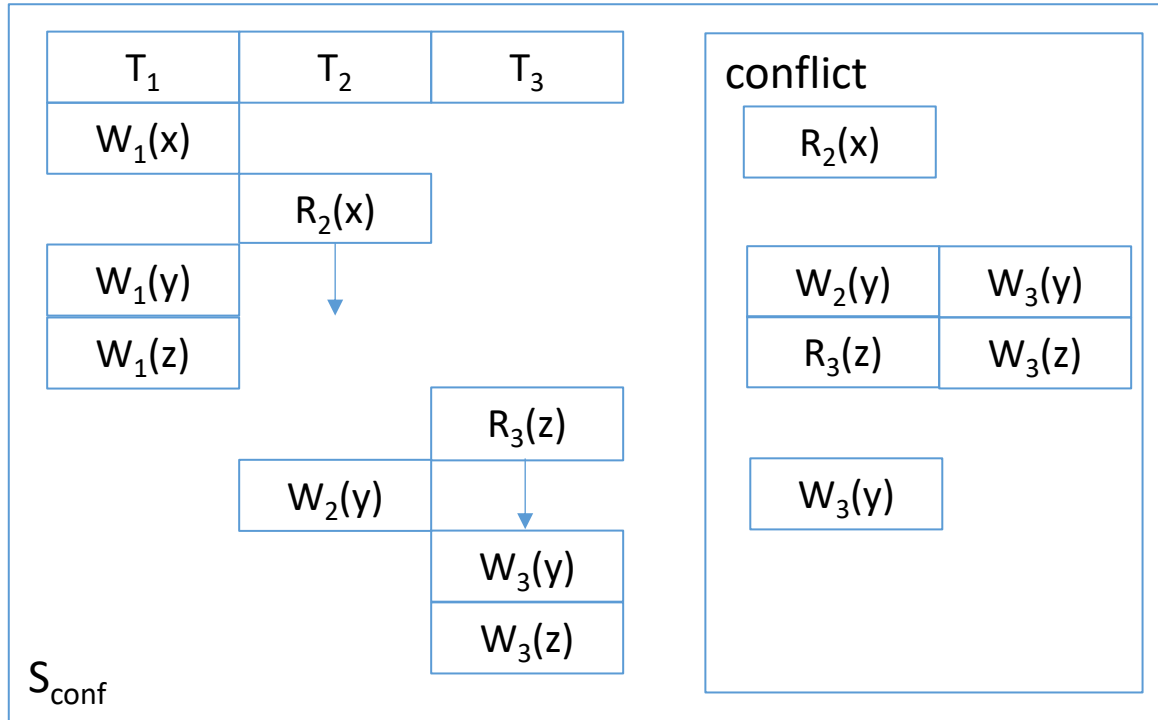
T1	T2
	Read(A)
	A := A * 0.2
	Write(A)
	Read(B)
	B := B + 300
	Write(B)
Read(A)	
A := A - 100	
Write(A)	
Read(B)	
B := B + 200	
Write(B)	

$\text{conflict}(S1) \neq \text{conflict}(S3)$
 $\Rightarrow S1 \not\equiv_c S3$

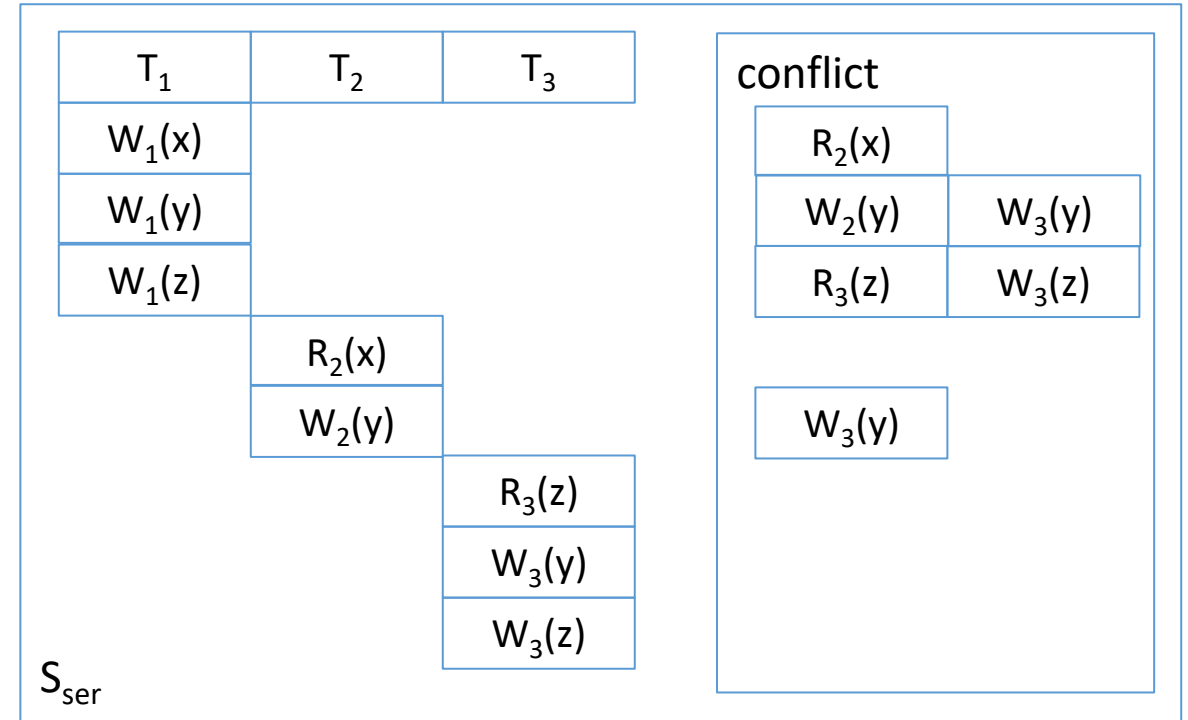
Conflict Serializability

- C – set of transactions
- $Sch(C)$ – the set of schedules for C
- let S be a schedule in $Sch(C)$
- schedule S is conflict serializable iff there exists a serial schedule $S_0 \in Sch(C)$ such that $S \equiv_c S_0$, i.e., S is conflict equivalent to some serial schedule

Conflict Serializability



conflict serializable schedule



serial schedule

Conflict Serializability - Precedence Graph

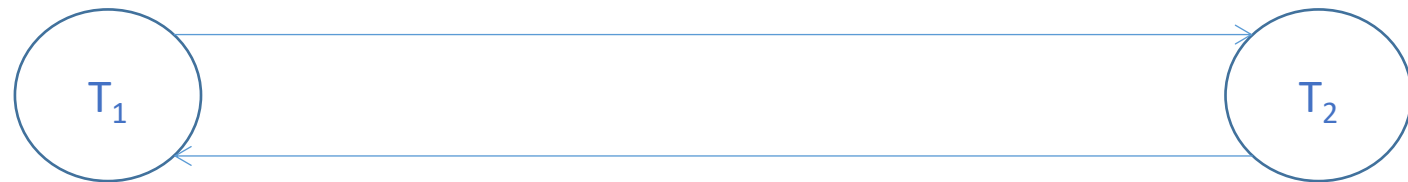
- let S be a schedule in $Sch(C)$
- the precedence graph (serializability graph) of S contains:
 - one node for every committed transaction in S
 - an arc from T_i to T_j if an action in T_i precedes and conflicts with one of the actions in T_j
- Theorem:
 - a schedule $S \in Sch(C)$ is conflict serializable if and only if its precedence graph is acyclic

Conflict Serializability - Precedence Graph

- example - a schedule that is not conflict serializable:



- the precedence graph has a cycle:

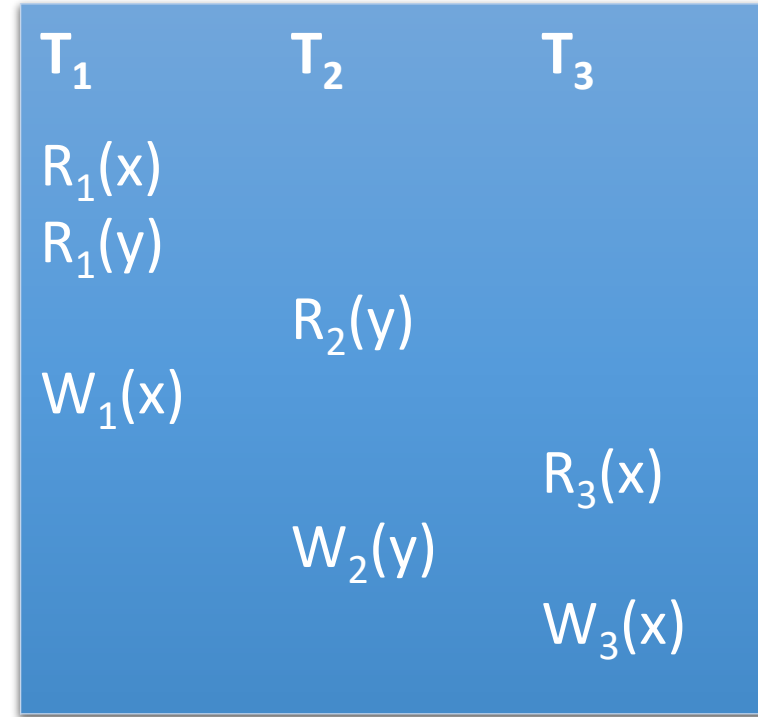


Conflict Serializability - Precedence Graph

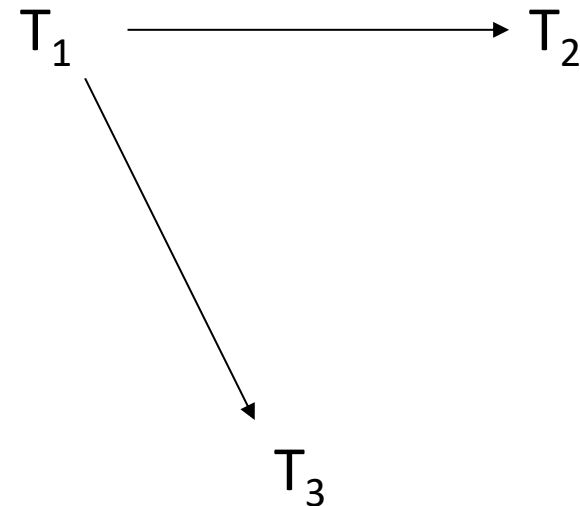
- algorithm to test the conflict serializability of a schedule $S \in Sch(C)$
 1. create a node labeled T_i in the precedence graph for every committed transaction T_i in the schedule
 2. create an arc (T_i, T_j) in the precedence graph if T_j executes a Read(A) after a Write(A) executed by T_i
 3. create an arc (T_i, T_j) in the precedence graph if T_j executes a Write(A) after a Read(A) executed by T_i
 4. create an arc (T_i, T_j) in the precedence graph if T_j executes a Write(A) after a Write(A) executed by T_i
 5. S is conflict serializable if and only if the resulting precedence graph has no cycles

Conflict Serializability - Precedence Graph

- examples
- let S_1 be a schedule over $\{T_1, T_2, T_3\}$:



- the precedence graph for S_1 :

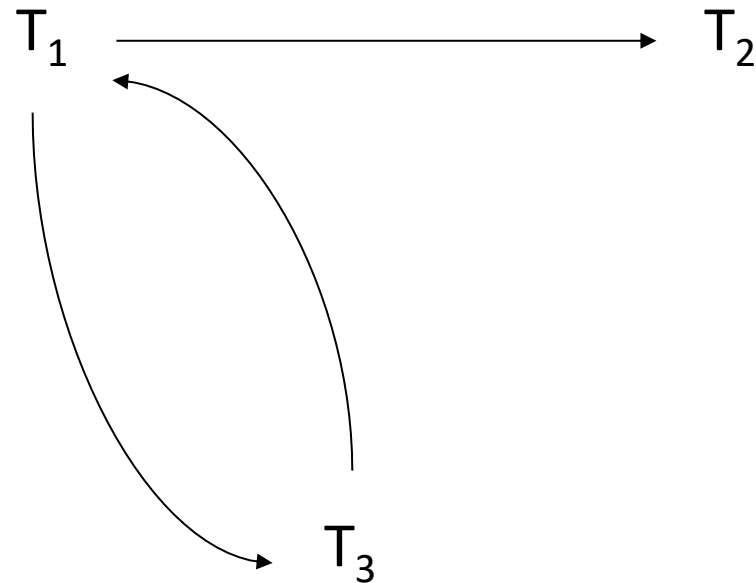


Conflict Serializability - Precedence Graph

- examples
- let S_2 be a schedule over $\{T_1, T_2, T_3\}$:

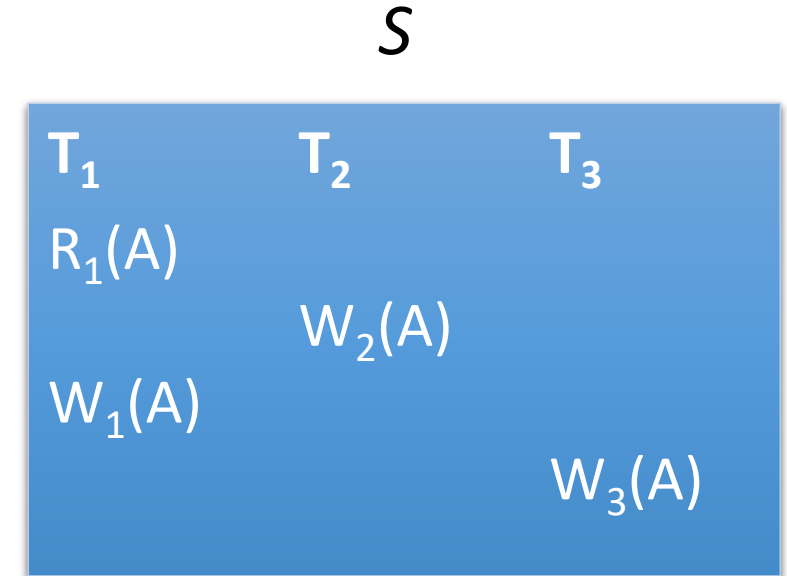
T_1	T_2	T_3
$R_1(x)$		
$R_1(y)$		
		$R_3(x)$
	$R_2(y)$	
$W_1(x)$		
	$W_2(y)$	
		$W_3(x)$

- the precedence graph for S_2 :



Conflict Serializability

- every conflict serializable schedule is serializable (in the absence of inserts / deletes, when items can only be updated)
- there are serializable schedules that are not conflict serializable
- S is serializable (it's equivalent to the serial execution of transactions T_1 , T_2 , T_3 – in this order), but it is not conflict serializable



View Serializability

- conflict serializability is a sufficient condition for serializability, but it is not a necessary one
- *view serializability*
 - a more general, sufficient condition for serializability
 - based on *view-equivalence*, a less stringent form of equivalence

View Serializability

- C – set of transactions
- $Sch(C)$ – the set of schedules for C
- let $T_i, T_j \in C, S_1, S_2 \in Sch(C)$; S_1 and S_2 are view equivalent, written $S_1 \equiv_v S_2$, iff the following conditions are met:
 - if T_i reads the initial value of V in S_1 , then T_i also reads the initial value of V in S_2 ;
 - if T_i reads the value of V written by T_j in S_1 , then T_i also reads the value of V written by T_j in S_2 ;
 - if T_i writes the final value of V in S_1 , then T_i also writes the final value of V in S_2 .
- i.e.:
 - each transaction performs the same computation in S_1 and S_2
 - and
 - S_1 and S_2 produce the same final database state.

View Serializability

S1

T1	T2
Read(A)	
A := A - 100	
Write(A)	
Read(B)	
B := B + 200	
Write(B)	
	Read(A)
	A := A * 0.2
	Write(A)
	Read(B)
	B := B + 300
	Write(B)

S2

T1	T2
Read(A)	
A := A - 100	
Write(A)	
	Read(A)
	A := A * 0.2
	Write(A)
Read(B)	
B := B + 200	
Write(B)	
	Read(B)
	B := B + 300
	Write(B)

$$S1 \equiv_v S2$$

View Serializability

S1

T1	T2
Read(A)	
A := A - 100	
Write(A)	
Read(B)	
B := B + 200	
Write(B)	
	Read(A)
	A := A * 0.2
	Write(A)
	Read(B)
	B := B + 300
	Write(B)

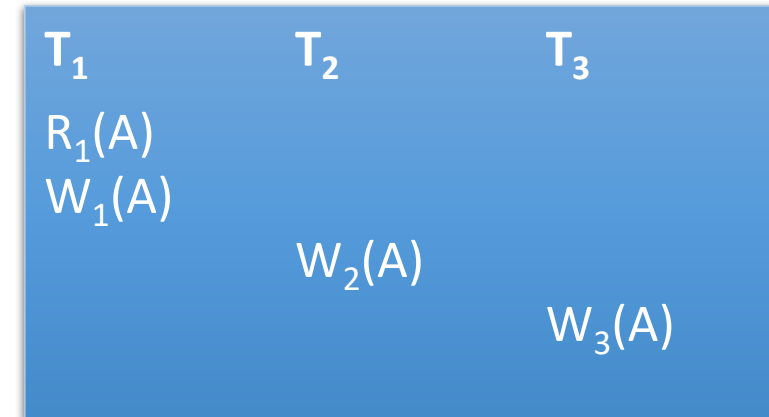
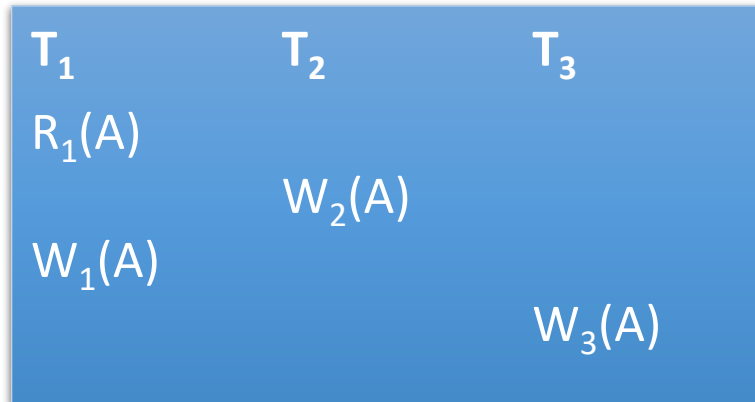
S3

T1	T2
	Read(A)
	A := A * 0.2
	Write(A)
	Read(B)
	B := B + 300
	Write(B)
Read(A)	
A := A - 100	
Write(A)	
Read(B)	
B := B + 200	
Write(B)	

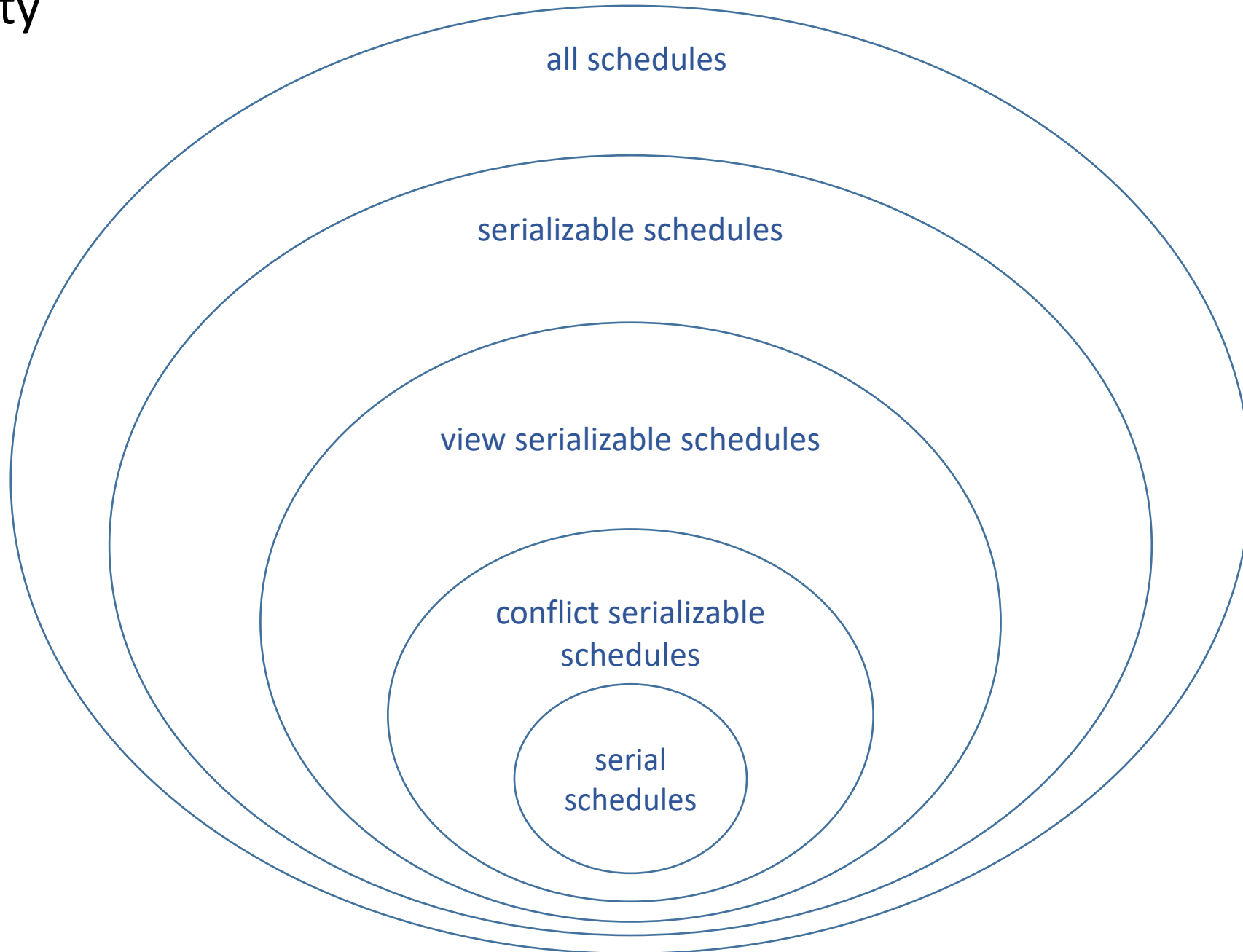
$S1 \not\equiv_v S3$

View Serializability

- C – set of transactions
- $Sch(C)$ – the set of schedules for C
- a schedule $S \in Sch(C)$ is view serializable iff there exists a serial schedule $S_0 \in Sch(C)$ such that $S \equiv_v S_0$, i.e., S is view equivalent to some serial schedule



Serializability



Recoverable Schedules

- consider schedule S over $\{T_1, T_2\}$

T_1	T_2
Read(x)	
$x := x + 10$	
Write(x)	
	Read(x)
	$x := x * 5$
	Write(x)
	Read(y)
	$y := y * 5$
	Write(y)
	commit
abort	

- T_2 operates on a value of x that shouldn't have been there (T_1 aborts)

Recoverable Schedules

T_1	T_2
Read(x)	
$x := x + 10$	
Write(x)	
	Read(x)
	$x := x * 5$
	Write(x)
	Read(y)
	$y := y * 5$
	Write(y)
	commit
abort	

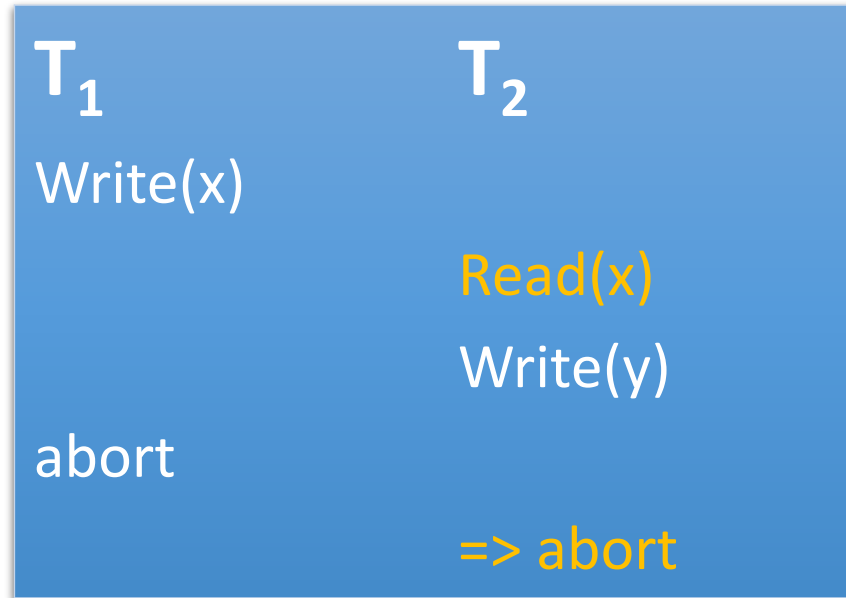
- cannot cascade the abort of T_1 , since T_2 has already committed
- schedule S is *unrecoverable*

Recoverable Schedules

- recoverable schedule

- each transaction T in the schedule commits only after all transactions whose changes T read commit

Avoiding Cascading Aborts



- a schedule in which each transaction T reads only changes of committed transactions is said to avoid cascading aborts
- avoiding cascading aborts => recoverable schedules

Lock-Based Concurrency Control

- technique used to guarantee serializable, recoverable schedules
- *lock*
 - a tool used by the transaction manager to control concurrent access to data
 - prevents a transaction from accessing a data object while another transaction is accessing the object
- *transaction protocol*
 - a set of rules enforced by the transaction manager and obeyed by all transactions
 - example – simple protocol: before a transaction can read / write an object, it must acquire an appropriate lock on the object
 - locks in conjunction with protocols allow interleaved executions

Lock-Based Concurrency Control

- SLock (*shared or read lock*)
 - if a transaction holds an SLock on an object, it can read the object, but it cannot modify it
- XLock (*exclusive or write lock*)
 - if a transaction holds an XLock on an object, it can both read and write the object
- if a transaction holds an SLock on an object, other transactions can be granted SLocks on the object, but they cannot acquire XLocks on it
- if a transaction holds an XLock on an object, other transactions cannot be granted either SLocks or XLocks on the object

	Shared	Exclusive
Shared	Yes	No
Exclusive	No	No

Lock-Based Concurrency Control

- *lock table*

- structure used by the lock manager to keep track of granted locks / lock requests
- entry in the lock table (corresponding to one data object):
 - number of transactions holding a lock on the data object
 - lock type (SLock / XLock)
 - pointer to a queue of lock requests

Lock-Based Concurrency Control

- *transaction table*
 - structure maintained by the DBMS
 - one entry / transaction
 - keeps a pointer to a list of locks held by the transaction
- lock upgrade
 - an SLock granted to a transaction can be upgraded to an XLock

* Jim Gray *

References

- [Ra02] RAMAKRISHNAN, R., GEHRKE, J., Database Management Systems (3rd Edition), McGraw-Hill, 2002
- [Le99] LEVENE, M., LOIZOU, G., A Guided Tour of Relational Databases and Beyond, Springer, 1999
- [Ga09] GARCIA-MOLINA, H., ULLMAN, J., WIDOM, J., Database Systems: The Complete Book (2nd Edition), Pearson Education, 2009
- [Ra02S] RAMAKRISHNAN, R., GEHRKE, J., Database Management Systems, Slides for the 3rd Edition,
<http://pages.cs.wisc.edu/~dbbook/openAccess/thirdEdition/slides/slides3ed.html>
- [Si19] SILBERSCHATZ, A., KORTH, H., SUDARSHAN, S., Database System Concepts (7th Edition), McGraw-Hill, 2019
- [Si19S] SILBERSCHATZ, A., KORTH, H., SUDARSHAN, S., Database System Concepts, Slides for the 7th Edition, <http://codex.cs.yale.edu/avi/db-book/>
- [Ul11] ULLMAN, J., WIDOM, J., A First Course in Database Systems,
<http://infolab.stanford.edu/~ullman/fcdb.html>

Database Management Systems

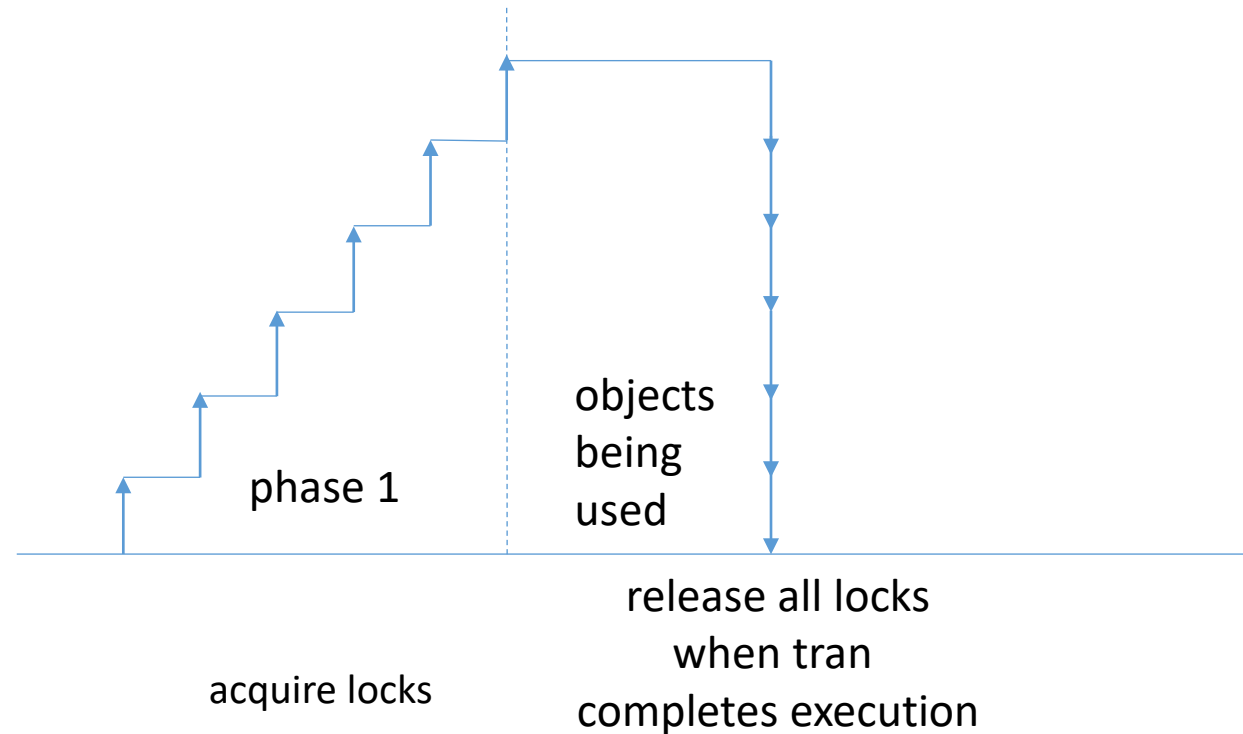
Lecture 3

Transactions. Concurrency Control

- locking *protocols*
 - Strict Two-Phase Locking
 - Two-Phase Locking
- *deadlocks*
 - prevention (Wait-die, Wound-wait)
 - detection (waits-for graph, timeout mechanism)
- the *phantom* problem
- *isolation levels*
 - READ UNCOMMITTED
 - READ COMMITTED
 - REPEATABLE READ
 - SERIALIZABLE

Strict Two-Phase Locking (*Strict 2PL*)

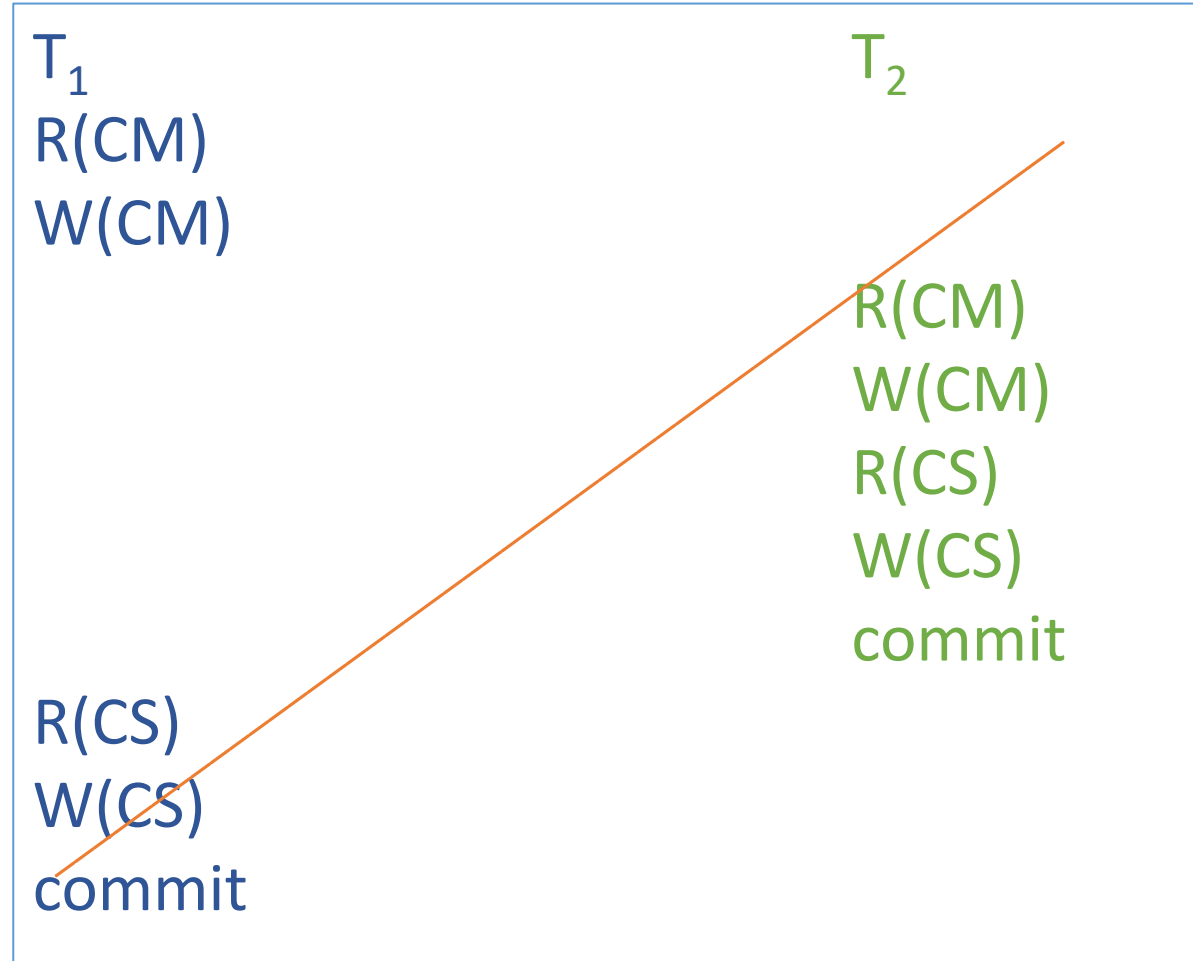
- * before a transaction can read / write an object, it must acquire a S / X lock on the object
- * all the locks held by a transaction are released when it completes execution



- the Strict 2PL protocol allows only serializable schedules (only schedules with acyclic precedence graphs are allowed by this protocol)

Strict Two-Phase Locking

- the interleaving below is not allowed by Strict 2PL:



->

Strict Two-Phase Locking

T_1
XLock(CM)
R(CM)
W(CM)
...

T_2

XLock(CM)
...

- T_1 acquires an X lock on object CM, reads and writes CM
 - T_1 is still in progress when T_2 requests a lock on the same object, CM
 - T_2 cannot acquire an exclusive lock on CM, since T_1 already holds a conflicting lock on this object
 - T_1 will release its lock on CM only when it completes execution (with *commit* or *abort*)
 - since it cannot grant T_2 the requested lock on CM, the DBMS suspends T_2
- in this example, we denote by $XLock(O)$ the action of the current transaction requesting an X lock on object O

Strict Two-Phase Locking

T_1

XLock(CM)

R(CM)

W(CM)

XLock(CS)

R(CS)

W(CS)

commit

T_2

XLock(CM)

R(CM)

W(CM)

XLock(CS)

R(CS)

W(CS)

commit

- T_1 continues execution
- when T_1 commits, it releases both locks (X lock on CM, X lock on CS)
- T_2 can now be granted an X lock on CM
- T_2 can now proceed

Strict Two-Phase Locking

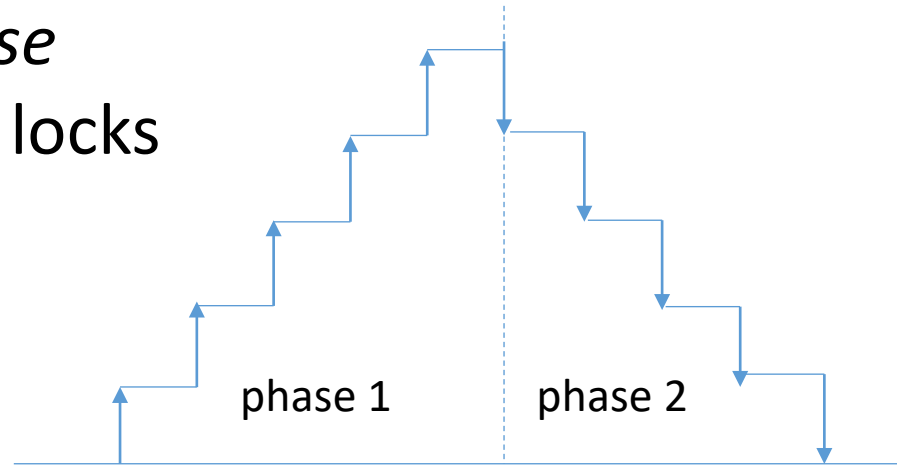
- the interleaving below is allowed by Strict 2PL:

T_1	T_2
XLock(CM)	
R(CM)	
W(CM)	
	XLock(CT)
	R(CT)
	W(CT)
	commit
XLock(CS)	
R(CS)	
W(CS)	
commit	

- in this example, since T_1 and T_2 are operating on separate data objects (CM, CT, CS), they can concurrently obtain all requested locks (an interleaved execution where T_1 and T_2 are only reading the same data object is also allowed)

Two-Phase Locking (2PL)

- variant of Strict Two-Phase Locking
 - * before a transaction can read / write an object, it must acquire a S / X lock on the object
 - * once a transaction releases a lock, it cannot request other locks
- phase 1 - *growing phase*
 - transaction acquires locks
- phase 2 - *shrinking phase*
 - transaction releases locks



Two-Phase Locking

- C – set of transactions
- $Sch(C)$ – set of schedules for C
- if all transactions in C obey 2PL, then any schedule $S \in Sch(C)$ that completes normally is serializable

Two-Phase Locking

- the following execution is allowed by the protocol:
- T1 can release its X lock on D prior to completion
- so T2 can acquire an X lock on D while T1 is still in progress
- however, T1 cannot acquire any other locks once it releases a lock (in this case, its X lock on D)

T1
R(D)
W(D)

T2

R(D)
W(D)
R(F)
W(F)
commit

...
commit

Two-Phase Locking

T1	T2
R(D)	
W(D)	
	R(D)
	W(D)
	R(F)
	W(F)
	commit

t

- suppose T1 is forced to terminate at time t
 - undo T1's updates => T2's update of D is lost (i.e., partial effects, as T2 also changed the value of F)
- problem – T1 released its exclusive lock on D prior to completion (under Strict 2PL, T1 can release its lock on D, as well as any other locks, only when it commits / aborts)

Strict Schedules

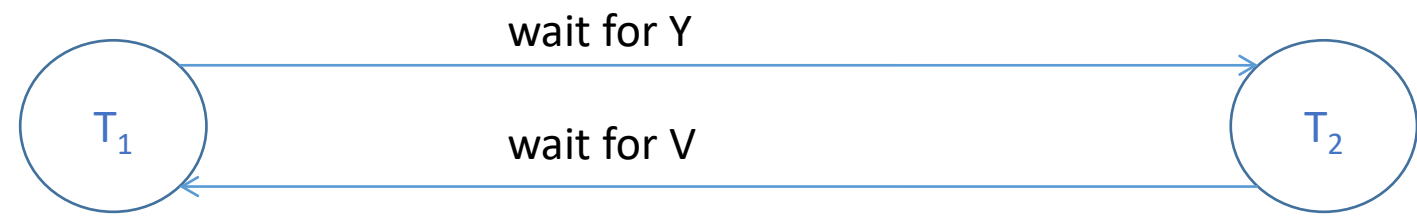
- if transaction T_i has written object A , then transaction T_j can read and / or write A only after T_i 's completion (commit / abort)
- strict schedules:
 - avoid cascading aborts
 - are recoverable schedules
 - if a transaction is aborted, its operations can be undone
- Strict 2PL only allows strict schedules

Deadlocks

- lock-based concurrency control techniques can lead to deadlocks
- deadlock
 - cycle of transactions waiting for one another to release a locked resource
 - normal execution can no longer continue without an external intervention, i.e., deadlocked transactions cannot proceed until the deadlock is resolved
- deadlock management
 - deadlock prevention
 - deadlock detection
 - allow deadlocks to occur and resolve them when they arise

Deadlocks

T1	T2
<code>XLock(V)</code>	
<code>Read(V)</code>	
<code>V := V + 100</code>	
<code>Write(V)</code>	
	<code>XLock(Y)</code>
	<code>Read(Y)</code>
	<code>Y := Y * 5</code>
	<code>Write(Y)</code>
<code>XLock(Y)</code>	<code>XLock(V)</code>
<code>Wait</code>	<code>Wait</code>
<code>Wait</code>	<code>Wait</code>
<code>...</code>	<code>...</code>



- T1 cannot obtain an X lock on Y, since T2 holds a conflicting lock on Y
- similarly, T2 cannot obtain an X lock on V, since T1 holds a conflicting lock on V

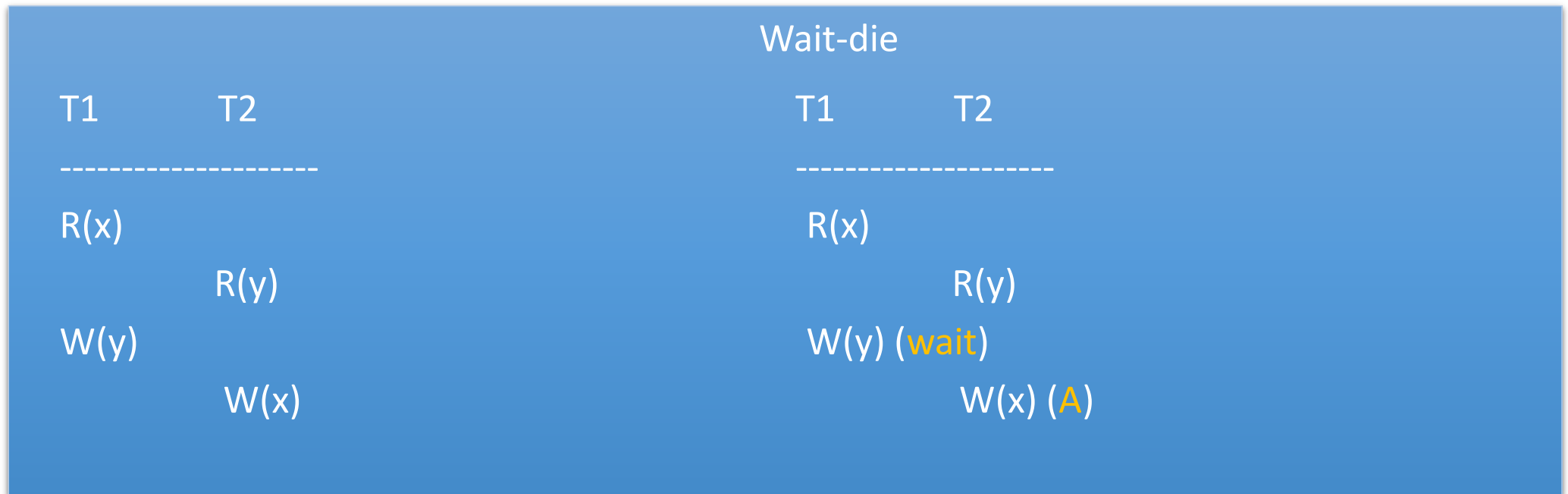
Deadlocks - Prevention

- assign transactions timestamp-based priorities (each transaction has a timestamp - the moment it begins execution)
- the lower the timestamp, the older the transaction
- the older a transaction is, the higher its priority, with the oldest transaction having the highest priority

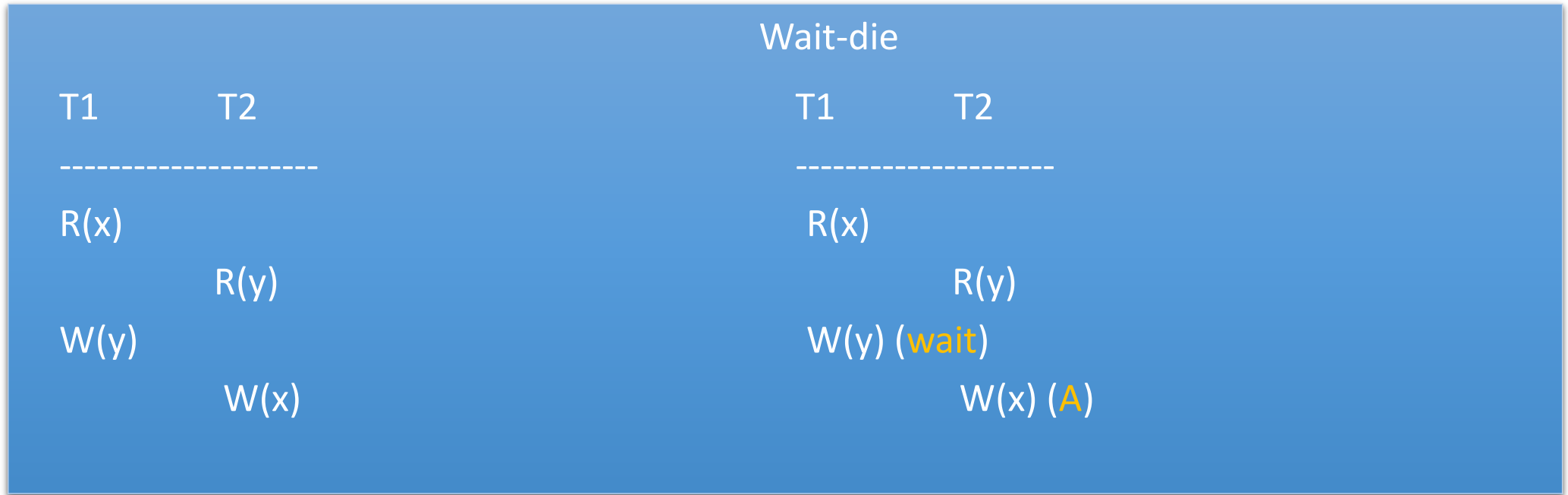
- 2 deadlock prevention policies: Wait-die and Wound-wait

Deadlocks - Prevention

- assume T_1 wants to access an object locked by T_2 (with a conflicting lock)
 - Wait-die
 - if T_1 's priority is higher, T_1 can wait; otherwise, T_1 is aborted
- in the following execution, 2 transactions are reading and / or writing 2 objects, x and y; T_1 's priority is higher:



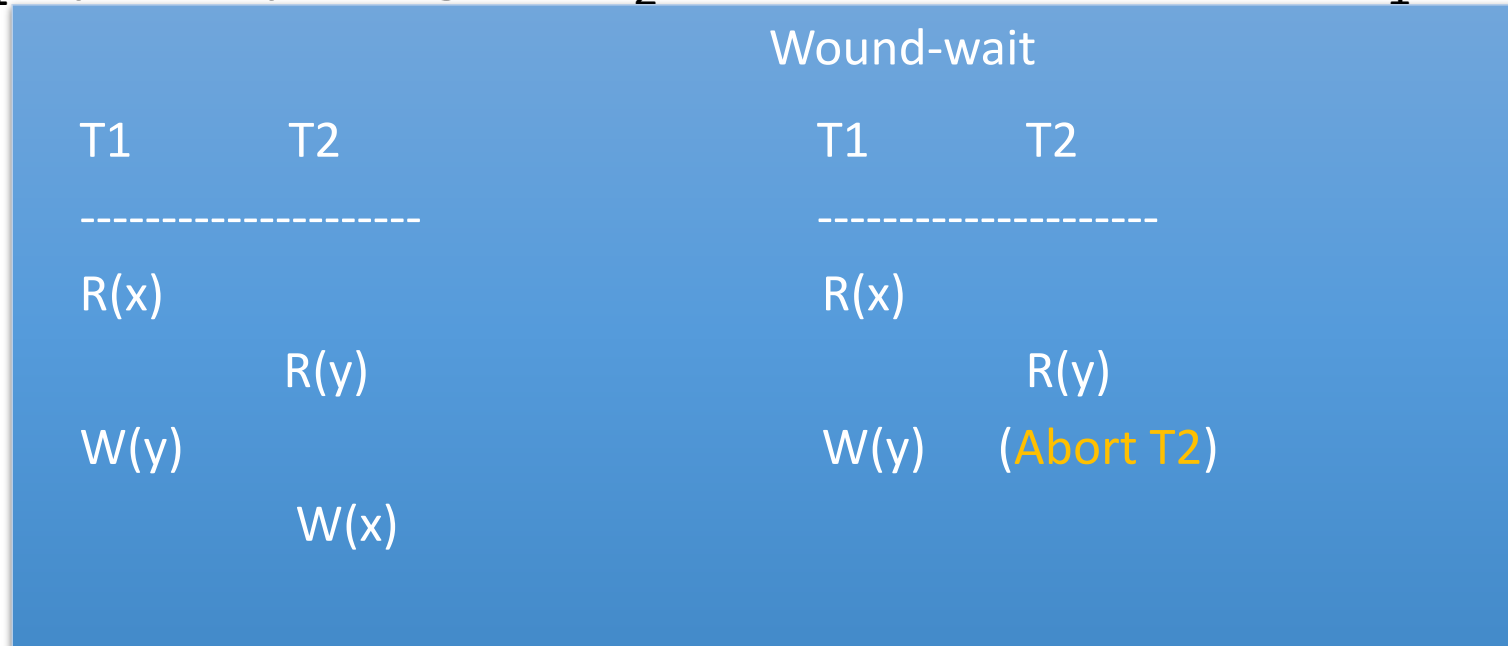
Deadlocks - Prevention



- T1 requests an X lock on object y, which is already locked with a conflicting lock by T2
- since T1 has a higher priority, it is allowed to wait
- T2 asks for an X lock on object x, already locked with a conflicting lock by T1
- since T2 has a lower priority, it is aborted
- T1 now obtains the requested lock on object y and proceeds with the write operation

Deadlocks - Prevention

- assume T_1 wants to access an object locked by T_2 (with a conflicting lock)
 - Wound-wait
 - if T_1 's priority is higher, T_2 is aborted; otherwise, T_1 can wait



- T1 requests an X lock on object y, which is already locked with a conflicting lock by T2
- since T1 has a higher priority, T2 is aborted
- T1 obtains the requested lock on object y and continues execution

Deadlocks - Prevention

- under these policies (Wait-die / Wound-wait), deadlock cycles cannot develop
- if an aborted transaction is restarted, it's assigned its original timestamp

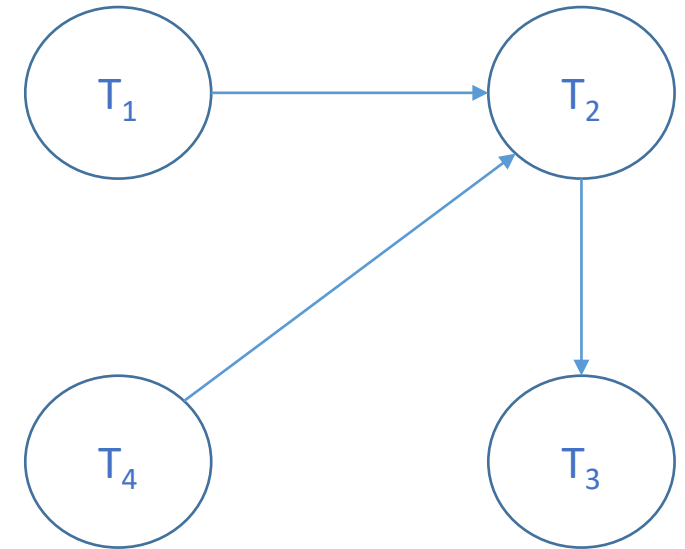
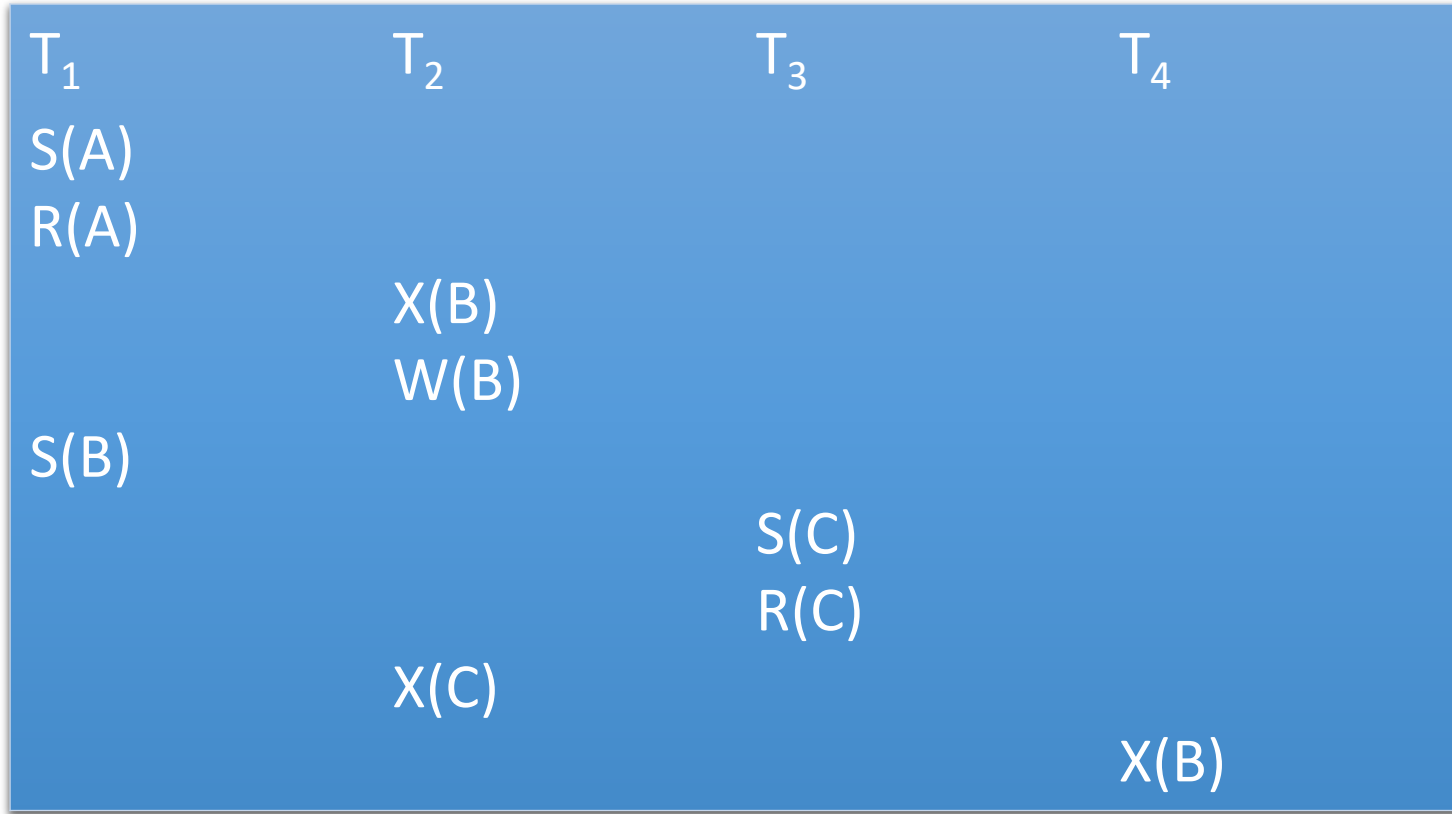
Deadlocks - Detection

a. waits-for graph

- structure maintained by the lock manager to detect deadlock cycles
 - a node / active transaction
 - arc from T_i to T_j if T_i is waiting for T_j to release a lock
- cycle in the graph $\Rightarrow$ deadlock
- DBMS periodically checks whether there are cycles in the waits-for graph

Deadlocks - Detection

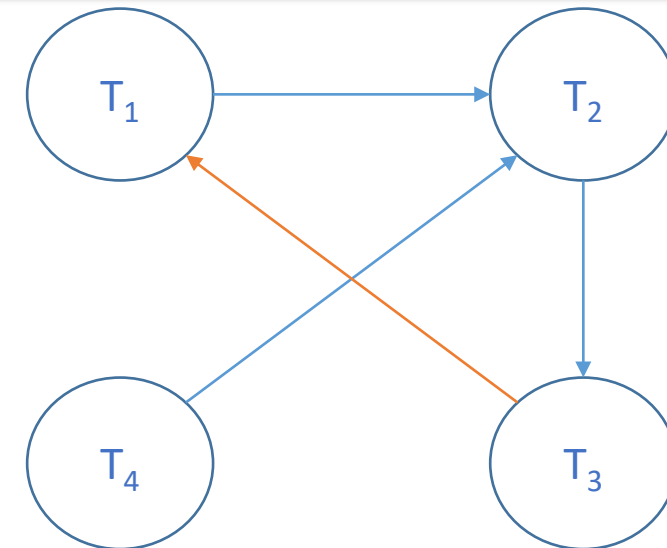
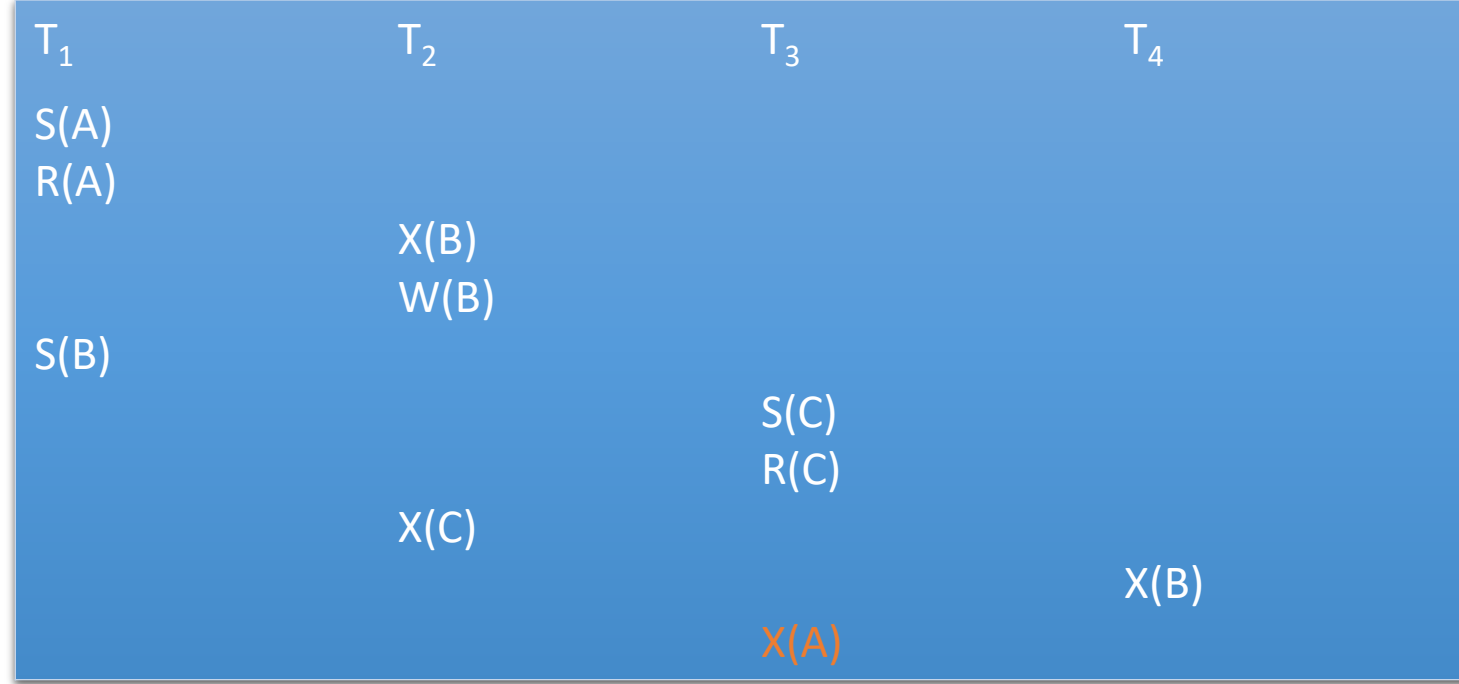
a. waits-for graph



Deadlocks - Detection

a. waits-for graph

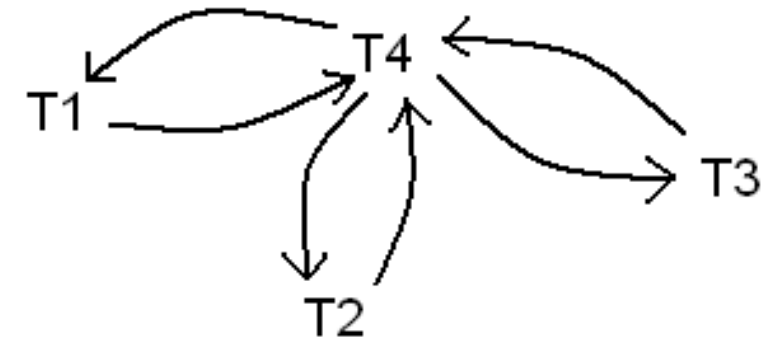
- if operation $X(A)$ is also part of T_3 , there will be an arc from T_3 to T_1 in the graph (since T_1 holds a conflicting lock on A , and T_3 is waiting for T_1 to release this lock)
- the graph has a cycle (T_1 is also waiting for T_2 to release a lock, which in turn is waiting for T_3 to release a lock)
=> deadlock
- aborting a transaction that appears on a cycle allows several other transactions to proceed



Deadlocks - Detection

a. waits-for graph

T_1	T_2	T_3	T_4
S(A) R(A)			
	S(A) R(A)		
		S(A) R(A)	
			S(B) R(B)
X(B)			
	X(B)		
...		X(B)	
	...		X(A)
		...	...



Deadlocks – Choosing the Deadlock Victim

- possible criteria to consider when choosing the deadlock victim
 - the number of objects modified by the transaction
 - the number of objects that are to be modified by the transaction
 - the number of locks held
- the policy should be “fair”, i.e., if a transaction is repeatedly chosen as a victim, it should be eventually allowed to proceed

Deadlocks - Detection

b. timeout mechanism

- very simple, practical method of detecting deadlocks
- if a transaction T has been waiting too long for a lock on an object, a deadlock is assumed to exist and T is terminated

The Phantom Problem

example 1. Researchers[RID, ..., ImpactFactor, Age]

- Page1: <R1, 5, 30>, <R2, 5, 20>
- Page2: <R3, 5, 100>, <R4, 5, 90>
- Page3: <R8, 6, 18>, <R9, 6, 19>
- concurrent transactions T1 and T2
 - transaction T1
 - retrieve the age of the oldest researcher for each of the impact factor values 5 and 6
 - transaction T2
 - add a new researcher with impact factor 5
 - remove researcher R9
- T1 and T2 obey Strict 2PL

The Phantom Problem

- Page1: <R1, 5, 30>, <R2, 5, 20>
- Page2: <R3, 5, 100>, <R4, 5, 90>
- Page3: <R8, 6, 18>, <R9, 6, 19>

- T1 identifies and locks pages holding researchers with IF 5 (Page1, Page2)
- T1 computes max age for IF 5 (100)
- T2 acquires X locks on: Page4 (onto which it adds a new researcher with IF 5 and age 102) and Page3 (from which it deletes researcher R9)
- T2 then commits, releasing all its locks
- T1 now obtains an S lock on Page3 (containing all researchers with IF 6), and computes max age for IF 6 (18)

T1

SLock(Page1)

SLock(Page2)

compute max age for IF 5 => 100

SLock(Page3)

compute max age for IF = 6 => 18

commit

T2

XLock(Page4)

add record <R5, 5, 102> to Page4

XLock(Page3)

delete researcher R9

commit – all locks are released

The Phantom Problem

- Page1: <R1, 5, 30>, <R2, 5, 20>
- Page2: <R3, 5, 100>, <R4, 5, 90>
- Page3: <R8, 6, 18>, <R9, 6, 19>
- outcome of interleaved schedule on the right:
 - IF 5, Max Age 100
 - IF 6, Max Age 18
- outcome of serial schedule (T1T2):
 - IF 5, Max Age 100
 - IF 6, Max Age 19
- outcome of serial schedule (T2T1):
 - IF 5, Max Age 102
 - IF 6, Max Age 18

T1

SLock(Page1)

SLock(Page2)

compute max age for IF 5 => 100

SLock(Page3)

compute max age for IF = 6 => 18

commit

T2

XLock(Page4)

add record <R5, 5, 102> to
Page4

XLock(Page3)

delete researcher R9

commit – all locks are released

->

The Phantom Problem

=> the interleaved schedule is not serializable (no serial schedule over the same set of transactions has the same outcome)

- however, the schedule is conflict serializable (the precedence graph is acyclic)

=> in the presence of insert operations, i.e., if new objects can be added to the database, conflict serializability does not guarantee serializability

The Phantom Problem example 2.

T1

```
SELECT *  
FROM Students  
WHERE GPA >= 8
```

```
SELECT *  
FROM Students  
WHERE GPA >= 8
```

...

T2

```
INSERT INTO Students VALUES  
(12, 'Ana', 'Ionescu', 10)  
COMMIT
```

result set for T1's query

row corresponding to student with sid 12
is not in the result set

row corresponding to student with sid 12
now appears in the result set

Transaction Support in SQL - Isolation Levels

- *isolation level*
 - determines the degree to which a transaction is exposed to the operations of other concurrently running transactions
- greater concurrency -> concurrency anomalies

Transaction Support in SQL - Isolation Levels

- isolation levels
 - READ UNCOMMITTED
 - READ COMMITTED
 - REPEATABLE READ
 - SERIALIZABLE
- isolation levels can be set with the following command:
 - SET TRANSACTION ISOLATION LEVEL *isolevel*
 - *e.g.*, SET TRANSACTION ISOLATION LEVEL REPEATABLE READ
- *dirty writes* are not allowed under any isolation level (*dirty write* - a transaction T1 modifies an object previously written by an ongoing transaction T2)

Isolation Levels

- READ UNCOMMITTED

- a transaction T can read data modified by an ongoing transaction (uncommitted data)
- lowest degree of isolation
- no S locks when reading data
- dirty reads ✓
- unrepeatable reads ✓
- phantoms ✓

Isolation Levels

- READ COMMITTED

- a transaction T can only read committed data
- however, an object read by T can be changed by another transaction while T is in progress
- a transaction must acquire an exclusive lock prior to writing an object
- a transaction must acquire a shared lock prior to reading an object (i.e., the last transaction that modified the object is finished)
- exclusive locks are released at the end of the transaction
- shared locks are immediately released
- dirty reads ✗
- unrepeatable reads ✓
- phantoms ✓

Isolation Levels

- REPEATABLE READ

- a transaction T can only read committed data
- no object read by T can be changed by another transaction while T is in progress (if T reads an object O twice, no transaction can modify O between T's reads)
- a transaction must acquire an exclusive lock prior to writing an object
- a transaction must acquire a shared lock prior to reading an object
- exclusive locks are released at the end of the transaction
- shared locks are released at the end of the transaction
- dirty reads ✗
- unrepeatable reads ✗
- phantoms ✓

Isolation Levels

- SERIALIZABLE

- a transaction T can only read committed data
- no object read by T can be changed by another transaction while T is in progress
- if T reads a set of objects based on a search predicate, this set cannot be changed by other transactions while T is in progress
- a transaction must acquire locks on objects before reading / writing them
- a transaction also acquires locks on sets of objects that must remain unmodified
 - if query `SELECT * FROM Students WHERE GPA >= 8` is executed twice within a transaction, it must return the same answer set
- locks are held until the end of the transaction

Isolation Levels

- SERIALIZABLE
 - highest degree of isolation
 - dirty reads ✕
 - unrepeatable reads ✕
 - phantoms ✕

References

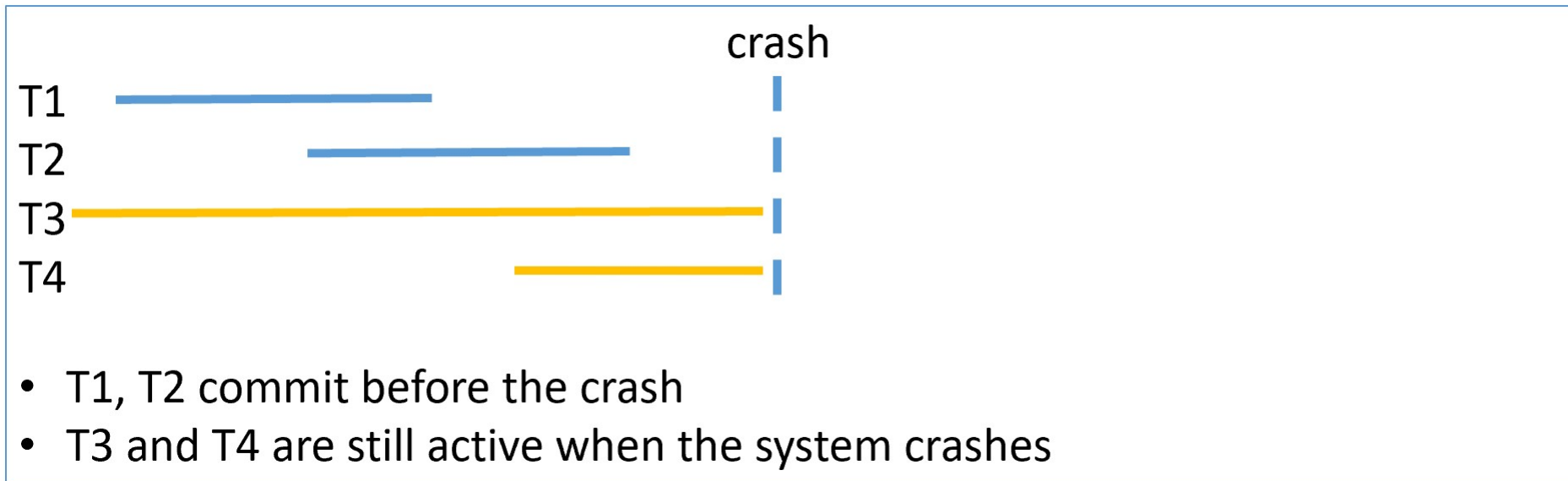
- [Ra02] RAMAKRISHNAN, R., GEHRKE, J., Database Management Systems (3rd Edition), McGraw-Hill, 2002
- [Le99] LEVENE, M., LOIZOU, G., A Guided Tour of Relational Databases and Beyond, Springer, 1999
- [Ga09] GARCIA-MOLINA, H., ULLMAN, J., WIDOM, J., Database Systems: The Complete Book (2nd Edition), Pearson Education, 2009
- [Ra02S] RAMAKRISHNAN, R., GEHRKE, J., Database Management Systems, Slides for the 3rd Edition,
<http://pages.cs.wisc.edu/~dbbook/openAccess/thirdEdition/slides/slides3ed.html>
- [Si19] SILBERSCHATZ, A., KORTH, H., SUDARSHAN, S., Database System Concepts (7th Edition), McGraw-Hill, 2019
- [Si19S] SILBERSCHATZ, A., KORTH, H., SUDARSHAN, S., Database System Concepts, Slides for the 7th Edition, <http://codex.cs.yale.edu/avi/db-book/>
- [Ul11] ULLMAN, J., WIDOM, J., A First Course in Database Systems,
<http://infolab.stanford.edu/~ullman/fcdb.html>

Database Management Systems

Lectures 4 and 5
Crash Recovery

Recovery Manager

- the Recovery Manager in a DBMS ensures two important properties of transactions:
 - atomicity - the effects of uncommitted transactions are undone
 - durability - the effects of committed transactions survive system crashes



- the system comes back up:
 - the effects of T1 & T2 must persist
 - T3 & T4 are undone (their effects are not persisted in the DB)

Transaction Failure - Causes

- system failure (hardware failures, bugs in the operating system, database system, etc.)
 - all running transactions terminate
 - contents of internal memory – affected (i.e., lost)
 - contents of external memory – not affected
- application error (“bug”, e.g., division by 0, infinite loop, etc.)
=> transaction fails; it should be executed again only after the error is corrected
- action by the Transaction Manager (TM)
 - e.g., deadlock resolution scheme
 - a transaction is chosen as the deadlock victim and terminated
 - the transaction might complete successfully if executed again

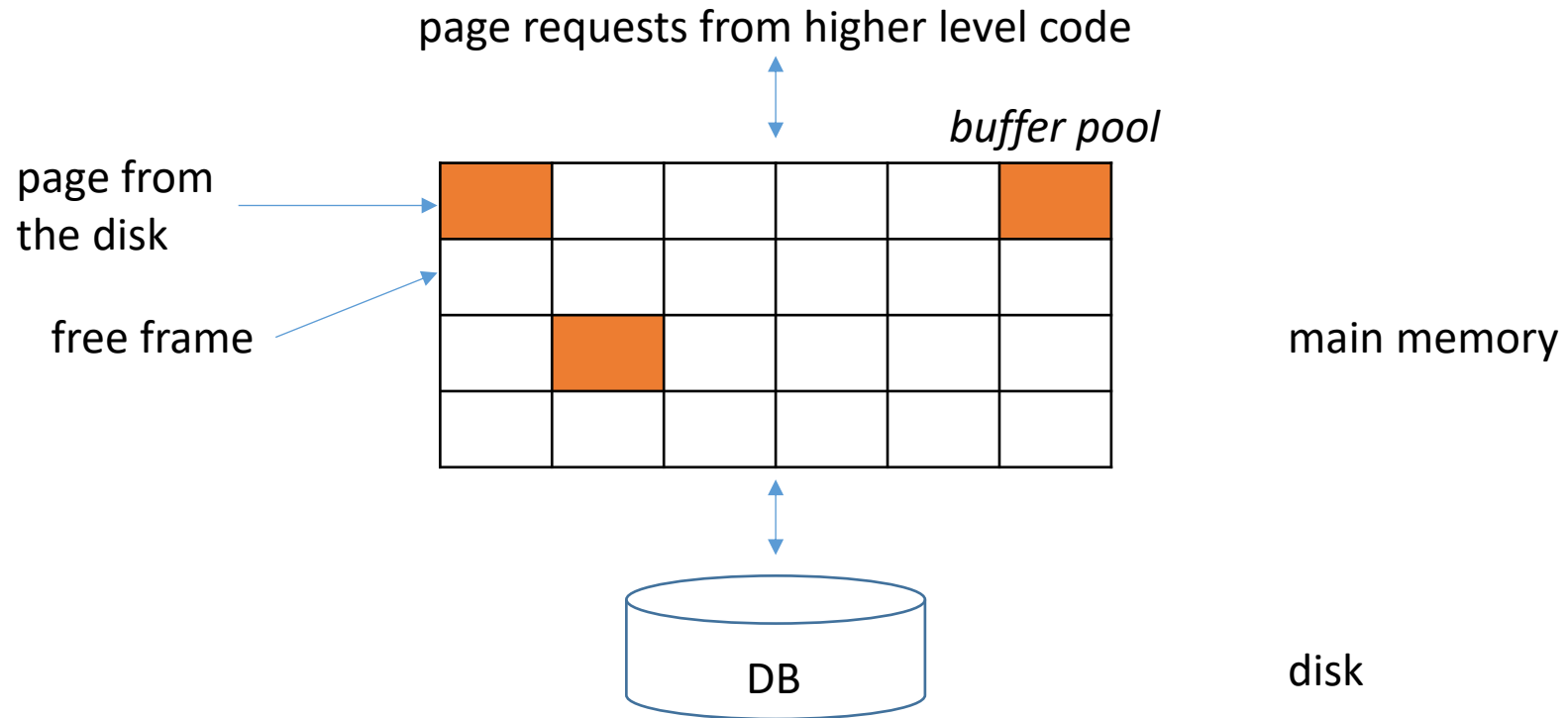
Transaction Failure - Causes

- self-abort
 - based on some computations, a transaction can decide to terminate and undo its actions
 - there are special statements for this purpose, e.g., *ABORT*, *ROLLBACK*
 - can be seen as a special case of *action by the TM*

Normal Execution

- during normal execution, transactions read / write database objects
- reading database object O:
 - bring O from disk into a frame in the Buffer Pool (BP)
 - copy O's value into a program variable
- writing database object O:
 - modify an in-memory copy of O (in the BP)
 - write the in-memory copy to disk

Buffer Manager*



*see the *Databases* course in the 1st semester (lecture 8 - Buffer Manager)

Writing Objects

- options: *steal / no-steal, force / no-force*
- transaction T changes object O (in frame F in the BP)
 - *steal* approach
 - T's changes can be written to disk before it commits
 - transaction T2 needs a page; the BM chooses F as a replacement frame (while T is in progress); T2 steals a frame from T
 - *no-steal* approach
 - T's changes cannot be written to disk before it commits
 - *force* approach
 - T's changes are immediately forced to disk when it commits
 - *no-force* approach
 - T's changes are not forced to disk when it commits

Writing Objects

- *no-steal* approach
 - advantage - changes of aborted transactions don't have to be undone (such changes are never written to disk!)
 - drawback - assumption: all pages modified by active transactions can fit in the BP
- *force* approach
 - advantage - actions of committed transactions don't have to be redone
 - by contrast, when using *no-force*, the following scenario is possible: transaction T commits at time t_0 ; its changes are not immediately forced to disk; the system crashes at time $t_1 \Rightarrow$ T's changes have to be redone!
 - drawback - can result in excessive I/O
- *steal, no-force* approach – used by most systems

Storage Media

- volatile storage
 - information doesn't usually survive system crashes (e.g., main memory)
- non-volatile storage
 - information survives system crashes (e.g., magnetic disks, flash storage)
- stable storage
 - information is never lost
 - techniques that approximate stable storage (e.g., store information on multiple disks, in several locations)

ARIES

- recovery algorithm; *steal, no-force* approach
- system restart after a crash - three phases:
 - analysis - determine:
 - active transactions at the time of the crash
 - *dirty pages*, i.e., pages in BP whose changes have not been written to disk
 - redo: reapply all changes (starting from a certain record in the log), i.e., bring the DB to the state it was in when the crash occurred
 - undo: undo changes of uncommitted transactions
- fundamental principle - *Write-Ahead Logging*
 - a change to an object O is first recorded in the log (in a log record LR)
 - LR must be written to stable storage before the change to O is written to disk

ARIES

* example

- analysis
 - active transactions at crash time: T1, T3 (to be undone)
 - committed transactions: T2 (its effects must persist)
 - potentially dirty pages: P1, P2, P3
- redo
 - reapply all changes in order (1, 2, ...)
- undo
 - undo changes of T1 and T3 in reverse order (6, 5, 1)

LSN	Log
1	update: T1 writes P1
2	update: T2 writes P2
3	T2 commit
4	T2 end
5	update: T3 writes P3
6	update: T3 writes P2
crash, restart	

The Log (journal)

- history of actions executed by the DBMS
- file of records
- stored in *stable storage* (keep ≥ 2 copies of the log on different disks (locations) - ensures the durability of the log)
- records are added to the end of the log
- *log tail*
 - the most recent fragment of the log
 - kept in main memory and periodically forced to stable storage
- *Log Sequence Number* (LSN)
 - unique id for every log record
 - monotonically increasing (e.g., address of 1st byte of log record)

The Log

- *pageLSN*
 - every page P in the DB contains the *pageLSN*: the LSN of the most recent record in the log describing a change to P
- *log record* – fields:
 - prevLSN - linking a transaction's log records
 - transID - id of the corresponding transaction
 - type - type of the log record
- a log record is written for each of the following actions:
 - *update page*
 - *commit*
 - *abort*
 - *end*
 - *undo an update*

The Log

- update page P
 - add an *update type* log record ULR to the log tail (with LSN_{ULR})
 - $pageLSN(P)$ is set to LSN_{ULR}
- transaction T commits*
 - add a *commit type* log record CoLR to the log
 - force log tail to stable storage (including CoLR)
 - complete subsequent actions (remove T from transaction table)
- transaction T aborts
 - add an *abort type* log record to the log
 - initiate Undo for T

* obs. committed transaction – a transaction whose log records (including the *commit log record*) have been written to stable storage

The Log

- transaction T ends
 - T commits / aborts - complete required actions
 - add an *end type* log record to the log
- undo an update
 - i.e., when the change described in an update log record is undone
 - write a *compensation log record* (CLR)
- *update log record*
 - additional fields
 - *pageID* (id of the changed page)
 - *length* (length of the change - in bytes)
 - *offset* (offset of the change)
 - *before-image* (value before the change)
 - *after-image* (value after the change)
 - can be used to undo / redo the change

The Log

- *compensation log record*
 - let U be an update log record describing an update of transaction T
 - let C be the compensation log record for U, i.e., C describes the action taken to undo the changes described by U
 - C has a field named *undoNextLSN*:
 - the LSN of the next log record to be undone for T
 - set to the value of prevLSN in U

->

The Log

- *compensation log record*

* example: undo T10's update to P10

=> CLR with:

transID = T10

pageID = P10

length = 2

offset = 10

before-image = JH

undoNextLSN = LSN of 1st log record (i.e., the next record that is to be undone for transaction T10)



prevLSN	transID	type	pageID	length	offset	before-image	after-image
	T10	update	P100	2	10	AB	CD
	T15	update	P2	2	10	YW	ZA
	T15	update	P100	2	9	EC	YW
	T10	update	P10	2	10	JH	AB

log

The Transaction Table and the Dirty Page Table

- contain important information for the recovery process
- *transaction table*: 1 entry / active transaction
 - fields
 - *transID*
 - *status* (in progress, committed, aborted)
 - *lastLSN*: LSN of the most recent log record for the transaction
- example (*status* = in progress, not displayed):

transID	lastLSN
T10	
T15	

transaction table

prevLSN	transID	type	pageID	length	offset	before-image	after-image
	T10	update	P100	2	10	AB	CD
	T15	update	P2	2	10	YW	ZA
	T15	update	P100	2	9	EC	YW
	T10	update	P10	2	10	JH	AB

log

The Transaction Table and the Dirty Page Table

- *dirty page table*: 1 entry / dirty page in the Buffer Pool
 - fields
 - *pageID*
 - *recLSN*: the LSN of the 1st log record that dirtied the page
 - example:

pageID	recLSN
P100	
P2	
P10	

dirty page table

transID	lastLSN
T10	
T15	

transaction table

prevLSN	transID	type	pageID	length	offset	before-image	after-image
	T10	update	P100	2	10	AB	CD
	T15	update	P2	2	10	YW	ZA
	T15	update	P100	2	9	EC	YW
	T10	update	P10	2	10	JH	AB

Checkpointing

- objective
 - reduce the amount of work performed by the system when it comes back up after a crash
- *checkpoints* taken periodically
- checkpointing in ARIES - 3 steps:
 - write a *begin_checkpoint* record (it indicates when the checkpoint starts)
 - LSN_{BCK} - LSN of *begin_checkpoint* record
 - write an *end_checkpoint* record
 - it includes the current Transaction Table and the current Dirty Page Table

Checkpointing

- checkpointing in ARIES - 3 steps:
 - after the *end_checkpoint* record is written to stable storage:
 - write a *master* record to a known place on stable storage
 - *master* record includes LSN_{BCK}
- crash -> restart -> system looks for the most recent checkpoint
- normal execution begins with a checkpoint with an empty Transaction Table and an empty Dirty Page Table

Recovery - overview

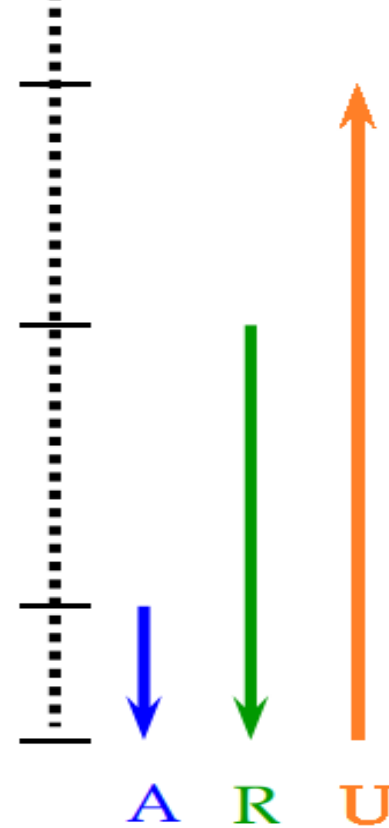
- system restart after a crash – 3 phases:
 - **Analysis**
 - reconstructs state at the most recent checkpoint
 - scans the log forward from the most recent checkpoint
 - identifies:
 - active transactions at the time of the crash (to be undone)
 - potentially dirty pages at the time of the crash
 - the starting point for the Redo pass

the oldest log record of transactions that were active at the time of the crash

the smallest recLSN in the dirty page table (determined during the Analysis pass)

the most recent checkpoint (master record)

system crash



Recovery - overview

- system restart after a crash – 3 phases:
 - **Redo**
 - repeats history, i.e., reapplies changes to dirty pages
 - all updates are reapplied (regardless of whether the corresponding transaction committed or not)
 - starting point is determined in the Analysis pass
 - scans the log forward until the last record
 - **Undo**
 - the effects of transactions that were active at the time of the crash are undone
 - such changes are undone in the opposite order (i.e., Undo scans the log backward from the last record)

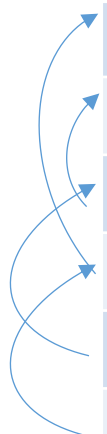
* Analysis

- investigate the most recent *begin_checkpoint* log record
 - get the next *end_checkpoint* log record EC
- set Dirty Page Table to the copy of the Dirty Page Table in EC
- set Transaction Table to the copy of the Transaction Table in EC
- scan the log forward from the most recent checkpoint:
 - transactions:
 - encounter **end log record** for transaction T:
 - remove T from Transaction Table
 - encounter **other log records (LR)** for transaction T:
 - add T to Transaction Table if not already there
 - set T.lastLSN to LR.LSN
 - if LR is a commit type log record:
 - set T's status to *C*
 - otherwise, set status to *U* (i.e., to be undone)

* Analysis

- scan the log forward from the most recent checkpoint:
 - pages:
 - encounter redoable log record (LR) for page P:
 - if P is not in the Dirty Page Table:
 - add P to Dirty Page Table
 - set P.recLSN to LR.LSN

Example 1



prevLSN	transID	type	pageID	length	offset	before-image	after-image
	T10	update	P100	2	10	AB	CD
	T15	update	P2	2	10	YW	ZA
	T15	update	P100	2	9	EC	YW
	T10	update	P10	2	10	JH	AB
	T15	commit					
	T10	update	P11	2	20	GF	YT

log

- first 5 log records are written to stable storage
- system crashes before the 6th log record is written to stable storage

Analysis

- most recent checkpoint – beginning of execution (empty Transaction Table, empty Dirty Page Table)
- 1st log record
 - add T10 to the Transaction Table
 - add P100 to the Dirty Page Table (recLSN = LSN(1st log record))

Example 1



prevLSN	transID	type	pageID	length	offset	before-image	after-image
	T10	update	P100	2	10	AB	CD
	T15	update	P2	2	10	YW	ZA
	T15	update	P100	2	9	EC	YW
	T10	update	P10	2	10	JH	AB
	T15	commit					

log

Analysis

- 2nd log record
 - add T15 to the Transaction Table
 - add P2 to the Dirty Page Table (recLSN = LSN(2nd log record))
- 4th log record
 - add P10 to the Dirty Page Table (recLSN = LSN(4th log record))
- active transactions at the time of the crash:
 - transactions with status *U*, i.e., T10 (T15 is a committed transaction)

Example 1

Analysis

- Dirty Page Table:
 - can include pages that were written to disk prior to the crash
 - assume P2's update is the only change written to disk before the crash, i.e., P2 is not dirty, but it's in the Dirty Page Table
 - the pageLSN on page P2 is equal to the LSN of the 2nd log record
- log record

T10	update	P11	2	20	GF	YT
-----	--------	-----	---	----	----	----

 is not seen during Analysis (it was not written to disk before the crash)
- Write-Ahead Logging protocol => the corresponding change to page P11 cannot have been written to disk

* Redo

- *repeat history*: reconstruct state at the time of the crash
 - reapply *all* updates (even those of aborted transactions!), reapply CLR
- scan the log forward from the log record with the smallest recLSN in the Dirty Page Table
- for each **redoable** log record **LR** affecting page P, redo the described action unless one of the conditions below is satisfied:
 - page P is not in the Dirty Page Table
 - page P is in the Dirty Page Table, but $P.\text{recLSN} > LR.\text{LSN}$
 - $P.\text{pageLSN (in DB)} \geq LR.\text{LSN}$
- to redo an action:
 - reapply the logged action
 - set $P.\text{pageLSN}$ to $LR.\text{LSN}$
 - no additional logging!

* Redo

- at the end of Redo:
 - for every transaction T with status C:
 - add an end log record
 - remove T from the Transaction Table

Example 1



prevLSN	transID	type	pageID	length	offset	before-image	after-image
	T10	update	P100	2	10	AB	CD
	T15	update	P2	2	10	YW	ZA
	T15	update	P100	2	9	EC	YW
	T10	update	P10	2	10	JH	AB
	T15	commit					

log

Redo

- previously stated assumption: P2's update is the only change written to disk before the crash, i.e., P2 is not dirty, but it's in the Dirty Page Table
- Dirty Page Table -> smallest recLSN is the LSN of the 1st log record
- 1st log record
 - fetch page P100 (its pageLSN is less than the LSN of the current log record) => reapply update, set P100.pageLSN to the LSN of the 1st log record

Example 1



prevLSN	transID	type	pageID	length	offset	before-image	after-image
	T10	update	P100	2	10	AB	CD
	T15	update	P2	2	10	YW	ZA
	T15	update	P100	2	9	EC	YW
	T10	update	P10	2	10	JH	AB
	T15	commit					

log

Redo

- 2nd log record
 - fetch page P2
 - P2.pageLSN = LSN of the current log record => update is not reapplied
- 3rd, 4th log records – processed similarly

* Undo

- *loser transaction* – transaction that was active at the time of the crash
- ToUndo = { l | l - lastLSN of a *loser* transaction }
- repeat:
 - choose the largest LSN in ToUndo and process the corresponding log record LR; let T be the corresponding transaction
 - if LR is a CLR:
 - if undoNextLSN == NULL
 - write an end log record for T
 - else
 - add undoNextLSN to ToUndo
 - else {LR is an update log record}
 - write a CLR
 - undo the update
 - add LR.prevLSN to ToUndo

until ToUndo is empty

Example 1



prevLSN	transID	type	pageID	length	offset	before-image	after-image
	T10	update	P100	2	10	AB	CD
	T15	update	P2	2	10	YW	ZA
	T15	update	P100	2	9	EC	YW
	T10	update	P10	2	10	JH	AB
	T15	commit					

log

Undo

- active transaction at the time of the crash: T10
- lastLSN of T10: LSN of the 4th log record
- 4th log record
 - write CLR, undo update
 - add LSN of 1st log record to ToUndo

Example 1



prevLSN	transID	type	pageID	length	offset	before-image	after-image
	T10	update	P100	2	10	AB	CD
	T15	update	P2	2	10	YW	ZA
	T15	update	P100	2	9	EC	YW
	T10	update	P10	2	10	JH	AB
	T15	commit					

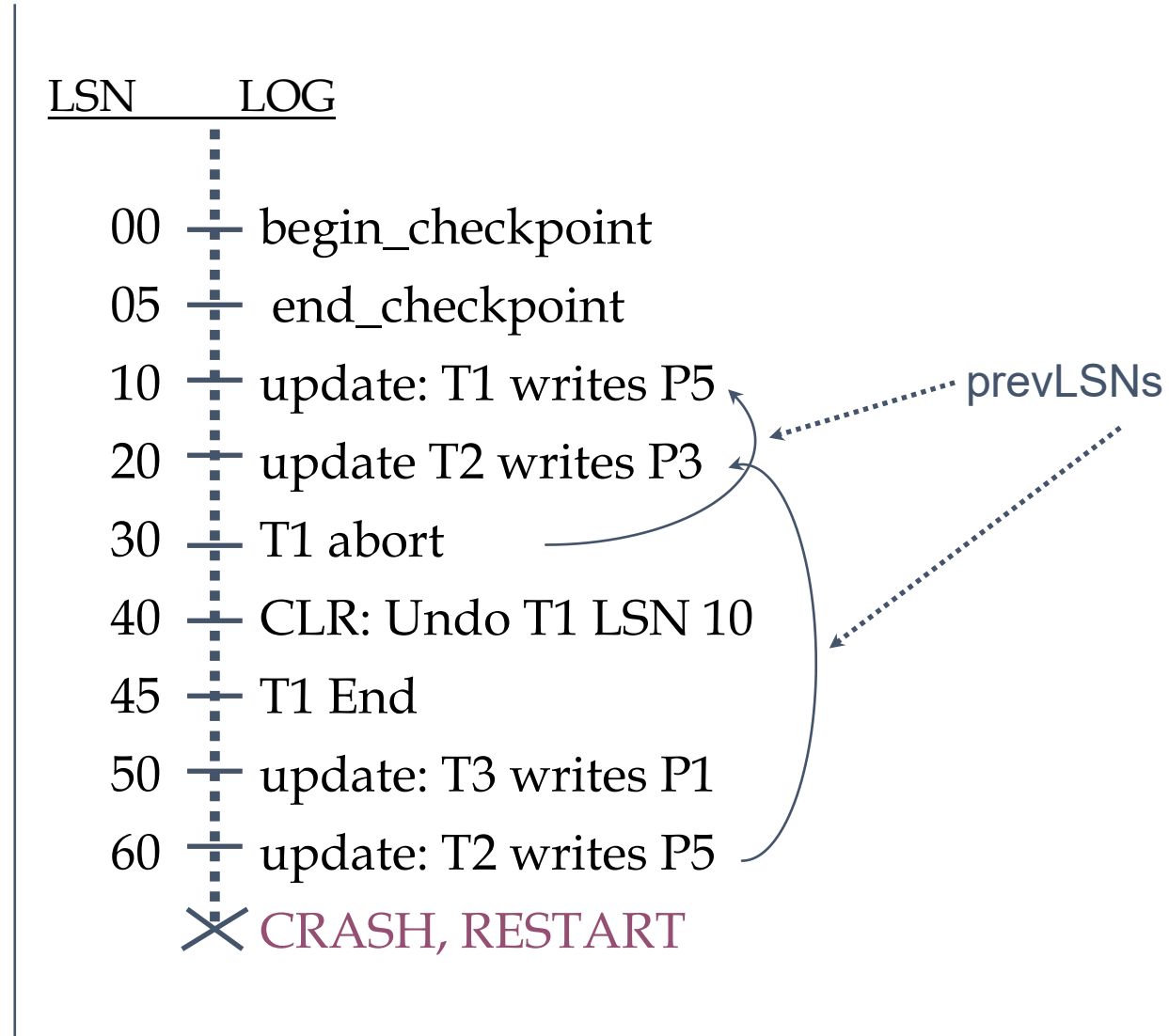
log

Undo

- 1st log record
 - write CLR, undo update (!T15's change to P100 is lost!), write end log record for T10
- obs. if Strict 2PL is used, T15 cannot write P100 while T10 is active (T10 also modified P100)

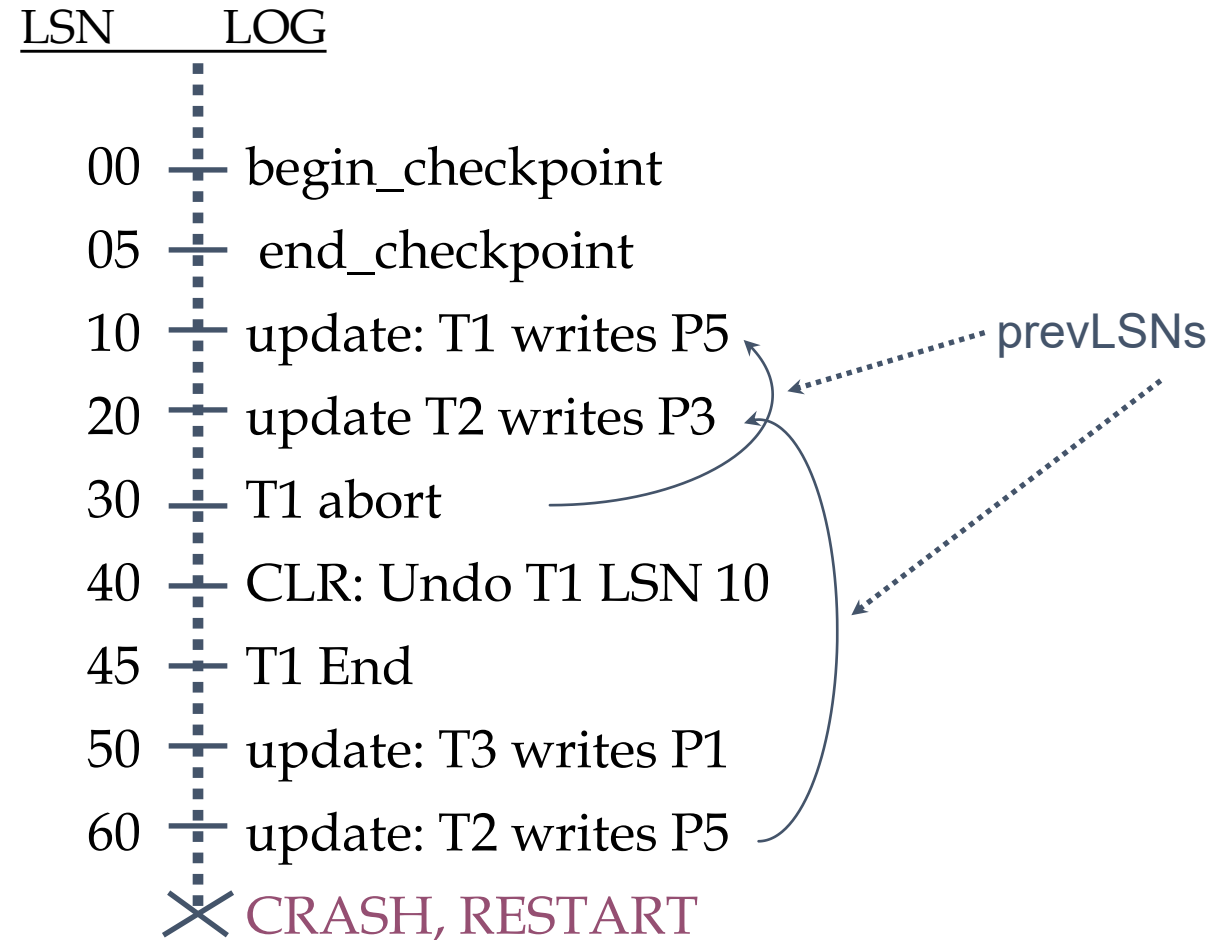
Example 2 – system crashes during Undo

- consider the execution history below:



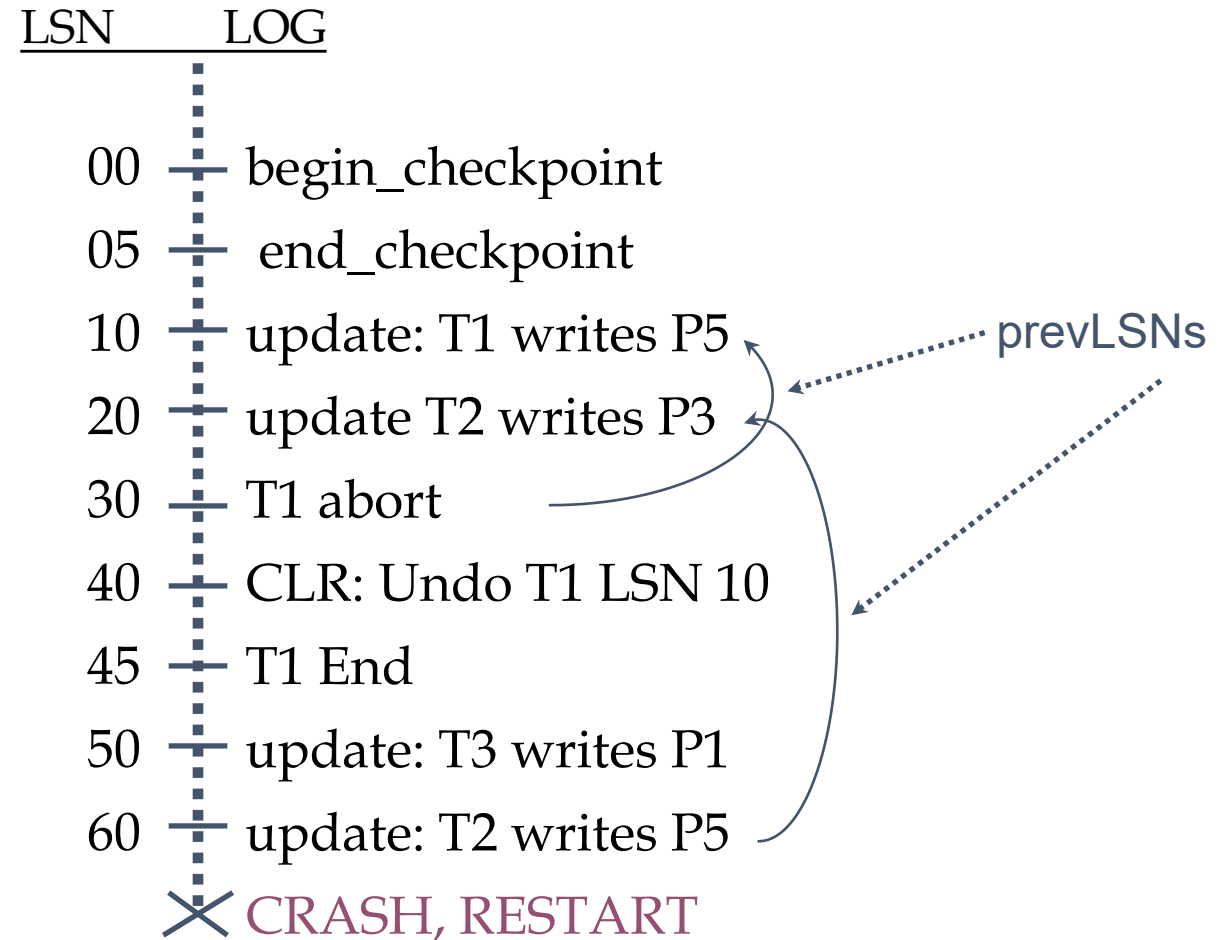
Example 2

- T1 aborts
=> its only update is undone (CLR with LSN 40)
- T1 - terminated
- **1st crash:**
 - **Analysis:**
 - dirty pages: P5 (recLSN 10), P3 (recLSN 20), P1 (recLSN 50)
 - active transactions at the time of the crash: T2 (lastLSN 60), T3 (lastLSN 50)



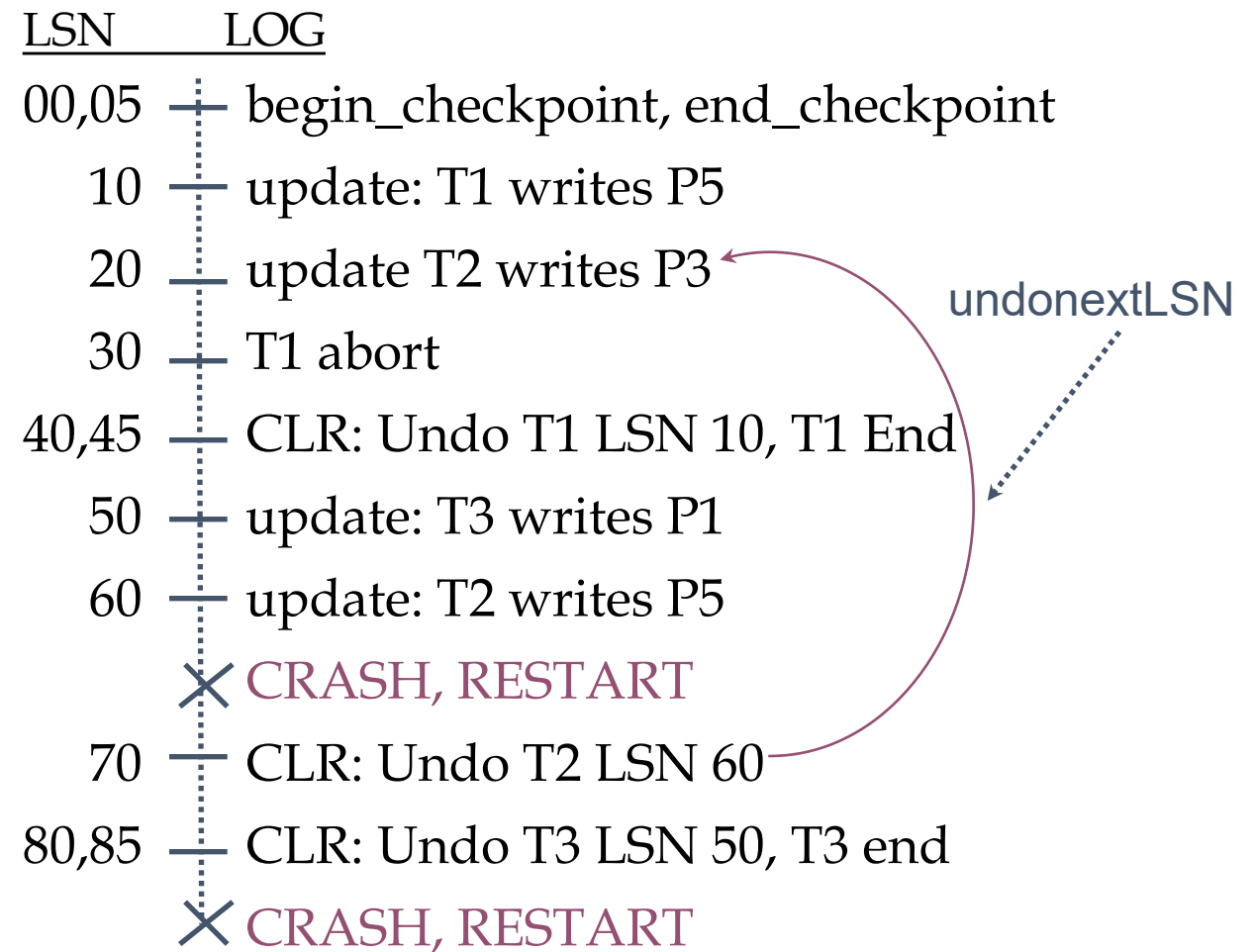
Example 2

- **1st crash:**
 - **Redo:**
 - starting point
 - log record with LSN = 10 (smallest recLSN in the Dirty Page Table)
 - reapply required actions in update log records / compensation log records



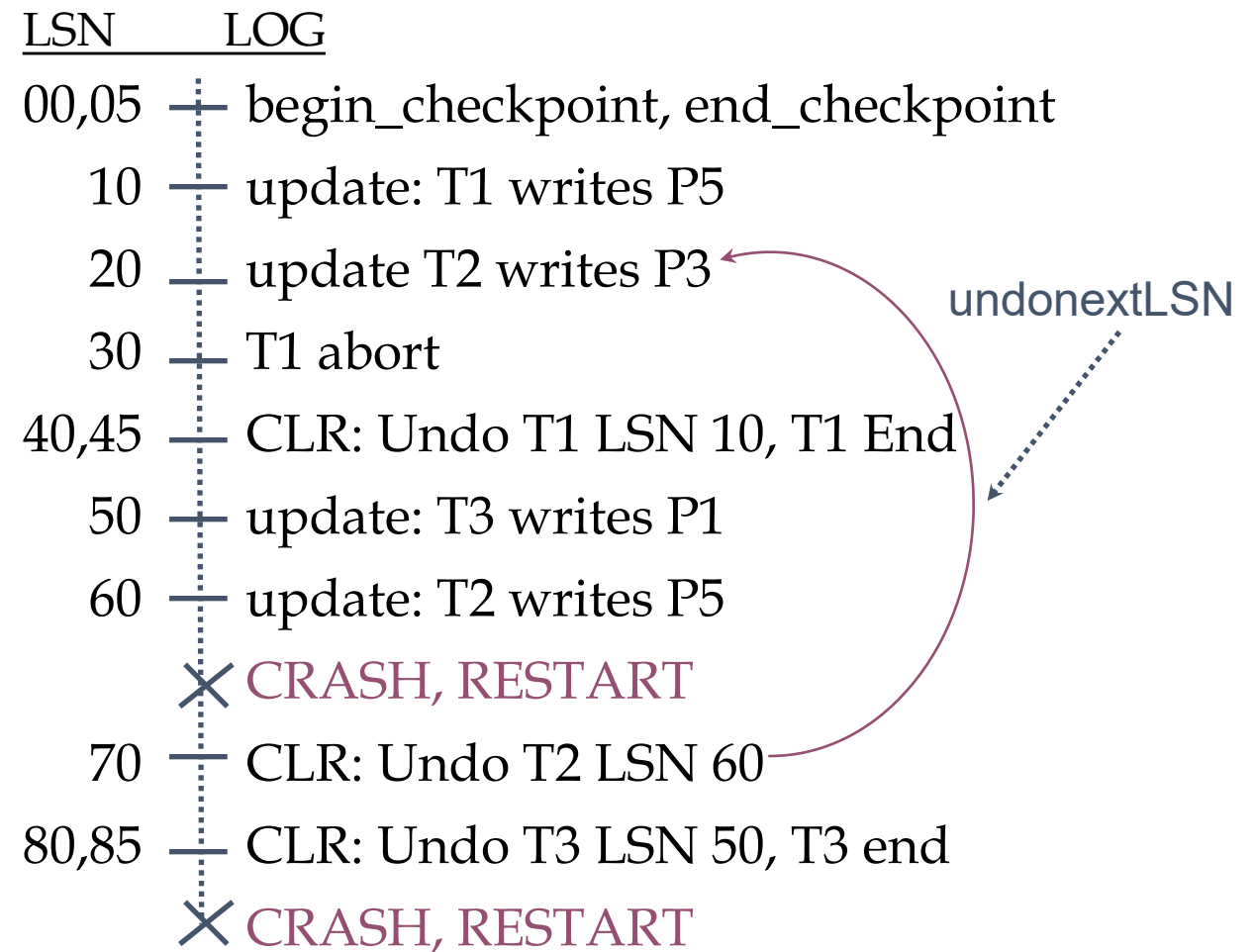
Example 2

- **1st crash:**
 - **Undo:**
 - T2, T3 – loser transactions
=> ToUndo = {60, 50}
 - process log record with LSN 60:
 - undo update, write CLR (LSN 70) with undoNextLSN 20 (i.e., the next log record that should be processed for T2)



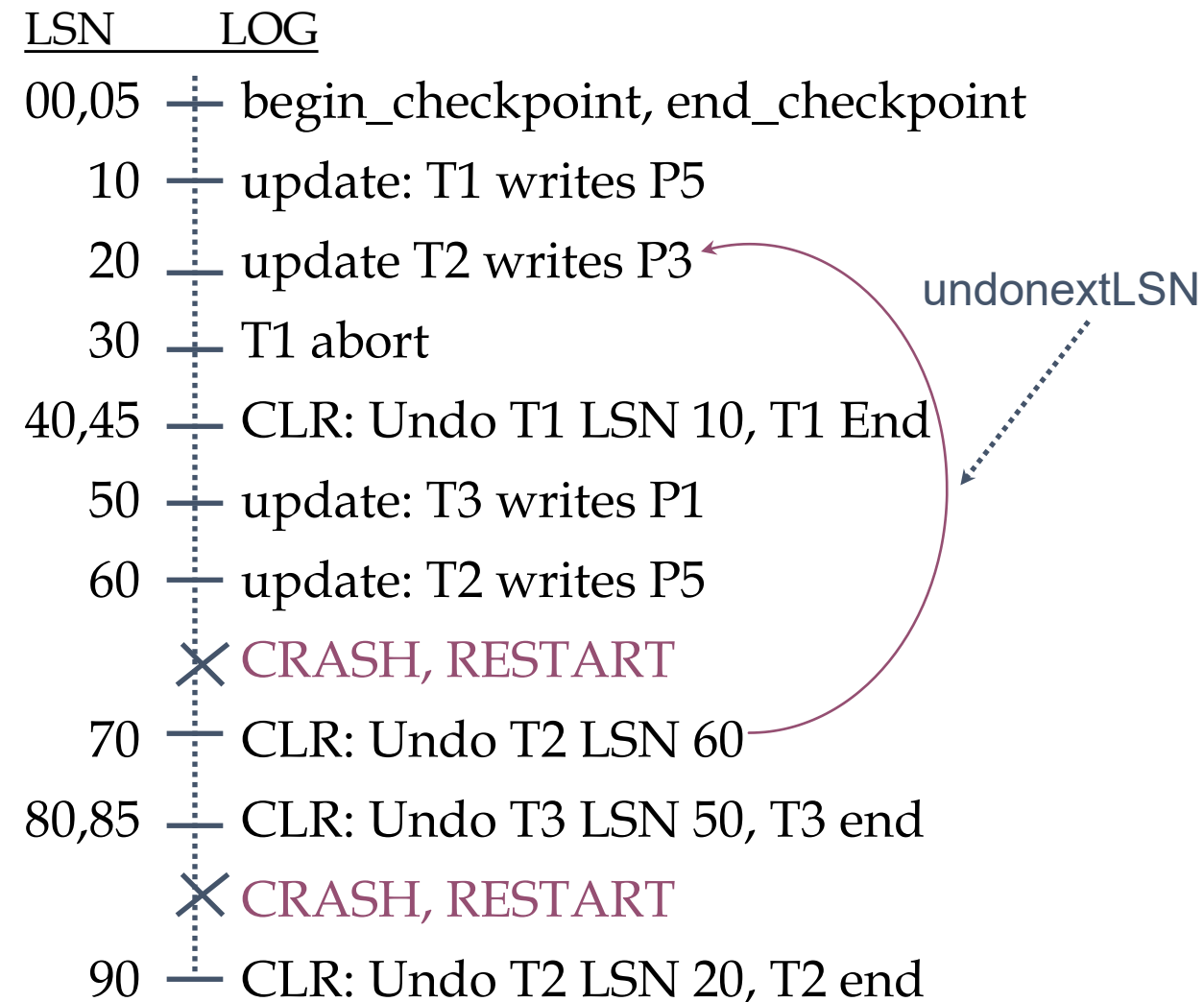
Example 2

- **1st crash:**
 - **Undo:**
 - process log record with LSN 50:
 - undo update, write CLR (LSN 80) with undoNextLSN *null* (i.e., T3 completely undone, write end log record for T3)
 - log records with LSN 70, 80, 85 are written to stable storage
- **2nd crash (during undo)!**



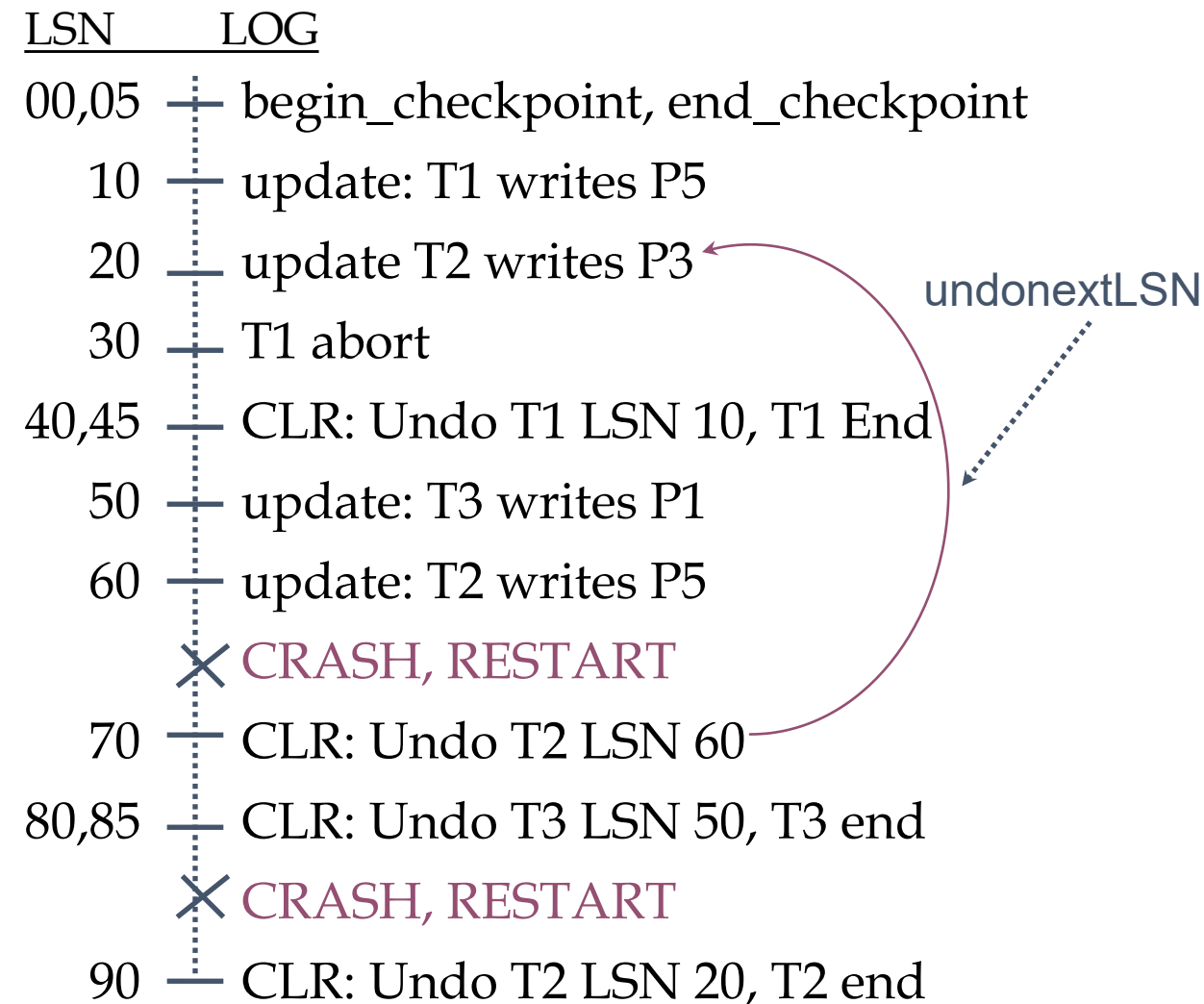
Example 2

- **2nd crash:**
 - **Analysis:**
 - the only active transaction: T2
 - dirty pages: P5 (recLSN 10), P3 (recLSN 20), P1 (recLSN 50)
 - **Redo:**
 - process log records with LSN between 10 and 85



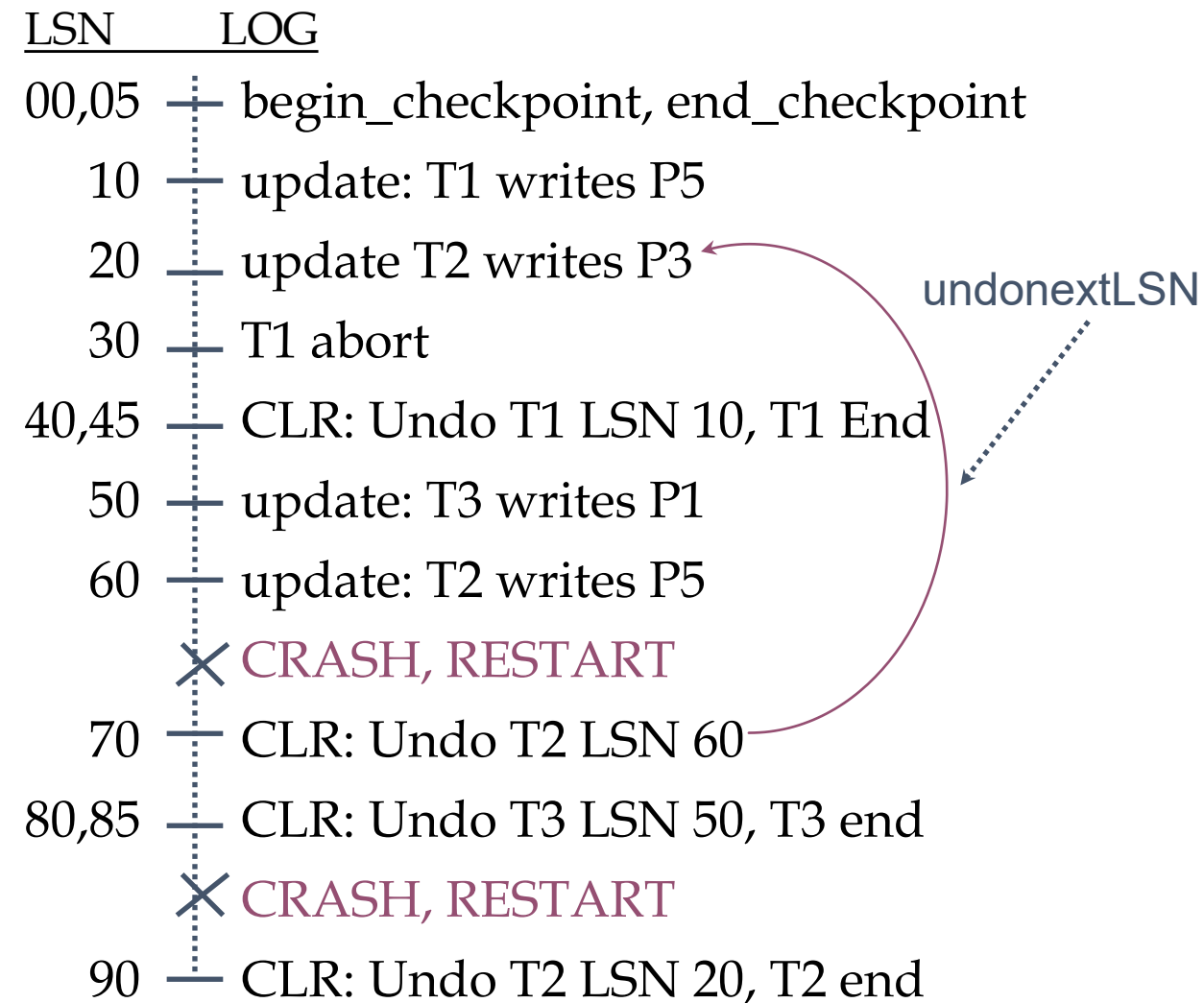
Example 2

- 2nd crash:
 - Undo:
 - lastLSN of T2: 70
 - ToUndo = {70}
 - process log record with LSN 70:
 - add 20 (undoNextLSN) to ToUndo
 - process log record with LSN 20:
 - undo update, write CLR (LSN 90) with undoNextLSN *null* => write end log record for T2



Example 2

- 2nd crash:
 - Undo:
 - ToUndo empty
- => recovery complete!



- obs. aborting a transaction
 - special case of Undo in which the actions of a single transaction are undone
- obs. system crash during the Analysis pass
 - all the work is lost
 - when the system comes back up, the Analysis phase has the same information as before
- obs. system crash during the Redo pass
 - some of the changes from the Redo pass may have been written to disk prior to the crash
 - the pageLSN will indicate such a situation, so these changes will not be reapplied in the subsequent Redo pass

References

- [Ra02] RAMAKRISHNAN, R., GEHRKE, J., Database Management Systems (3rd Edition), McGraw-Hill, 2002
- [Le99] LEVENE, M., LOIZOU, G., A Guided Tour of Relational Databases and Beyond, Springer, 1999
- [Ra02S] RAMAKRISHNAN, R., GEHRKE, J., Database Management Systems, Slides for the 3rd Edition, <http://pages.cs.wisc.edu/~dbbook/openAccess/thirdEdition/slides/slides3ed.html>
- [Si19] SILBERSCHATZ, A., KORTH, H., SUDARSHAN, S., Database System Concepts (7th Edition), McGraw-Hill, 2019
- [Si19S] SILBERSCHATZ, A., KORTH, H., SUDARSHAN, S., Database System Concepts, Slides for the 7th Edition, <http://codex.cs.yale.edu/avi/db-book/>

Database Management Systems

Lecture 6

Evaluating Relational Operators

Query Optimization

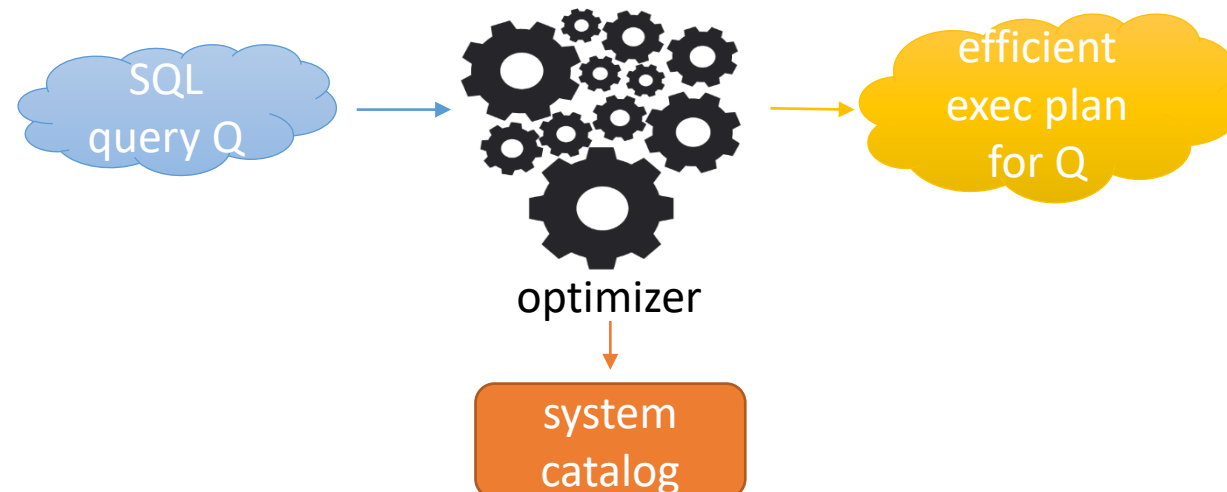
- * queries – composed of relational operators:
 - selection (σ)
 - selects a subset of records from a relation
 - projection (π)
 - eliminates certain columns from a relation
 - join ($\otimes$)
 - combines data from two relations
 - cross-product ($R1 \times R2$)
 - returns every record in R1 concatenated with every record in R2
 - set-difference ($R1 - R2$)
 - returns records that belong to R1 and don't belong to R2
 - union ($R1 \cup R2$)
 - returns all records in relations R1 and R2

*Review lecture notes on *Relational Algebra* (Databases course)

- * queries – composed of relational operators:
 - intersection ($R1 \cap R2$)
 - returns records that belong to both R1 and R2
 - grouping and aggregate operators (algebra extensions)
 - every operation returns a relation => operations can be composed
 - an operator can have several implementation algorithms

- * optimizer

- input: SQL query Q
- output: an efficient execution plan for evaluating Q



* algorithms for operators - based on 3 techniques:

- iteration:
 - examine iteratively:
 - all tuples in input relationsor
 - data entries in indexes, provided they contain all the necessary fields (data entries are smaller than data records)
- indexing:
 - used when the query contains a selection condition or a join condition
 - examine only the tuples that meet the condition, using an index
- partitioning:
 - partition the tuples
 - decompose operation into collection of cheaper operations on partitions
 - partitioning techniques: sorting, hashing

* access paths

- *access path* = way of retrieving tuples from a relation
 - file scan
- or
- an index I + a matching selection condition C
- condition C matches index I if I can be used to retrieve just the tuples satisfying C
- if relation R has an index I that matches selection condition C , then there are at least 2 access paths for R (file scan; index)

*Review lecture notes on *Indexes (Databases course)*

* access paths - example:

- relation *Students*[*SID*, *Name*, *City*]
- *I* - tree index on *Students* with search key $\langle \textit{Name} \rangle$
- query Q:
SELECT *
FROM *Students*
WHERE *Name* = 'Ionescu'
- condition *C*: *Name* = 'Ionescu'
- *C* matches *I*, i.e., index *I* can be used to retrieve only the *Students* tuples satisfying *C*
- the following condition also matches the index: *Name* > 'Ionescu'

* access paths - example:

- relation *Students*[*SID*, *Name*, *City*]
- *I* - hash index on *Students* with search key $\langle \textit{Name} \rangle$
- query Q:
SELECT *
FROM *Students*
WHERE *Name* = 'Ionescu'
- condition *C*: *Name* = 'Ionescu'
- *C* matches *I*, i.e., index *I* can be used to retrieve only the *Students* tuples satisfying *C*
- condition *Name* > 'Ionescu' doesn't match *I* (since *I* is a hash index; it cannot be used to retrieve just the tuples satisfying *Name* > 'Ionescu')

* access paths

- to sum up:
 - condition $C: attr\ op\ value$, $op \in \{<, <=, =, <>, >=, >\}$
 - condition C matches index I if:
 - the search key of I is $attr$ and:
 - I is a tree index or
 - I is a hash index and op is $=$

* access paths

- index I , selection condition C
- I - hash index
- condition C of the form:
 - $\bigwedge_{i=1}^n T_i$
 - term T_i : $attr = value$
- I matches C if C has one term for each attribute in the search key of I

Condition	Hash index with search key <a, b, c>
a = 10 AND b = 5 AND c = 2	Yes
a = 10 AND b = 5	No
b = 5	No
b = 5 AND c = 2	No

* access paths

- index I , selection condition C
- I - tree index
- condition C of the form:
 - $\bigwedge_{i=1}^n T_i$
 - term T_i : *attr op value*; $op \in \{<, <=, =, <>, >=, >\}$
- I matches C if C has one term for each attribute in a prefix of the search key of I

Condition	B+ tree index with search key <a, b, c>
a = 10 AND b = 5 AND c = 2	Yes
a = 10 AND b = 5	Yes
b = 5	No
b = 5 AND c = 2	No

* access paths

- selectivity of an access path
 - the number of retrieved pages when using the access path to obtain the desired tuples
 - both data pages and index pages are counted
- example:
SELECT *
FROM Students
WHERE Name = 'Ionescu'
 - access paths:
 - file scan – selectivity could be 1000
 - matching index I with search key <Name> – selectivity could be 3
- most selective access path
 - retrieves the fewest pages, i.e., data retrieval cost is minimized

* general selection conditions

- in general, a selection condition can contain one or several terms of the form:
 - *attr op constant*
 - *attr1 op attr2*,combined with $\wedge$ and $\vee$

```
SELECT *  
FROM Exams  
WHERE SID = 7 AND EDate = '04-01-2021'
```

$$\sigma_{SID=7 \wedge EDate='04-01-2021'}(Exams)$$

* general selection conditions

- process a selection operation with a general selection condition
 $C \rightarrow$ express C in CNF (conjunctive normal form)
- condition in CNF:
 - collection of conjuncts connected with the $\wedge$ operator
 - a *conjunct* has one or more terms connected with the $\vee$ operator
 - *term*:
 - *attr op constant*
 - *attr1 op attr2*
- example:
condition $(EDate < '4-1-2021' \wedge Grade = 10) \vee CID = 5 \vee SID = 3$
is rewritten in CNF:
 $(EDate < '4-1-2021' \vee CID = 5 \vee SID = 3) \wedge (Grade = 10 \vee CID = 5 \vee SID = 3)$

* general selection conditions matching an index

- relation R[a, b, c, d, e], index I with search key <a, b, c>

Condition	B+ tree index	Hash index
a = 10 AND b = 5 AND c = 2	Yes	Yes
a = 10 AND b = 5	Yes	No
b = 5	No	No
b = 5 AND c = 2	No	No
d = 2	No	No
a = 20 AND b = 10 AND c = 5 AND d = 11	Partly	Partly

Condition – CNF selection condition

B+ tree index / Hash index – B+ tree / hash index I matches (Yes) / doesn't match (No) / matches a part of (Partly) the selection condition

- for the condition in the last row (a = 20 AND b = 10 AND c = 5 AND d = 11):
 - use index I to retrieve tuples satisfying *a = 20 AND b = 10 AND c = 5*, then apply *d = 11* to each retrieved tuple

* general selection conditions matching an index

- relation R[a, b, c, d]
- index I1 with search key <a, b>
- B+ tree index I2 with search key <c>

Condition	Indexes
$c < 100 \text{ AND } a = 3 \text{ AND } b = 5$	- use I1 or I2 to retrieve tuples - then check terms in the selection condition that do not match the index for each retrieved tuple - e.g., use the B+ tree index to retrieve tuples where $c < 100$; then apply $a = 3 \text{ AND } b = 5$ to each retrieved tuple

* running example - schema

- Students (SID: integer, SName: string, Age: integer)
- Courses (CID: integer, CName: string, Description: string)
- Exams (SID: integer, CID: integer, EDate: date, Grade: integer, FacultyMember: string)

- Students

- every record has 50 bytes
- there are 80 records / page
- 500 pages of Students tuples

- Courses

- every record has 50 bytes
- there are 80 records / page
- 100 pages of Courses tuples

- Exams

- every record has 40 bytes
- there are 100 records / page
- 1000 pages of Exams tuples

* joins

SELECT *

FROM Exams E, Students S

WHERE E.SID = S.SID

- algebra: $E \bowtie_{E.SID=S.SID} S$
 - to be carefully optimized
 - size of $E \times S$ is large, so computing $E \times S$ followed by selection is inefficient
- E
 - M pages
 - p_E records / page
- S
 - N pages
 - p_S records / page
- evaluation: number of I/O operations

* joins – implementation techniques

- iteration
 - Simple/Page-Oriented Nested Loops Join
 - Block Nested Loops Join
- indexing
 - Index Nested Loops Join
- partitioning
 - Sort-Merge Join
 - Hash Join
- equality join, one join column
 - join condition: $E_i = S_j$

Simple Nested Loops Join

```
foreach tuple e ∈ E do
    foreach tuple s ∈ S do
        if ei == sj then add <e, s> to the result
```

- for each record in the outer relation E, scan the entire inner relation S
- cost
 - $M + p_E * M * N = 1000 + 100 * 1000 * 500 \text{ I/Os} = 1000 + (5 * 10^7) \text{ I/Os}$
 - M I/Os – cost of scanning E
 - N I/Os – cost of scanning S
 - S is scanned $p_E * M$ times (there are $p_E * M$ records in the outer relation E)

* E - M pages, p_E records / page *

* 1000 pages * * 100 records / page*

* S - N pages, p_S records / page *

* 500 pages * * 80 records / page *

Page-Oriented Nested Loops Join

```
foreach page  $pe \in E$  do
    foreach page  $ps \in S$  do
        if  $e_i == s_j$  then add  $\langle e, s \rangle$  to the result
```

- for each page in E read each page in S
- pairs of records $\langle e, s \rangle$ that meet the join condition are added to the result (where record e is on page pe , and record s – on page ps)
- refinement of Simple Nested Loops Join

Page-Oriented Nested Loops Join

```
foreach page  $p_e \in E$  do
    foreach page  $p_s \in S$  do
        if  $e_i == s_j$  then add  $\langle e, s \rangle$  to the result
```

- cost

- **$M + M*N = 1000 + 1000*500$ I/Os = 501.000 I/Os**
 - M I/Os – cost of scanning E; N I/Os – cost of scanning S
 - S is scanned M times
 - significantly lower than the cost of Simple Nested Loops Join (improvement - factor of p_E)
- if the smaller table (S) is chosen as outer table:
 $\Rightarrow \text{cost} = 500 + 500 * 1000 \text{ I/Os} = 500.500 \text{ I/Os}$

* E - M pages, p_E records / page *

* 1000 pages * * 100 records / page*

* S - N pages, p_S records / page *

* 500 pages * * 80 records / page *

Block Nested Loops Join

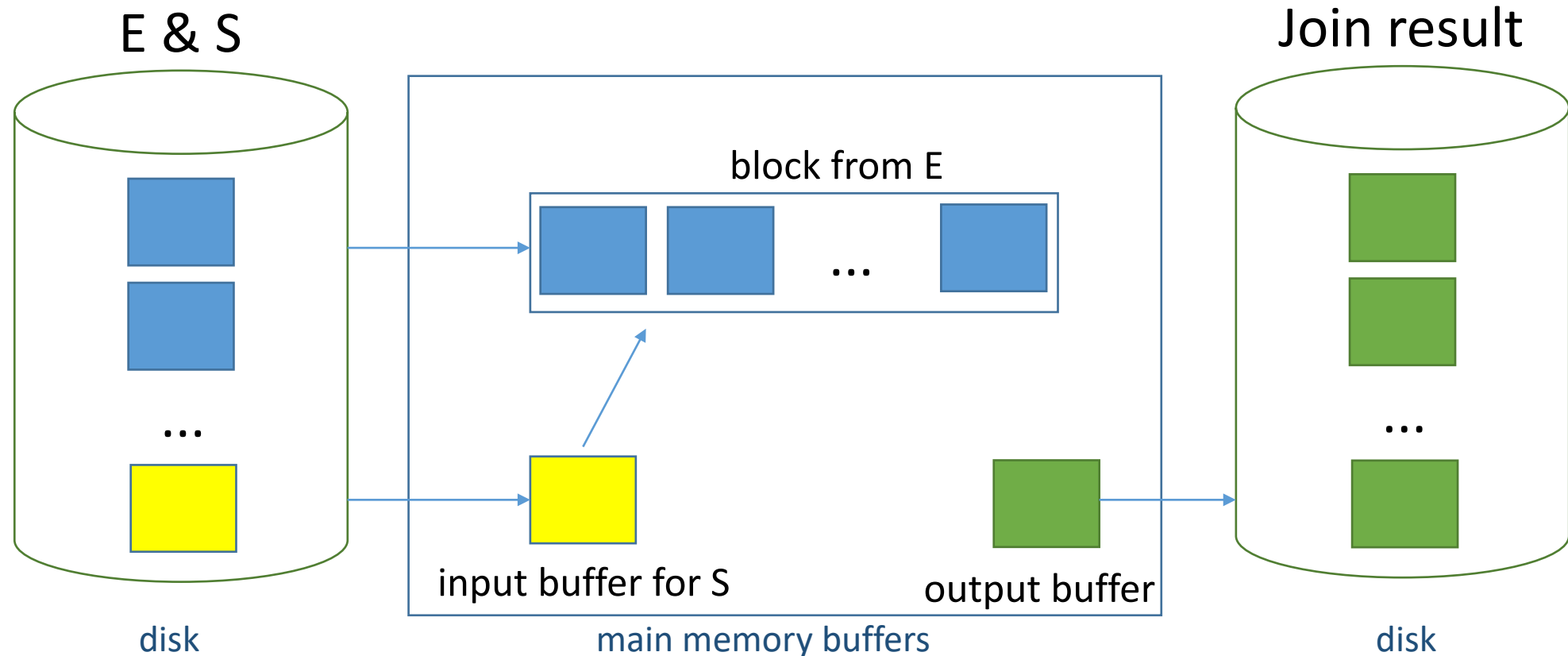
- previously presented join algorithms do not use buffer pages effectively
 - join relations R1 and R2; R1 – the smaller relation
 - assumption – the smaller relation fits in main memory
 - improvement:
 - store smaller relation R1 in memory
 - keep at least 2 extra buffer pages B1 and B2
 - use B1 to read the larger relation R2 (one page at a time)
 - use B2 as the output buffer (i.e., for tuples in the result of the join)
 - for each tuple in R2, search R1 for matching tuples
- => optimal cost: *number of pages in R1 + number of pages in R2*, since R1 is scanned only once, R2 is also scanned only once

Block Nested Loops Join

- refinement
 - don't store the smaller relation in main memory as is, build an in-memory hash table for it instead
 - the I/O cost remains unchanged, but the CPU cost is usually much lower (since for each tuple in the larger relation, the smaller relation is examined to find matching tuples)

Block Nested Loops Join

- if there isn't enough main memory to hold one of the input relations:
 - use one buffer page to scan the inner table (e.g., S)
 - use one page for the result
 - use all remaining pages to read a *block* from the outer table (e.g., E)
 - block – set of pages from E that fit in main memory

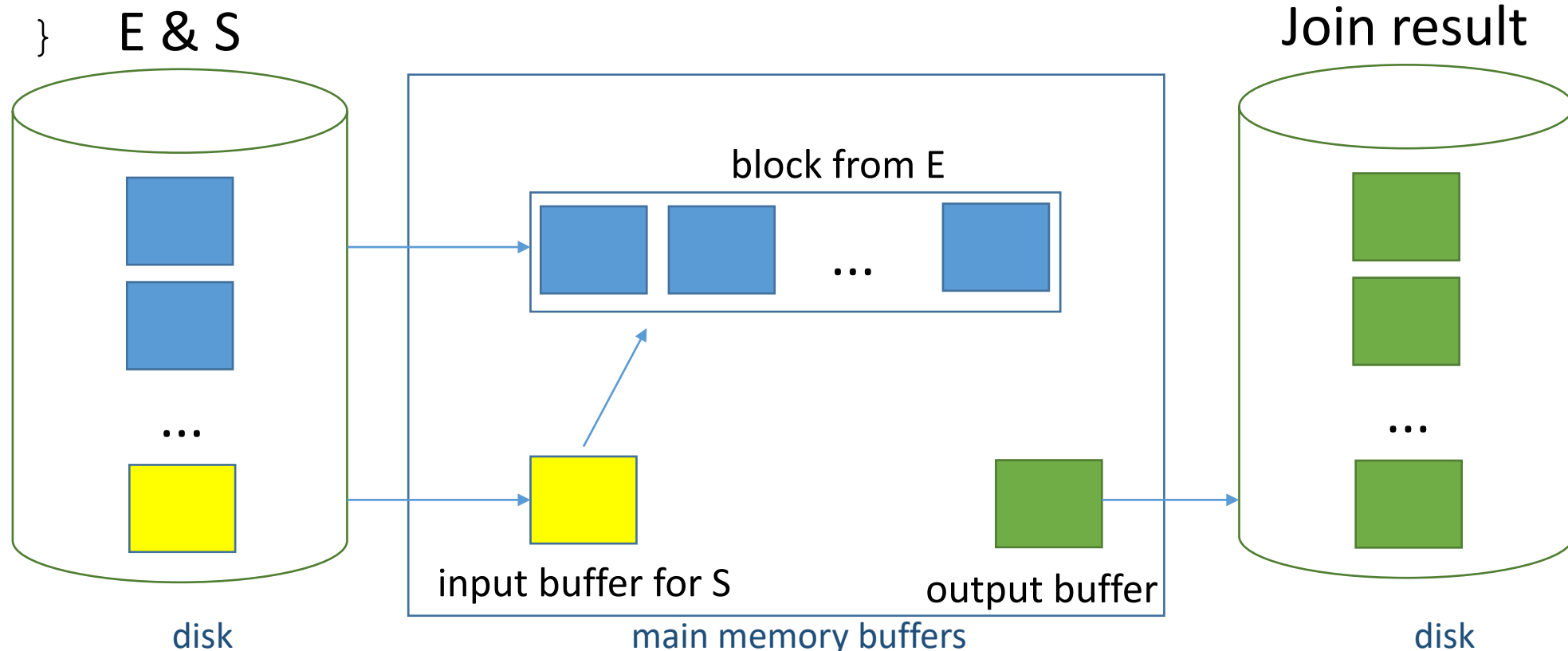


Block Nested Loops Join

```
foreach block be ∈ E do  
  foreach page ps ∈ S do  
  {
```

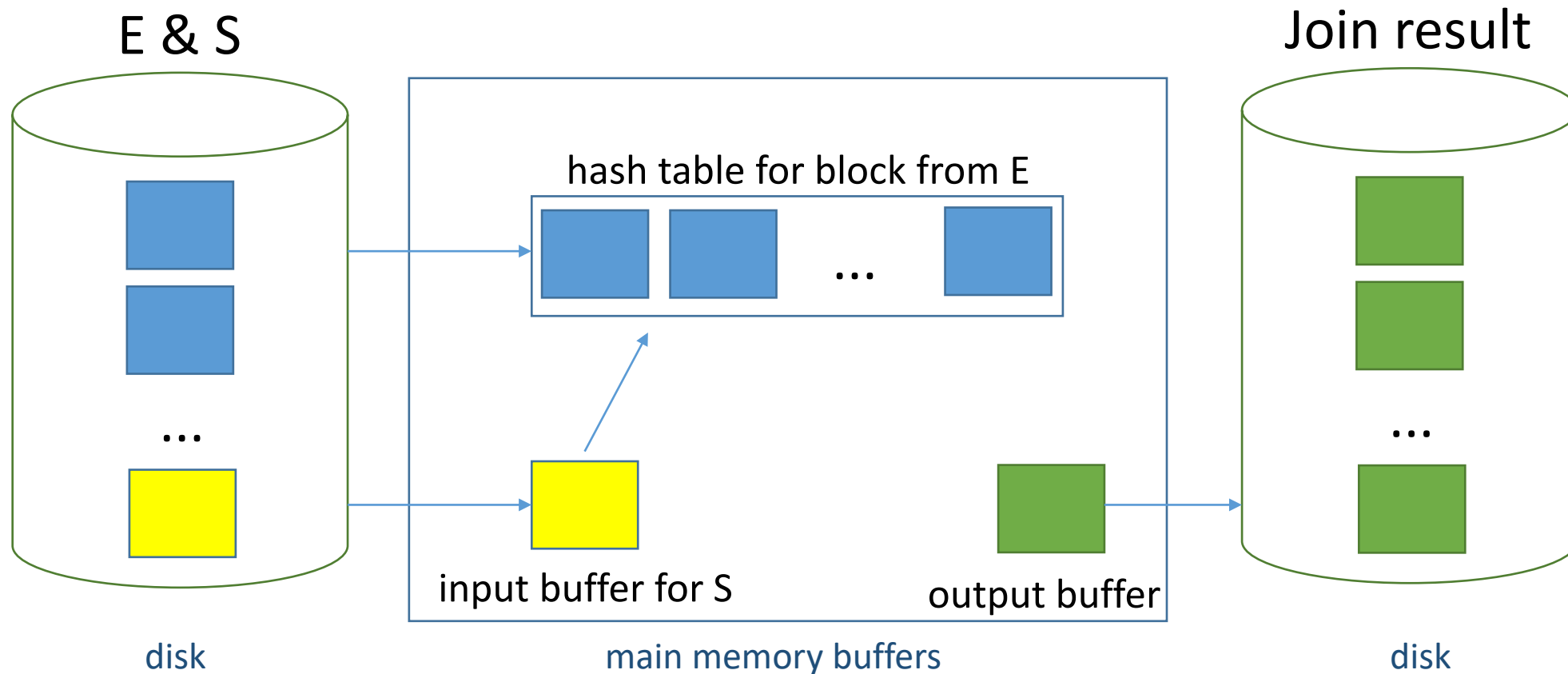
```
    for all pairs of tuples <e, s> that meet the join  
      condition, where e ∈ be and s ∈ ps,  
        add <e, s> to the result  
  }
```

- inner relation S is scanned once for each block in outer relation E
- outer relation E is scanned once



Block Nested Loops Join

- refinement to efficiently find matching tuples
 - build main-memory hash table for the block of E
 - trade-off: reduce size of E block



Block Nested Loops Join

- cost
 - scan of outer table + number of blocks in outer table * scan of inner table
 - number of outer blocks = $\left\lceil \frac{\text{number of pages in outer table}}{\text{size of block}} \right\rceil$
 - outer table: Exams (E), a block can hold 100 pages
 - scan cost for E: 1000 I/Os
 - number of blocks: $\left\lceil \frac{1000}{100} \right\rceil = 10$
 - foreach block in E, scan Students (S): 10*500 I/Os
- => total cost = 1000 + 10 * 500 = **6000 I/Os**

* E - M pages, p_E records / page *

* 1000 pages * * 100 records / page *

* S - N pages, p_S records / page *

* 500 pages * * 80 records / page *

Block Nested Loops Join

- cost
 - scan of outer table + number of blocks in outer table * scan of inner table
 - number of outer blocks = $\left\lceil \frac{\text{number of pages in outer table}}{\text{size of block}} \right\rceil$
 - outer table: Exams (E)
 - suppose the buffer has 90 pages available for E, i.e., block of 90 pages
- => number of blocks: $\left\lceil \frac{1000}{90} \right\rceil = 12$
- => S is scanned 12 times
- scan cost for E: 1000 I/Os
 - foreach block in E, scan Students (S): 12*500 I/Os
- => total cost = 1000 + 12 * 500 = **7000 I/Os**

* E - M pages, p_E records / page *

* 1000 pages * * 100 records / page *

* S - N pages, p_S records / page *

* 500 pages * * 80 records / page *

Block Nested Loops Join

- cost
 - scan of outer table + number of blocks in outer table * scan of inner table
 - number of outer blocks = $\left\lceil \frac{\text{number of pages in outer table}}{\text{size of block}} \right\rceil$
 - outer table: Students (S), block of 100 pages
 - scan cost for S: 500 I/Os
 - number of blocks: $\left\lceil \frac{500}{100} \right\rceil = 5$
 - for each block in S, scan E: 5 * 1000 I/Os
- => total cost = 500 + 5 * 1000 = **5500 I/Os**

* E - M pages, p_E records / page *

* 1000 pages * * 100 records / page *

* S - N pages, p_S records / page *

* 500 pages * * 80 records / page *

Index Nested Loops Join

```
foreach tuple e in E do
    foreach tuple s in S where  $e_i == s_j$ 
        add  $\langle e, s \rangle$  to the result
```

- if there is an index on the join column of S, S can be considered as inner table and the index can be used
- cost
 - $M + (M * p_E) * \text{cost of finding corresponding records in S}$

* E - M pages, p_E records / page *

* S - N pages, p_S records / page *

* 1000 pages * * 100 records / page *

* 500 pages * * 80 records / page *

Index Nested Loops Join

- for a record e in E :
 - the cost of examining the index on S is:
 - approx. 1.2 for a hash index (typical cost for hash indexes)
 - typically 2-4 for a B+-tree index
 - the cost of reading corresponding records in S :
 - for a clustered index:
 - plus one I/O for each outer tuple in E (typically)
 - for an unclustered index:
 - up to one I/O for each corresponding record in S
(worst case – there are n matching records in S located on n different pages!)

Index Nested Loops Join

- hash index on SID in Students (Students – inner table)
- scan Exams:
 - cost = 1000 I/Os, with a total of 100*1000 records
- for each record in Exams:
 - (on average) 1.2 I/Os to obtain the page in the hash index (i.e., the page containing the rid of the matching Students tuple)
and
 - 1 I/O to retrieve the page in Students that contains the matching tuple (exactly one! – since SID is a key in Students, i.e., there is one matching Students tuple for an exam)

=> cost to retrieve matching Students tuples: $1000 * 100 * (1.2 + 1) = 220.000$

- total cost: $1000 + 220.000 = 221.000$ I/Os

* E - M pages, p_E records / page * * 1000 pages * * 100 records / page*

* S - N pages, p_S records / page * * 500 pages * * 80 records / page *

* sorting

- can be explicitly required (SELECT ... ORDER BY list), used to eliminate duplicates (SELECT DISTINCT), used by operators like:
 - join
 - union
 - intersection
 - set-difference
 - grouping

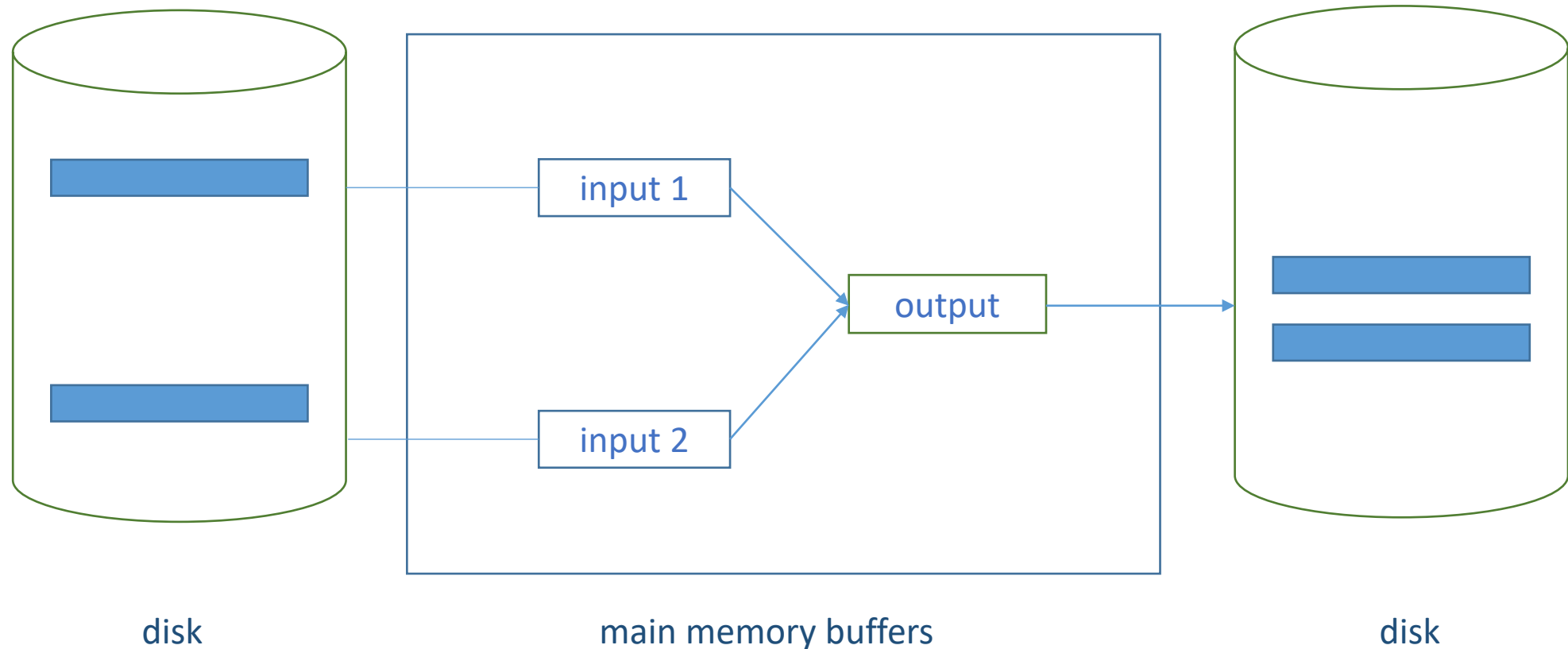
* sorting

e.g., the user wants to sort the collection of Courses records by name

- if the data to be sorted fits into available main memory:
 - use an internal sorting algorithm (Quick Sort or any other in-memory sorting algorithm can be used to sort a collection of records that fits into main memory)
- if the data to be sorted doesn't fit into available main memory:
 - use an external sorting algorithm
 - minimizes the cost of accessing the disk
 - breaks the data collection into subcollections of records
 - sorts the subcollections; a sorted subcollection of records is called a *run*
 - writes runs to disk
 - merges runs

Simple Two-Way Merge Sort

- uses 3 buffer pages
- passes over the data multiple times
- can sort large data collections using a small amount of main memory



Simple Two-Way Merge Sort

- pass 0:
 - for each page P in the data collection:
 - read in page P -> sort page P -> save page P to disk

=> 1-page runs (runs that are 1 page long)

example: - read in the 1st page from Courses, sort the 80 records on it by course name, write out the sorted page to disk (i.e., a *run* that is one page long);

- read in the 2nd page from Courses, sort the 80 records on it by course name, write out sorted page to disk;

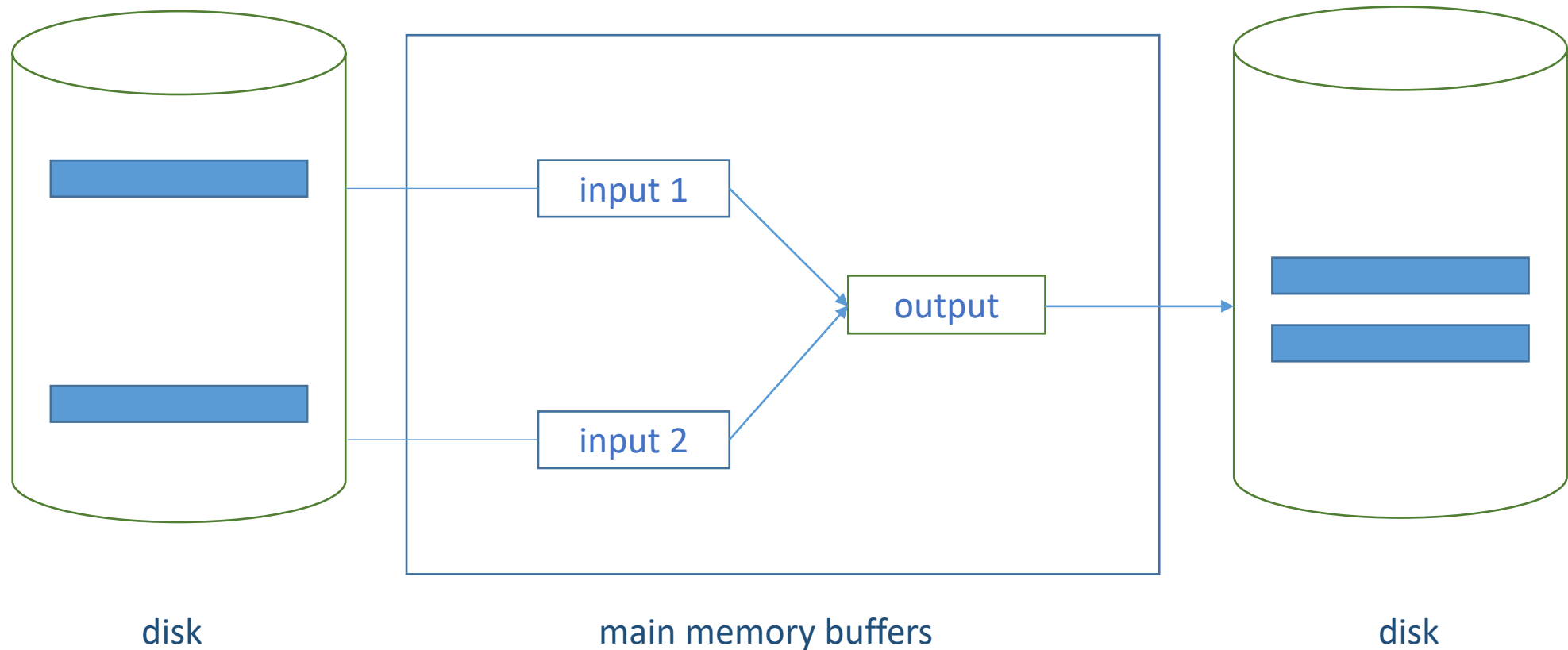
...

- read in the 100th page from Courses, sort the 80 records on it by course name, write out sorted page to disk

=> 100 1-page runs saved on disk

Simple Two-Way Merge Sort

- pass 1, 2, ... etc.:
 - use 3 buffer pages
 - read and merge pairs of runs from the previous pass
 - produce runs that are twice as long



Simple Two-Way Merge Sort

- e.g., pass 1 (pass 0 produced 100 1-page runs):
 - read in 2 runs from pass 0 (i.e., two pages holding Courses records, each of them sorted in pass 0), using 2 buffer pages
 - merge these runs writing to the 3rd available buffer page (the *output* buffer); when the output buffer fills up, write it out to disk (i.e., write a page of 80 sorted records to disk)
=> a run that is 2 pages long (it contains 160 Courses records, sorted by name)
- read in and merge the next 2 runs from pass 0 ... => another run that is 2 pages long
- continue while there are runs to be processed (read in and merged) from pass 0
- at the end of pass 1 there are 50 2-page runs (each run consists of 2 pages holding 160 records sorted by course name)

Simple Two-Way Merge Sort

- another example – sort data collection (file of records) with 7 pages:

3, 4	6, 2	9, 4	8, 7	5, 6	3, 1	2
page 1	page 2	...				

- only the value of the key is displayed (the key on which the user wants to sort the collection, an integer number in the example)
- simplifying assumption that allows us to focus on the idea of the algorithm: a page can hold 2 records

- pass 0

- read in the collection one page at a time
- sort each page that is read in
- write out each sorted page to disk

=> 7 sorted runs that are 1 page long:

3, 4	2, 6	4, 9	7, 8	5, 6	1, 3	2
------	------	------	------	------	------	---

Simple Two-Way Merge Sort

- runs at the end of pass 0:

3, 4

2, 6

4, 9

7, 8

5, 6

1, 3

2

- pass 1

- read in & merge pairs of runs from pass 0
- produce runs that are twice as long
- read in runs 3, 4 and 2, 6 :
 - merge the runs and write to the output buffer
 - write the output buffer to disk one page at a time

=> run

2, 3

4, 6

- read in runs 4, 9 and 7, 8 :

- merge the runs and write to the output buffer
- write the output buffer to disk one page at a time

=> run

4, 7

8, 9

Simple Two-Way Merge Sort

- runs at the end of pass 0:

3, 4

2, 6

4, 9

7, 8

5, 6

1, 3

2

- pass 1

- read in runs 5, 6 and 1, 3 ...

=> run 1, 3 5, 6

- read in run 2 (the last run from pass 0) ...

=> run 2

=> 4 sorted runs that are 2 pages long (except for the last run):

2, 3 4, 6

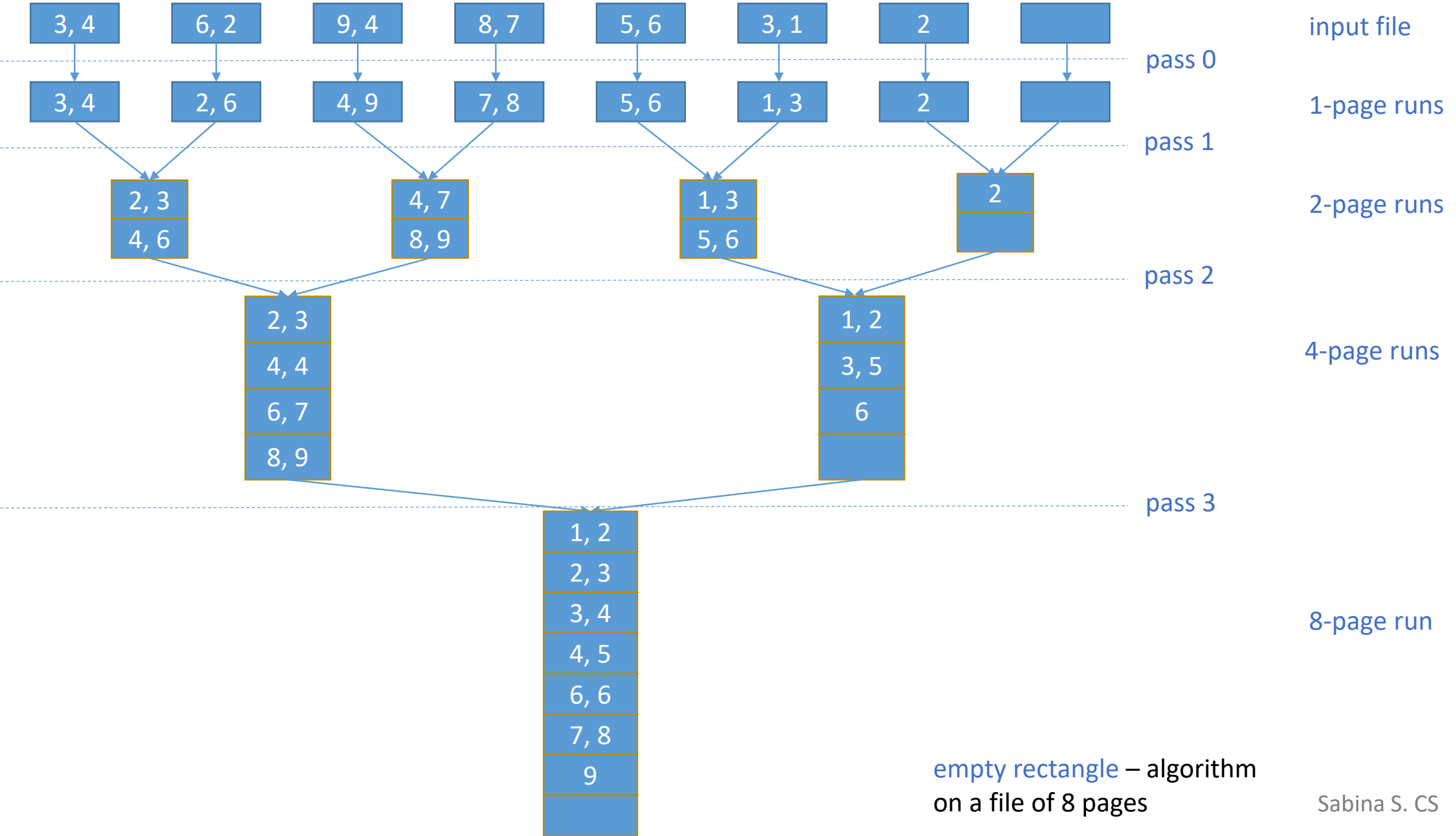
4, 7 8, 9

1, 3 5, 6

2

Simple Two-Way Merge Sort

- pass 2
 - read in & merge pairs of runs from pass 1
 - produce runs that are twice as long
- ...
- complete example, with all passes of the algorithm, on the next page ->



input file: 2^k pages

pass 0

=> 2^k sorted runs (1-page)

pass 1

=> 2^{k-1} sorted runs (2-pages)

pass 2

=> 2^{k-2} sorted runs (4-pages)

pass 3

=> 2^{k-3} sorted runs (8-pages)

i.e.,

pass k

=> one sorted run (2^k pages)

Simple Two-Way Merge Sort

- in each pass, each page in the input file is: read in, processed, and written out; there are 2 I/O operations per page, per pass

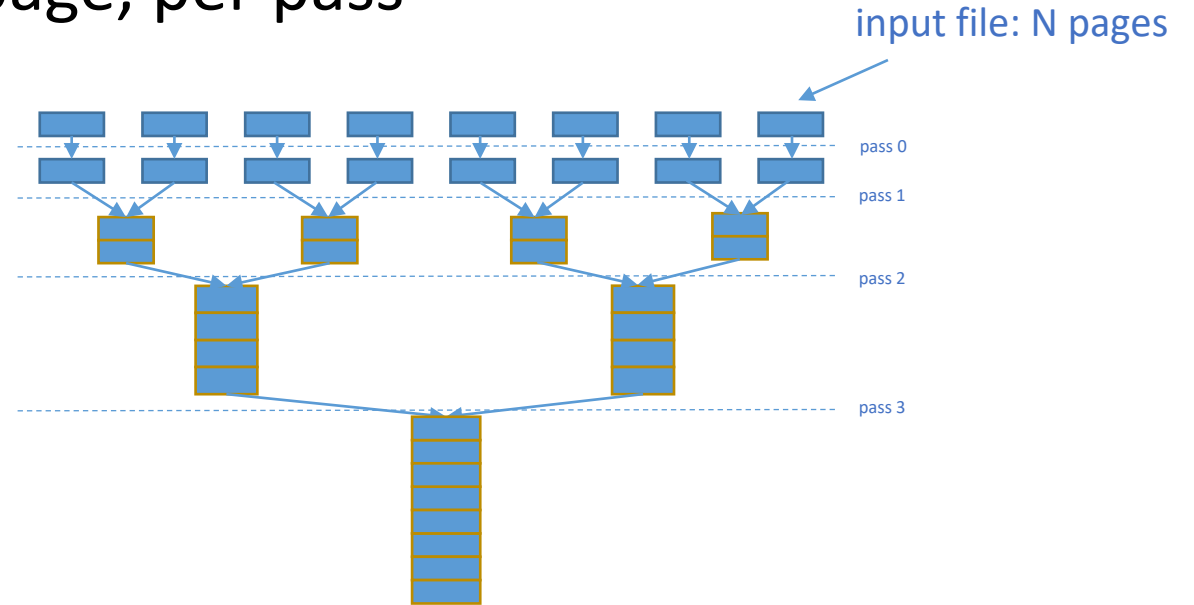
- number of passes:

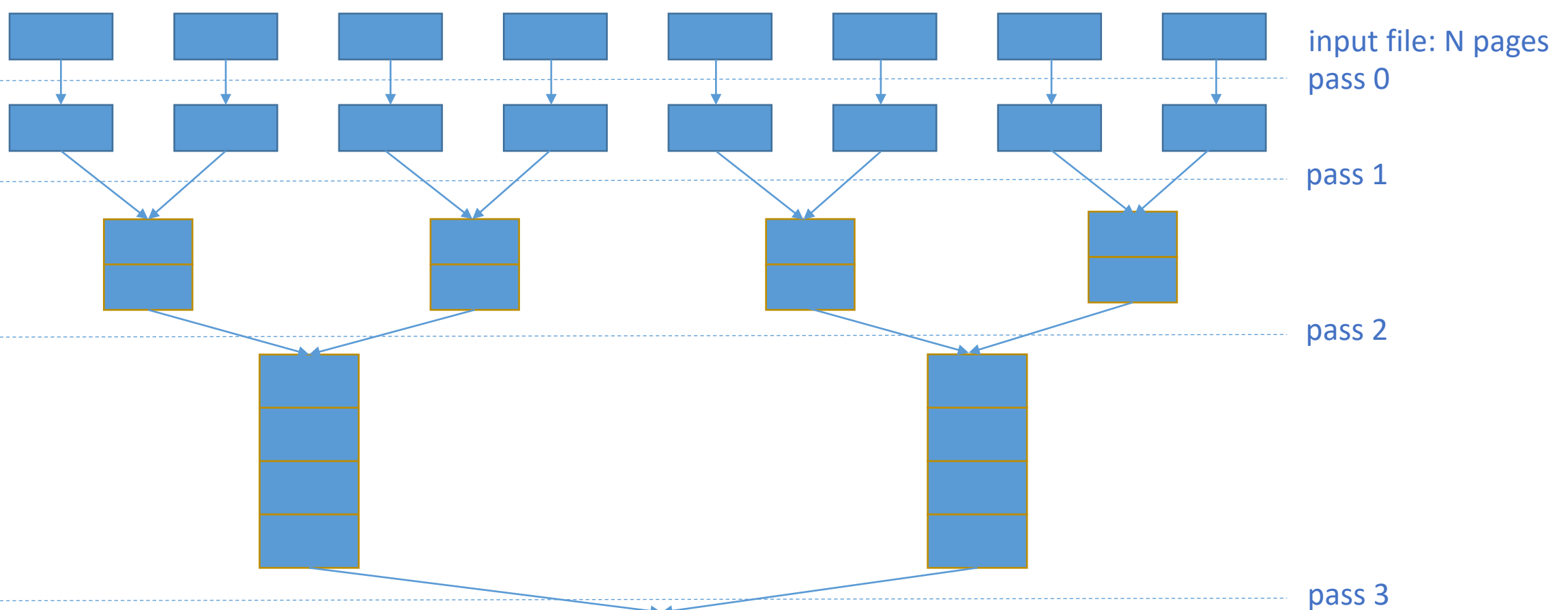
- $\lceil \log_2 N \rceil + 1$

where N is the number of pages in the file to be sorted

- total cost:

- $2 * \text{number of pages} * \text{number of passes}$
 - $2 * N * (\lceil \log_2 N \rceil + 1) \text{ I/Os}$





- in each pass: read / process / write each page in the file
- number of passes:
 - $\lceil \log_2 N \rceil + 1$
- total cost:
 - $2 * N * (\lceil \log_2 N \rceil + 1)$ I/Os

- there are $N = 8$ pages, 4 passes
 - $\Rightarrow 2 * 8 * 4 = 64$ I/Os
 - $2 * 8 * (\lceil \log_2 8 \rceil + 1) = 2 * 8 * 4 = 64$ I/Os
- $N = 7$ pages, 4 passes
 - $\Rightarrow 2 * 7 * 4 = 56$ I/Os
 - $2 * 7 * (\lceil \log_2 7 \rceil + 1) = 56$ I/Os

References

- [Ra02] RAMAKRISHNAN, R., GEHRKE, J., Database Management Systems (3rd Edition), McGraw-Hill, 2002
- [Da03] DATE, C.J., An Introduction to Database Systems (8th Edition), Addison-Wesley, 2003
- [Ga09] GARCIA-MOLINA, H., ULLMAN, J., WIDOM, J., Database Systems: The Complete Book (2nd Edition), Pearson Education, 2009
- [Ra02S] RAMAKRISHNAN, R., GEHRKE, J., Database Management Systems, Slides for the 3rd Edition,
<http://pages.cs.wisc.edu/~dbbook/openAccess/thirdEdition/slides/slides3ed.html>
- [Si19] SILBERSCHATZ, A., KORTH, H., SUDARSHAN, S., Database System Concepts (7th Edition), McGraw-Hill, 2019
- [Si19S] SILBERSCHATZ, A., KORTH, H., SUDARSHAN, S., Database System Concepts, Slides for the 7th Edition, <http://codex.cs.yale.edu/avi/db-book/>
- [Ul11] ULLMAN, J., WIDOM, J., A First Course in Database Systems,
<http://infolab.stanford.edu/~ullman/fcdb.html>

Database Management Systems

Lecture 7

Evaluating Relational Operators

Query Optimization

- running example - schema
 - Students (SID: integer, SName: string, Age: integer)
 - Courses (CID: integer, CName: string, Description: string)
 - Exams (SID: integer, CID: integer, EDate: date, Grade: integer, FacultyMember: string)
- Students
 - every record has 50 bytes
 - there are 80 records / page
 - 500 pages of Students tuples
- Courses
 - every record has 50 bytes
 - there are 80 records / page
 - 100 pages of Courses tuples
- Exams
 - every record has 40 bytes
 - there are 100 records / page
 - 1000 pages of Exams tuples

External Merge Sort

- Simple Two-Way Merge Sort: buffer pages are not used effectively
 - for instance, if 200 buffer pages are available, this algorithm still uses only 2 input buffers for passes 1, 2, ...
- generalize the Two-Way Merge Sort algorithm to effectively use the available main memory and minimize the number of passes
- input file to be sorted: N pages
- B buffer pages are available
- pass 0:
 - use B buffer pages
 - read in B pages at a time and sort them in memory
 - $\Rightarrow \left\lceil \frac{N}{B} \right\rceil$ runs of B pages each (except for the last one, which may be smaller)

External Merge Sort

- consider again the input file in the previous example:

3, 4	6, 2	9, 4	8, 7	5, 6	3, 1	2
page 1	page 2	...				

- $N = 7$ (number of pages in the file)
- $B = 4$ (there are 4 available buffer pages)

- pass 0 produces $\left\lceil \frac{N}{B} \right\rceil = \left\lceil \frac{7}{4} \right\rceil = 2$ runs:

- use all 4 buffer pages

- read in 4 pages:

3, 4	6, 2	9, 4	8, 7
------	------	------	------

- sort the pages in memory, write to disk a run that is 4 pages long:

2, 3	4, 4	6, 7	8, 9
------	------	------	------

- read in remaining 3 pages:

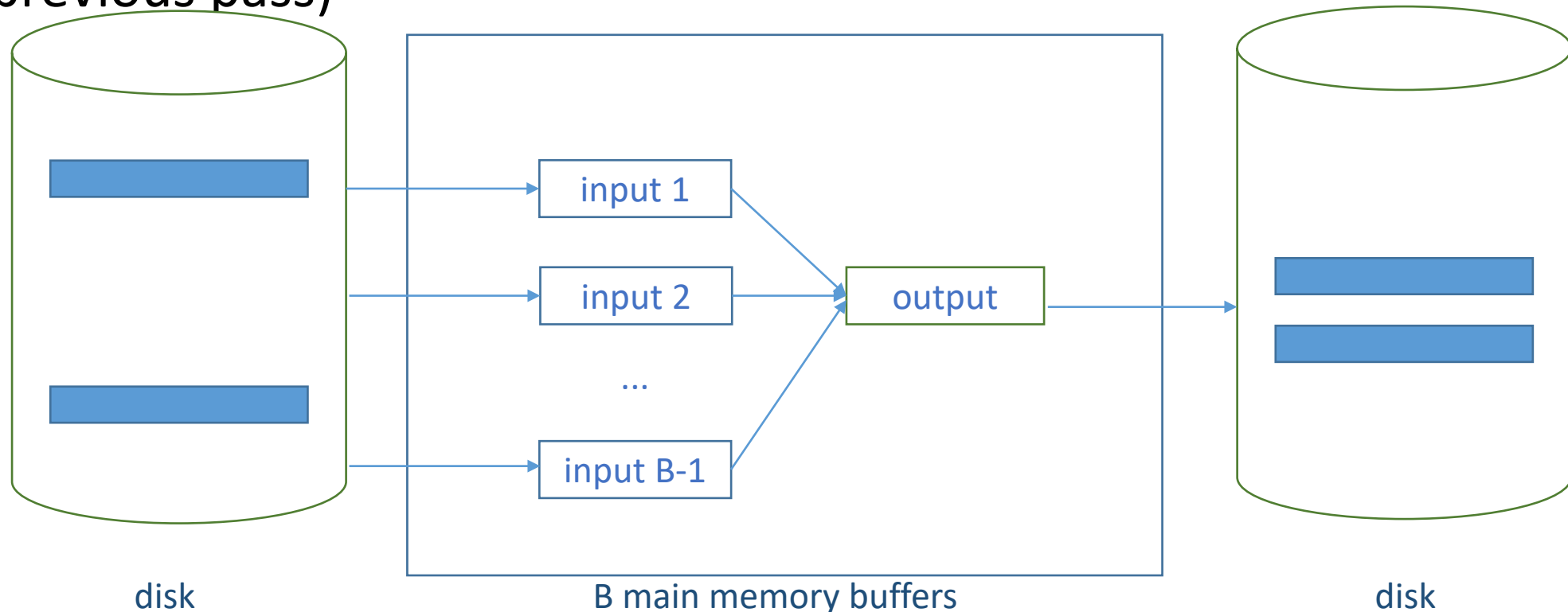
5, 6	3, 1	2
------	------	---

- sort pages in memory, write to disk run of 3 pages:

1, 2	3, 5	6
------	------	---

External Merge Sort

- input file to be sorted: N pages
- B buffer pages are available
- pass 1, 2 ...:
 - use $B-1$ pages for input, and one page for output
 - perform a $(B-1)$ -way merge in each pass (i.e., merge $B-1$ runs from the previous pass)



External Merge Sort

- runs at the end of pass 0:

2, 3	4, 4	6, 7	8, 9
------	------	------	------

1, 2	3, 5	6
------	------	---

- pass 1

- read in & merge the first $B-1 = 4-1 = 3$ runs from pass 0
- pass 0 produced only 2 runs in this example; read in and merge these 2 runs:

=> run

1, 2	2, 3	3, 4	4, 5	6, 6	7, 8	9
------	------	------	------	------	------	---

External Merge Sort

- another example:
 - 5 buffer pages $B = 5$
 - sort file with 108 pages $N = 108$
- pass 0
 - use all 5 buffer pages
 - read in the first 5 pages of the file, sort them in memory, write the resulting run to disk (5 pages long)
 - read in the next 5 pages of the file, sort them in memory, write the resulting run to disk (5 pages long)
 - ...
 - read in the remaining 3 pages of the file, sort them in memory, write the resulting run to disk (3 pages long)
 - 21 runs are 5 pages long; 1 run is 3 pages long

External Merge Sort

- another example: $B = 5$, $N = 108$
- pass 0
 - at the end of pass 0 there are $\left\lceil \frac{N}{B} \right\rceil = \left\lceil \frac{108}{5} \right\rceil = 22$ runs
- pass 1
 - use $B-1 = 5-1 = 4$ pages for input, and one page for output
 - do a 4-way merge: read in and merge 4 runs from the previous pass
 - read in the first 4 runs from pass 0 (each run into an input buffer)
 - merge the runs and write to the output buffer
 - write the output buffer to disk one page at a time
 - => a run that is 20 pages long (4 runs from pass 0 times 5 pages per run)
 - read in the next 4 runs from pass 0; merge the runs and write to the output buffer; write the output buffer to disk one page at a time
 - => another run (20 pages long)

...

External Merge Sort

- another example: $B = 5$, $N = 108$
- pass 0
 - at the end of pass 0 there are 22 runs
- pass 1
 - read in the last 2 runs from pass 0 (one has 5 pages, the other one has 3 pages)
 - merge the runs and write to the output buffer; write the output buffer to disk one page at a time
 - => the last run (8 pages long)
 - at the end of pass 1 there are $\left\lceil \frac{22}{4} \right\rceil = 6$ runs
 - 5 runs are 20 pages long; 1 run is 8 pages long

External Merge Sort

- another example: $B = 5$, $N = 108$
- pass 1
 - at the end of pass 1 there are 6 runs
- pass 2
 - 4-way merge
 - read in the first 4 runs from pass 1
 - merge the runs and write to the output buffer; write the output buffer to disk one page at a time
 - => a run that is 80 pages long (4 runs from pass 1 times 20 pages per run)
 - read in the remaining 2 runs from pass 1 (20 and 8 pages, respectively)
 - => a run that is 28 pages long
 - at the end of pass 2 there are $\left\lceil \frac{6}{4} \right\rceil = 2$ runs

External Merge Sort

- another example: $B = 5$, $N = 108$
- pass 2
 - at the end of pass 2 there are 2 runs
- pass 3
 - read in the 2 runs from pass 2 and merge them
=> a run that is 108 pages long, representing the sorted file

External Merge Sort

- cost
 - N – number of pages in the input file, B – number of available pages in the buffer
 - in each pass: read / process / write each page
 - number of passes: $\lceil \log_{B-1} \lceil N/B \rceil \rceil + 1$
 - total cost: $2 * N * \left(\lceil \log_{B-1} \left\lceil \frac{N}{B} \right\rceil \rceil + 1 \right)$ I/Os
- previous example: $B = 5$ and $N = 108$, with 4 passes over the data
 - cost:
$$2 * 108 * 4 = 864 \text{ I/Os}$$
 - $2 * 108 * \left(\lceil \log_{5-1} \left\lceil \frac{108}{5} \right\rceil \rceil + 1 \right) = 216 * (\lceil \log_4 22 \rceil + 1) = 216 * 4 = 864 \text{ I/Os}$

- B buffer pages
- sort file with N pages

Simple Two-Way Merge Sort

pass 0 => N runs

number of passes = $\lceil \log_2 N \rceil + 1$

External Merge Sort

pass 0 => $\left\lceil \frac{N}{B} \right\rceil$ runs

number of passes = $\left\lceil \log_{B-1} \left\lceil \frac{N}{B} \right\rceil \right\rceil + 1$

- External Merge Sort – reduced number of:
 - runs produces by the 1st pass
 - passes over the data
- B is usually large => significant performance gains

External Merge Sort – number of passes for different values of N and B

N	B = 3	B = 5	B = 9	B = 17	B = 129	B = 257
100	7	4	3	2	1	1
1,000	10	5	4	3	2	2
10,000	13	7	5	4	2	2
100,000	17	9	6	5	3	3
1,000,000	20	10	7	5	3	3
10,000,000	23	12	8	6	4	3
100,000,000	26	14	9	7	4	4
1,000,000,000	30	15	10	8	5	4

Sort-Merge Join

- equality join, one join column: $E \bowtie_{i=j} S$ (i^{th} column's value in $E = j^{\text{th}}$ column's value in S)
- sort E and S on the join column (if not already sorted):
 - for instance, by using External Merge Sort

$\Rightarrow$ *partitions* = groups of tuples with the same value in the join column
- merge E and S ; look for tuples e in E , s in S such that $e_i = s_j$:
 - while *current* $e_i <$ *current* s_j
 - advance the scan of E
 - while *current* $e_i >$ *current* s_j
 - advance the scan of S
 - if *current* $e_i =$ *current* s_j
 - output joined tuples $\langle e, s \rangle$, where e and s are in the current partition (i.e., they have the same value in the i^{th} and j^{th} column, respectively)
 - there could be multiple tuples in E with the same value in the i^{th} column as the current tuple e (same is true for S)

Sort-Merge Join

- partitions are illustrated on tables Students and Exams below (join column SID in both tables):

SID	SName	Age
20	Ana	20
30	Dana	20
40	Dan	20
45	Daniel	20
50	Ina	20

SID	CID	EDate	Grade	FacultyMember
30	2	20/1/2018	10	Ionescu
30	1	21/1/2018	9.99	Pop
45	2	20/1/2018	9.98	Ionescu
45	1	21/1/2018	9.98	Pop
45	3	22/1/2018	10	Stan
50	2	20/1/2018	10	Ionescu

Sort-Merge Join

- during the merging phase, E is scanned once; every partition in S is scanned as many times as there are matching tuples in the corresponding partition in E

E	...	i th column	...		...	j th column	...	S
		1				2		
		3				3		
		3				3	partition P	
		3				4		
		8				...		
		...				...		

- for instance, partition P in the above table S is scanned 3 times, once per matching tuple in the corresponding partition in E
- there are 6 output joined tuples $\langle e, s \rangle$ for partition P
- this algorithm avoids the enumeration of the cross-product: tuples in a partition in E are compared only with the S tuples in the same partition!

Sort-Merge Join

- cost:
 - sorting E
 - cost: $O(M \log M)$
 - sorting S
 - cost: $O(N \log N)$
 - cost of merging: $M + N$ I/Os, assuming partitions in S are scanned only once
 - worst-case scenario: $O(M * N)$ I/Os (when all records in E and S have the same value in the join column)

* E - M pages; S - N pages*

Sort-Merge Join ($\text{Exams} \bowtie_{\text{Exams.SID}=\text{Students.SID}} \text{Students}$)

- 100 buffer pages
 - sort Exams
 - 2 passes $\Rightarrow$ cost: $2 * 2 * 1000 = 4000$ I/Os
 - sort Students
 - 2 passes $\Rightarrow$ cost: $2 * 2 * 500 = 2000$ I/Os
 - merging phase
 - cost: $1000 + 500 = 1500$ I/Os
 - total cost: $4000 + 2000 + 1500 = 7500$ I/Os
 - similar to the cost of Block Nested Loops Join
- 35 buffer pages, 300 buffer pages – cost remains unchanged (need 2 passes to sort Exams, 2 passes to sort Students)
 - ex: compute cost of BNLJ and compare

* E - M pages, p_E records / page * * 1000 pages * * 100 records / page *

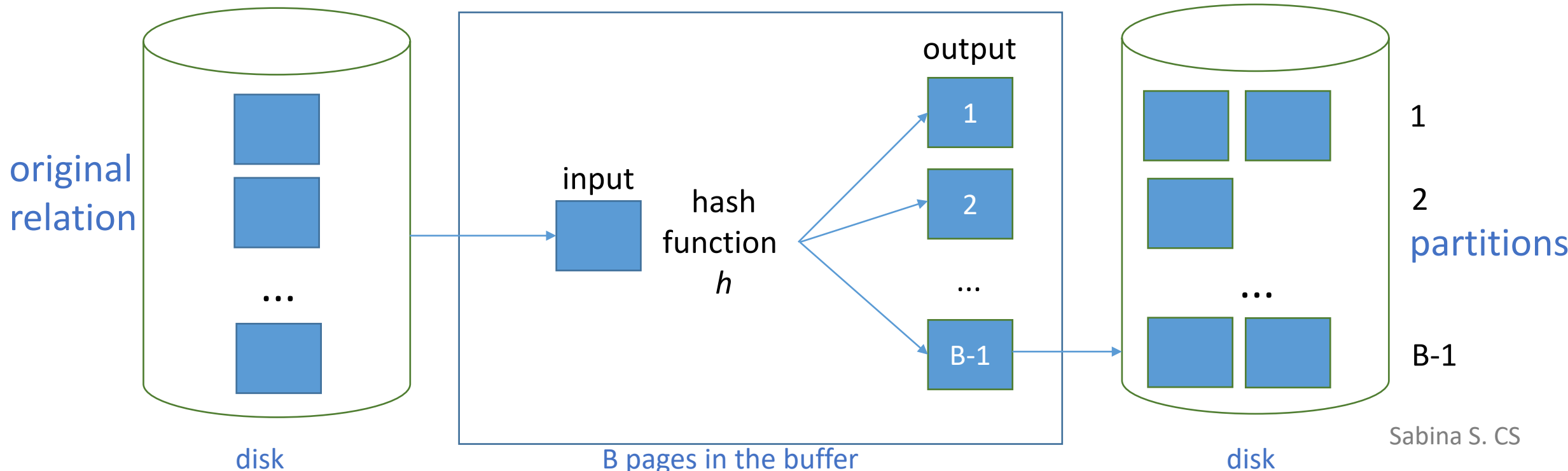
* S - N pages, p_S records / page * * 500 pages * * 80 records / page *

Hash Join - equality join, one join column: $E \bowtie_{i=j} S$

- phases: partitioning (building phase) & probing (matching phase)
- partitioning phase:
 - there are B pages available in the buffer:
 - use one page as the input buffer page
 - and the remaining $B-1$ pages as output buffer pages
 - choose a hash function h that distributes tuples uniformly to one of $B-1$ partitions
 - hash E and S on the join column (the i^{th} column of E , the j^{th} column of S) with the same hash function h

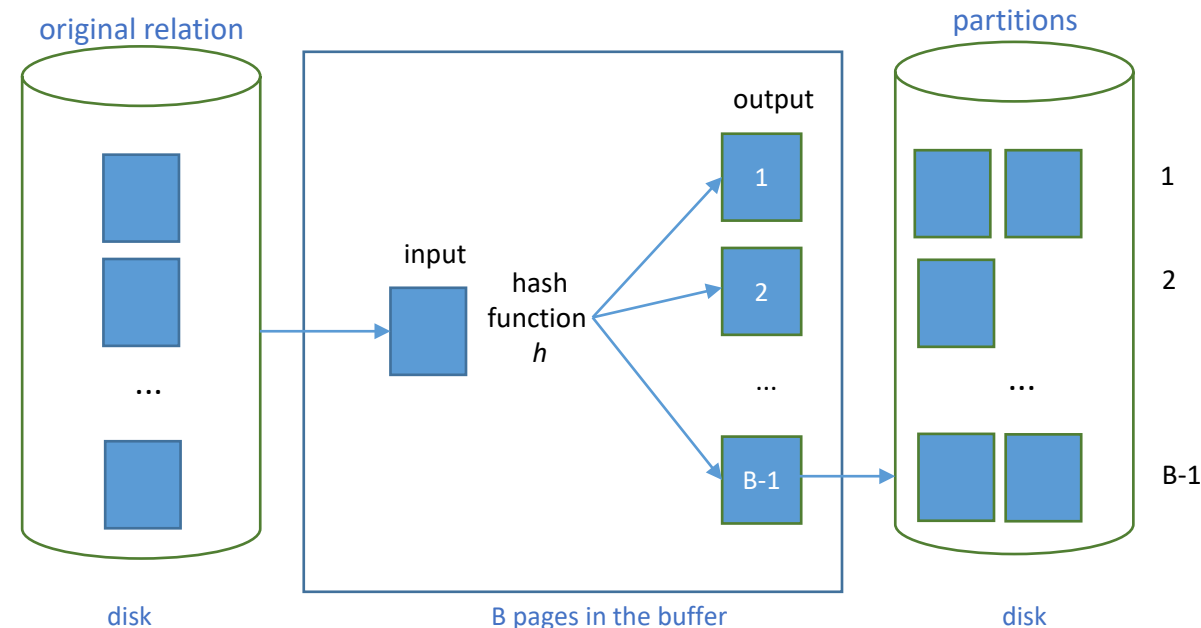
Hash Join

- hash E on the join column with hash function h (similarly for S):
 - for each tuple e in E, compute $h(e_i)$
(e_i : the value of the i^{th} column in tuple e)
 - add tuple e to the output buffer page that it is hashed to by h (buffer page $h(e_i)$)
 - when an output buffer page fills up, flush the page to disk



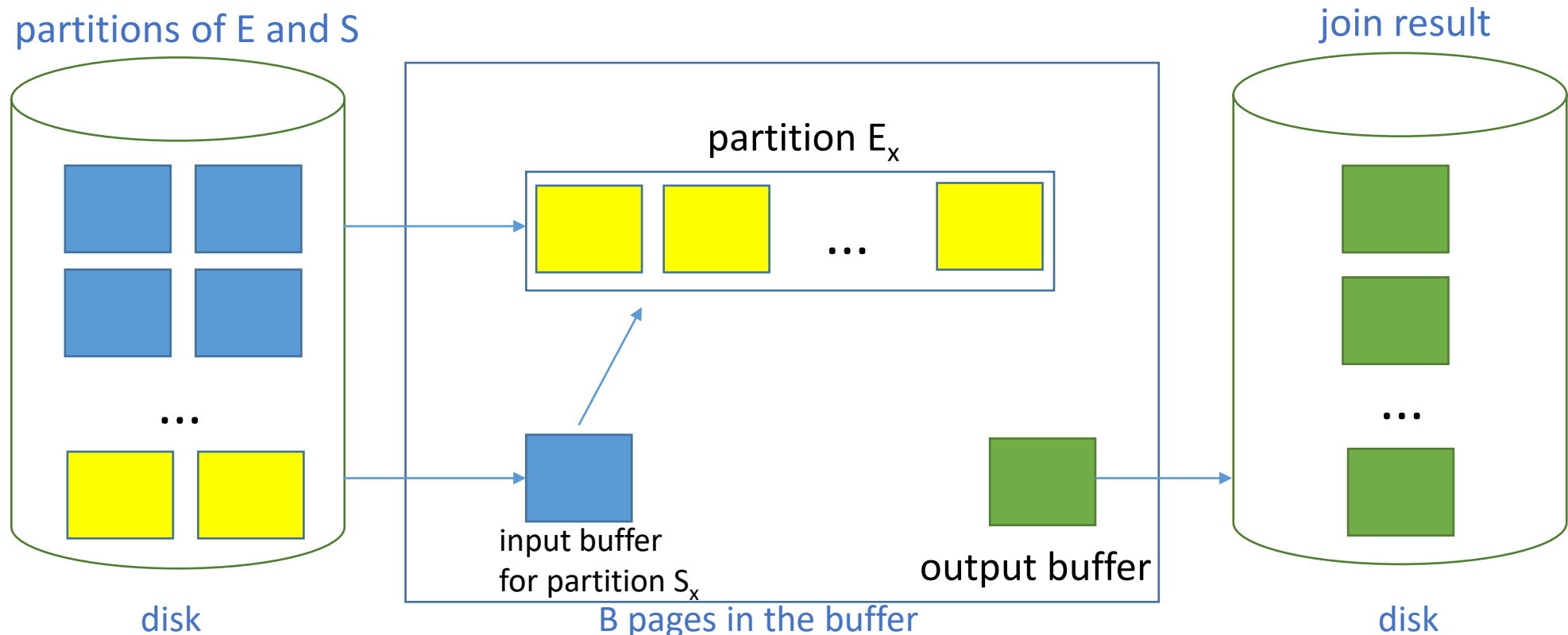
Hash Join

- partitioning phase $\Rightarrow$ *partitions* of E (E_1, E_2 , etc.) and S (S_1, S_2 , etc.) on disk
- partition = collection of tuples that have the same hash value
- tuples in partition E_1 can only join with tuples in partition S_1 (they cannot join with tuples in partitions S_2 or S_3 , for instance, since these tuples have a different hash value)
- so to compute the join, we need to scan E and S only once (provided any partition of E fits in main memory)
- when reading in a partition E_k of E , we must scan only the corresponding partition S_k of S to find matching tuples (compare tuples e in E_k with tuples s in S_k to test the join condition *value of i^{th} column in E = value of j^{th} column in S*)



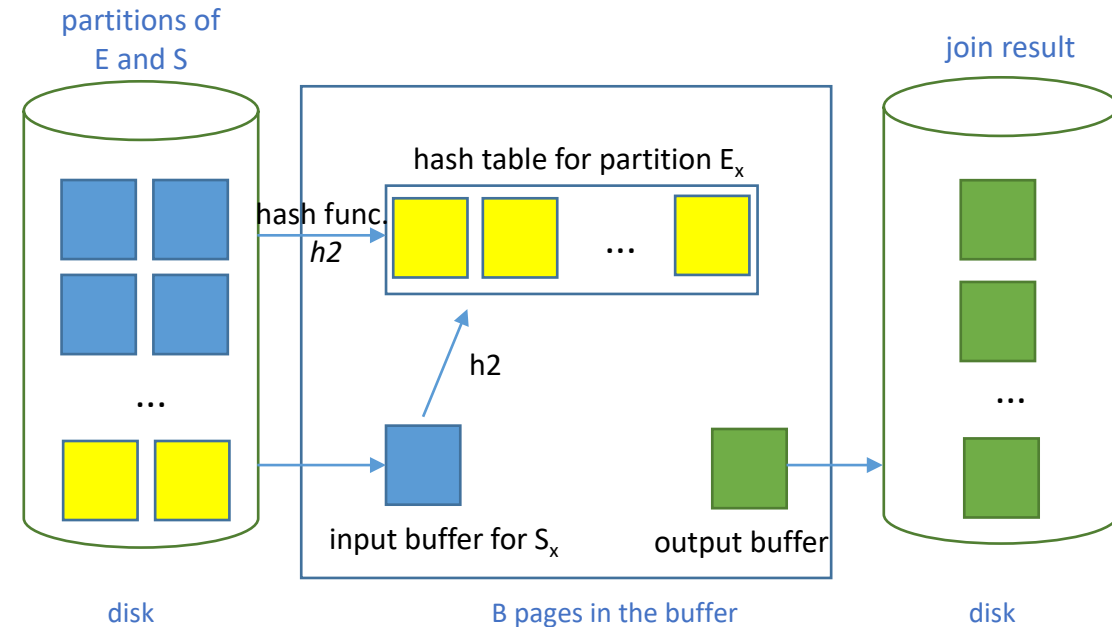
Hash Join

- probing phase:
 - read in a partition of the smaller relation (e.g., E) and scan the corresponding partition of S for matching tuples
 - use one page as the input buffer for S, one page as the output buffer, and the remaining pages to read in partitions of E



Hash Join

- probing phase:
 - in practice, to reduce CPU costs, an in-memory hash table is built, using a different function $h2$, for the E partition
- consider a partition E_x of E
- build in-memory hash table for E_x using hash function $h2$ (the function is applied to the join column of E)
- for each tuple s in partition S_x , find matching tuples in the hash table using the hash value $h2(s_j)$
- result tuples $\langle e, s \rangle$ are written to output buffer
- once partitions E_x and S_x are processed, the hash table is emptied (to prepare for the next partition)



Hash Join

- cost:
 - partitioning: both E and S are read and written once => cost: $2*(M+N)$ I/Os
 - probing: scan each partition once => cost: $M+N$ I/Os=> total cost: $3*(M+N)$ I/Os
 - assumption: each partition fits into memory during probing
 - $3*(1000 + 500) = 4500$ I/Os
- * E - M pages, p_E records / page * * 1000 pages * * 100 records / page*
- * S - N pages, p_S records / page * * 500 pages * * 80 records / page *
- *partition overflow* – an E partition does not fit in memory during probing:
apply hash join technique recursively:
 - divide E, S into subpartitions
 - join subpartitions pairwise
 - if subpartitions don't fit in memory, apply hash join technique recursively

Hash Join

- memory requirements - objective: partition in E fits into main memory (S - similarly)
 - B buffer pages; need one input buffer $\Rightarrow$ maximum number of partitions: B-1
 - size of largest partition: B - 2 (need one input buffer for S, one output buffer)
 - assume uniformly sized partitions $\Rightarrow$ size of each E partition: $M/(B-1)$
 $\Rightarrow M/(B-1) < B-2 \Rightarrow$ we need approximately $B > \sqrt{M}$
- if an in-memory hash table is used to speed up tuple matching $\Rightarrow$ need a little more memory (because the hash table for a collection of tuples will be a little larger than the collection itself)

* E - M pages, p_E records / page *

* 1000 pages * * 100 records / page*

general join conditions

- equalities over several attributes
 - $E.SID = S.SID \text{ AND } E.attrE = S.attrS$
 - index nested loops join
 - Exams – inner relation:
 - build index on Exams with search key $\langle SID, attrE \rangle$ (if not already created)
 - can also use index on SID or index on attrE
 - Students – inner relation (similar)
 - sort-merge join
 - sort Exams on $\langle SID, attrE \rangle$, sort Students on $\langle SID, attrS \rangle$
 - hash join
 - partition Exams on $\langle SID, attrE \rangle$, partition Students on $\langle SID, attrS \rangle$
 - other join algorithms
 - essentially unaffected

general join conditions

- inequality comparison
 - $E.attrE < S.attrS$
 - index nested loops join: B+ tree index required
 - sort-merge join: not applicable
 - hash join: not applicable
 - other join algorithms: essentially unaffected
- * no join algorithm is uniformly superior to others
- choice of a good algorithm depends on:
 - size(s) of:
 - joined relations
 - buffer pool
 - available access methods

Selection

Q:

```
SELECT *  
FROM Exams E  
WHERE E.FacultyMember = 'Ionescu'
```

- use information in the selection condition to reduce the number of retrieved tuples
- e.g., $|Q| = 4$ (result set has 4 tuples), there's a B+ tree index on FacultyMember
 - it's expensive to scan E (1000 I/Os) to evaluate the query
 - should use the index instead
- selection algorithms based on the following techniques:
 - iteration, indexing

* E - M pages, p_E records / page * * 1000 pages * * 100 records / page*

Selection

- simple selections
 - $\sigma_{E.attr \text{ op } val}(E)$
- no index on *attr*, data not sorted on *attr*
 - must scan E and test the condition for each tuple
 - access path: file scan

=> cost: M I/Os = 1000 I/Os
- no index, sorted data (E physically sorted on *attr*)
 - binary search to locate 1st tuple that satisfies condition
and
 - scan E starting at this position until condition is no longer satisfied
 - access method: sorted file scan

Review lecture notes on *Relational Algebra, Indexes, DB – Physical Structure* (Databases course)

Selection

- simple selections
 - $\sigma_{E.attr \text{ op } val}(E)$
- no index, sorted data (E physically sorted on *attr*)
=> cost:
 - binary search: $O(\log_2 M)$
 - scan cost: varies from 0 to M
 - binary search on E
 - $\log_2 1000 \approx 10$ I/Os

Selection

- simple selections
 - $\sigma_{E.attr \text{ op } val}(E)$
- B+ tree index on *attr*
 - * search tree to find 1st index entry pointing to a qualifying E tuple
 - cost: typically 2, 3 I/Os
 - * scan leaf pages to retrieve all qualifying entries
 - cost: depends on the number of qualifying entries
 - * for each qualifying entry - retrieve corresponding tuple in E
 - cost: depends on the number of tuples and the nature of the index (clustered / unclustered)

Selection

- simple selections
 - $\sigma_{E.attr \text{ op } val}(E)$
- B+ tree index on *attr*
 - assumption
 - indexes use a2 or a3
 - a1-based index => data entry contains the data record => the cost of retrieving records = the cost of retrieving the data entries!
 - access path: B+ tree index
 - clustered index:
 - best access path when *op* is not *equality*
 - good access path when *op* is *equality*

Selection

- simple selections: $\sigma_{E.attr \text{ op } val}(E)$

- B+ tree index on *attr*

Q

```
SELECT *
```

```
FROM Exams E
```

```
WHERE E.FacultyMember < 'C%'
```

- names uniformly distributed with respect to 1st letter

=> $|Q| \approx 10,000$ tuples = 100 pages

- clustered B+ tree index on FacultyMember

=> cost of retrieving tuples: ≈ 100 I/Os (a few I/Os to get from root to leaf)

- non-clustered B+ tree index on FacultyMember

=> cost of retrieving tuples: up to 1 I/O per tuple (worst case) => up to 10.000 I/Os

* E - M pages, p_E records / page *

* 1000 pages * * 100 records / page*

Selection

- simple selections: $\sigma_{E.attr \text{ op } val}(E)$

- B+ tree index on *attr*

SELECT *

FROM Exams E

WHERE E.FacultyMember < 'C%'

- non-clustered B+ tree index on FacultyMember
 - refinement - sort rids in qualifying data entries by page-id
=> a page containing qualifying tuples is retrieved only once
 - cost of retrieving tuples: number of pages containing qualifying tuples (but such tuples are probably stored on more than 100 pages)
- range selections
 - non-clustered indexes can be expensive
 - could be less costly to scan the relation (in our example: 1000 I/Os)

Selection

- general selections
 - selections without disjunctions
- C - CNF condition without disjunctions
 - evaluation options:
 1. use the most selective access path
 - if it's an index I:
 - apply conjuncts in C that match I
 - apply rest of conjuncts to retrieved tuples
 - example
 - $c < 100 \text{ AND } a = 3 \text{ AND } b = 5$
 - can use a B+ tree index on c and check $a = 3 \text{ AND } b = 5$ for each retrieved tuple
 - can use a hash index on a and b and check $c < 100$ for each retrieved tuple

Selection

- general selections - selections without disjunctions
 - evaluation options:
 2. use several indexes - when several conjuncts match indexes using a2 / a3
 - compute sets of rids of candidate tuples using indexes
 - intersect sets of rids, retrieve corresponding tuples
 - apply remaining conjuncts (if any)
 - example: $c < 100 \text{ AND } a = 3 \text{ AND } b = 5$
 - use a B+ tree index on c to obtain rids of records that meet condition $c < 100$ (R_1)
 - use a hash index on a to retrieve rids of records that meet condition $a = 3$ (R_2)
 - compute $R_1 \cap R_2 = R_{int}$
 - retrieve records with rids in R_{int} (R)
 - check $b = 5$ for each record in R

Selection

- general selections
 - selections with disjunctions
- C - CNF condition with disjunctions, i.e., some conjunct J is a disjunction of terms
 - if some term T in J requires a file scan, testing J by itself requires a file scan
 - example: $a < 100 \vee b = 5$
 - hash index on b , hash index on c
 - => check both terms using a file scan (i.e., best access path: file scan)
- compare with the example below:
 - $(a < 100 \vee b = 5) \wedge c = 7$
 - hash index on b , hash index on c
 - => use index on c , apply $a < 100 \vee b = 5$ to each retrieved tuple (i.e., most selective access path: index)

Selection

- general selections
 - selections with disjunctions
- C - CNF condition with disjunctions
 - every term T in a disjunction matches an index

=> retrieve tuples using indexes, compute union

 - example
 - $a < 100 \vee b = 5$
 - B+ tree indexes on a and b
 - use index on a to retrieve records that meet condition $a < 100$ (R_1)
 - use index on b to retrieve records that meet condition $b = 5$ (R_2)
 - compute $R_1 \cup R_2 = R$
 - if all matching indexes use a2 or a3 => take union of rids, retrieve corresponding tuples

References

- [Ra02] RAMAKRISHNAN, R., GEHRKE, J., Database Management Systems (3rd Edition), McGraw-Hill, 2002
- [Da03] DATE, C.J., An Introduction to Database Systems (8th Edition), Addison-Wesley, 2003
- [Ga09] GARCIA-MOLINA, H., ULLMAN, J., WIDOM, J., Database Systems: The Complete Book (2nd Edition), Pearson Education, 2009
- [Ra02S] RAMAKRISHNAN, R., GEHRKE, J., Database Management Systems, Slides for the 3rd Edition,
<http://pages.cs.wisc.edu/~dbbook/openAccess/thirdEdition/slides/slides3ed.html>
- [Si19] SILBERSCHATZ, A., KORTH, H., SUDARSHAN, S., Database System Concepts (7th Edition), McGraw-Hill, 2019
- [Si19S] SILBERSCHATZ, A., KORTH, H., SUDARSHAN, S., Database System Concepts, Slides for the 7th Edition, <http://codex.cs.yale.edu/avi/db-book/>
- [Ul11] ULLMAN, J., WIDOM, J., A First Course in Database Systems,
<http://infolab.stanford.edu/~ullman/fcdb.html>

Database Management Systems

Lecture 8

Evaluating Relational Operators

Query Optimization

- running example - schema
 - Students (SID: integer, SName: string, Age: integer)
 - Courses (CID: integer, CName: string, Description: string)
 - Exams (SID: integer, CID: integer, EDate: date, Grade: integer)
- Students
 - every record has 50 bytes
 - there are 80 records / page
 - 500 pages
- Courses
 - every record has 40 bytes
 - there are 100 records / page
 - 1 page
- Exams
 - every record has 40 bytes
 - there are 100 records / page
 - 1000 pages

Projection

- $\Pi_{\text{SID, CID}}(\text{Exams})$

```
SELECT DISTINCT E.SID, E.CID  
FROM Exams E
```

- to implement projection:
 - eliminate:
 - unwanted columns
 - duplicates
- projection algorithms - *partitioning* technique:
 - sorting
 - hashing

Projection Based on Sorting

- step 1
 - scan $E \Rightarrow$ set of tuples containing only desired attributes (E')
 - cost:
 - scan E : M I/Os
 - write temporary relation E' : T I/Os
 - T depends on: number of columns and their sizes, T is $O(M)$
- step 2
 - sort tuples in E'
 - sort key: all columns
 - cost: $O(T \log T)$ (also $O(M \log M)$)
- step 3
 - scan sorted E' , compare adjacent tuples, eliminate duplicates
 - cost: T
- total cost: $O(M \log M)$

Projection Based on Sorting

* example

```
SELECT DISTINCT E.SID, E.CID  
FROM Exams E
```

- scan Exams: 1000 I/Os
- size of tuple in E': 10 bytes

=> cost of writing temporary relation E': 250 I/Os

- available buffer pages: 20
 - E' can be sorted in 2 passes
 - sorting cost: $2 * 2 * 250 = 1000$ I/Os
- final scan of E' - cost: 250 I/Os

=> total cost: $1000 + 250 + 1000 + 250 = 2500$ I/Os

* E – record size = 40 bytes * * 1000 pages * * 100 records / page*

Projection Based on Sorting

* example

```
SELECT DISTINCT E.SID, E.CID  
FROM Exams E
```

- scan Exams: 1000 I/Os
- size of tuple in E': 10 bytes

=> cost of writing temporary relation E': 250 I/Os

- available buffer pages: 257
 - E' can be sorted in 1 pass
 - sorting cost: $2 * 1 * 250 = 500$ I/Os
- final scan of E' - cost: 250 I/Os

=> total cost: $1000 + 250 + 500 + 250 = 2000$ I/Os

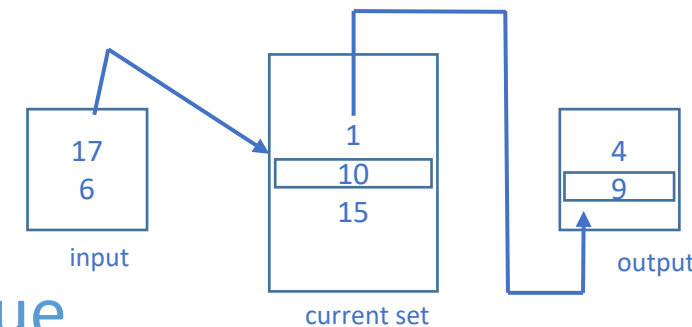
* E – record size = 40 bytes * * 1000 pages * * 100 records / page*

Projection Based on Sorting

- improvement
 - adapt the sorting algorithm to do projection with duplicate elimination
 - modify pass 0 of External Merge Sort: eliminate unwanted columns
 - read in B pages from E
 - write out $(T/M) * B$ internally sorted pages of E'
 - refinement: write out $2*B$ internally sorted pages of E' (on average)
 - tuples in runs - smaller than input tuples
 - modify merging passes: eliminate duplicates
 - number of result tuples is smaller than number of input tuples

* minimize the number of runs - optional

- external merge sort
 - N pages in the file, B buffer pages $\Rightarrow \lceil N/B \rceil$ runs of B pages each
- improvement
 - algorithm known to produce runs of approximately $2*B$ pages (on average)
 - use 1 page as an input buffer, 1 page as an output buffer
 - the remaining buffer pages are collectively referred to as the *current set*
- example - sort file in ascending order on some key k :
 - repeatedly pick record r in the current set and append it to the output buffer
 - keep output buffer sorted: r 's k value $\geq$ largest k value in the output buffer
 - multiple such records in current set - choose the smallest one



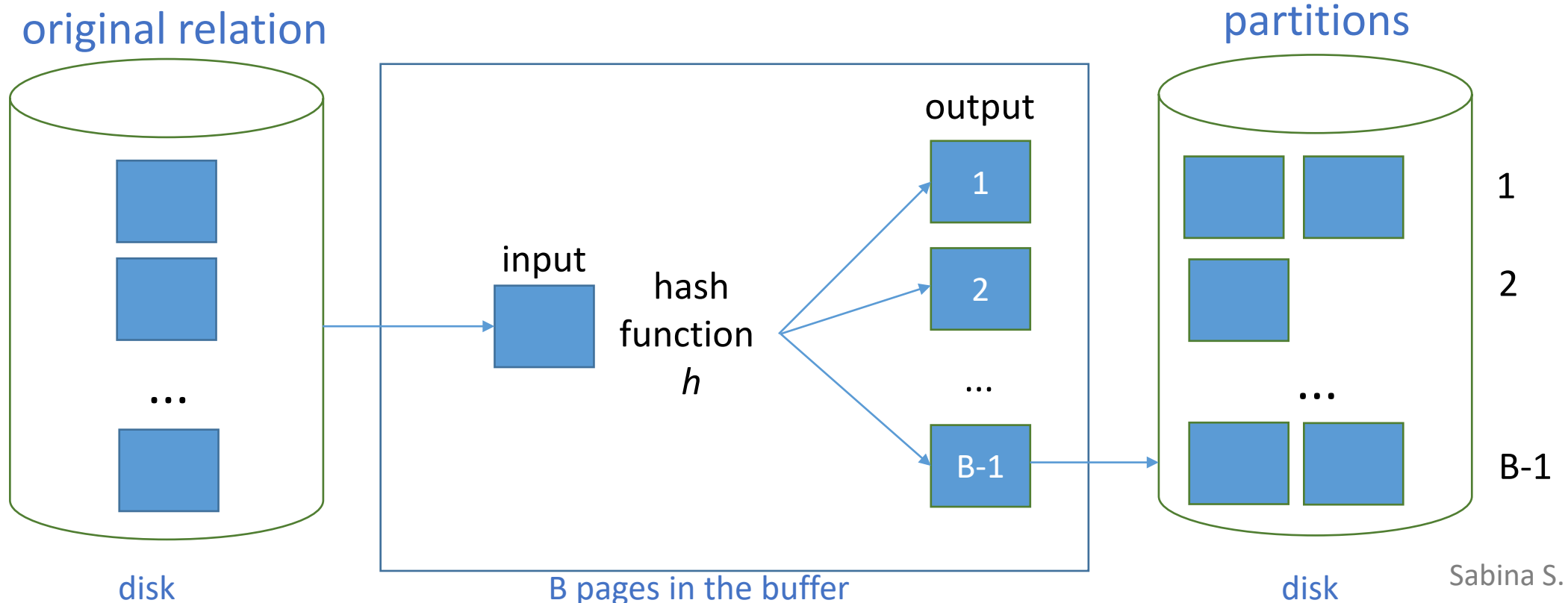
- * minimize the number of runs - optional
- example - sort file in ascending order on some key k :
 - use the extra space in the current set to bring in the next tuple from the input buffer
 - process all tuples in the input buffer, then read in the next page of the file
 - when the output buffer fills up, write it to disk (add its content to the run that is currently being built)
 - the current run is completed when every k value in the current set is $<$ the largest k value in the output buffer; when this happens, the output buffer is written out (its content becomes the last page in the current run), and a new run is started

Projection Based on Sorting

- improvement
 - * example
 - pass 0:
 - scan Exams: 1000 I/Os
 - write out 250 pages:
 - 20 available buffer pages
 - 250 pages => 7 sorted runs about 40 pages long (except the last one, which is about 10 pages long)
 - pass 1:
 - read in all runs – cost: 250 I/Os
 - merge runs
 - total cost : $1000 + 250 + 250 = 1500$ I/Os

Projection Based on Hashing

- phases: partitioning & duplicate elimination
- partitioning phase:
 - 1 input buffer page – read in the relation one page at a time
 - hash function h – distribute tuples uniformly to one of $B-1$ partitions
 - $B-1$ output buffer pages – one output page / partition



Projection Based on Hashing

- partitioning phase:
 - read the relation using the input buffer page
 - for each tuple t :
 - discard unwanted fields $\Rightarrow$ tuple t'
 - apply hash function h to t'
 - write t' to the output buffer page that it is hashed to by h

$\Rightarrow$ B-1 partitions

- partition:
 - collection of tuples with:
 - common hash value
 - no unwanted fields
- tuples in different partitions are guaranteed to be distinct

Projection Based on Hashing

- duplicate elimination phase:
 - process all partitions:
 - read in partition P, one page at a time
 - build in-memory hash table with hash function $h_2 (\neq h)$ on all fields:
 - if a new tuple hashes to the same value as an existing tuple, compare them to check if they are distinct
 - eliminate duplicates
 - write duplicate-free hash table to result file
 - clear in-memory hash table
- partition overflow
 - apply hash-based projection technique recursively (subpartitions)

Projection Based on Hashing

- cost
 - partitioning:
 - read E: M I/Os
 - write E': T I/Os
 - duplicate elimination:
 - read in partitions: T I/Os

=> total cost: $M + 2 * T$ I/Os
- Exams:
 - $1000 + 2 * 250 = 1500$ I/Os

Set Operations

- intersection, cross-product
 - special cases of join (join condition for intersection - equality on all fields, no join condition for cross-product)
- union, set-difference
 - similar
- union: $R \cup S$
 - sorting
 - sort R and S on all attributes
 - scan the sorted relations in parallel; merge them, eliminating duplicates
 - refinement
 - produce sorted runs of R and S, merge runs in parallel

Set Operations

- union: $R \cup S$
 - hashing
 - partition R and S with the same hash function h
 - for each S -partition
 - build in-memory hash table (using h_2) for the S -partition
 - scan corresponding R -partition, add tuples to hash table, discard duplicates
 - write out hash table
 - clear hash table

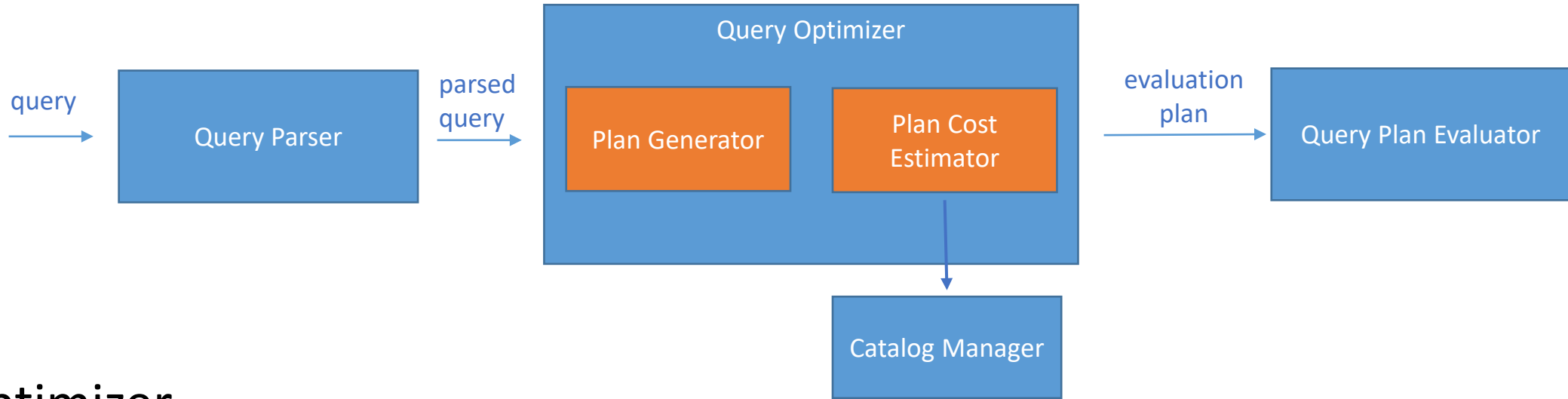
Aggregate Operations

- without grouping
 - scan relation
 - maintain *running information* about scanned tuples
 - COUNT - count of values retrieved
 - SUM - *total* of values retrieved
 - AVG - $\langle total, count \rangle$ of values retrieved
 - MIN, MAX - smallest / largest value retrieved
- with grouping
 - sort relation on the grouping attributes
 - scan relation to compute aggregate operations for each group
 - improvement: combine sorting with aggregation computation
 - alternative approach based on hashing

Aggregate Operations

- using existing indexes
 - index with a search key that includes all the attributes required by the query
 - work with the data entries in the index (instead of the data records)
 - attribute list in the GROUP BY clause is a prefix of the index search key (tree index)
 - get data entries (and records, if necessary) in the required order (i.e., avoid sorting)

Query Optimization



- optimizer
 - objective
 - given a query Q , find a good evaluation plan for Q
 - generates alternative plans for Q , estimates their costs, and chooses the one with the least estimated cost
 - uses information from the system catalogs

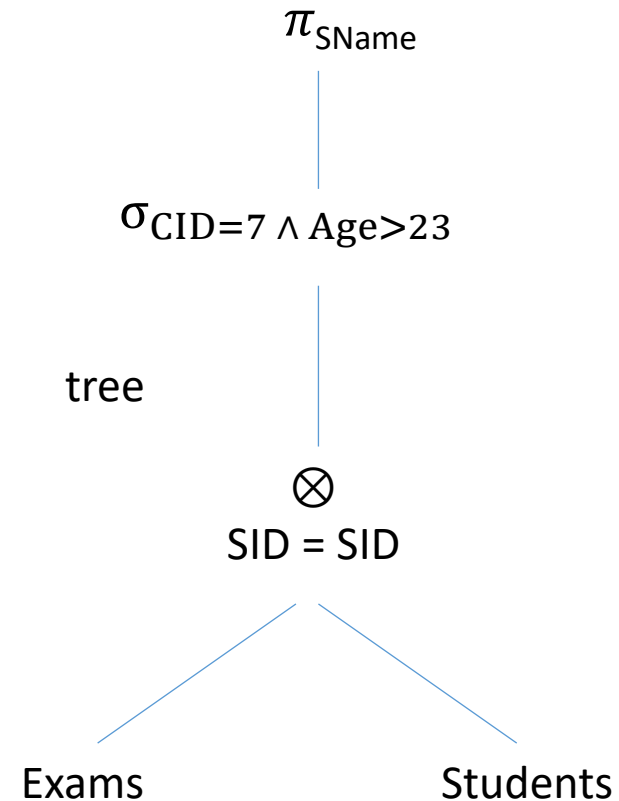
Query Evaluation Plans

```
SELECT S.SName
FROM Exams E, Students S
WHERE E.SID = S.SID AND E.CID = 7
      AND S.Age > 23
```

query

$\pi_{SName}(\sigma_{CID=7 \wedge Age>23}(Exams \otimes_{SID=SID} Students))$

relational algebra expression

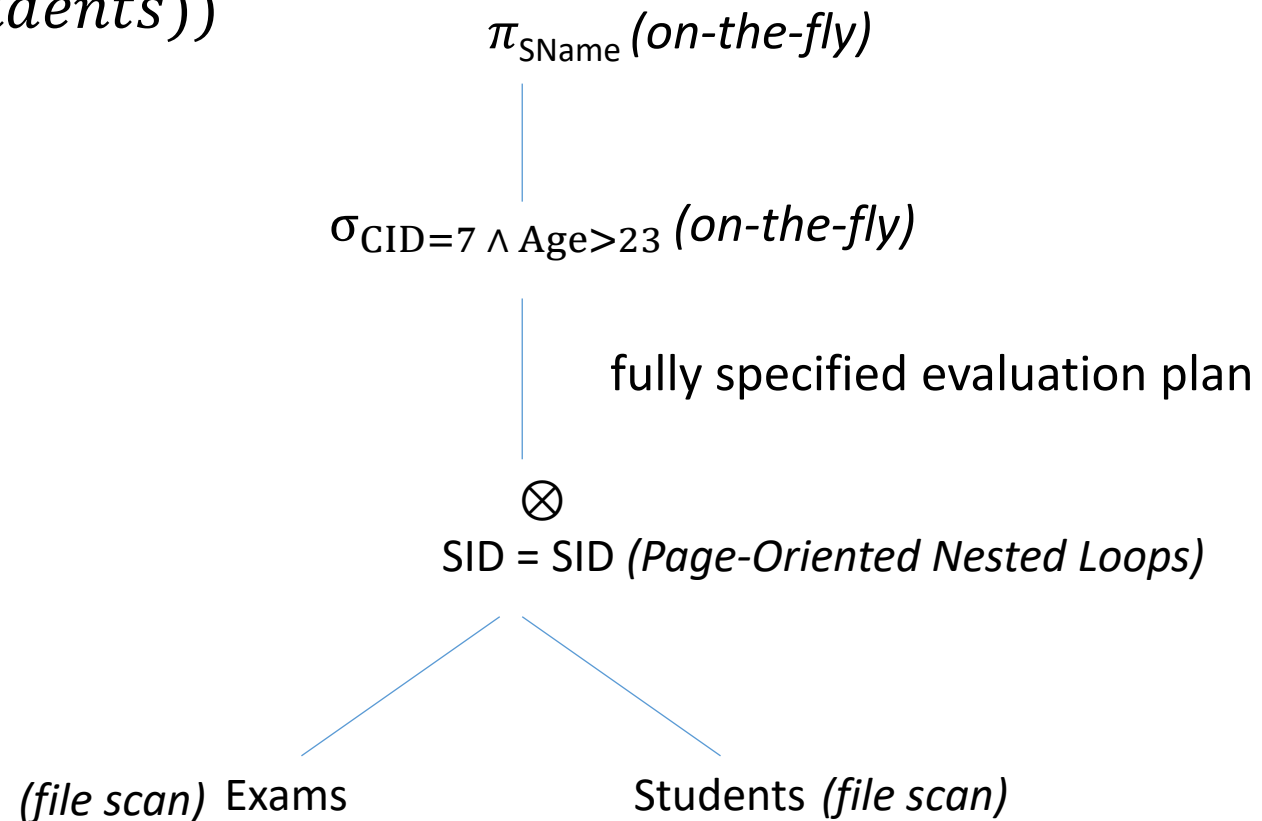


Query Evaluation Plans

```
SELECT S.SName
FROM Exams E, Students S
WHERE E.SID = S.SID AND E.CID = 7
      AND S.Age > 23
```

$\pi_{SName}(\sigma_{CID=7 \wedge Age>23}(Exams \otimes_{SID=SID} Students))$

- query evaluation plan
 - extended relational algebra tree
 - node – annotations
 - relation
 - access method
 - relational operator
 - implementation method

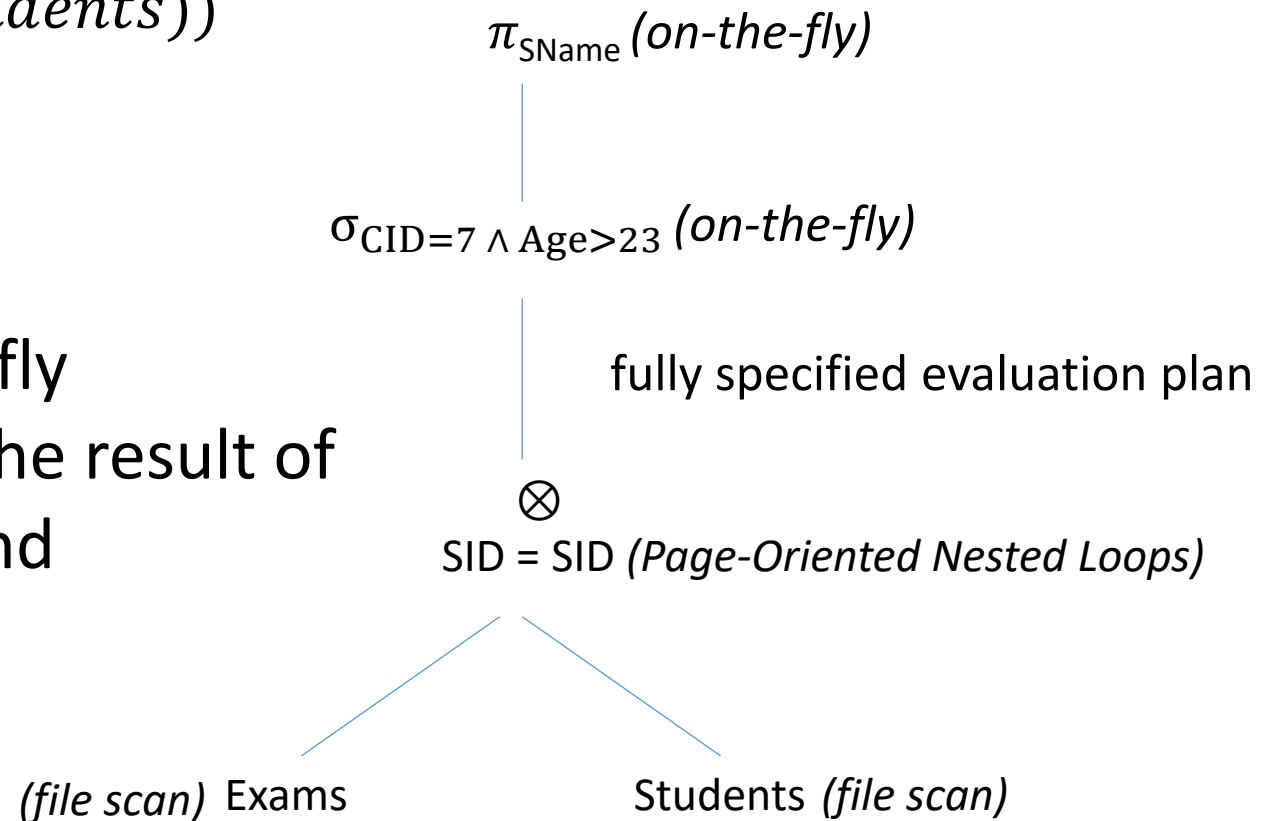


Query Evaluation Plans

```
SELECT S.SName
FROM Exams E, Students S
WHERE E.SID = S.SID AND E.CID = 7
      AND S.Age > 23
```

$\pi_{SName}(\sigma_{CID=7 \wedge Age>23}(Exams \otimes_{SID=SID} Students))$

- Page-Oriented Nested Loops Join
 - Exams – outer relation
- selection, projection applied on-the-fly to each tuple in the join result, i.e., the result of the join (before applying selection and projection) is not stored



Pipelined Evaluation

SELECT *

FROM Exams

WHERE $\frac{EDate > '1-1-2020'}{T1}$ AND $\frac{Grade > 8}{T2}$

$$\sigma_{Grade>8}(\sigma_{EDate>'1-1-2020'}(Exams))$$

- index / matches $T1$
- $v1$ - *materialization*
 - evaluate $T1$
 - write out result tuples to temporary relation R , i.e., tuples are *materialized*
 - apply the 2nd selection to R
 - cost: read and write R

Pipelined Evaluation

SELECT *

FROM Exams

WHERE $\frac{\text{EDate} > '1-1-2020'}{T1}$ AND $\frac{\text{Grade} > 8}{T2}$

- *v2 – pipelined evaluation*
 - apply the 2nd selection to each tuple in the result of the 1st selection as it is produced
 - i.e., 2nd selection operator is applied *on-the-fly*
 - saves the cost of writing out / reading in the temporary relation *R*

Query Blocks – Units of Optimization

- parse $Q \Rightarrow$ collection of query *blocks* $\rightarrow$ passed on to the optimizer
- optimizer:
 - optimize one block at a time
- query *block* - SQL query:
 - without nesting
 - with exactly: one SELECT clause, one FROM clause
 - with at most: one WHERE clause, one GROUP BY clause, one HAVING clause
 - WHERE condition - CNF

Query Blocks – Units of Optimization

- query Q:

```
SELECT S.SID, MIN(E.EDate)
FROM Students S, Exams E, Courses C
WHERE S.SID = E.SID AND E.CID = C.CID AND C.Description = 'Elective' AND
      S.Age = (SELECT MAX(S2.Age)
               FROM Students S2)
GROUP BY S.SID
HAVING COUNT(*) > 2
```

nested block

- decompose query into a collection of blocks without nesting

```
SELECT S.SID, MIN(E.EDate)
FROM Students S, Exams E, Courses C
WHERE S.SID = E.SID AND E.CID = C.CID AND C.Description = 'Elective' AND
      S.Age = Reference to nested block
GROUP BY S.SID
HAVING COUNT(*) > 2
```

Query Blocks – Units of Optimization

* block optimization

- express query block as a relational algebra expression

```
SELECT S.SID, MIN(E.EDate)
FROM Students S, Exams E, Courses C
WHERE S.SID = E.SID AND E.CID = C.CID AND C.Description = 'Elective' AND
      S.Age = Reference to nested block
GROUP BY S.SID
HAVING COUNT(*) > 2
```

$$\pi_{S.SID, MIN(E.EDate)}($$
$$HAVING_{COUNT(*) > 2}($$
$$GROUP BY_{S.SID}($$
$$\sigma_{S.SID = E.SID \wedge E.CID = C.CID \wedge C.Description = 'Elective' \wedge S.Age = value_from_nested_block}($$
$$Students \times Exams \times Courses)))))$$

- GROUP BY, HAVING – operators in the extended algebra used for plans
- argument list of projection can include aggregate operations

Query Blocks – Units of Optimization

- query Q treated as a $\sigma \pi \times$ algebra expression
- the remaining operations in Q are performed on the result of the $\sigma \pi \times$ expression

```
SELECT S.SID, MIN(E.EDate)
FROM Students S, Exams E, Courses C
WHERE S.SID = E.SID AND E.CID = C.CID AND C.Description = 'Elective' AND
      S.Age = Reference to nested block
GROUP BY S.SID
HAVING COUNT(*) > 2
```

$$\pi_{S.SID, E.EDate}(\sigma_{S.SID = E.SID \wedge E.CID = C.CID \wedge C.Description = 'Elective' \wedge S.Age = value_from_nested_block}(Students \times Exams \times Courses))$$

- attributes in GROUP BY, HAVING are added to the argument list of projection
- aggregate expressions in the argument list of projection are replaced by their argument attributes

Query Blocks – Units of Optimization

* block optimization

- find best plan P for the $\sigma \pi \times$ expression
- evaluate $P \Rightarrow$ result set RS
- sort/hash $RS \Rightarrow$ groups
- apply HAVING to eliminate some groups
- compute aggregate expressions in SELECT for each remaining group

$\pi_{S.SID, MIN(E.EDate)}(\$
 $HAVING_{COUNT(*) > 2}(\$
 $GROUP BY_{S.SID}(\$
 $\pi_{S.SID, E.EDate}(\$
 $\sigma_{S.SID = E.SID \wedge E.CID = C.CID \wedge C.Description = 'Elective' \wedge S.Age = value_from_nested_block}(\$
 $Students \times Exams \times Courses))))$

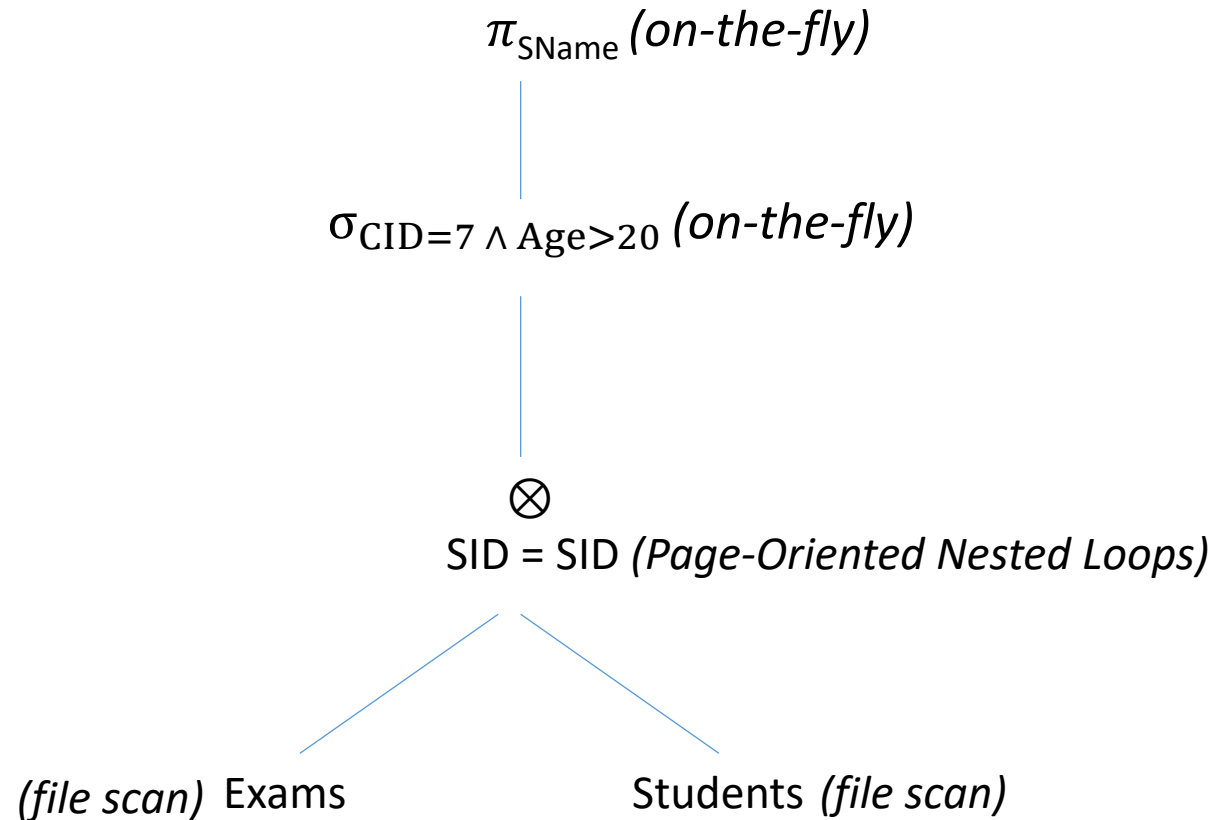
Motivating Example

* E - 1000 pages *

* S - 500 pages *

```
SELECT S.SName
FROM Exams E, Students S
WHERE E.SID = S.SID AND E.CID = 7
      AND S.Age > 20
```

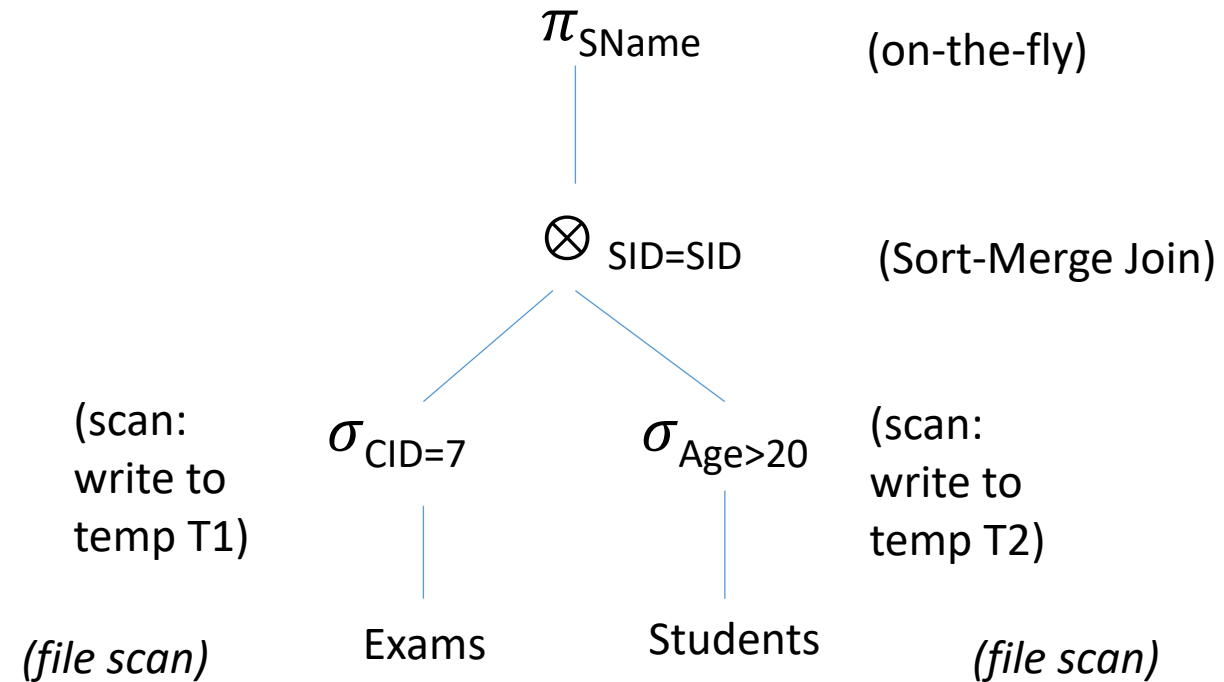
- σ, π – on-the-fly
- cost of plan – very high:
 - $1000 + 1000 * 500 = 501,000$ I/Os



Motivating Example

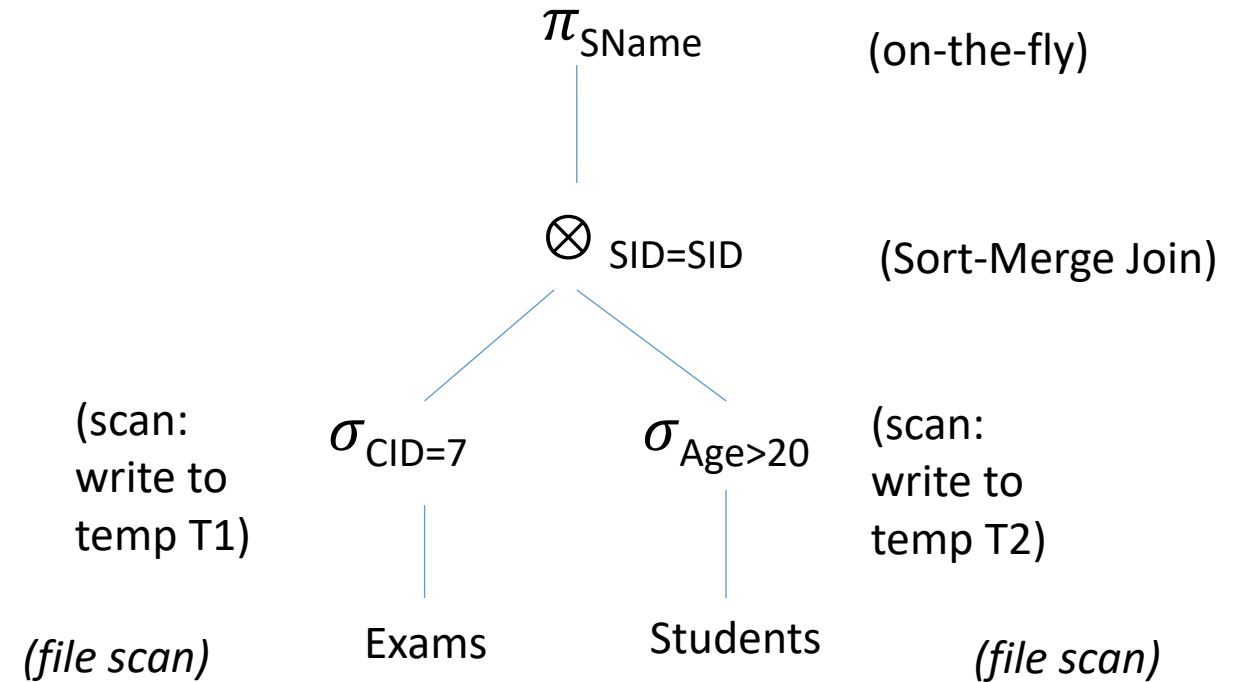
* optimizations

- reduce sizes of the relations to be joined
 - push selections, projections ahead of the join
- alternative plans
 - push selections ahead of joins
- selection
 - file scan
 - write the result to a temporary relation on disk
- join the temporary relations using Sort-Merge Join



Motivating Example

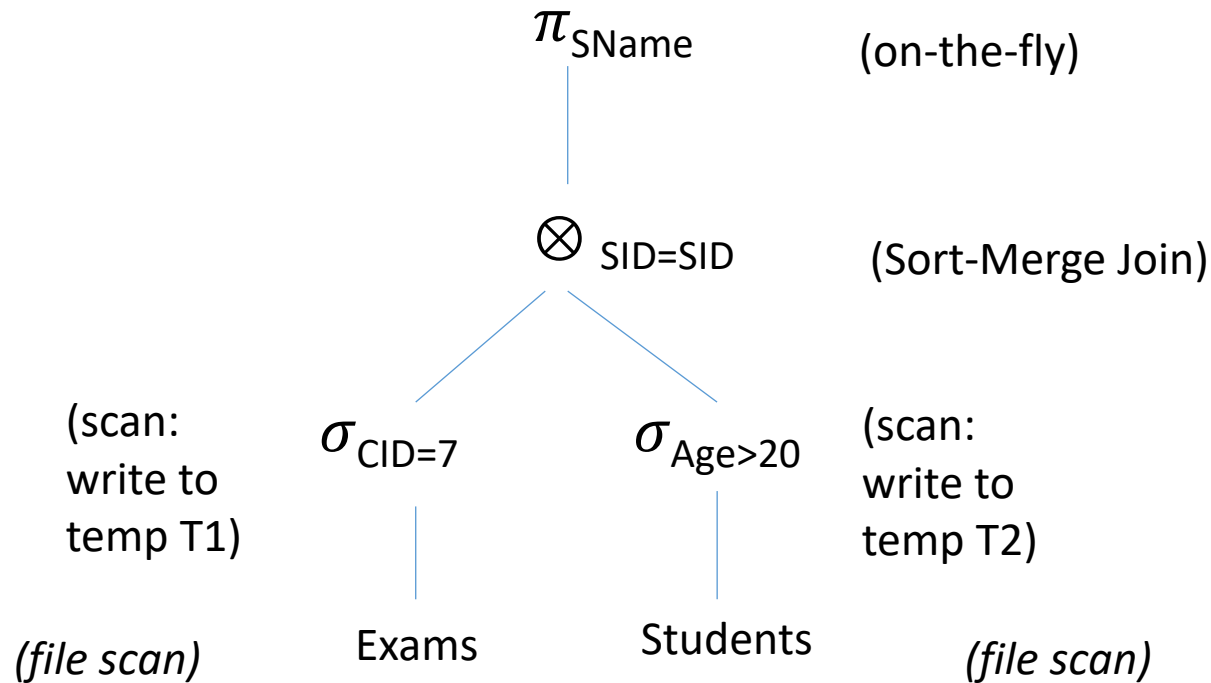
- 5 available buffer pages
- cost
 - $\sigma_{CID=7}$
 - scan Exams: 1000 I/Os
 - write T1
 - assume exams are uniformly distributed across all courses, i.e., T1 has 10 pages (there are 100 courses)
 - $\sigma_{Age>20}$
 - scan Students: 500 I/Os
 - write T2
 - assume ages are uniformly distributed over the range 19 to 22, i.e., T2 has 250 pages



Motivating Example

- 5 available buffer pages
- cost
 - Sort-Merge Join
 - T1 - 10 pages
 - sort T1: $2 * 2 * 10 = 40$ I/Os
 - T2 - 250 pages
 - sort T2: $2 * 4 * 250 = 2000$ I/Os
 - merge sorted T1 and T2
 - $10 + 250 = 260$ I/Os
 - π - on the fly

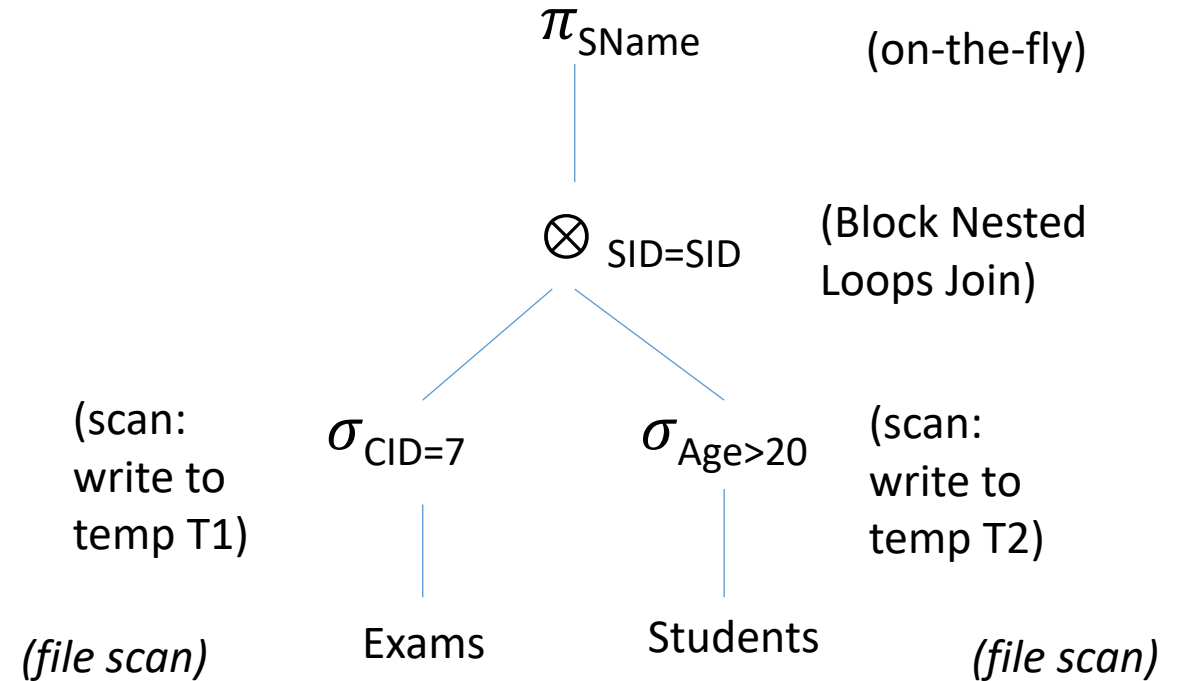
=> **total cost:** $\underbrace{1000 + 10 + 500 + 250}_{\text{selection}} + \underbrace{40 + 2000 + 260}_{\text{join}} = 4060 \text{ I/Os}$



Motivating Example

- 5 available buffer pages
- cost
 - Block Nested Loops Join
 - T1 - 10 pages, T2 - 250 pages
 - T1 - outer relation
=> scan T1: 10 I/Os
 - $\lceil 10/3 \rceil = 4$ T1 blocks
=> T2 scanned 4 times: $4 * 250 = 1000$ I/Os
 - BNLJ cost: $10 + 1000 = 1010$ I/Os
 - π - on the fly

=> **total cost:** $\underbrace{1000 + 10 + 500 + 250}_{\text{selection}} + \underbrace{10 + 1000}_{\text{join}} = \mathbf{2770 \text{ I/Os}}$



Motivating Example

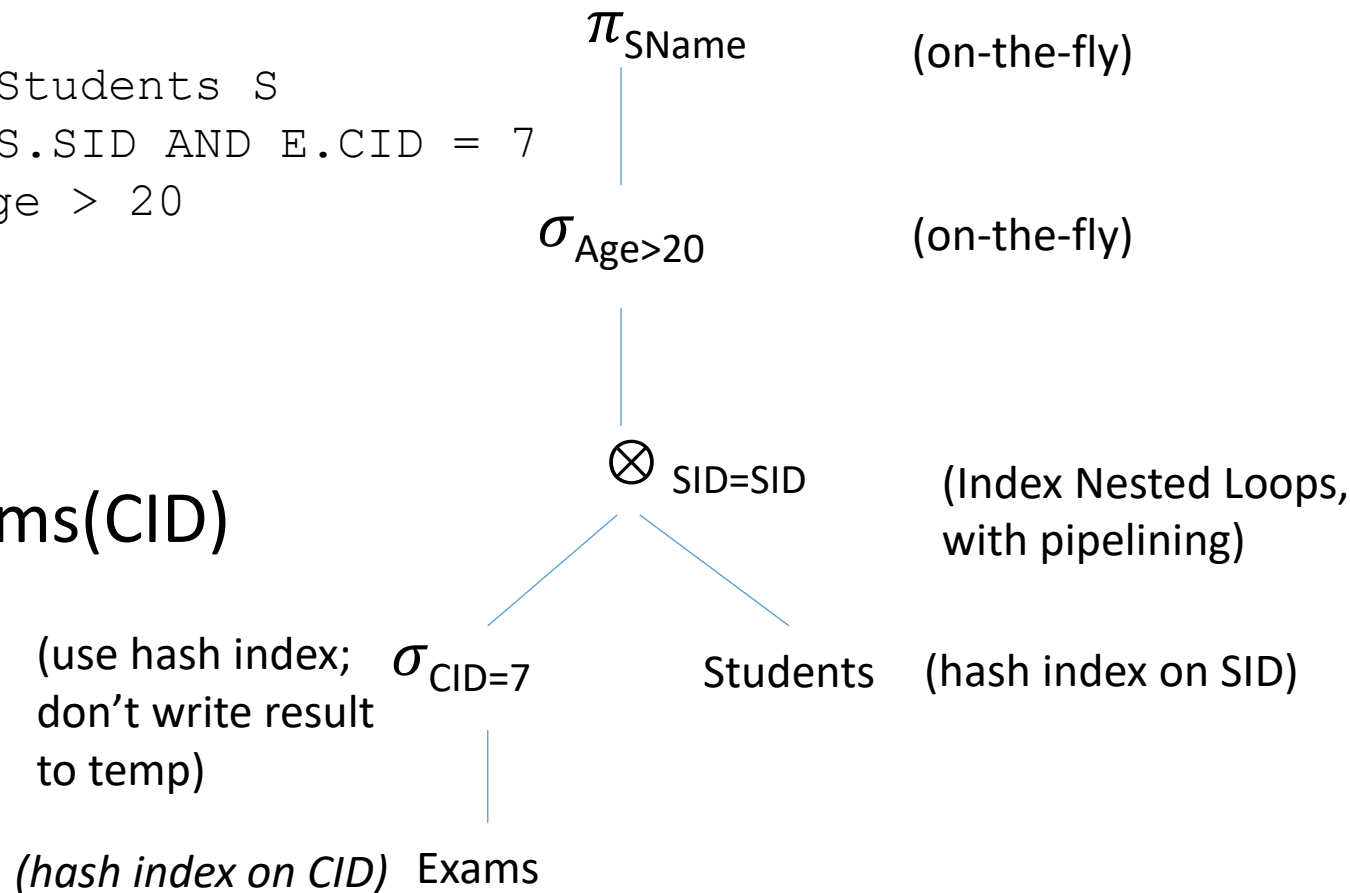
- push projections ahead of joins
 - drop unwanted columns while scanning Exams and Students to evaluate selections => T1[SID], T2[SID, SName]
- T1 fits within 3 buffer pages
 - => T2 scanned only once
 - => **total cost**: about **2000 I/Os**

Motivating Example

* optimizations

- investigate the use of indexes
- clustered static hash index on Exams(CID)
- hash index on Students(SID)
- cost
 - $\sigma_{CID=7}$
 - assume exams are uniformly distributed across all courses => 100,000 exams / 100 courses => 1,000 exams / course
 - clustered index on CID => 1,000 tuples for course with CID=7 appear consecutively within the same bucket => cost: 10 I/Os
 - the result of the selection is not materialized, the join is pipelined

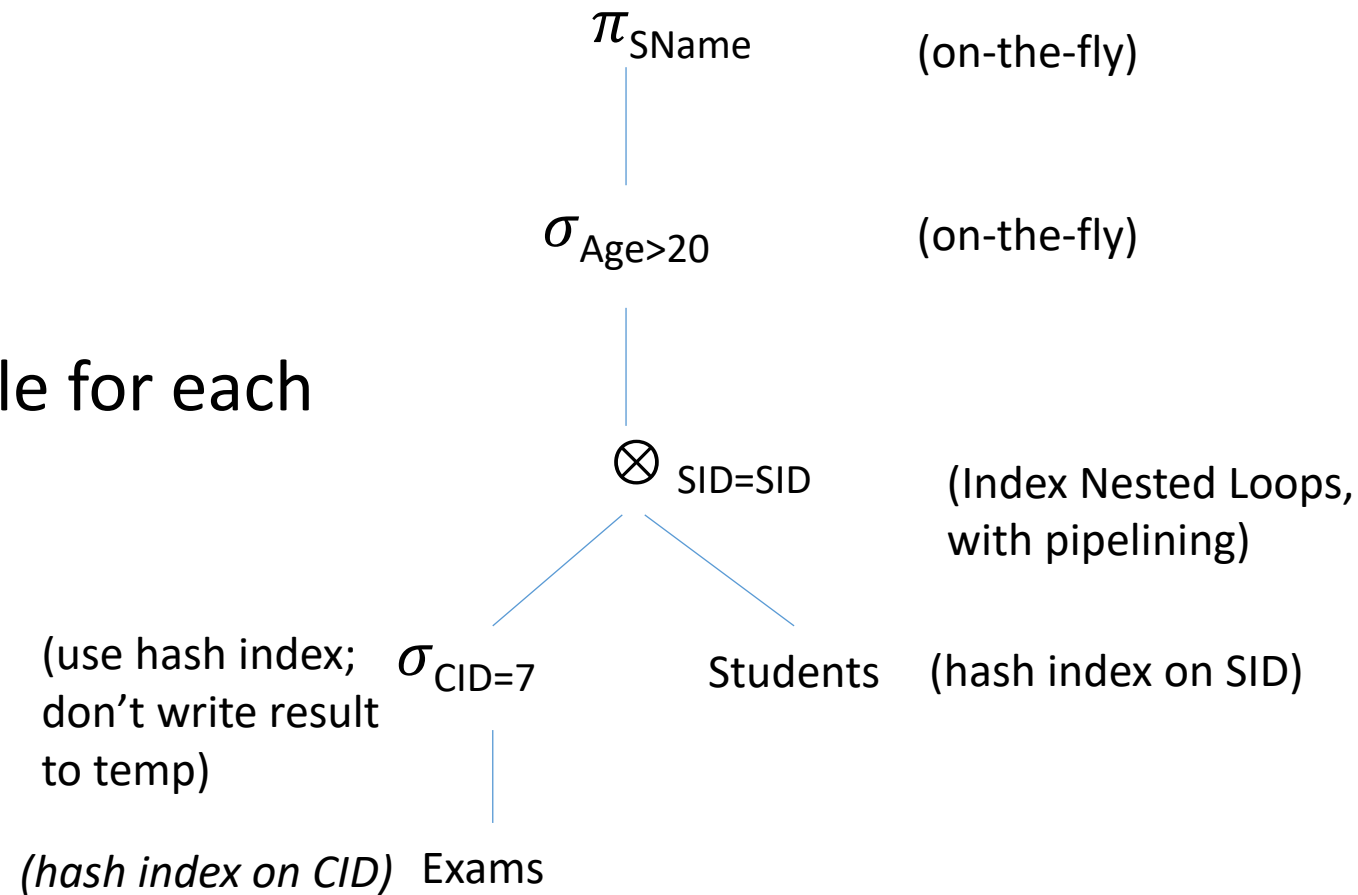
```
SELECT S.SName
FROM Exams E, Students S
WHERE E.SID = S.SID AND E.CID = 7
      AND S.Age > 20
```



Motivating Example

- cost
 - Index Nested Loops
 - find matching Students tuple for each selected exam
 - use hash index on SID
 - assume the index uses a1 => cost of 1.2 I/Os (on avg.) per exam
- σ, π – performed on-the-fly on each tuple in the result of the join

$$\Rightarrow \text{total cost} = \underbrace{10}_{\sigma \text{ on Exams}} + \underbrace{1000}_{\text{num. of Exams tuples}} * \underbrace{1.2}_{\text{find matching Students tuple (on avg.)}} = \mathbf{1210 \text{ I/Os}}$$



* can we push the selection *Age>20* ahead of the join?

References

- [Ra02] RAMAKRISHNAN, R., GEHRKE, J., Database Management Systems (3rd Edition), McGraw-Hill, 2002
- [Da03] DATE, C.J., An Introduction to Database Systems (8th Edition), Addison-Wesley, 2003
- [Ga09] GARCIA-MOLINA, H., ULLMAN, J., WIDOM, J., Database Systems: The Complete Book (2nd Edition), Pearson Education, 2009
- [Ra02S] RAMAKRISHNAN, R., GEHRKE, J., Database Management Systems, Slides for the 3rd Edition,
<http://pages.cs.wisc.edu/~dbbook/openAccess/thirdEdition/slides/slides3ed.html>
- [Si19] SILBERSCHATZ, A., KORTH, H., SUDARSHAN, S., Database System Concepts (7th Edition), McGraw-Hill, 2019
- [Si19S] SILBERSCHATZ, A., KORTH, H., SUDARSHAN, S., Database System Concepts, Slides for the 7th Edition, <http://codex.cs.yale.edu/avi/db-book/>
- [Ul11] ULLMAN, J., WIDOM, J., A First Course in Database Systems,
<http://infolab.stanford.edu/~ullman/fcdb.html>

Database Management Systems

Lecture 9

Evaluating Relational Operators

Query Optimization

- running example - schema
 - Students (SID: integer, SName: string, Age: integer, RoundedGPA: integer)
 - Courses (CID: integer, CName: string, Description: string)
 - Exams (SID: integer, CID: integer, EDate: date, Grade: integer)
- Students
 - every record has 50 bytes
 - there are 80 records / page
 - 500 pages
- Courses
 - every record has 40 bytes
 - there are 100 records / page
 - 1 page
- Exams
 - every record has 40 bytes
 - there are 100 records / page
 - 1000 pages

IBM's System R Optimizer

- tremendous influence on subsequent relational optimizers
- design choices:
 - use statistics to estimate the costs of query evaluation plans
 - consider only plans with binary joins in which the inner relation is a base relation
 - focus optimization on SQL queries without nesting
 - don't eliminate duplicates when performing projections (unless DISTINCT is used)

Estimating the Cost of a Plan

- estimating the cost of an evaluation plan for a query block
 - for each node N in the tree:
 - estimate the cost of the corresponding operation (pipelining versus temporary relations)
 - estimate the size of N's result and whether it is sorted
 - N's result is the input of N's parent node
 - these estimates affect the estimation of cost, size, and sort order for N's parent

Estimating the Cost of a Plan

- estimating costs
 - use data about the input relations (such statistics are stored in the DBMS's system catalogs)
 - number of pages, existing indexes, etc.
 - obtained estimates are at best approximations to actual sizes and costs
- => one shouldn't expect the optimizer to find the best possible plan
- optimizer - goals:
 - avoid the worst plans
 - find a good plan

Statistics Maintained by the DBMS

- updated periodically, not every time the data is changed
 - relation R
 - cardinality - $NTuples(R)$
 - the number of tuples in R
 - size - $NPages(R)$
 - the number of pages in R
 - index I
 - cardinality - $NKeys(I)$
 - the number of distinct key values for I
 - size - $INPages(I)$
 - the number of pages for I
 - B+ tree index
 - number of leaf pages

Statistics Maintained by the DBMS

- index I
 - height - $IHeight(I)$
 - maintained for tree indexes
 - the number of nonleaf levels in I
 - range - $ILow(I), IHigh(I)$
 - the minimum / maximum key value in I

Estimating Result Sizes

- query Q
SELECT attribute list
FROM relation list
WHERE $term_1 \wedge \dots \wedge term_k$
- the maximum number of tuples in Q's result:
 - $\prod |R_i|$
where $R_i \in \text{relation list}$
- each $term_j$ in the WHERE clause eliminates some candidate tuples
 - associate a reduction factor RF_j with each term $term_j$
 - RF_j models the impact $term_j$ has on the result size
- estimate the actual size of the result:
 - $\prod |R_i| * \prod RF_j$
 - i.e., the maximum result size times the product of the reduction factors for the terms in the WHERE clause

Estimating Result Sizes

- query Q
SELECT attribute list
FROM relation list
WHERE $\text{term}_1 \wedge \dots \wedge \text{term}_k$
- assumption
 - the conditions tested by the terms in the WHERE clause are statistically independent

Estimating Result Sizes

- compute reduction factors for terms in the WHERE clause
- assumptions:
 - uniform distribution of values
 - independent distribution of values in different columns

SELECT attribute list

FROM relation list

WHERE term₁ AND ... AND term_k

- *column = value*
 - index I on *column*
=> RF approximated by $1/NKeys(I)$
 - no index on *column*
=> RF: $1/10$
 - maintain statistics on *column* (e.g., number of distinct values in *column*) to obtain a better value

Estimating Result Sizes

- *column1 = column2*
 - indexes *I1* on *column1*, *I2* on *column2*
=> RF: $1/\text{MAX}(\text{NKeys}(I1), \text{NKeys}(I2))$
 - only one index *I* (on one of the 2 columns)
=> RF: $1/\text{NKeys}(I)$
 - no indexes
=> RF: $1/10$
- *column > value*
 - index *I* on *column*
=> RF: $(\text{IHigh}(I) - \text{value}) / (\text{IHigh}(I) - \text{ILow}(I))$
 - no index on *column* or *column* not of an arithmetic type
=> a value less than 0.5 is arbitrarily chosen
 - similar formulas can be obtained for other range selections

Estimating Result Sizes

- *column IN (list of values)*
=> RF: (RF for *column = value*) * number of items in list (but at most 0.5)
- *NOT condition*
=> RF: 1 - RF for *condition*
- obtain better estimates
 - use more detailed statistics (e.g., histograms of the values in a column)

Relational Algebra Equivalences

- central role in generating alternative plans
- different join orders can be considered
- selections, projections can be pushed ahead of joins
- cross-products can be converted to joins
- selections
 - *cascading selections*
 - $\sigma_{c1 \wedge \dots \wedge cn}(R) \equiv \sigma_{c1}(\sigma_{c2}(\dots(\sigma_{cn}(R))\dots))$
 - *commutativity*
 - $\sigma_{c1}(\sigma_{c2}(R)) \equiv \sigma_{c2}(\sigma_{c1}(R))$
- projections
 - *cascading projections*
 - $\pi_{a1}(R) \equiv \pi_{a1}(\pi_{a2}(\dots(\pi_{an}(R))\dots))$
 - a_i – set of attributes in R
 - $a_i \subseteq a_{i+1}$, for $i = 1..n-1$

Relational Algebra Equivalences

- joins and cross-products
 - assumption
 - fields are identified by their name, not by their position
 - *associativity*
 - $R \times (S \times T) \equiv (R \times S) \times T$
 - $R * (S * T) \equiv (R * S) * T$
 - *commutativity*
 - $R \times S \equiv S \times R$
 - $R * S \equiv S * R$
 - can choose the inner / outer relation in a join

Relational Algebra Equivalences

- joins and cross-products
 - e.g., check that $R * (S * T) \equiv (T * R) * S$
 - commutativity
 - $R * (S * T) \equiv R * (T * S)$
 - associativity
 - $R * (T * S) \equiv (R * T) * S$
 - commutativity
 - $(R * T) * S \equiv (T * R) * S$

Relational Algebra Equivalences

- can commute σ with π if σ uses only attributes retained by π
 - $\pi_a(\sigma_c(R)) \equiv \sigma_c(\pi_a(R))$
- can combine σ with $\times$ to form a join
 - $R \otimes_c S \equiv \sigma_c(R \times S)$
- can commute σ with $\times$ or a join when the selection condition includes only fields of one of the arguments (to the cross-product or join)
 - for instance:
 - $\sigma_c(R * S) \equiv \sigma_c(R) * S$
 - $\sigma_c(R \times S) \equiv \sigma_c(R) \times S$
 - condition c must include only fields from R
- in general: $\sigma_c(R \times S) \equiv \sigma_{c_1}(\sigma_{c_2}(R) \times \sigma_{c_3}(S))$
 - c_1 – attributes of both R and S
 - c_2 – only attributes of R
 - c_3 – only attributes of S

Relational Algebra Equivalences

- can commute π with $\times$
 - $\pi_a(R \times S) \equiv \pi_{a_1}(R) \times \pi_{a_2}(S)$
 - a_1 – attributes in a that appear in R
 - a_2 – attributes in a that appear in S
- can commute π with join
 - $\pi_a(R \bowtie_c S) \equiv \pi_{a_1}(R) \bowtie_c \pi_{a_2}(S)$
 - every attribute in c must appear in a
 - a_1 – attributes in a that appear in R
 - a_2 – attributes in a that appear in S
 - a doesn't contain all the attributes in c – generalization
 - eliminate unwanted fields, compute join, eliminate fields not in a
 - $\pi_a(R \bowtie_c S) \equiv \pi_a(\pi_{a_1}(R) \bowtie_c \pi_{a_2}(S))$
 - a_1 – attributes of R that appear in either a or c
 - a_2 – attributes of S that appear in either a or c

Enumeration of Alternative Plans

- query Q
 - consider a certain set of plans
 - choose the plan with the least estimated cost
 - algebraic equivalences
 - implementation techniques for Q's operators
- not all algebraically equivalent plans are enumerated (optimization costs would be too high)
- two main cases:
 - queries with one relation in the FROM clause
 - queries with two or more relations in the FROM clause

Enumeration of Alternative Plans

- queries with one relation in the FROM clause
 - i.e., no joins; only σ , π , grouping, aggregate operations
 - if there is only one σ or π or aggregate operation: consider implementation techniques and cost estimates discussed in previous lectures
 - if there is a combination of operations:
 - plans with / without indexes
 - example query:

```
SELECT S.RoundedGPA, COUNT (*)  
FROM Students S  
WHERE S.RoundedGPA > 5 AND S.Age = 20  
GROUP BY S.RoundedGPA  
HAVING COUNT (DISTINCT S.SName) > 5
```

$\pi_{S.RoundedGPA, COUNT(*)}(\text{HAVING}_{COUNT\ DISTINCT\ (S.SName) > 5}(\text{GROUP BY}_{S.RoundedGPA}(\pi_{S.RoundedGPA, S.SName}(\sigma_{S.RoundedGPA > 5 \wedge S.Age = 20}(Students))))))$

Enumeration of Alternative Plans

* plans without indexes:

- apply σ, π while scanning Students
 - file scan
 - $\text{NPages}(\text{Students})$
 - 500 I/Os
 - write out tuples to a temporary relation T:
 - $\text{NPages}(\text{Students}) * \text{RF}(\text{RoundedGPA} > 5) * \text{RF}(\text{Age} = 20) *$
(size of a *pair* $\langle \text{RoundedGPA}, \text{SName} \rangle$ / size of a *Students tuple*)
 - RF for *RoundedGPA* > 5
 - 0.5
 - RF for *Age* = 20
 - 0.1
 - size of $\langle \text{RoundedGPA}, \text{SName} \rangle$
 - about $0.8 * \text{size of a Students tuple}$

Enumeration of Alternative Plans

* plans without indexes:

- apply σ, π while scanning Students
 - write out tuples to a temporary relation T:
 $\Rightarrow 500 * 0.5 * 0.1 * 0.8 = 20$ I/Os (temporary relation T)
- GROUP BY:
 - sort T in 2 passes
 - $4 * 20 = 80$ I/Os
- HAVING, aggregations
 - no additional I/O
- **total cost**
 - $500 + 20 + 80 = \mathbf{600}$ I/Os

Enumeration of Alternative Plans

* plans that use an index:

- available indexes on Students - a2
 - hash index on <Age>
 - B+ tree index on <RoundedGPA>
 - B+ tree index on <RoundedGPA, SName, Age>
- single-index access path:
 - choose the index that provides the most selective access path
 - apply π , nonprimary selection terms (i.e., that don't match the index)
 - compute grouping and aggregation operations
 - example:
 - use the hash index on Age to retrieve Students with Age = 20
 - cost: retrieve index entries and corresponding Students tuples
 - apply condition *RoundedGPA* > 5 to each retrieved tuple
 - retain RoundedGPA and SName

Enumeration of Alternative Plans

* plans that use an index:

- single-index access path – example:
 - write out tuples to a temporary relation
 - sort the temporary relation by RoundedGPA to identify groups
 - apply the HAVING condition (to eliminate some groups)
- multiple-index access path
 - several indexes using a2 / a3 match the selection condition, e.g., I1, I2
 - retrieve $Rids_{I1}$, $Rids_{I2}$ using I1, I2
 - get tuples with rids in $Rids_{I1} \cap Rids_{I2}$ (tuples satisfying the primary selection terms of I1 and I2)
 - apply π , nonprimary selection terms
 - compute grouping and aggregation operations
 - example:
 - use the index on Age $\Rightarrow$ rids of tuples with Age = 20 (R1)

Enumeration of Alternative Plans

* plans that use an index:

- multiple-index access path – example:
 - index on RoundedGPA $\Rightarrow$ rids of tuples with RoundedGPA > 5 (R2)
 - retrieve tuples with rids in $R1 \cap R2$
 - keep only RoundedGPA and SName
 - write out tuples to a temporary relation
 - sort the temporary relation by RoundedGPA to identify groups
 - apply the HAVING condition (to eliminate some groups)
- sorted index access path:
 - works well when the index is clustered
 - B+ tree index I with search key K
 - GROUP BY attributes – prefix of K
 - use the index to retrieve tuples in the order required by the GROUP BY clause

Enumeration of Alternative Plans

* plans that use an index:

- apply σ , π
 - compute aggregation operations
 - sorted index access path – example:
 - use the B+ tree index on RoundedGPA to retrieve Students tuples with $\text{RoundedGPA} > 5$, ordered by RoundedGPA
 - aggregations in HAVING, SELECT - computed on-the-fly
 - index-only access path:
 - index I with search key K
 - all the attributes in the query are included in K
- => index-only scan, don't need to retrieve tuples from the relation
- data entries: apply σ , perform π , sort the result (to identify groups), compute aggregate operations
- * obs. index I doesn't have to match the selections in the WHERE clause

Enumeration of Alternative Plans

- * plans that use an index:

- index-only access path

- * obs. 1 – tree index, GROUP BY attributes – prefix of K

- => can avoid sorting

- example:

- use the B+ tree index on <RoundedGPA, SName, Age> to retrieve entries with RoundedGPA > 5, ordered by RoundedGPA
 - select entries with Age = 20
 - aggregation operations in the HAVING and SELECT clauses - computed on-the-fly

Enumeration of Alternative Plans

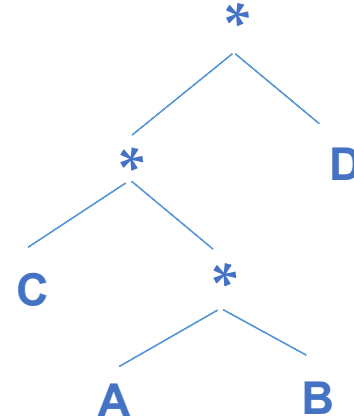
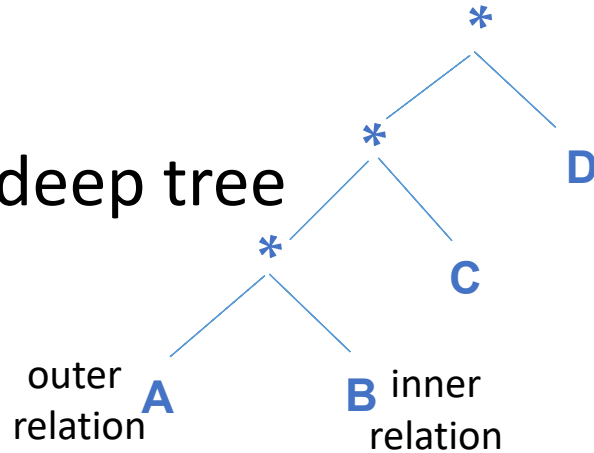
- queries with several relations in the FROM clause:
 - joins, cross-products => queries can be quite expensive
 - different join orders => intermediate relations of widely varying sizes => plans with very different costs
- class of plans considered by the optimizer
- plan enumeration

Enumeration of Alternative Plans

- queries with several relations in the FROM clause

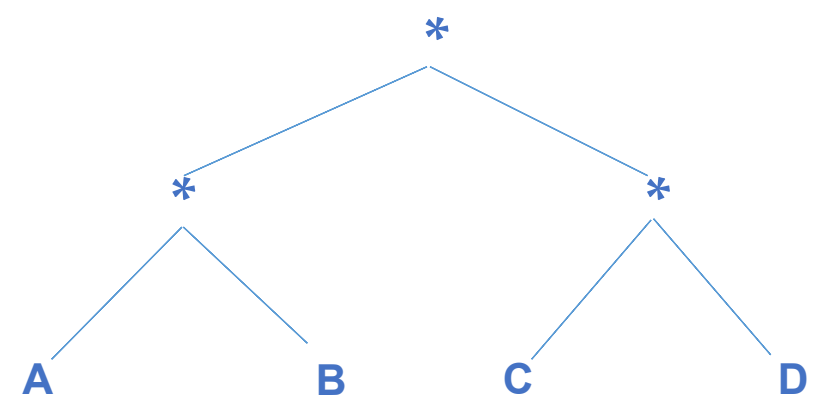
$A * B * C * D$

left-deep tree



linear trees

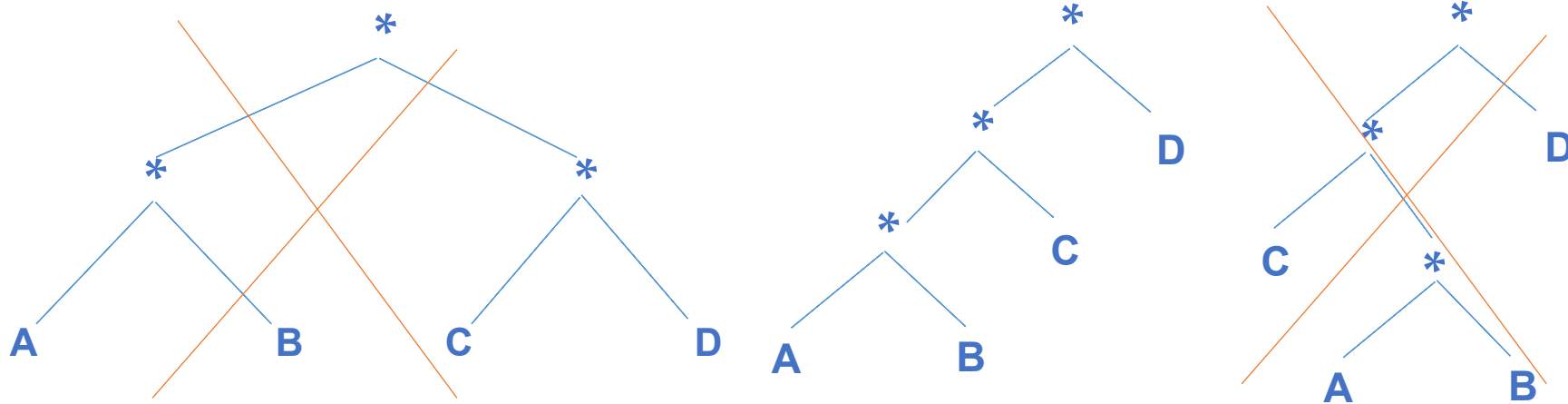
bushy tree



- linear trees:
 - at least one child of a join node is a base relation
- left-deep trees:
 - the right child of each join node is a base relation
- bushy tree: not linear

Enumeration of Alternative Plans

- queries with several relations in the FROM clause
- fundamental decision in System R:
 - only *left-deep trees* are considered



- motivation:
 - number of joins increases => number of alternative plans increases quickly => must prune the search space
 - left-deep trees generate all fully pipelined plans (all joins are evaluated using pipelining)

Enumeration of Alternative Plans

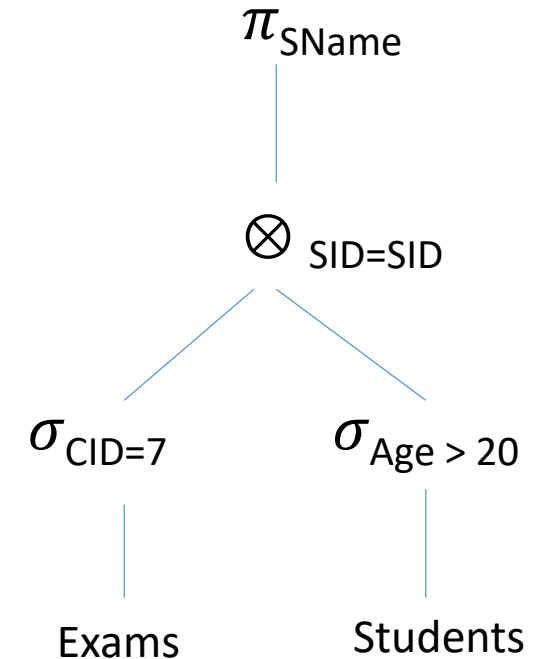
- queries with several relations in the FROM clause
- multiple passes:
 - pass 1:
 - find best 1-relation plan for each relation
 - pass 2:
 - find best way to join the result of each 1-relation plan (as the outer argument) with another relation (all 2-relation plans)
 - ...
 - repeat until the obtained plans contain all the relations in the query
- for each subset of relations, retain: the cheapest plan overall & the cheapest plan for each interesting ordering of tuples

Enumeration of Alternative Plans

- queries with several relations in the FROM clause
 - GROUP BY, aggregates are handled as a final step:
 - use a plan with an interesting ordering of tuples
 - use an additional sorting operator
- obs. avoid cross-products if possible

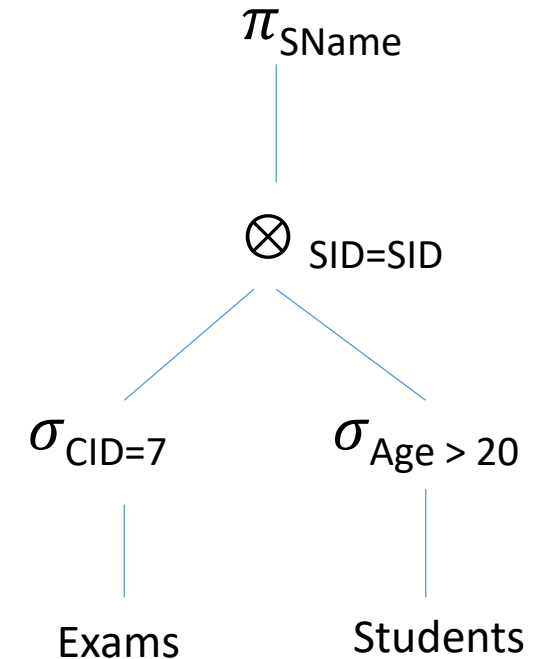
Example

- unclustered indexes using a2
- Students: B+ tree index on Age, hash index on SID
- Exams: B+ tree index on CID
- pass 1:
 - Students – 3 possible access paths:
 - B+ tree index, hash index, file scan
 - *Age > 20* matches the B+ tree index on Age
 - hash index / file scan => probably much higher cost
 - keep the plan using the B+ tree index => retrieved tuples are ordered by Age
 - Exams – 2 possible access paths
 - *CID = 7* matches the B+ tree index on CID
 - better than file scan => keep the plan using the B+ tree index



Example

- Students: B+ tree index on Age, hash index on SID
- Exams: B+ tree index on CID
- pass 2:
 - consider (the result of) each plan from pass 1 as the outer argument and analyze how to join it with the other relation
 - e.g., Exams – outer argument
 - examine alternative access methods / join methods
 - access methods:
 - need Students tuples s.t. $SID = value$ from outer tuple and $Age > 20$
 - hash index $\Rightarrow$ Students tuples s.t. $SID = value$ from outer tuple
 - B+ tree index $\Rightarrow$ Students tuples s.t. $Age > 20$
 - join methods:
 - consider all available methods



Example

- Students: B+ tree index on Age, hash index on SID
- Exams: B+ tree index on CID
- pass 2:
 - consider (the result of) each plan from pass 1 as the outer argument and analyze how to join it with the other relation
 - e.g., Students – outer argument
 - access / join methods
 - access methods:
 - need Exams tuples s.t. $SID = value$ from outer tuple and $CID = 7$
 - join methods:
 - consider all available methods
 - retain cheapest plan overall!

Nested Queries - optional

- usually handled using some form of nested loops evaluation
- correlated query - typical evaluation strategy:
 - the inner block is evaluated for each tuple of the outer block
- some strategies are not considered
 - e.g., index on SID in Students
 - best plan could be INLJ with Exams as the outer argument, and Students as the inner one; such a plan is never considered by the optimizer
- the unnested version of the query is typically optimized better

```
SELECT S.SName  
FROM Students S  
WHERE EXISTS  
  (SELECT *  
   FROM Exams E  
   WHERE E.CID=7  
        AND E.SID=S.SID)
```

Equivalent unnested query:
SELECT S.SName
FROM Students S, Exams E
WHERE E.SID=S.SID AND E.CID = 7

References

- [Ra02] RAMAKRISHNAN, R., GEHRKE, J., Database Management Systems (3rd Edition), McGraw-Hill, 2002
- [Da03] DATE, C.J., An Introduction to Database Systems (8th Edition), Addison-Wesley, 2003
- [Ga09] GARCIA-MOLINA, H., ULLMAN, J., WIDOM, J., Database Systems: The Complete Book (2nd Edition), Pearson Education, 2009
- [Ra02S] RAMAKRISHNAN, R., GEHRKE, J., Database Management Systems, Slides for the 3rd Edition,
<http://pages.cs.wisc.edu/~dbbook/openAccess/thirdEdition/slides/slides3ed.html>
- [Si19] SILBERSCHATZ, A., KORTH, H., SUDARSHAN, S., Database System Concepts (7th Edition), McGraw-Hill, 2019
- [Si19S] SILBERSCHATZ, A., KORTH, H., SUDARSHAN, S., Database System Concepts, Slides for the 7th Edition, <http://codex.cs.yale.edu/avi/db-book/>
- [Ul11] ULLMAN, J., WIDOM, J., A First Course in Database Systems,
<http://infolab.stanford.edu/~ullman/fcdb.html>

Database Management Systems

Lecture 10

Distributed Databases

* centralized DB systems

- all data at a single site
- each transaction is processed sequentially
- centralized lock management
- processor fails => entire system fails

* distributed systems

- the data is stored at several sites
- each site is managed by a DBMS that can run independently; these autonomous components can also be heterogeneous

=> impact on: query processing, query optimization, concurrency control, recovery

Distributed Database Systems

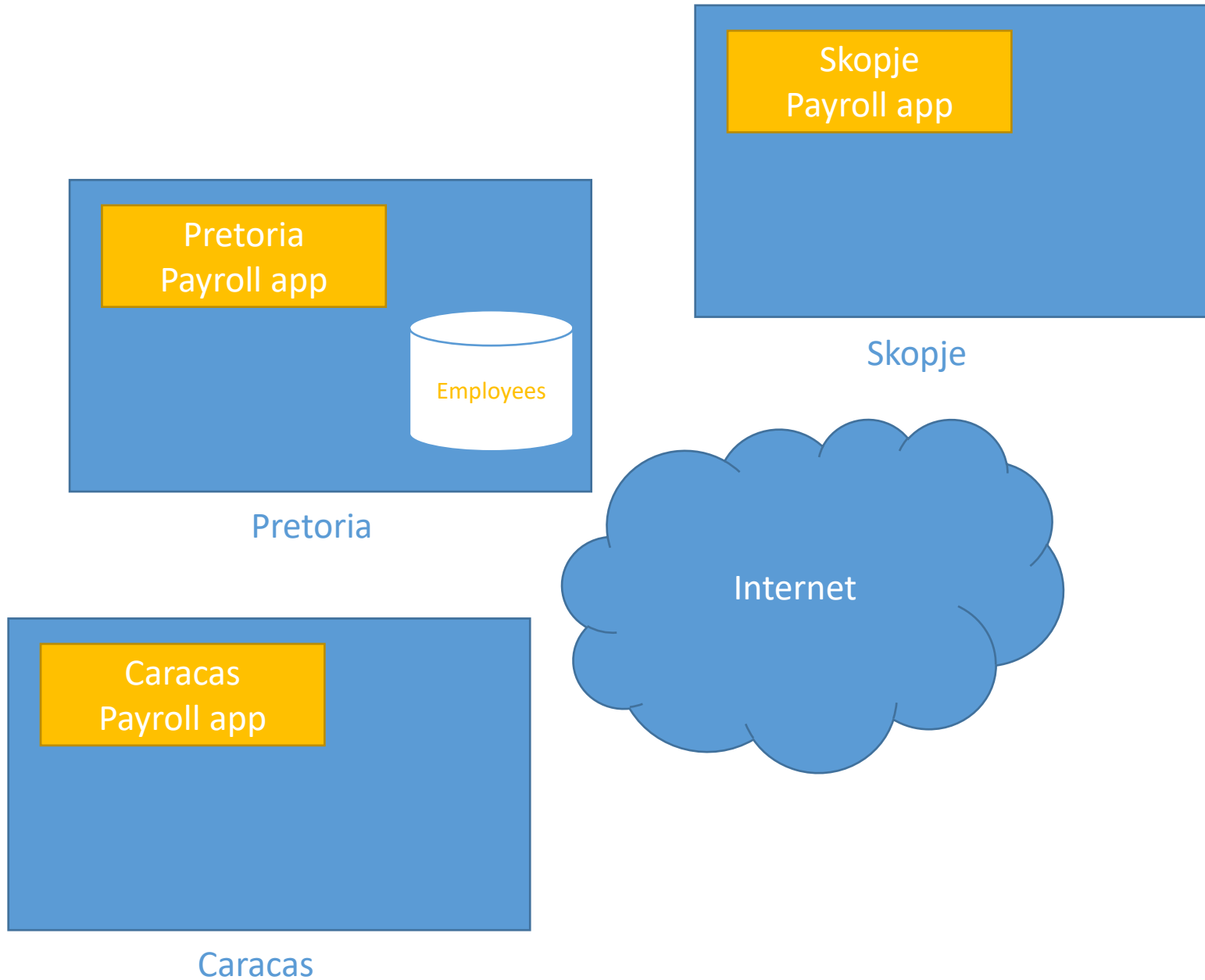
* properties:

- distributed data independence (extension of the physical and logical data independence principles):
 - users can write queries without knowing / specifying the actual location of the data
 - cost-based query optimization that takes into account communication costs & differences in computation costs across sites
- distributed transaction atomicity
 - users can write transactions accessing multiple sites just as they would write local transactions
 - transactions are still atomic (if the transaction commits, all its changes persist; if it aborts, none of its changes persist)

Distributed Databases - Motivating Example

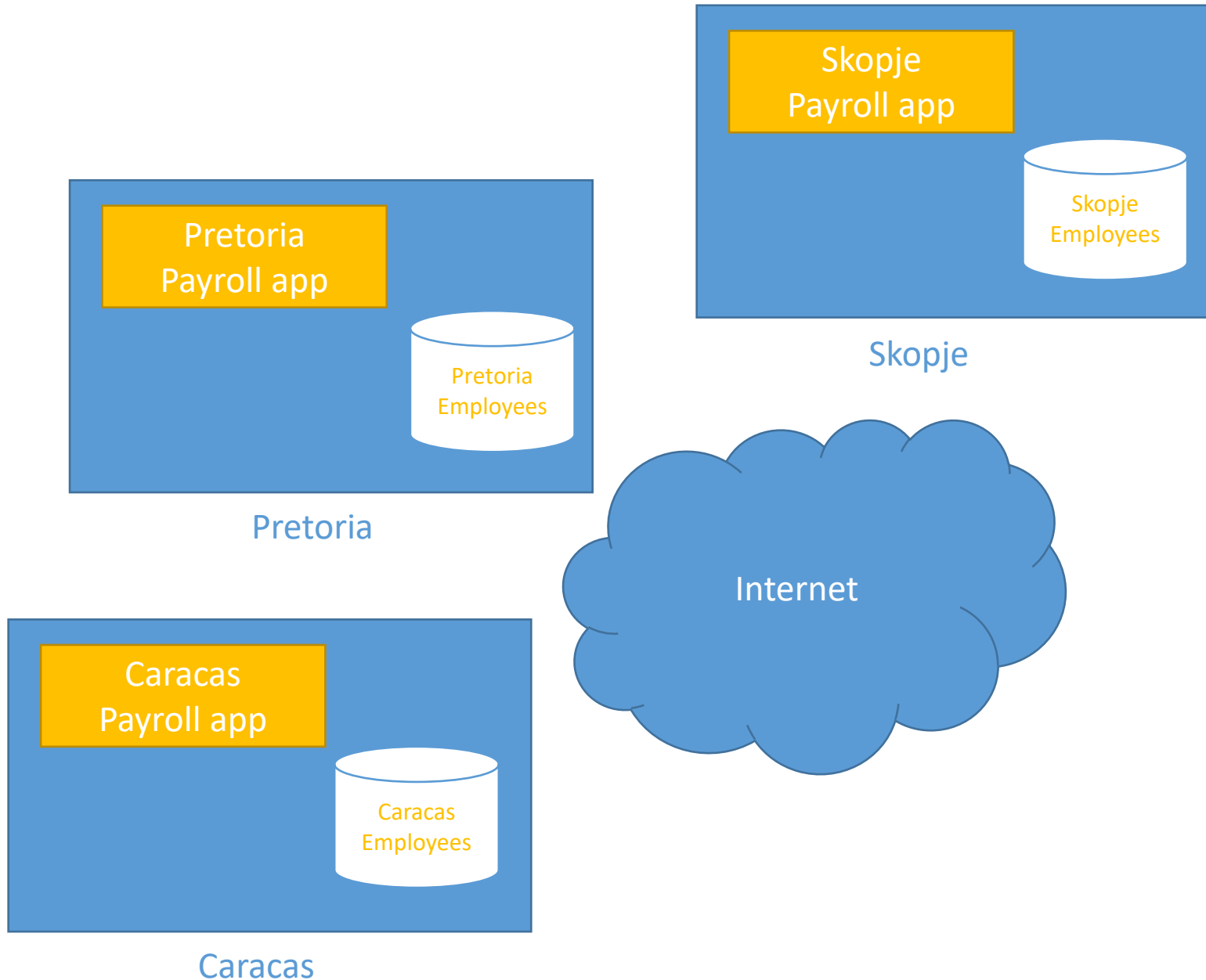
- company with offices in Pretoria, Skopje, Caracas
- in general, an employee's data is managed from the office where the employee works (payroll, benefits, hiring data, etc.)
 - for instance, data about Skopje employees is managed from the Skopje office
- periodically, the company needs access to all employees' data, e.g.:
 - compute the total payroll expenses
 - compute the annual bonus
- where should we store employee data?

Distributed Databases - Motivating Example



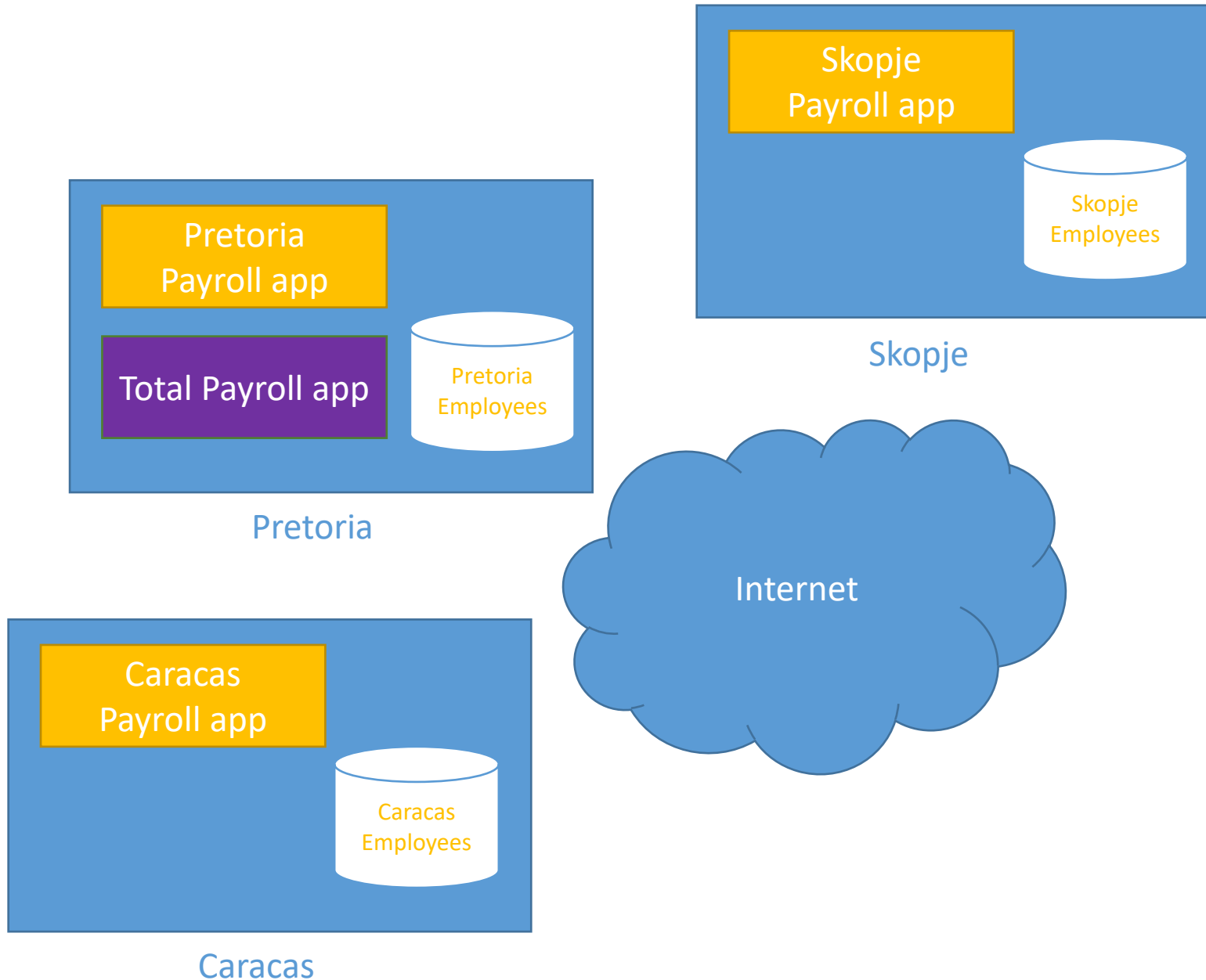
- *SiteX* payroll app – computing payrolls for *SiteX* employees
 - suppose the DB is stored at the company's headquarters in Pretoria
- => slow-running payroll apps in Caracas and Skopje
- moreover, if the Pretoria site becomes unavailable, payroll apps in Caracas and Skopje cannot access the required data

Distributed Databases - Motivating Example



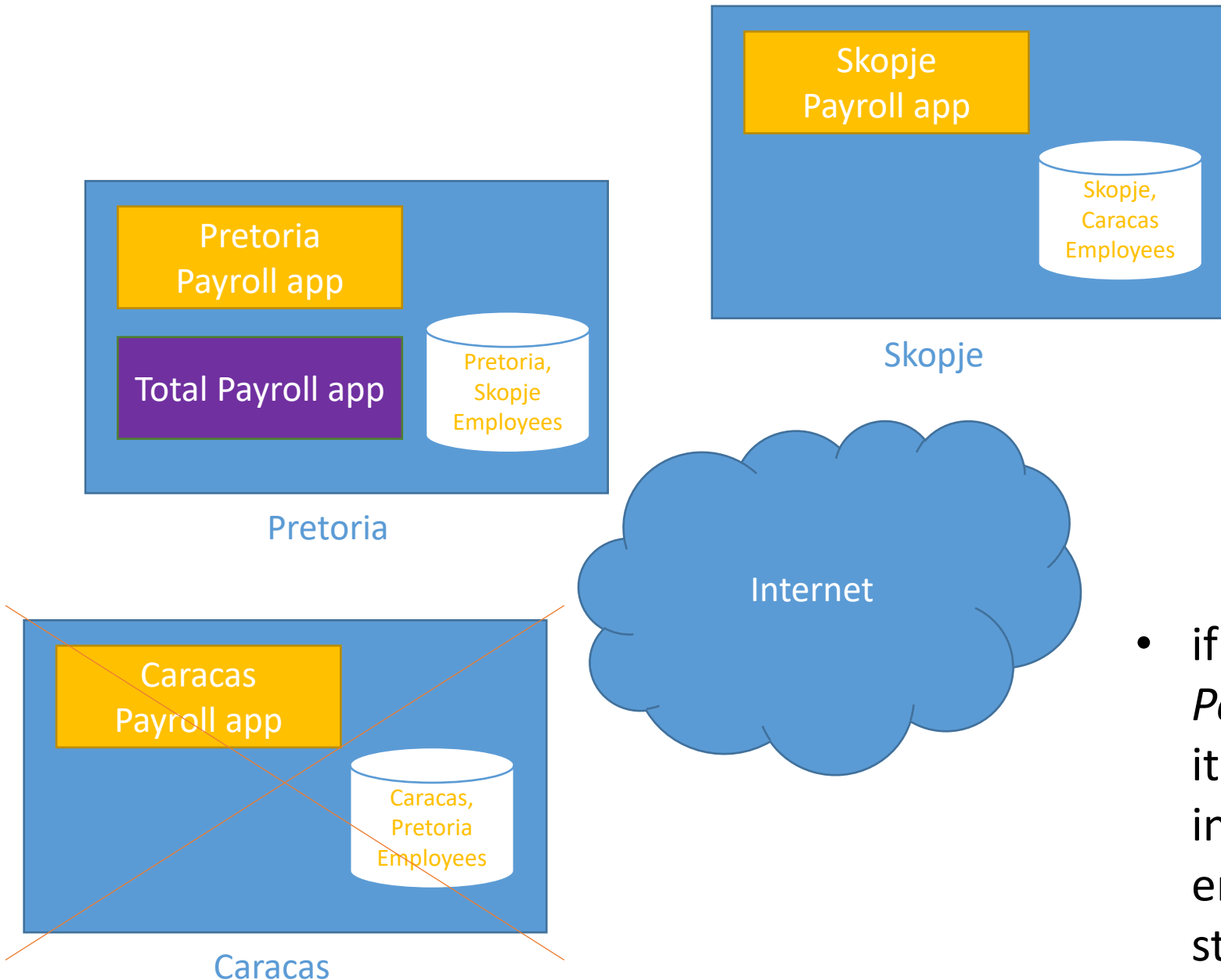
- store data about Pretoria employees in Pretoria, about Skopje employees in Skopje, etc.
- => improved performance for payroll apps in Caracas and Skopje
- if the Pretoria site becomes unavailable, payroll apps in Caracas and Skopje are still operational (as these apps only need data about Caracas employees and about Skopje employees, respectively)

Distributed Databases - Motivating Example



- *Total Payroll app* (at the company's headquarters in Pretoria) computing the total payroll expenses
- this app needs to access employee data at all 3 sites, so generating the final results will last a little longer
- if the Caracas site crashes, the *Total Payroll app* cannot access all required data
- opportunities for parallel execution

Distributed Databases - Motivating Example



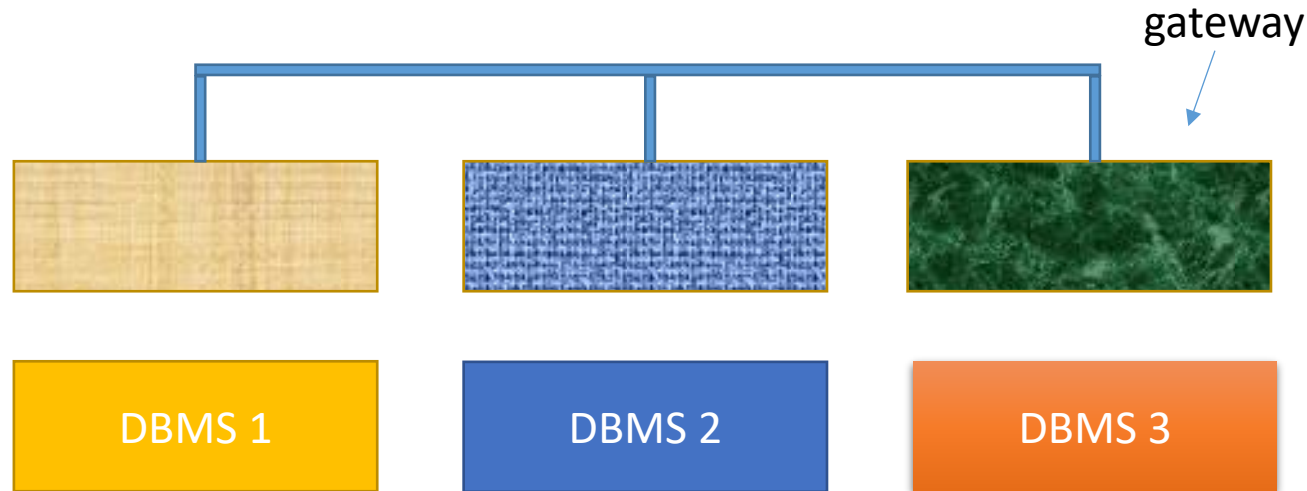
- data replication: at *Site X*, store data about *Site X* employees and data about employees from a different site, e.g., store data about Pretoria employees and Skopje employees in Pretoria (a copy of the Skopje employees data, which is stored in Skopje)
- if the Caracas site crashes, the *Total Payroll app* can continue to work, as it can access all required data, including data about the Caracas employees (a copy of this data being store in Skopje)

Types of Distributed Databases

- homogeneous:
 - the same DBMS software at every site
- heterogeneous (multidatabase system):
 - different DBMSs at different sites

* *gateway protocols*:

- mask differences between different database servers



Distributed Databases - Challenges

* distributed database design:

- deciding where to store the data
- depends on the data access patterns of the most important applications (how are they accessing the data)
- two sub-problems: *fragmentation* and *allocation*

* distributed query processing:

- centralized query plan
 - objective: minimize the number of disk I/Os; execute the query as fast as possible
- distributed context – there are additional factors to consider:
 - communication costs
 - opportunity for parallelism

=> the space of possible query plans for a given query is much larger!

Distributed Databases - Challenges

* distributed concurrency control:

- serializability
- distributed deadlock management (e.g., deadlock involving transactions T1 and T2, with T1 executing at site S1, and T2 executing at site S2)

* reliability of distributed databases:

- transaction failures
 - one or more processors may fail
 - the network may fail
- data must be synchronized

Storing Data in a Distributed DBMS

- accessing relations at remote sites => communication costs
 - * example:
 - Pretoria: the site stores the *Employees* relation, holding data about all employees in the company
 - Skopje: a local manager wants to obtain the average salary for Skopje employees; she must access the relation stored in Pretoria
- reduce such costs: *fragmentation / replication*
 - a relation can be partitioned into *fragments*, which are stored across several sites (a fragment is kept where it's most often accessed)
 - * example:
 - partition the *Employees* relation into fragments *PretoriaEmployees*, *SkopjeEmployees*, etc.
 - store fragment *PretoriaEmployees* in Pretoria, fragment *SkopjeEmployees* in Skopje, etc.

Storing Data in a Distributed DBMS

- accessing relations at remote sites => communication costs
- reduce such costs: *fragmentation / replication*
 - a relation can be *replicated* at each site where it's needed the most
- * example:
 - suppose the *Employees* relation is frequently needed in Beijing, New York, and Bucharest
 - *Employees* can be replicated at the Bucharest site, the New York one, and in Beijing

Storing Data in a Distributed DBMS

* *fragmentation*: break a relation into smaller relations (fragments); store the fragments instead of the relation itself

- *horizontal / vertical / hybrid*

* example – relation Accounts(accnum, name, balance, branch):

- horizontal fragmentation

- fragment: subset of rows
- n selection predicates => n fragments (n record sets)
- horizontal fragments should be disjoint
- reconstruct the original relation: take the union of the horizontal fragments

- $\sigma_{\text{branch}='Eroilor'}(\text{Accounts}),$
 $\sigma_{\text{branch}='Napoca'}(\text{Accounts}),$
 $\sigma_{\text{branch}='Motilor'}(\text{Accounts}) \Rightarrow$

R
1, Radu, 250, Eroilor
2, Ana, 200, Napoca
3, Ionel, 150, Motilor
4, Maria, 400, Eroilor
5, Andi, 600, Napoca
6, Calin, 250, Eroilor
7, Iulia, 350, Motilor

R1	1, Radu, 250, Eroilor 4, Maria, 400, Eroilor 6, Calin, 250, Eroilor
R2	2, Ana, 200, Napoca 5, Andi, 600, Napoca
R3	3, Ionel, 150, Motilor 7, Iulia, 350, Motilor

Storing Data in a Distributed DBMS

- vertical fragmentation
 - fragment: subset of columns
 - performed using projection operators
 - must obtain a good decomposition*
 - reconstruction operator: natural join

- $\pi_{\{\text{accnum}, \text{name}\}}(\text{Accounts})$
 $\pi_{\{\text{accnum}, \text{balance}, \text{branch}\}}(\text{Accounts})$

R1	R2
1, Radu	1, 250, Eroilor
2, Ana	2, 200, Napoca
3, Ionel	3, 150, Motilor
4, Maria	4, 400, Eroilor
5, Andi	5, 600, Napoca
6, Calin	6, 250, Eroilor
7, Iulia	7, 350, Motilor

- hybrid fragmentation
 - horizontal fragmentation + vertical fragmentation

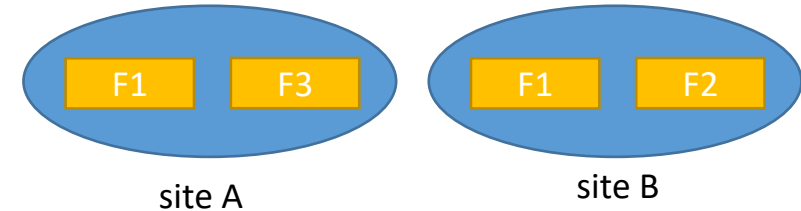
R1	R2
1, Radu	1, 250, Eroilor
2, Ana	2, 200, Napoca
3, Ionel	3, 150, Motilor
6, Calin	6, 250, Eroilor
R3	R4
4, Maria	4, 400, Eroilor
5, Andi	5, 600, Napoca
7, Iulia	7, 350, Motilor

* see *Databases – Normal Forms*

Storing Data in a Distributed DBMS

* *replication*: store multiple copies of a relation or of a relation fragment; an entire relation or one or several fragments of a relation can be replicated at one or several sites

* example: relation R is partitioned into fragments F1, F2, F3; fragment F1 is stored at site A and at site B:



- motivation:
 - increased availability of data: query Q uses fragment F1 from site A; if site A goes down or a communication link fails, the query can use another active server (e.g., site B)
 - faster query evaluation: can use a local copy of the data to avoid communication costs, e.g., query Q at site A can use the local copy of F1, it doesn't need to access the copy of F1 from site B
- types of replication: synchronous versus asynchronous
 - how are the copies of the data kept current when the relation is changed

Updating Distributed Data

- synchronous replication:

- transaction T modifies relation R
- before T commits, it synchronizes R's copies

=> data distribution is transparent to the user

- asynchronous replication:

- transaction T modifies relation R
- R's copies are synchronized periodically, i.e., it's possible that some of R's copies are outdated for brief periods of time
- a transaction T2 reading 2 different copies of R may see different data; but in the end, all copies of R will be synchronized

=> users must be aware of the fact that the data is distributed, i.e., distributed data independence is compromised

- a lot of current systems are using this approach

Updating Distributed Data – Synchronous Replication

- 2 basic techniques: *voting* and *read-any write-all*

* *voting*

- to modify object O, a transaction T1 must write a majority of its copies
- when reading O, a transaction T2 must read enough copies to make sure it's seeing at least one current copy
- e.g., O has 10 copies; T1 changes O: suppose T1 writes 7 copies of O; T2 reads O: it should read at least 4 copies to make sure one of them is current
- each copy has a version number (the copy that is current has the highest version number)
- not an attractive approach in most cases, because reads are usually much more common than writes (and reads are expensive in this approach)

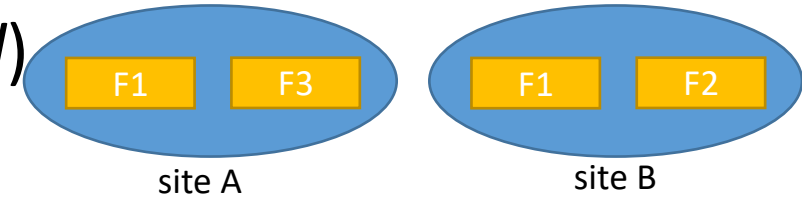
Updating Distributed Data – Synchronous Replication

* *read-any write-all*

- transaction T1 modifies O: T1 must write all copies of O
- transaction T2 reads O: T2 can read any copy of O (no matter which copy T2 reads, it will see current data, as T1 wrote all copies of O)
- fast reads (only one copy is read), slower writes (compared with the voting technique)
- most common approach to synchronous replication

Updating Distributed Data – Synchronous Replication Costs

- before an update transaction T can commit, it must lock all copies of the modified relation / fragment (with *read-any write-all*)
 - suppose relation R is partitioned into fragments F1, F2, F3, stored at sites A and B (with F1 being stored at both sites)
 - if transaction T changes fragment F1 at site A, it must also lock the copy of F1 stored at site B!
 - such a transaction sends lock requests to remote sites; while waiting for the response, the transaction holds on to its other locks
- if there's a site or link failure, transaction T cannot commit until the network is / involved sites are back up (if site B becomes unavailable in the example above, transaction T cannot commit until site B comes back up)
- even if there are no failures and locks are immediately obtained, T must follow an expensive commit protocol when committing (with several messages being exchanged) => asynchronous replication is more used



Updating Distributed Data – Asynchronous Replication

- two approaches:
 - *primary site replication*
 - *peer-to-peer replication*
- * difference: number of *updatable copies (master copies)*

Updating Distributed Data – Asynchronous Replication

* *peer-to-peer* replication

- several copies of an object *O* can be *master copies* (i.e., updatable)
- changes to a master copy are propagated to the other copies of object *O*
- need a conflict resolution strategy for cases when two master copies are changed in a conflicting manner:
 - e.g., master copies at site 1 and site 2; at site 1: Dana's city is changed to Cluj-Napoca; at site 2: Dana's city is changed to Timișoara; which value is correct?
 - in general - ad hoc approaches to conflict resolution

Updating Distributed Data – Asynchronous Replication

* *peer-to-peer* replication

- best utilized when conflicts do not arise:
 - each master site owns a fragment (usually a horizontal fragment) and any 2 fragments that can be updated by different master sites are disjoint
 - * example: store relation *Employees* at Skopje and Caracas
 - data about Skopje employees can only be updated in Skopje
 - data about Caracas employees can only be updated in Caracas
- => no conflicts
- updating rights are held by only one master site at a time

Updating Distributed Data – Asynchronous Replication

* *primary site* replication

- exactly one copy of an object *O* is chosen as the *primary (master)* copy; this copy is published at the *primary* site
- secondary copies of the object (copies of the relation or copies of relation fragments) can be created at other sites (*secondary* sites)
- can subscribe to the primary copy or to fragments of the primary copy
- changes to the primary copy are propagated to the secondary copies in 2 steps:
 - *capture* the changes made by committed transactions
 - *apply* these changes to secondary copies

Updating Distributed Data – Asynchronous Replication

* *primary site* replication

- capture: *log-based capture / procedural capture*
 - log-based capture:
 - the log (kept for recovery purposes) is used to generate the *Change Data Table* (CDT) structure: write log tail to stable storage -> write all log records affecting replicated relations to the CDT
 - changes of aborted transactions must be removed from the CDT
 - in the end, CDT contains only update log records of committed transactions
 - suppose committed transaction T has executed 3 UPDATE operations on (the replicated) relation *Employees*; then the CDT will include 3 update log records, describing the UPDATE operations

Updating Distributed Data – Asynchronous Replication

* *primary site* replication

- capture: *log-based capture / procedural capture*
 - procedural capture
 - capture is performed through a procedure that is automatically invoked (a trigger)
 - the procedure takes a snapshot of the primary copy
 - *snapshot*: a copy of the relation
- log-based capture:
 - smaller overhead and smaller delay, but it depends on proprietary log details

Updating Distributed Data – Asynchronous Replication

* *primary site* replication

- apply
 - applies changes collected in the Capture step (from the Change Data Table or snapshot) to the secondary copies:
 - the primary site can continuously send the CDT
 - or the secondary sites can periodically request a snapshot or (the latest portion of) the CDT from the primary site
 - each secondary site runs a copy of the Apply process

Updating Distributed Data – Asynchronous Replication

* *primary site* replication

- log-based capture + continuous apply
 - minimizes delay in propagating changes
- procedural capture + application-driven apply
 - most flexible way to process changes
- the replica could be a view over the modified relation - optional
 - the view is incrementally updated as the relation changes

Distributed Query Processing

Researchers(RID: integer, Name: string, ImpactF: integer, Age: real)

AuthorContribution(RID: integer, PID: integer, Year: integer, Descr: string)

- Researchers
 - 1 tuple - 50 bytes
 - 1 page - 80 tuples
 - 500 pages
- AuthorContribution
 - 1 tuple - 40 bytes
 - 1 page - 100 tuples
 - 1000 pages
- estimate the cost of evaluation strategies:
 - number of I/O operations and number of pages shipped among sites, i.e., take into account communication costs
 - use t_d to denote the time to read / write a page from / to disk
 - use t_s to denote the time to ship a page from one site to another (e.g., from Skopje to Caracas)

Distributed Query Processing

* nonjoin queries in a distributed DBMS

- impact of fragmentation / replication on simple operations
- scanning a relation, selection, projection
- horizontal fragmentation
 - all Researchers tuples with ImpactF < 6 - stored at New York
 - all Researchers tuples with ImpactF >= 6 - stored at Lisbon

Q1 .

```
SELECT R.Age
```

```
FROM Researchers R
```

```
WHERE R.ImpactF > 4 AND R.ImpactF < 10
```

- the DBMS evaluates the query at New York and Lisbon, then takes the union of the obtained results to produce the result set for query Q1

Distributed Query Processing

* nonjoin queries in a distributed DBMS

- horizontal fragmentation

Q2.

```
SELECT AVG (R.Age)
```

```
FROM Researchers R
```

```
WHERE R.ImpactF > 4 AND R.ImpactF < 10
```

- the DBMS computes SUM(Age) and number of Age values at New York and Lisbon; then based on this information, it computes the average age of all researchers with ImpactF in the specified range

Q3.

```
SELECT ...
```

```
FROM Researchers R
```

```
WHERE R.ImpactF > 7
```

- in this case, the DBMS evaluates the query only at Lisbon

Distributed Query Processing

* nonjoin queries in a distributed DBMS

- vertical fragmentation

- RID, ImpactF - stored at New York (for all researchers)
- Name, Age - stored at Lisbon (for all researchers)
- the DBMS adds a field that contains the id of the corresponding tuple from Researchers to both fragments and rebuilds the Researchers relation by joining the 2 fragments on the common field (the tuple-id field)
- the DBMS then evaluates the query over the reconstructed relation

Distributed Query Processing

* nonjoin queries in a distributed DBMS

- replication

- Researchers relation stored at both New York and Lisbon
- then Q1, Q2, Q3 can be executed at either New York or Lisbon
- choosing the execution site - factors to consider:
 - the cost of shipping the result to the query site (e.g., the query site could be New York, Lisbon, or a 3rd, distinct site)
 - local processing costs:
 - can vary from one site to another – check the available indexes on Researchers at New York and Lisbon

Distributed Query Processing

* join queries in a distributed DBMS

- can be quite expensive if the relations are stored at different sites
- Researchers stored at New York
- AuthorContribution stored at Lisbon
- evaluate *Researchers join AuthorContribution*

->

Distributed Query Processing

* join queries in a distributed DBMS

- Researchers R - New York, AuthorContribution A - Lisbon, R join A

* fetch as needed

- page-oriented nested loops in New York
 - Researchers - outer relation
 - for each page in Researchers, bring in all the AuthorContribution pages from Lisbon
 - cost
 - scan Researchers: $500t_d$
 - scan AuthorContribution + ship all AuthorContribution pages (for each Researchers page): $1000(t_d + t_s)$
- => total cost: $500t_d + 500,000(t_d + t_s)$

Distributed Query Processing

* join queries in a distributed DBMS

- Researchers R - New York, AuthorContribution A - Lisbon, R join A

* fetch as needed

- page-oriented nested loops in New York
 - obs. bring in all AuthorContribution pages for each Researchers tuple => much higher cost
 - optimization
 - bring in AuthorContribution pages only once from Lisbon to New York
 - cache AuthorContribution pages at New York until the join is complete

Distributed Query Processing

* join queries in a distributed DBMS

- Researchers R - New York, AuthorContribution A - Lisbon, R join A

* fetch as needed

- page-oriented nested loops in New York
 - query not submitted at New York
 - => add the cost of shipping the result to the query site
 - RID - key in Researchers
 - => the result has 100,000 tuples (the number of tuples in AuthorContribution)
 - the size of a tuple in the result
 - $40 + 50 = 90$ bytes
 - the number of result tuples / page
 - $4000 / 90 = 44$

Distributed Query Processing

* join queries in a distributed DBMS

- Researchers R - New York, AuthorContribution A - Lisbon, R join A

* fetch as needed

- page-oriented nested loops in New York
 - query not submitted at New York
 - number of pages necessary to hold all the result tuples
 - $100,000/44 = 2273$ pages
 - the cost of shipping the result to another site (if necessary)
 - $2273 t_s$
 - higher than the cost of shipping both Researchers and AuthorContribution to the site ($1500 t_s$)

Distributed Query Processing

* join queries in a distributed DBMS

- Researchers R - New York, AuthorContribution A - Lisbon, R join A

* fetch as needed

- index nested loops join in New York
 - AuthorContribution - unclustered hash index on RID
 - 100,000 AuthorContribution tuples, 40,000 Researchers tuples
 - on average, a researcher has 2.5 corresponding tuples in AuthorContribution
 - for each Researchers tuple, retrieve the 2.5 corresponding tuples in AuthorContribution:
 - obtain the index page: $1.2 t_d$ (on average)
 - +
 - read the matching records in AuthorContribution: $2.5 t_d$

Distributed Query Processing

* join queries in a distributed DBMS

- Researchers R - New York, AuthorContribution A - Lisbon, R join A

* fetch as needed

- index nested loops join in New York
 - for each Researchers tuple, retrieve the 2.5 corresponding tuples in AuthorContribution:
 - => cost per Researchers tuple: $(1.2 + 2.5)t_d$
 - the pages containing these 2.5 tuples must also be shipped from Lisbon to New York
- => total cost: $500t_d + 40,000(3.7t_d + 2.5t_s)$ (there are 40,000 records in Researchers)

Distributed Query Processing

* join queries in a distributed DBMS

- Researchers R - New York, AuthorContribution A - Lisbon, R join A

* ship to one site

- ship Researchers to Lisbon, compute the join at Lisbon
 - scan Researchers, ship it to Lisbon, save Researchers at Lisbon:
 - cost: $500(2t_d + t_s)$
 - compute *Researchers join AuthorContribution* at Lisbon
 - example: use improved version of Sort-Merge Join with
cost = $3(\text{number of R pages} + \text{number of A pages})$
SMJ cost: $3(500 + 1000) = 4500t_d$
- => total cost: $500(2t_d + t_s) + 4500t_d$

Distributed Query Processing

* join queries in a distributed DBMS

- Researchers R - New York, AuthorContribution A - Lisbon, R join A

* ship to one site

- ship Researchers to Lisbon, compute the join at Lisbon
 - total cost: $500(2t_d + t_s) + 4500t_d$
- ship AuthorContribution to New York, compute the join at New York
 - total cost: $1000(2t_d + t_s) + 4500t_d$

Distributed Query Processing

* join queries in a distributed DBMS

- Researchers R - New York, AuthorContribution A - Lisbon, R join A

* semijoin

- at New York:
 - project Researchers onto the join columns (RID)
 - ship the projection to Lisbon
- at Lisbon:
 - join the Researchers projection with AuthorContribution
=> the so-called *reduction of AuthorContribution with respect to Researchers*
 - ship the reduction of AuthorContribution to New York
- at New York:
 - join Researchers with the reduction of AuthorContribution

Distributed Query Processing

* join queries in a distributed DBMS

- Researchers R - New York, AuthorContribution A - Lisbon, R join A

* semijoin

- tradeoff:
 - the cost of computing and shipping the projection
+
 - the cost of computing and shipping the reductionversus
 - the cost of shipping the entire AuthorContribution relation
- very useful if there is a selection on one of the relations

Distributed Query Processing

* join queries in a distributed DBMS

- Researchers R - New York, AuthorContribution A - Lisbon, R join A

* bloomjoin

- at New York:
 - compute a bit-vector of some size k
 - hash Researchers tuples (using the join column RID) into the range 0 to $k-1$
 - if some tuple hashes to i , set bit i to 1 (i from 0 to $k-1$)
 - otherwise (no tuple hashes to i), set bit i to 0
 - ship the bit-vector to Lisbon
 - at Lisbon:
 - hash each AuthorContribution tuple (using the join column RID) into the range 0 to $k-1$, with the same hash function

Distributed Query Processing

* join queries in a distributed DBMS

- Researchers R - New York, AuthorContribution A - Lisbon, R join A

* bloomjoin

- at Lisbon:
 - discard tuples with a hash value i that corresponds to a 0 bit in the Researchers bit-vector
- => reduction of AuthorContribution with respect to Researchers*
- ship the reduction to New York
- at New York:
 - join Researchers with the reduction

References

- [Ra02] RAMAKRISHNAN, R., GEHRKE, J., Database Management Systems (3rd Edition), McGraw-Hill, 2002
- [Da03] DATE, C.J., An Introduction to Database Systems (8th Edition), Addison-Wesley, 2003
- [Ga09] GARCIA-MOLINA, H., ULLMAN, J., WIDOM, J., Database Systems: The Complete Book (2nd Edition), Pearson Education, 2009
- [Ra02S] RAMAKRISHNAN, R., GEHRKE, J., Database Management Systems, Slides for the 3rd Edition,
<http://pages.cs.wisc.edu/~dbbook/openAccess/thirdEdition/slides/slides3ed.html>
- [Si19] SILBERSCHATZ, A., KORTH, H., SUDARSHAN, S., Database System Concepts (7th Edition), McGraw-Hill, 2019
- [Si19S] SILBERSCHATZ, A., KORTH, H., SUDARSHAN, S., Database System Concepts, Slides for the 7th Edition, <http://codex.cs.yale.edu/avi/db-book/>
- [Ul11] ULLMAN, J., WIDOM, J., A First Course in Database Systems,
<http://infolab.stanford.edu/~ullman/fcdb.html>

Database Management Systems

Lecture 11

Problems

I

Let R and S be 2 relations. R has 10,000 records; a page can hold 10 R records. S has 2,000 records; a page can hold 10 S records.

1. 52 buffer pages are available. Compute the cost of:

```
SELECT *  
FROM R INNER JOIN S ON R.a = S.b
```

using *page-oriented nested loops join* and *block nested loops join*; S is the outer relation.

R – 10,000 records; a page can hold 10 R records => 1,000 pages

S – 2,000 records; a page can hold 10 S records => 200 pages

- page-oriented nested loops join
 - $200 + 200 * 1000 = 200,200$ I/Os
- block nested loops join
 - block size: 50 => $\left\lceil \frac{200}{50} \right\rceil = 4$ S blocks
 - $200 + 4 * 1000 = 4,200$ I/Os

Let R and S be 2 relations. R has 10,000 records; a page can hold 10 R records. S has 2,000 records; a page can hold 10 S records.

2. Compute the cost of sorting R using *external merge sort* with 200 buffer pages.

- $2 * 1000 * 2 = 4,000$ I/Os
- $2 * N * \left(\left\lceil \log_{B-1} \left\lceil \frac{N}{B} \right\rceil \right\rceil + 1 \right)$ I/Os

Let R and S be 2 relations. R has 10,000 records; a page can hold 10 R records. S has 2,000 records; a page can hold 10 S records.

3. R is stored at București, S is stored at Cluj-Napoca. Compute the cost of:

```
SELECT *  
FROM R INNER JOIN S ON R.a = S.b
```

using *simple nested loops join (tuple-oriented)* in Cluj-Napoca, without caching; S is the outer relation.

- t_d time to R / W a page from / to disk
- t_s time to ship a page
 - $200t_d + 2000 * 1000 (t_d + t_s) = 200t_d + 2,000,000 (t_d + t_s)$
 $= 2,000,200t_d + 2,000,000t_s$

4. Encode the data *de gustibus non disputandum* using the secret encryption key *metallica* and the table of codes below. Write the last 5 characters in the result.

	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z	-
00	01	02	03	04	05	06	07	08	09	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27

||

1. T1 and T2 are 2 concurrent transactions, both active at time t . Choose the correct answer(s):
- a. The following execution describes a *write read* conflict: At time t , T2 is reading a data object previously written by T1.
 - b. The following execution describes a *write read* conflict: At time t , T2 is writing a data object previously read by T1.
 - c. The following execution describes a *read write* conflict: At time t , T2 is reading a data object previously written by T1.
 - d. The following execution describes a *read write* conflict: At time t , T2 is writing a data object previously read by T1.
 - e. none of the above answers is correct.

2. A schedule S:

- a. is conflict serializable if and only if its precedence graph has exactly one cycle.
- b. is conflict serializable if and only if its precedence graph is acyclic.
- c. is conflict serializable if and only if its precedence graph has exactly two cycles.
- d. is conflict serializable if and only if its precedence graph has exactly three cycles.
- e. none of the above answers is correct.

3. In SQL Server, under the READ UNCOMMITTED isolation level:
- a. S locks must be acquired to perform read operations.
 - b. read operations are performed without acquiring S locks.
 - c. X locks must be acquired to perform write operations.
 - d. write operations are performed without acquiring X locks.
 - e. none of the above answers is correct.

4. In horizontal fragmentation:

- a. the reconstruction operator is the natural join.
- b. the union of the horizontal fragments must be equal to the original relation.
- c. fragmentation is performed with projection operators.
- d. fragmentation is performed with selection predicates.
- e. none of the above answers is correct.

5. I is an index with search key $\langle C1, C2, C3, C4 \rangle$.

a. If I is a hash index, I matches condition $C1 > 10 \text{ AND } C2 > 7$.

b. If I is a hash index, I matches condition $C1 = 10 \text{ AND } C2 = 7 \text{ AND } C3 = 1 \text{ AND } C4 = 5$.

c. If I is a B+ tree index, I matches condition $C1 = 10 \text{ AND } C2 = 7$.

d. If I is a B+ tree index, I matches condition $C2 = 7 \text{ AND } C3 = 9$.

e. none of the above answers is correct.

6. Let R be a relation with P pages. The cost of sorting R using *simple two-way merge sort* (i.e., with 3 pages in the buffer pool) is:

- a. π^P
- b. $2P(\lceil \log_4 P \rceil + 1)$
- c. $2P(\lceil \log_2 P \rceil + 1)$
- d. $2P(\lceil \log_3 P \rceil + 1)$
- e. none of the above answers is correct.

7. Consider the query:

SELECT *

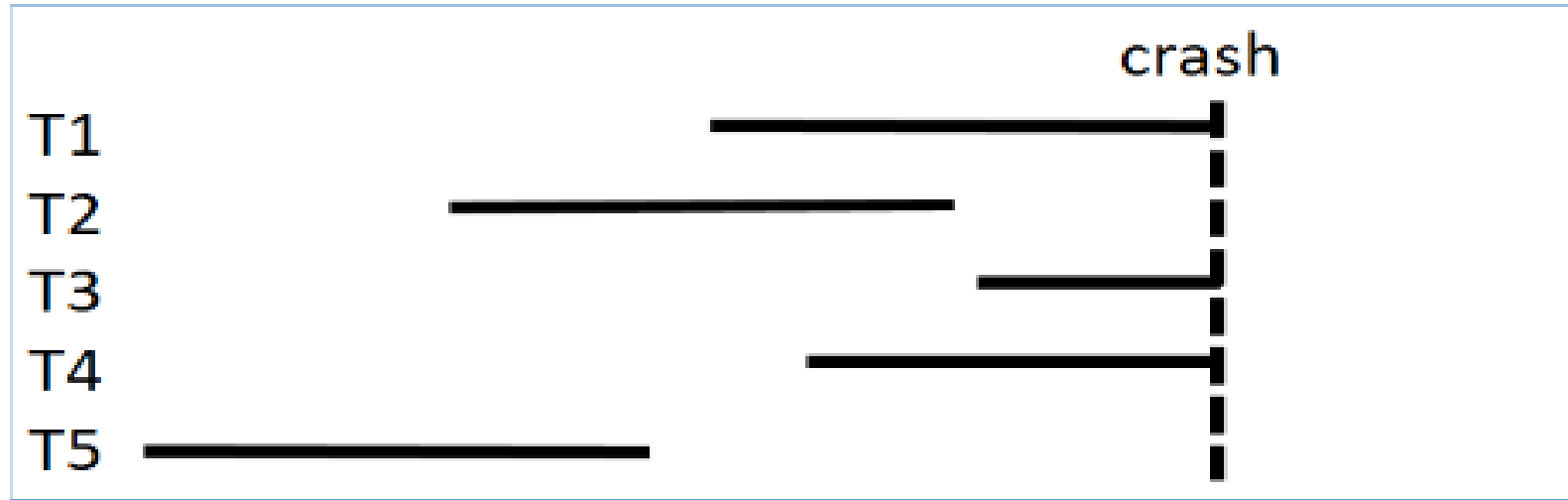
FROM R1, R2, R3

WHERE p1 AND p2 AND p3

The conditions tested by the predicates in the WHERE clause are statistically independent. The cardinality of a relation R is denoted by $|R|$. The reduction factor associated with predicate p is denoted by $RF(p)$. The cardinality of the query's result set can be estimated by:

- a. $\frac{|R1| * |R2| * |R3|}{RF(p1) + RF(p2) + RF(p3)}$
- b. $|R1| * |R2| * |R3| * RF(p1) * RF(p2) * RF(p3)$
- c. $RF(p1) * RF(p2) * RF(p3) - (|R1| + |R2| + |R3|)$
- d. $|R1| + |R2| + |R3| + RF(p1) + RF(p2) + RF(p3)$
- e. none of the above answers is correct.

8. Consider the execution below. When the system comes back up after the crash, it must ensure that:



- a. T1, T3, T4 are durable; T2 and T5 are undone.
- b. T1, T3, T4 are undone; T2 and T5 are durable.
- c. T1 is undone only if T2 and T4 are also undone.
- d. T2 is durable only if T5 is undone.
- e. none of the above answers is correct.

9. In data replication:

- a. *primary site replication* is an asynchronous replication technique.
- b. *primary site replication* is a synchronous replication technique.
- c. *read-any write-all* is a synchronous replication technique.
- d. *read-any write-all* is an asynchronous replication technique.
- e. none of the above answers is correct.

10. A database access request contains:
- a. the requesting user.
 - b. the criminal record of the requesting user.
 - c. the operation the user wants to perform.
 - d. the requested object.
 - e. none of the above answers is correct.

11. Consider schedule S below over transactions T1, T2, T3, T4 (all transactions commit):

T1	T2	T3	T4
W(A)			
			R(C)
	R(B)		
		W(D)	
	R(A)		
R(D)			
			W(B)
R(C)			

- a. S is conflict serializable.
- b. S is not conflict serializable.
- c. (R(T4, C), R(T1, C)) belongs to the conflict relation of S.
- d. (W(T1, A), R(T2, A)) belongs to the conflict relation of S.
- e. none of the above answers is correct.

Database Management Systems

Lecture 12

Distributed Databases

Distributed Transaction Management

- a transaction submitted at a site S could ask for data stored at several other sites
- *subtransaction* - the activity of a transaction at a given site
- context: Strict 2PL with deadlock detection
- problems:
 - distributed concurrency control
 - lock management when objects are stored across several sites
 - deadlock detection
 - distributed recovery
 - transaction atomicity
 - all the effects of a committed transaction (across all the sites it executes at) are permanent
 - none of the actions of an aborted transaction are allowed to persist

Distributed Transaction Management

- distributed concurrency control
 - lock management
 - techniques – synchronous / asynchronous replication
 - which objects will be locked
 - concurrency control protocols
 - when are locks acquired / released
 - lock management
 - *centralized*
 - *primary copy*
 - *fully distributed*

Distributed Transaction Management

- distributed concurrency control
 - lock management
 - centralized:
 - one site does all the locking for all the objects
 - vulnerable to single-site failures
 - primary copy:
 - object O, primary copy PC of O stored at site S with lock manager L
 - all requests to lock / unlock some copy of O are handled by L
 - not vulnerable to single-site failures
 - read some copy C of O stored at site S2:
=> communicate with both S and S2

Distributed Transaction Management

- distributed concurrency control
 - lock management
 - fully distributed:
 - object O, some copy C of O stored at site S with Lock Manager L
 - requests to lock / unlock C are handled by L (the site where the copy is stored)
 - one doesn't need to access 2 sites when reading some copy of O

Distributed Transaction Management

- distributed concurrency control
 - detect and resolve deadlocks
 - each site maintains a local waits-for graph
 - a cycle in such a graph indicates a deadlock
 - but a global deadlock can exist even if none of the local graphs contains a cycle

->

Distributed Transaction Management

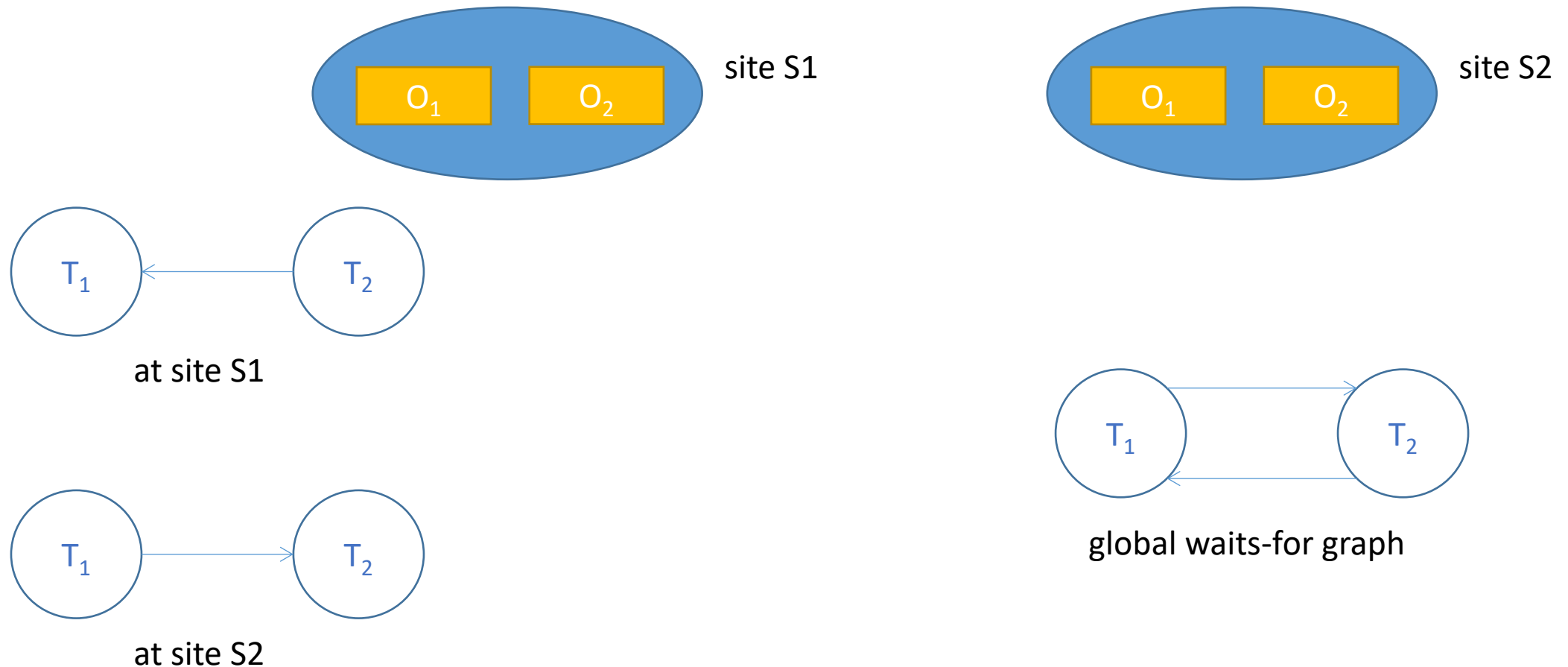
- distributed concurrency control – distributed deadlock
 - e.g., using *read-any write-all*



- T_1 wants to read O_1 and write O_2
- T_2 wants to read O_2 and write O_1
- T_1 acquires a S lock on O_1 and an X lock on O_2 at site S1
- T_2 obtains a S lock on O_2 and an X lock on O_1 at site S2
- T_1 asks for an X lock on O_2 at site S2
- T_2 asks for an X lock on O_1 at site S1

Distributed Transaction Management

- distributed concurrency control – distributed deadlock
 - e.g., using *read-any write-all*



Distributed Transaction Management

- distributed concurrency control – distributed deadlock
 - distributed deadlock detection algorithms
 - *centralized*
 - *hierarchical*
 - based on a *timeout* mechanism

Distributed Transaction Management

- distributed concurrency control – distributed deadlock
 - distributed deadlock detection algorithms
 - centralized:
 - all the local waits-for graphs are periodically sent to a single site S
 - S - responsible for global deadlock detection
 - the global waits-for graph is generated at site S
 - nodes
 - the union of the nodes in the local graphs
 - edges
 - there is an edge from node N1 to node N2 if such an edge exists in one of the local graphs

Distributed Transaction Management

- distributed concurrency control – distributed deadlock
 - distributed deadlock detection algorithms
 - hierarchical:
 - sites are organized into a hierarchy, e.g., grouped by city, county, country, etc.
 - each site periodically sends its local waits-for graph to its parent site
 - assumption: more deadlocks are likely across related sites
 - all the deadlocks are detected in the end

Distributed Transaction Management

- distributed concurrency control – distributed deadlock
 - distributed deadlock detection algorithms

- hierarchical:

- example:

RO (CJ (Cluj-Napoca, Dej, Turda), BN (Bistrita, Beclean))

Cluj-Napoca : T1 -> T2

Dej: T2 -> T3

Turda: T3 -> T4 <- T7

Bistrita: T5 -> T6

Beclean: T4 -> T7 -> T6 -> T5

CJ: T1 -> T2 -> T3 -> T4 <- T7

BN: T5 <-> T6 <- T7 <- T4 (*)

RO: T1 -> T2 -> T3 -> T4 <-> T7 -> T6 <-> T5

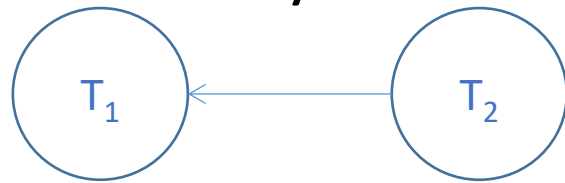
Obs RO: T5 or T6 was aborted at (*)

Distributed Transaction Management

- distributed concurrency control – distributed deadlock
 - distributed deadlock detection algorithms
 - based on a timeout mechanism:
 - a transaction is aborted if it lasts longer than a specified interval
 - can lead to unnecessary restarts
 - however, the deadlock detection overhead is low
 - could be the only available option in a heterogeneous system (if the participating sites cannot cooperate, i.e., they cannot share their local waits-for graphs)

Distributed Transaction Management

- distributed concurrency control – distributed deadlock
- phantom deadlocks
 - "deadlocks" that don't exist, but are detected due to delays in propagating local information
 - lead to unnecessary aborts
 - example:



at site S1



global waits-for graph



at site S2

- generate local waits-for graphs, send them to the site responsible for global deadlock detection
- T₂ aborts (not because of the deadlock) => local waits-for graphs are changed, there is no cycle in the "real" global waits-for graph
- but the built waits-for graph does have a cycle, T₁ could be chosen as a victim

Distributed Transaction Management

- distributed recovery
 - more complex than in a centralized DBMS
 - new types of failure
 - network failure
 - site failure
 - commit protocol
 - either all the subtransactions of a transaction commit, or none of them does
- normal execution
 - ensure all the necessary information is provided to recover from failures
 - a log is maintained at each site; it contains:
 - data logged in a centralized DBMS
 - actions carried out as part of the commit protocol

Distributed Transaction Management

- distributed recovery
 - transaction T
 - coordinator
 - the Transaction Manager at the site where T originated
 - subordinates
 - the Transaction Managers at the sites where T's subtransactions execute

Distributed Transaction Management

- distributed recovery
 - two-phase commit protocol (2PC)
 - exchanged messages + records written in the log
 - 2 rounds of messages, both initiated by the coordinator
 - *voting phase*
 - *termination phase*
 - any Transaction Manager can abort a transaction
 - however, for a transaction to commit, all Transaction Managers must decide to commit

Distributed Transaction Management

- distributed recovery
 - two-phase commit protocol
 - the user decides to commit transaction T
- => the commit command is sent to T's coordinator, 2PC is initiated
1. the coordinator sends a *prepare* message to each subordinate
 2. upon receiving a *prepare* message, a subordinate decides whether to commit / abort its subtransaction
- the subordinate force-writes an *abort* or a *prepare** log record
 - then it sends a *no* or *yes* message to the coordinator

* *prepare* log records are specific to the commit protocol, they are not used in centralized DBMSs

Distributed Transaction Management

- distributed recovery
 - two-phase commit protocol

3.

- if the coordinator receives *yes* messages from all subordinates:
 - it force-writes a *commit* log record
 - it then sends *commit* messages to all subordinates
- otherwise (i.e., if it receives at least one *no* message or if it doesn't receive any message from a subordinate for a predetermined timeout interval)
 - it force-writes an *abort* log record
 - it then sends an *abort* message to each subordinate

Distributed Transaction Management

- distributed recovery
 - two-phase commit protocol

4.

- upon receiving an *abort* message, a subordinate:
 - force-writes an *abort* log record
 - sends an *ack* message to the coordinator
 - aborts the subtransaction
- upon receiving a *commit* message, a subordinate:
 - force-writes a *commit* log record
 - sends an *ack* message to the coordinator
 - commits the subtransaction

Distributed Transaction Management

- distributed recovery
 - two-phase commit protocol
 5. after it receives *ack* messages from all subordinates, the coordinator writes an *end* log record for the transaction
- * obs. sending a message – the sender has made a decision
 - the message is sent only after the corresponding log record has been forced to stable storage (to ensure the corresponding decision can survive a crash)
- * obs. T is a committed transaction if the commit log record of T's coordinator has been forced to stable storage

Distributed Transaction Management

- distributed recovery
 - two-phase commit protocol
 - log records for the commit protocol
 - record type
 - transaction id
 - coordinator's identity
 - the commit / abort log record for the coordinator also contains the identities of the subordinates

Distributed Transaction Management

- distributed recovery
 - restart after a failure – site S comes back up after a crash
 - if there is a *commit* or an *abort* log record for transaction T:
 - must redo / undo T
 - if S is T's coordinator:
 - periodically send *commit* / *abort* messages to subordinates until *ack* messages are received
 - write an *end* log record after receiving all *ack* messages
 - if there is a *prepare* log record for transaction T, but no *commit* / *abort*, S is one of T's subordinates
 - contact T's coordinator repeatedly until T's status is obtained
 - write a *commit* / an *abort* log record
 - redo / undo T

Distributed Transaction Management

- distributed recovery
 - restart after a failure – site S
 - if there are no *commit* / *abort* / *prepare* log records for T:
 - abort T, undo T
 - if S is T's coordinator, T's subordinates may subsequently contact S

Distributed Transaction Management

- distributed recovery
 - link and remote site failures
 - current site S, remote site R, transaction T
 - if R doesn't respond during the commit protocol for T, either because R failed or the link failed:
 - if S is T's coordinator:
 - S should abort T
 - if S is one of T's subordinates, and has not voted yet:
 - S should abort T
 - if S is one of T's subordinates and has voted yes:
 - S is blocked until T's coordinator responds

Distributed Transaction Management - optional

- distributed recovery
 - 2PC – observations
 - *ack* messages
 - used to determine when can a coordinator C “forget” about a transaction T
 - C must keep T in the transaction table until it receives all *ack* messages
 - C fails after sending *prepare* messages, but before writing a *commit* / an *abort* log record
 - when C comes back up it aborts T
 - i.e., absence of information => T is presumed to have aborted
 - if a subtransaction doesn't change any data, its commit / abort status is irrelevant

Distributed Transaction Management - optional

- distributed recovery
 - 2PC with Presumed Abort
 - coordinator C, transaction T, some subordinate S, some subtransaction t
 - C aborts T
 - T is undone
 - C immediately removes T from the Transaction Table, i.e., it doesn't wait for *ack* messages
 - subordinates' names need not be recorded in C's abort log record
 - S doesn't send an *ack* message when it receives an *abort* message

Distributed Transaction Management - optional

- distributed recovery
 - 2PC with Presumed Abort
 - coordinator C, transaction T, some subordinate S, some subtransaction t
 - t doesn't change any data
 - t responds to *prepare* messages with a *reader* message, instead of *yes / no*
 - C subsequently ignores readers
 - if all subtransactions are readers, the 2nd phase of the protocol is not needed

Distributed Catalog Management - optional

- keeping track of data distribution across sites
- one should be able to identify each replica of each fragment for a relation that is fragmented and replicated
- local autonomy should not be compromised
 - solution - names containing several fields:
 - global relation name:
 - <local-name, birth-site>
 - global replica name:
 - <local-name, birth-site, replica_id>

Distributed Catalog Management - optional

- centralized system catalog
 - stored at a single site
 - contains data about all the relations, fragments, replicas
 - vulnerable to single-site failures
 - can overload the server
- global system catalog maintained at each site
 - every copy of the catalog describes all the data
 - not vulnerable to single-site failures (the data can be obtained from a different site)
 - local autonomy is compromised:
 - changes to a local catalog must be propagated to all the other sites

Distributed Catalog Management - optional

- local catalog maintained at each site
 - each site keeps a catalog that describes local data, i.e., copies of data stored at the site
 - the catalog at the birth-site for a relation keeps track of all the fragments / replicas of the relation
 - create a new replica / move a replica to another site:
 - must update the catalog at the birth-site
 - not vulnerable to single-site failures & doesn't compromise local autonomy

References

- [Ra02] RAMAKRISHNAN, R., GEHRKE, J., Database Management Systems (3rd Edition), McGraw-Hill, 2002
- [Da03] DATE, C.J., An Introduction to Database Systems (8th Edition), Addison-Wesley, 2003
- [Ga09] GARCIA-MOLINA, H., ULLMAN, J., WIDOM, J., Database Systems: The Complete Book (2nd Edition), Pearson Education, 2009
- [Ra02S] RAMAKRISHNAN, R., GEHRKE, J., Database Management Systems, Slides for the 3rd Edition,
<http://pages.cs.wisc.edu/~dbbook/openAccess/thirdEdition/slides/slides3ed.html>
- [Si19] SILBERSCHATZ, A., KORTH, H., SUDARSHAN, S., Database System Concepts (7th Edition), McGraw-Hill, 2019
- [Si19S] SILBERSCHATZ, A., KORTH, H., SUDARSHAN, S., Database System Concepts, Slides for the 7th Edition, <http://codex.cs.yale.edu/avi/db-book/>
- [Ul11] ULLMAN, J., WIDOM, J., A First Course in Database Systems,
<http://infolab.stanford.edu/~ullman/fcdb.html>

Database Management Systems

Lecture 12

Security

* database protection

- security & integrity
- security
 - protecting the data against unauthorized users (who may want to read, modify, destroy the data)
 - i.e., users have the right to do what they are trying to do
- integrity
 - protecting the data against authorized users
 - i.e., the operations that users are trying to execute are correct
 - the consistency of the database must be maintained at all times

* aspects related to security:

- legal, ethical aspects
 - certain rules are being broken
 - e.g., a person searches for a password or accidentally finds a password and uses it
- physical controls
 - e.g., the computer room is locked (or guarded)
- software controls
 - files are protected (unauthorized access is not allowed)
 - there are no programs intercepting the actions of authorized users
- operational problems
 - if a password mechanism is used, how are the passwords kept secret and how often should a user change his / her password?

- * data security and data integrity – similarities:
 - the system enforces certain constraints (users cannot violate these constraints)
 - security constraints, integrity constraints
 - constraints are:
 - specified in a declarative language
 - saved in the system catalog
- example: *primary key constraint*, specified in SQL and saved in the system catalog (retrieving info about PK constraints in SQL Server:

```
SELECT *  
FROM sys.objects  
WHERE type = 'PK')
```
- the system monitors users' actions to enforce the specified constraints
example: INSERT, UPDATE statements cannot violate a PK constraint

- * the DBMS's authorization subsystem (security subsystem)
 - checks any given access request against the applicable constraints
 - access request:
 - requested object + requested operation + requesting userexample: *Alice wants to delete 10 rows from table Customers.*
 - identifying the applicable constraints for an access request:
 - the system must recognize the source of the request, i.e., the requesting user
 - an authentication mechanism is used for this purpose, e.g.:
 - password scheme
 - users supply their user ID (users say who they are) and their password (users prove they are who they say they are)
 - fingerprint readers, voice verifiers, retinal scanners, etc.

- * main approaches to data security in a DBMS:
 - discretionary control & mandatory control
 - discretionary control
 - users have different access rights (privileges) on different database objects
 - very flexible schemes, e.g.:
 - user U1 sees object O1, but doesn't see object O2
 - user U2 sees object O2, but doesn't see object O1

- * main approaches to data security in a DBMS:
 - discretionary control & mandatory control
 - mandatory control
 - each object has a classification level
 - each user has a clearance level
 - an object O can be accessed by user U only if U has the right clearance
 - schemes - more rigid, e.g.:
 - user U1 can see object O1, but cannot see object O2
 - i.e., object O2's classification level is higher than O1's classification level
 - so no user U2 can see O2, but not O1

* discretionary control

- the operations users are allowed to perform are explicitly specified
- anything not explicitly authorized is implicitly forbidden

- examples:

Alice and Bob are both allowed to SELECT from table Customers.

Jane is allowed to SELECT, INSERT, DELETE from / into table Orders.

* discretionary control

- possible language to define *authorities*

AUTHORITY CA1

GRANT RETRIEVE
ON Customers
TO Alice, Bob

AUTHORITY CA2

GRANT RETRIEVE {CID, LastName, FirstName, DOB}, DELETE
ON Customers
TO Jane

- an authority has:
 - a name
 - a set of privileges (GRANT clause)
 - the relation to which it applies (ON clause)
 - a set of users (user IDs) who will be granted the specified privileges on the specified relation (TO clause)

- privileges
 - RETRIEVE [attr_list]
 - INSERT [attr_list]
 - DELETE
 - UPDATE [attr_list]

* discretionary control

- SQL support
 - views
 - users are allowed to see only a certain subset of the data
 - authorization subsystem
 - users can grant privileges to / revoke privileges from other users
- user U created object O => U automatically receives all privileges associated with O
 - e.g., O is a table => U is automatically granted the following privileges: SELECT, INSERT, UPDATE, DELETE, REFERENCES, TRIGGER
 - TRIGGER - allows the user to create a trigger on O
 - REFERENCES - allows the user to refer to O in an integrity constraint
 - U can also grant these privileges to other users

* discretionary control

- SQL support
 - granting privileges - GRANT statement

GRANT privilege_list

ON object

TO user_list

[WITH GRANT OPTION]

- WITH GRANT OPTION - users (in user_list) can grant the specified privileges on the specified object to other users

* discretionary control

- SQL support
 - revoking privileges - REVOKE statement

```
REVOKE [GRANT OPTION FOR] privilege_list
      ON object
      FROM user_list
      behavior
```

```
REVOKE SELECT, DELETE, UPDATE {FirstName, LastName}
      ON Customers
      FROM Jane RESTRICT
```

```
REVOKE SELECT
      ON Customers
      FROM Alice, Bob CASCADE
```

* discretionary control

- SQL support
 - revoking privileges - REVOKE statement
- GRANT OPTION FOR
 - revoke the grant authority
- behavior - RESTRICT / CASCADE
 - let p be a privilege on object O
 - user U1 grants privilege p to user U2, user U2 grants privilege p to user U3; U1 tries to revoke p from U2 => privilege p of U3 would be *abandoned*
 - RESTRICT / CASCADE - avoid abandoned privileges
 - RESTRICT - operation fails
 - CASCADE - abandoned privileges are also revoked

* discretionary control

- SQL support
 - revoking privileges - REVOKE statement
- behavior - RESTRICT / CASCADE
 - user U1 grants privilege p to user U2, user U2 grants privilege p to user U3
 - U1 tries to revoke p from U2 with RESTRICT => REVOKE fails
 - U1 tries to revoke p from U2 with CASCADE => p is revoked from U2 and U3

* mandatory control

- every object has a classification level (e.g., *top secret*, *secret*, etc.)
 - levels – strict ordering (e.g., *top secret* > *secret*)
- every user has a clearance level (same options as for classification levels)
- Bell and La Padula rules:
 - user *x* can retrieve object *y* only if the clearance level of *x* is $\geq$ the classification level of *y*
example: *Alice's clearance level is top secret, Bob's is secret.*
Object O1's classification level is top secret, O2's is secret.
Alice can retrieve both O1 and O2, Bob can only retrieve O2.
 - user *x* can update object *y* only if the clearance level of *x* is equal to the classification level of *y*
example: *Alice can only update O1, Bob can only update O2.*

* audit trail

- database administrator should monitor the database and keep track of all operations performed by users via an audit trail
 - if there is any suspicion of wrongdoing, the audit trail can be analyzed
 - an audit trail can contain:
 - executed SQL statement, user name of the user who executed the statement, IP address, date and time, affected object, old values and new values for changed records

* SQL injection

- application -> execution of a SQL statement (fixed statement, statement that is generated using input data from the user)
- a statement can be changed (when being generated) due to data supplied by the user
- such a change is attempted when the user is trying to obtain additional access rights
- the user enters code into input variables; the code is concatenated with SQL statements and executed
- the DBMS executes all statements that are syntactically valid
- obs.
 - string separators: single quotes, double quotes
 - statement separator (if a command can include multiple statements): semicolon
 - comments in SQL: `statement --comment`

* SQL injection

1. changing a user's authentication statement

- errors when executing the statement => use the error messages in subsequent executions
- authenticating with a username and a password - in many cases, the following statement is executed:

```
SELECT ... WHERE user = "uname" AND password = "upassword"
```

columns in the *users*
table (can have various
other names)

could be an email

supplied by the client

- if the statement is successfully executed and returns at least one record, the authentication is successful

* SQL injection

1. changing a user's authentication statement

- values that change the SQL statement (modify the action intended by the programmer):

uname	upassword	condition in the statement
a	" OR 1 = 1 --	user="a" AND password="" OR 1=1 -- "
admin" --	?	user="admin" --" AND password ... (other values besides <i>admin</i> can be chosen from various lists on the internet)
" OR 0=0 --	?	user="" OR 0=0 -- ...

SELECT ... WHERE user = "uname" AND password = "upassword"

- for the values in the 1st row, the statement becomes:

SELECT ... WHERE user = "a" AND password = "" OR 1 = 1 --"

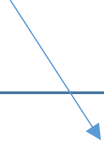
SELECT ... WHERE user = "a" AND password = "" OR 1 = 1 --" => all rows are retrieved from the DB, successful authentication

* SQL injection

2. obtaining information from the database

- the client is asked to provide value v :
- possible values entered by the user:

provided by the client



```
SELECT ... WHERE ... = " $v$ " ...
```

- `x" OR 1 = (SELECT COUNT(*) FROM table) --`
 - if an error occurs, the statement can be executed again, with a different table name; a malevolent user can obtain, in this way, the name of a table in the DB (for instance, a table with passwords)
- `x" AND user IS NULL --`
 - if an error occurs, the statement can be executed again, with a different column name (instead of *user* in the example); a malevolent user can obtain the name of a column
- `x" AND user LIKE "%pop%" --`
 - if an error occurs or no data is returned => change the column name or the string in the condition (*%pop%*); one can obtain the name of a user

* SQL injection

2. obtaining information from the database

- the client is asked to provide value v :
- possible value entered by the user:

provided by the client



SELECT ... WHERE ... v ...

The diagram shows a rectangular box containing the SQL query 'SELECT ... WHERE ... v ...'. A blue arrow points from the text 'provided by the client' above to the variable 'v' in the query, indicating that 'v' is the value supplied by the client.

- 0 UNION SELECT CONCAT(name, password) FROM users
- retrieving all usernames and passwords while executing the SELECT statement

* SQL injection

3. changing data in tables

- the client is asked to provide value v :
- possible values entered by the user:

- `0; INSERT INTO users ...`

- the SELECT statement is executed, then data about a new user is inserted into the *users* table

- `0; DROP TABLE users`

- the SELECT statement is executed, then table *users* is dropped

supplied by the client



SELECT ... WHERE ... v ...

The diagram shows a rectangular box containing the text "SELECT ... WHERE ... v ...". A blue arrow points from the text "supplied by the client" above the box to the variable v inside the box.

* SQL injection

4. changing a user's password

- the client is asked to provide value v :
- possible values entered by the user for v (the new password):
 - $x'' --$
 - setting the password for all users to x
 - $x'' \text{ WHERE user LIKE } "\%admin\%" --$
 - setting the password for all usernames matching the specified criterion

```
UPDATE table  
SET password = "v"  
WHERE user = " " ...
```

supplied by the client

* SQL injection

- prevention

- data validation

- use regular expressions to validate data
 - allow the client to use only certain types of characters in the input data

- modify problematic characters

- double single quotes and double quotes; or precede single and double quotes with "\" (one can use functions for such changes)

=> single and double quotes in the statement won't modify the statement (like in the previous examples)

example: SELECT ... WHERE user = "uname" AND password = "upassword"

user enters: *a* and " *OR 1 = 1 --* for *uname* and *upassword* (like in a prev. ex.)

double the "

=> statement: SELECT ... WHERE user = "a" AND password = " " " OR 1 = 1 -- ", which retrieves rows with user *a* and password " *OR 1 = 1 --*

* SQL injection

- prevention

- use parameterized statements

SELECT ... WHERE user=? AND password=?, where “?” is a parameter

- such a statement has a collection of parameters

* data encryption

- protecting the data when the normal security mechanisms in the DBMS are insufficient (e.g., an intruder gets physical access to the server or the data center and steals the drives containing the data, an enemy taps into a communication line, etc.)
- storing / transmitting encrypted data => the stolen data is illegible for the intruder
- some DBMSs store data (all data or only a part of the data) in an encrypted form; if the files containing the database can be copied, using the data in these files is impossible (or extremely difficult, as the decryption cost is enormous)

* codes and ciphers - optional

- code
 - replace one word or phrase with another word / number / symbol
 - e.g., replace expression *The enemy is moving* with code word *Binary*
- cipher
 - replace each letter with another letter (number / symbol)
 - e.g., replace each letter in the alphabet with the previous one (B with A, C with B, etc.); *The enemy is moving* becomes *Sgd dmdlx hr lnuhmf*

* steganography and cryptography - optional

- steganography - hide the *existence* of the message
 - e.g., shave messenger's head, write message on scalp, wait for the hair to grow back; use invisible ink; etc.
- cryptography - hide the *meaning* of the message – encryption
 - transposition
 - rearrange letters in the message (create anagrams)
 - every letter retains its identity, but changes its position
 - e.g., 6 ways of rearranging 3 letters: *cat*, *cta*, *act*, *atc*, *tca*, *tac*
 - substitution
 - pair the letters of the alphabet (randomly, e.g., A's pair could be X)
 - replace each letter in the message with its pair
 - every letter retains its position, but changes its identity

* substitution - optional

- *plaintext* - message before encryption
- *ciphertext* - message after encryption
- *plain alphabet* - alphabet used to write the message
- *cipher alphabet* - alphabet used to encrypt the message
- examples:
 - shift the original alphabet:
 - by 1 position (replace *A* with *B*, *B* with *C*, etc.)
 - by 2 positions (replace *A* with *C*, *B* with *D*, etc.), etc.

=> there are 25 possible ciphers

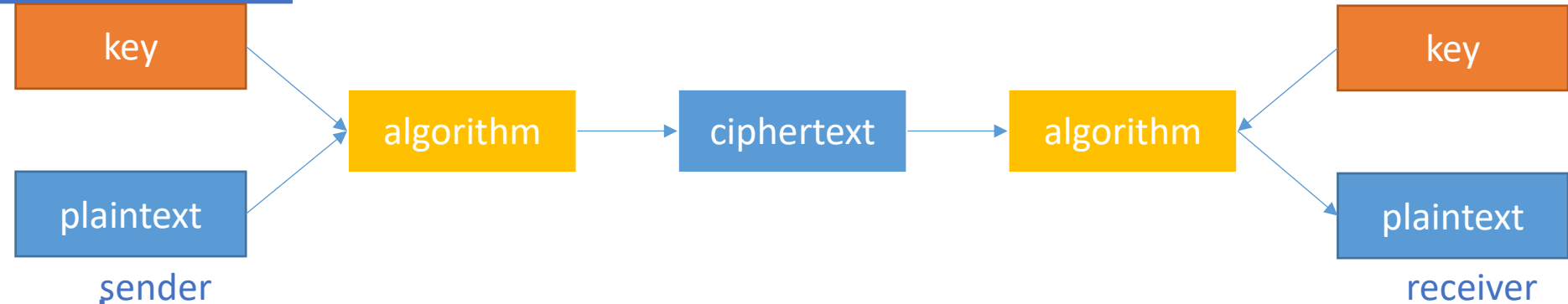
- if any rearrangement of the plain alphabet is allowed (don't restrict to shift-based rearrangements) => over $4 \cdot 10^{26}$ possible rearrangements (i.e., over $4 \cdot 10^{26}$ ciphers)

* substitution - optional

example:

- *plain alphabet:*
 - *cipher alphabet:*
- | | | | | |
|---|---|---|---|-----|
| a | b | c | d | ... |
| B | C | D | E | ... |
- (shift the original alphabet by one position)
- *plaintext: the enemy is moving*
 - *ciphertext: UIF FOFNZ JT NPWJOH*

* algorithms and keys - optional



- algorithm - not secret
 - general encryption method (e.g., replace each letter in the plain alphabet with a letter from the cipher alphabet; cipher alphabet can be any rearrangement of the plain alphabet)
- key - must be kept secret!
 - details of a particular encryption (e.g., the chosen cipher alphabet)
 - important - large number of keys:
 - suppose the enemy intercepts a ciphertext; if they know a shift cypher has been used, they only need to check 25 keys; but if any rearrangement of the original alphabet is possible, they need to check 4×10^{26} keys!

* the key distribution problem - optional

- if two parties want to exchange a message, they must first exchange a secret key:
 - meet in person, use couriers (banks used such couriers back in the '70s to communicate securely with their customers)
 - logistical difficulties, costs way too high
- Whitfield Diffie, Martin Hellman, Ralph Merkle
 - came up with a solution that would enable 2 people to exchange a key without meeting in person and without relying on a 3rd party
 - solution based on one-way functions in modular arithmetic
 - one-way functions - easy to do and extremely difficult to undo

one-way function: $Y^x \pmod P$ e.g., $7^x \pmod{11}$ 7, 11 – numbers chosen by Alice & Bob; not secret	Alice	Bob
1	Alice chooses a number $A = 3$ A - secret	Bob chooses a number $B = 6$ B - secret
2	Alice computes: $\alpha = 7^A \pmod{11} = 2$	Bob computes: $\beta = 7^B \pmod{11} = 4$
3	Alice sends α to Bob	Bob sends β to Alice
exchange	An eavesdropper can intercept α and β . This poses no problems, since these numbers are not the key!	
4	Alice computes: $\beta^A \pmod{11} = 4^3 \pmod{11} = 9$	Bob computes: $\alpha^B \pmod{11} = 2^6 \pmod{11} = 9$
the key	Alice and Bob obtained the same number: 9 - the key.	

- Alice and Bob are able to establish a key without meeting beforehand
- Mark (eavesdropper, trying to intercept exchanged messages) only knows the $7^x \pmod{11}$ function, $\alpha = 2$, $\beta = 4$
- Mark doesn't know the key and cannot obtain it, since:
 - he doesn't know A and B (these numbers are kept secret);
 - it's extremely difficult for him to obtain A from α (or B from β), since $7^x \pmod{11}$ is a one-way function; this is especially true if the chosen numbers are very large

* encryption algorithm - example

- use a secret encryption key

data:	disciplina baze de date
secret key:	student

a. create a table of codes

- every character is associated with a number, for instance:

	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
00	01	02	03	04	05	06	07	08	09	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26

* encryption algorithm - example

b. divide the message into blocks of length L , where L is the number of characters in the key

d	i	s	c	i	p	l	i	n	a		b	a	z	e		d	e		d	a	t	e
s	t	u	d	e	n	t																

- replace every character in the message and every character in the key with their associated codes

	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
00	01	02	03	04	05	06	07	08	09	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26

s	t	u	d	e	n	t
19	20	21	04	05	14	20

d	i	s	c	i	p	l	i	n	a		b	a	z	e		d	e		d	a	t	e
04	09	19	03	09	16	12	09	14	01	00	02	01	26	05	00	04	05	00	04	01	20	05

* encryption algorithm - example

b. add every number that corresponds to a character in a block with the number of the corresponding character in the key

- if the obtained value is greater than n (in the example, $n = 27$), compute the remainder of the division by n

d	i	s	c	i	p	l	i	n	a		b	a	z	e		d	e		d	a	t	e
04	09	19	03	09	16	12	09	14	01	00	02	01	26	05	00	04	05	00	04	01	20	05
19	20	21	04	05	14	20	19	20	21	04	05	14	20	19	20	21	04	05	14	20	19	20
23	02	13	07	14	03	05	01	07	22	04	07	15	19	24	20	25	09	05	18	21	12	25

c. replace the obtained numbers with their corresponding characters in the table of codes

=> a string that can be stored / transmitted

- the obtained string in the example: *wbmgnceagvldgosxtyieruly*

* encryption algorithm - example

- decryption
 - similar
 - in step b, perform *subtraction (modulo n)* (instead of addition)
- one could also permute the original message and add values (to the characters' codes) for every position; or combine this method with the previous one

References

- [Ta13] ȚÂMBULEA, L., Curs Baze de date, Facultatea de Matematică și Informatică, UBB, 2013-2014
- [Da03] DATE, C.J., An Introduction to Database Systems (8th Edition), Addison-Wesley, 2003
- [Si05] SINGH, Simon, Cartea codurilor – istoria secretă a codurilor și a spargerii lor, Humanitas, 2005
- [Ra02] RAMAKRISHNAN, R., GEHRKE, J., Database Management Systems (3rd Edition), McGraw-Hill, 2002
- [Ra02S] RAMAKRISHNAN, R., GEHRKE, J., Database Management Systems, Slides for the 3rd Edition,
<http://pages.cs.wisc.edu/~dbbook/openAccess/thirdEdition/slides/slides3ed.html>
- [WWW1] The Open Web Application Security Project,
https://www.owasp.org/index.php/SQL_Injection
- [WWW4] SQL injection, http://en.wikipedia.org/wiki/Sql_injection

Database Management Systems

Lecture 13

Parallel Databases*

Spatial Databases*

* **not** among the exam topics

Parallel Databases

Parallel Database Systems

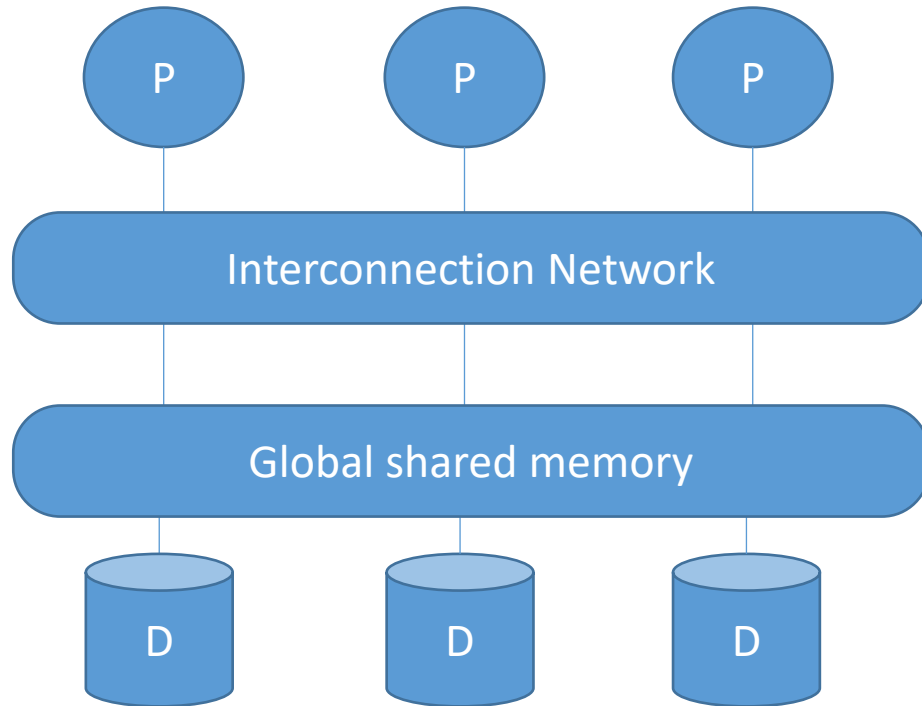
- performance improvement
 - parallelize operations:
 - loading data
 - building indexes
 - query evaluation
 - data can be distributed, but distribution is dictated solely by performance reasons

Parallel Databases - Architectures

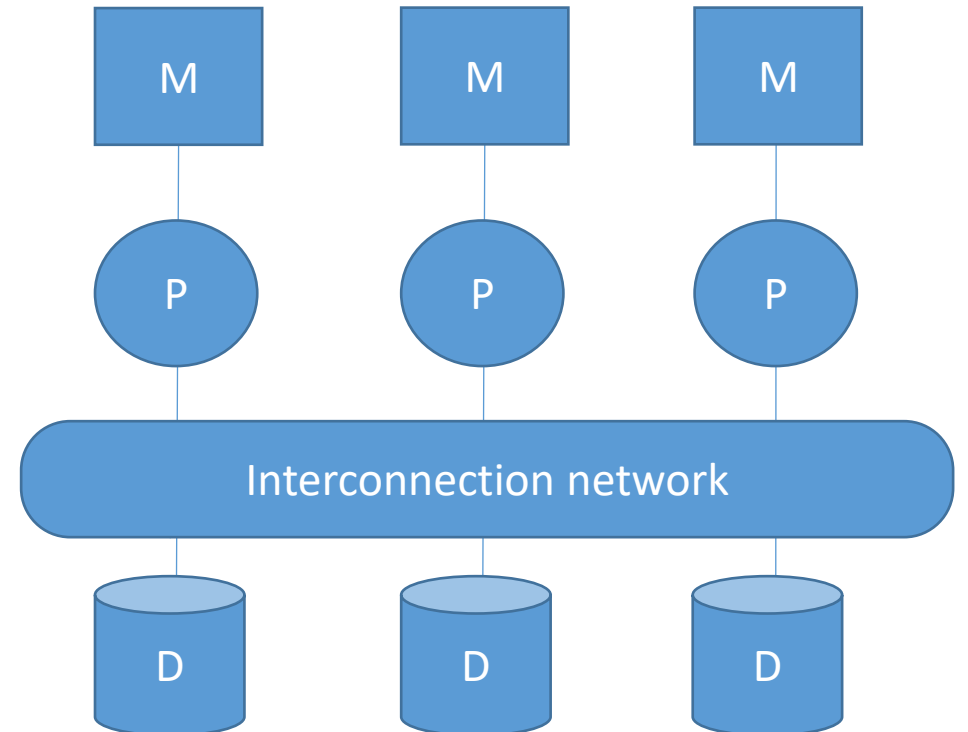
- *shared-memory*
- *shared-disk*
- *shared-nothing*

Parallel Databases - Architectures

- *shared-memory*
 - several CPUs:
 - attached to an interconnection network
 - can access a common region in the main memory

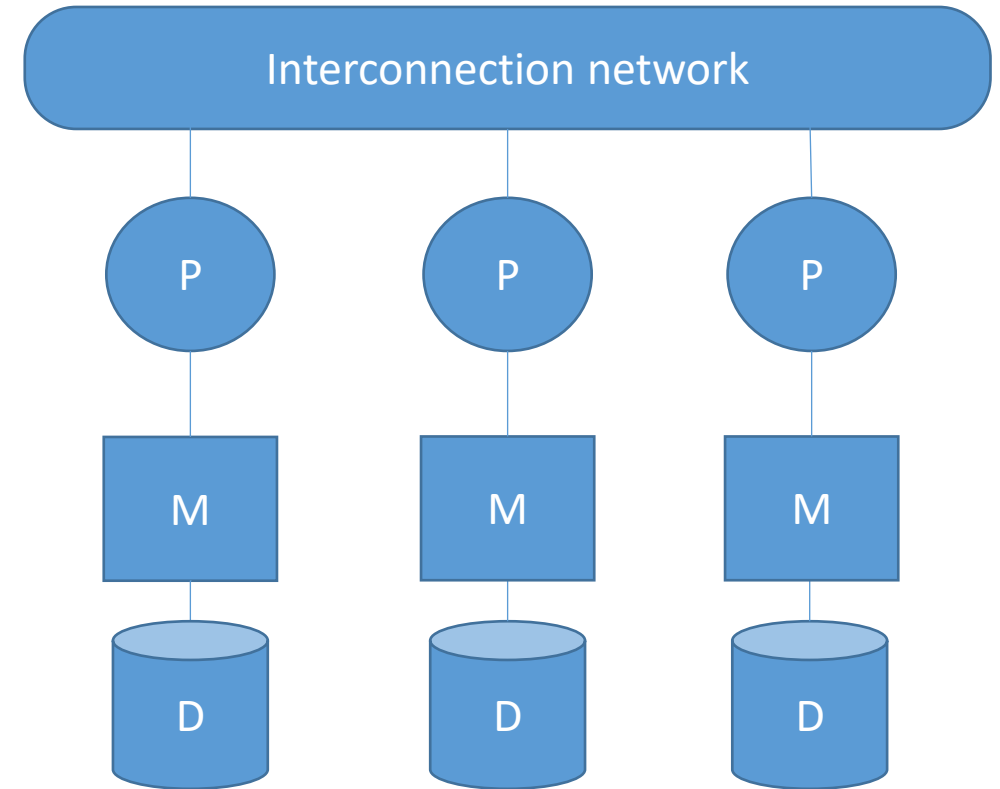


- *shared-disk*
 - a CPU:
 - its own private memory
 - can access all disks through a network



Parallel Databases - Architectures

- *shared-nothing*
 - a CPU:
 - its own local main memory
 - its own disk space
 - 2 different CPUs cannot access the same storage area
 - CPUs communicate through a network



Interference

- specific to shared-memory and shared-disk architectures
- add CPUs:
 - increased contention for memory and network bandwidth
=> existing CPUs are slowing down
- main reason that led to the shared-nothing architecture, currently considered as the best option for large parallel database systems

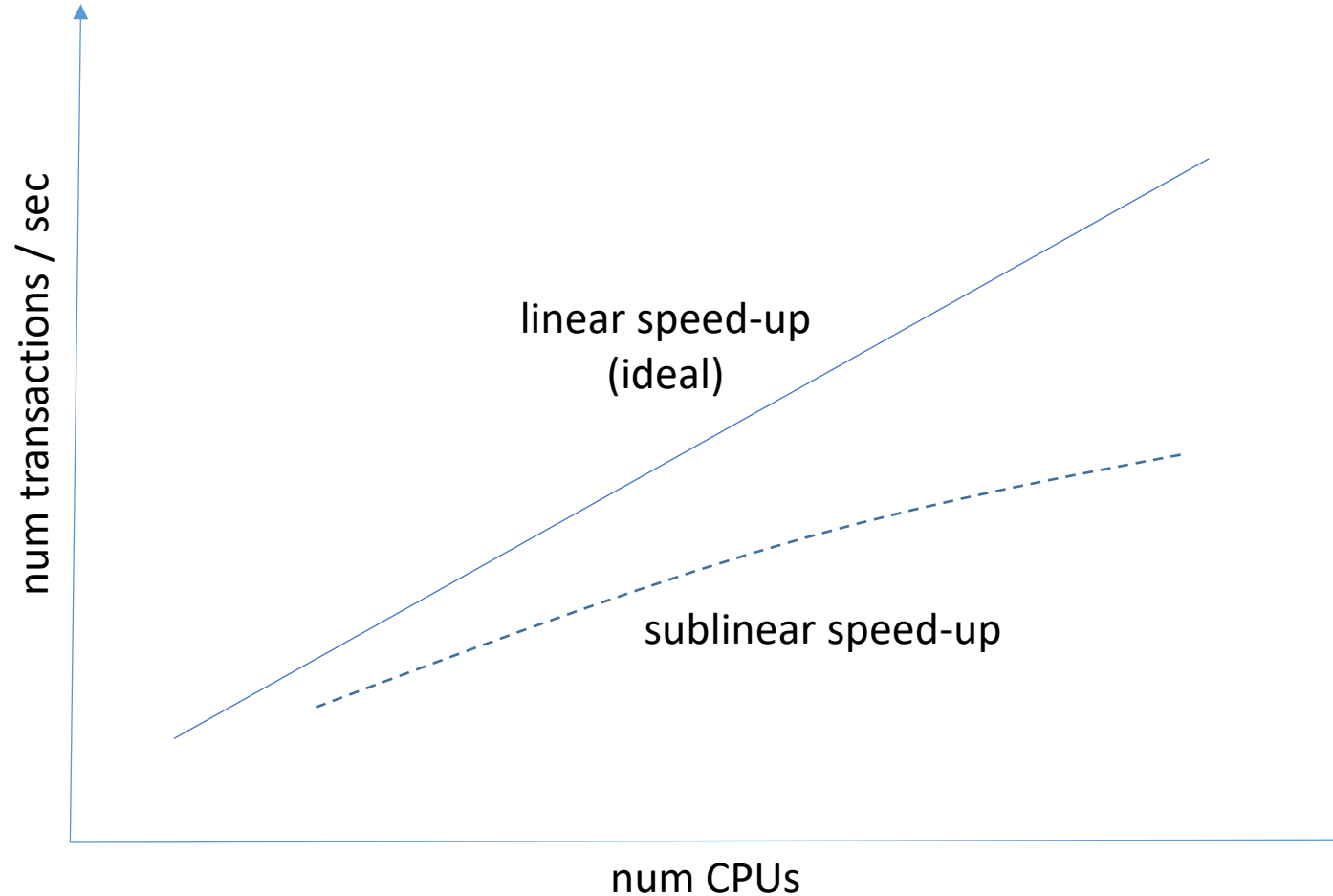
The Shared-Nothing Architecture

- linear speed-up & linear scale-up
- linear speed-up
 - required processing time for operations decreases proportionally to the increase in the number of CPUs and disks
- linear scale-up
 - num. of CPUs and disks grows proportionally to the amount of data

=> performance is sustained

The Shared-Nothing Architecture

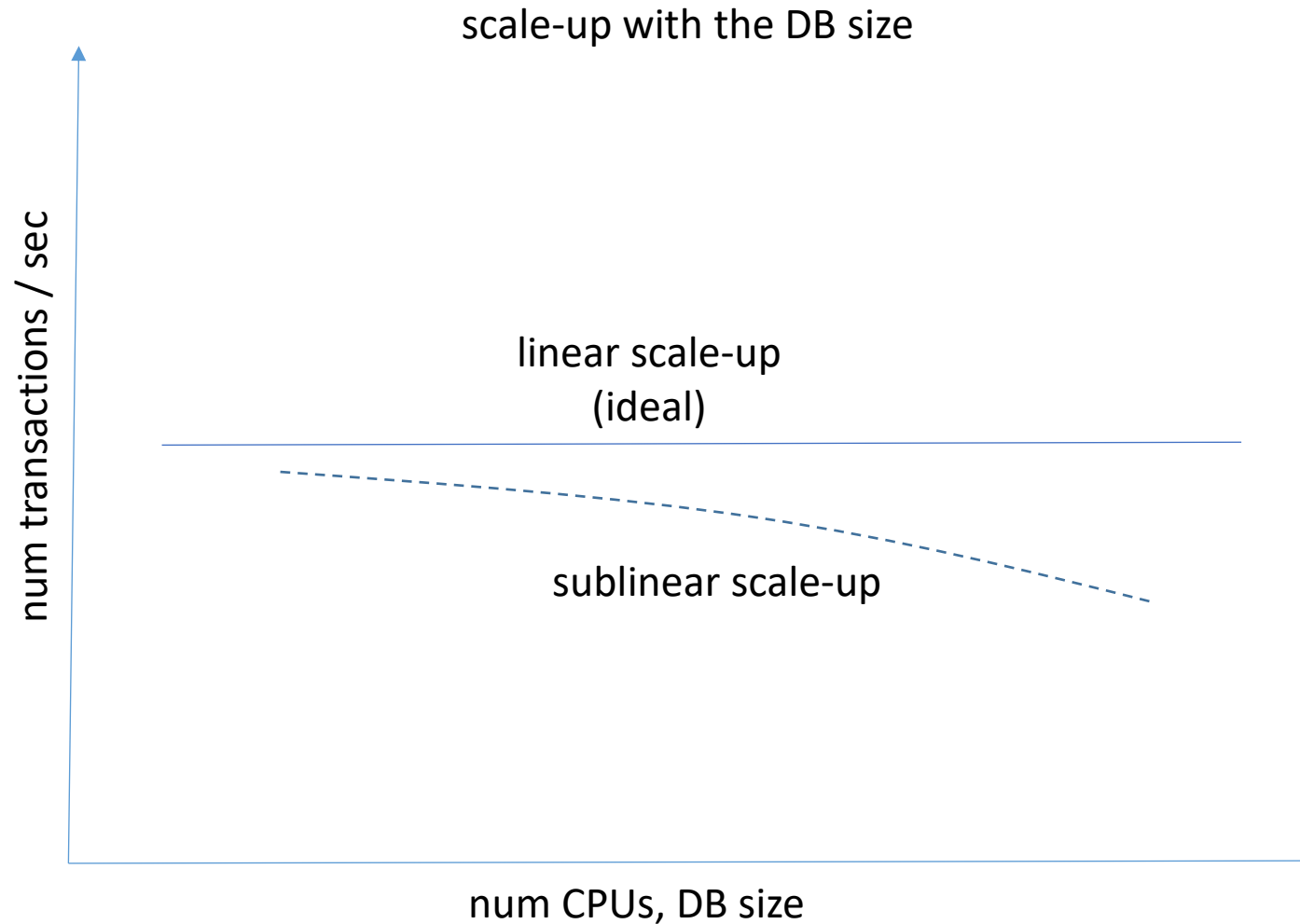
- speed-up



- DB size - fixed
 - add CPUs
- => more transactions can be executed per second

The Shared-Nothing Architecture

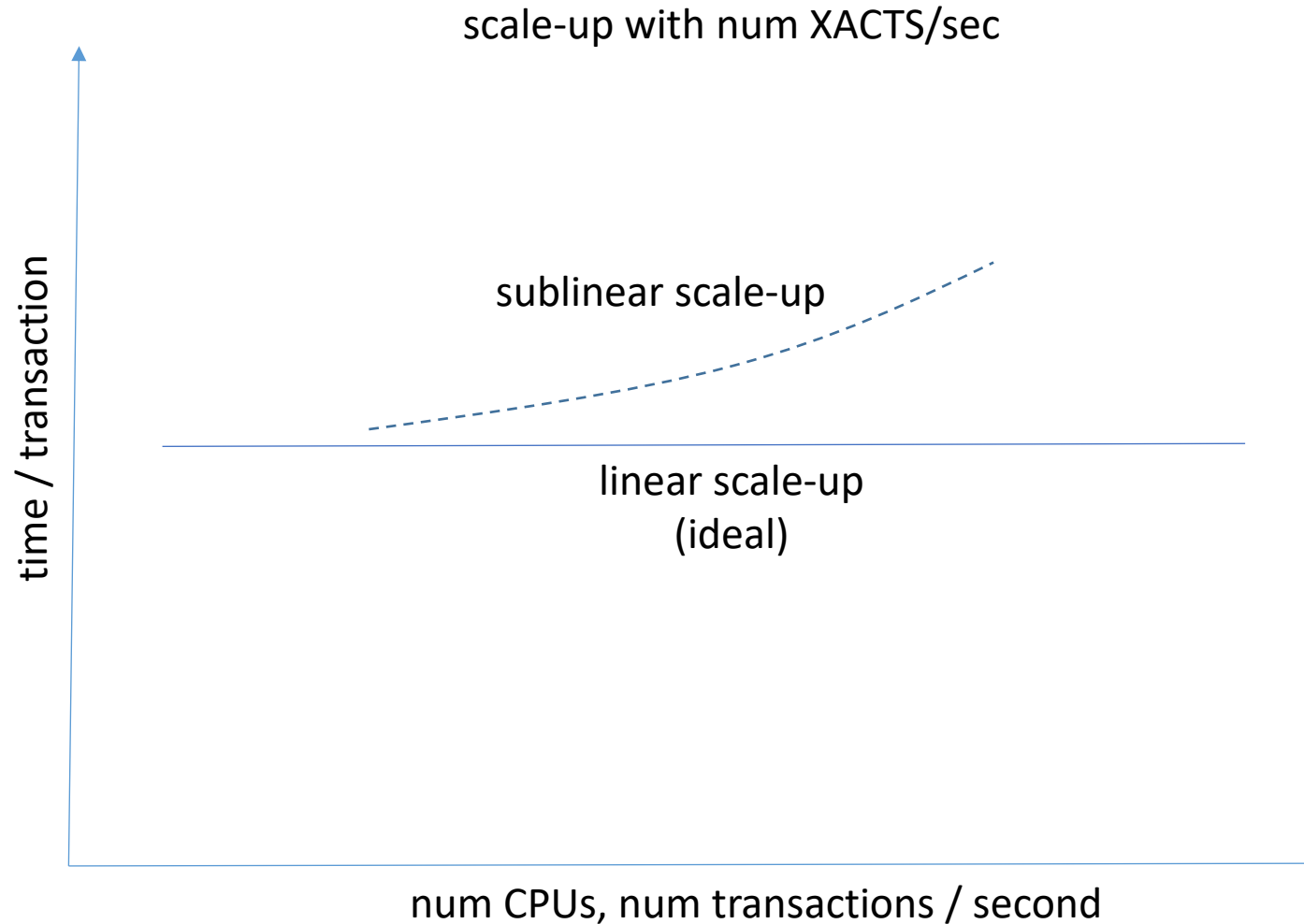
- scale-up



- the number of transactions executed per second, as the DB size and the number of CPUs increase

The Shared-Nothing Architecture

- scale-up



* alternative

- add CPUs as the number of transactions executed per second increases
- evaluate the time required for a transaction

Parallel Query Evaluation

- context
 - DBMS based on a shared-nothing architecture
- evaluate a query in a parallel manner
- operators in an execution plan can be evaluated in parallel
 - 2 operators are evaluated in parallel
 - one operator is evaluated in a parallel manner
- an operator is said to *block* if it doesn't produce results until it consumes all its inputs (e.g., sorting, aggregation)
- pipelined parallelism
 - an operator consumes the output of another operator
 - limited by blocking operators

Parallel Query Evaluation

- parallel evaluation on partitioned data
 - every operator in a plan can be evaluated in a parallel manner by partitioning the input data
 - partitions are processed in parallel, the results are then combined
- processor = CPU + its local disk

Parallel Query Evaluation – Data Partitioning

- horizontally partition a large dataset on several disks
- partitions are then read / written in parallel
- *round-robin* partitioning
 - n processors
 - the i^{th} tuple is assigned to processor $i \% n$
- hash partitioning
 - determine the processor for a tuple t
 - apply a hash function to t ((some of) its attributes)
- range partitioning
 - n processors
 - order tuples conceptually
 - choose n ranges for the sorting key values s.t. each range contains approximately the same number of tuples
 - tuples in range i are assigned to processor i

Parallel Query Evaluation – Data Partitioning

- queries that scan the entire relation
 - round-robin partitioning - suitable
- queries that operate on a subset of tuples
 - equality selection, e.g., $\text{age} = 30$
 - tuples partitioned on the attributes in the selection condition, e.g., age
 - hash and range partitioning are better than round-robin (one can access only the disks containing the desired tuples)
 - range selection, e.g., $20 < \text{age} < 30$
 - range partitioning is better than hash partitioning (it's likely that the desired tuples are grouped on several processors)

Parallelizing Individual Operations

- context
 - DBMS based on a shared-nothing architecture
- each relation is horizontally partitioned on several disks
- scanning a relation
 - pages can be read in parallel
 - obtained tuples can then be reunited
 - similarly – obtain all tuples that meet a selection condition

Parallelizing Individual Operations

- sorting
 - v1
 - each CPU sorts the relation fragment on its disk
 - subsequently, the sorted tuple sets are merged
 - v2
 - redistribute tuples in the relation using range partitioning
 - each processor sorts its tuples with a sequential sorting algorithm => several sorted runs on the disk
 - merge runs => sorted version of the set of tuples assigned to the current processor
 - obtain the entire sorted relation
 - visit processors in an order corresponding to their assigned ranges and scan the tuples

Parallelizing Individual Operations

- sorting
 - v2
 - challenges
 - range partitioning – assign approximately the same number of tuples to each processor
 - a processor that receives a disproportionately large number of tuples will limit scalability

Spatial Databases

Types of Spatial Data and Queries

- spatial data
 - multidimensional points
 - lines
 - rectangles
 - cubes
 - etc.
- spatial extent (SE)
 - region of space occupied by an object
 - characterized by location + boundary

Types of Spatial Data and Queries

- DBMS
 - point data
 - region data
- point data
 - collection of points in a multidimensional space
 - point
 - SE – only location
 - no space, area, volume
 - direct measurements (e.g., MRI)
 - transforming data objects (e.g., feature vectors)

Types of Spatial Data and Queries

- region data
 - collection of regions
 - region - SE
 - location
 - position of a fixed anchor point for the region
 - boundary (line, surface)
 - geometric approximations to objects, built with line segments, polygons, spheres, cubes, etc.
- e.g., in geographic apps
 - line segments – roads, railways, rivers, etc.
 - polygons – lakes, counties, countries, etc.

Types of Spatial Data and Queries

- spatial queries
 - spatial range queries
 - nearest neighbor queries
 - spatial join queries
- spatial range queries
 - associated region
 - location + boundary
 - find all regions that overlap / are contained within the specified range
 - e.g., Find all cities within 150 km of Constanța.
 - e.g., Find all rivers in Bihor.

Types of Spatial Data and Queries

- spatial queries
 - nearest neighbor queries
 - e.g., Find the 5 cities that are nearest to Paris.
 - usually, answers are ordered by proximity
- spatial join queries
 - e.g., Find pairs of cities within 150 km of each other.
 - meaning of queries can be determined by the level of detail in the representation of objects
 - e.g., relation in which each tuple is a point that represents a city
 - answer the queries with a self join
 - join condition – distance between 2 tuples in the query result

Types of Spatial Data and Queries

- spatial queries
 - spatial join queries
 - e.g., cities have a boundary
- => meaning of queries and evaluation strategies become more complex
- cities whose centroids are within 150 km of each other?
 - or cities whose boundaries come within 150 km of each other?

Applications

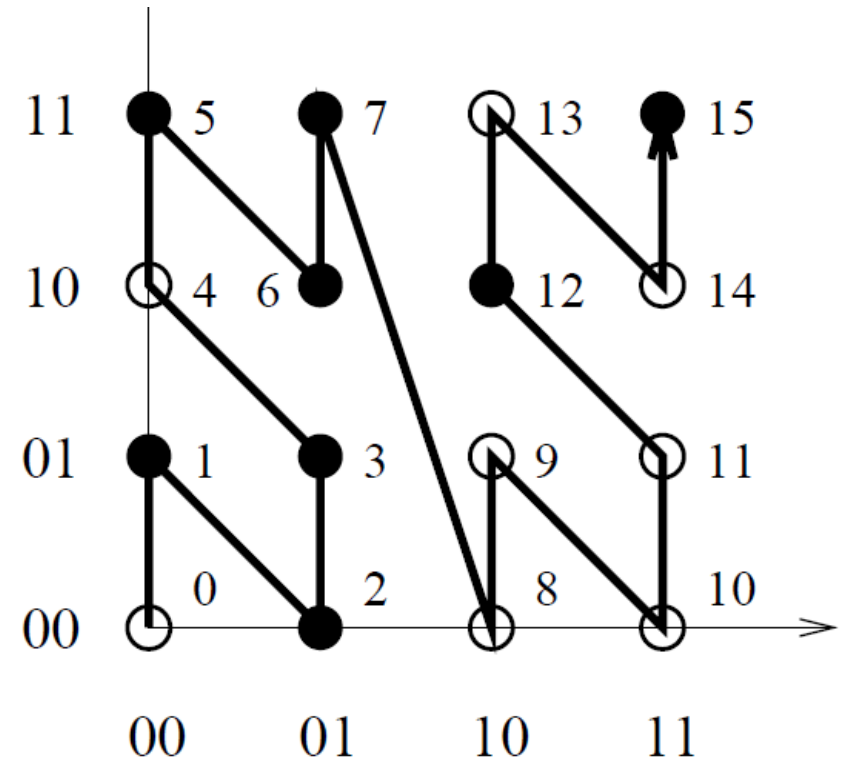
- relation R with k attributes seen as a collection of k -dimensional points
- Geographic Information Systems (GIS)
 - point & region data
 - e.g., a map that contains the locations of several small objects (points), highways (lines), cities (regions)
 - range, nearest neighbor, join queries
- Computer-aided design (CAD) systems, medical imaging systems
 - store spatial objects
 - point & region data
 - most frequent queries - range & join
 - spatial integrity constraints

Applications

- multimedia DBs
 - multimedia objects
 - images, text, *time-series* data (e.g., audio)
 - object
 - point in a multidimensional space
 - similarity of 2 multimedia objects
 - distance between the corresponding points
 - similarity queries seen as nearest neighbor queries
 - point data & nearest neighbor queries – most common

Indexing - *space-filling curves*

- assumption
 - any attribute value can be represented with a fixed num. of bits, e.g., k bits
- ⇒ max. num. of values per dimension = 2^k
- the point with $X = 01, Y = 11$ has Z-value = 0111, obtained by interleaving the bits of X and Y
 - the 8th point visited by the space-filling curve, which starts at point $X=00, Y=00$
- points in the dataset are stored in the order of their Z-values
- Z-ordering curve
 - linear ordering on the domain



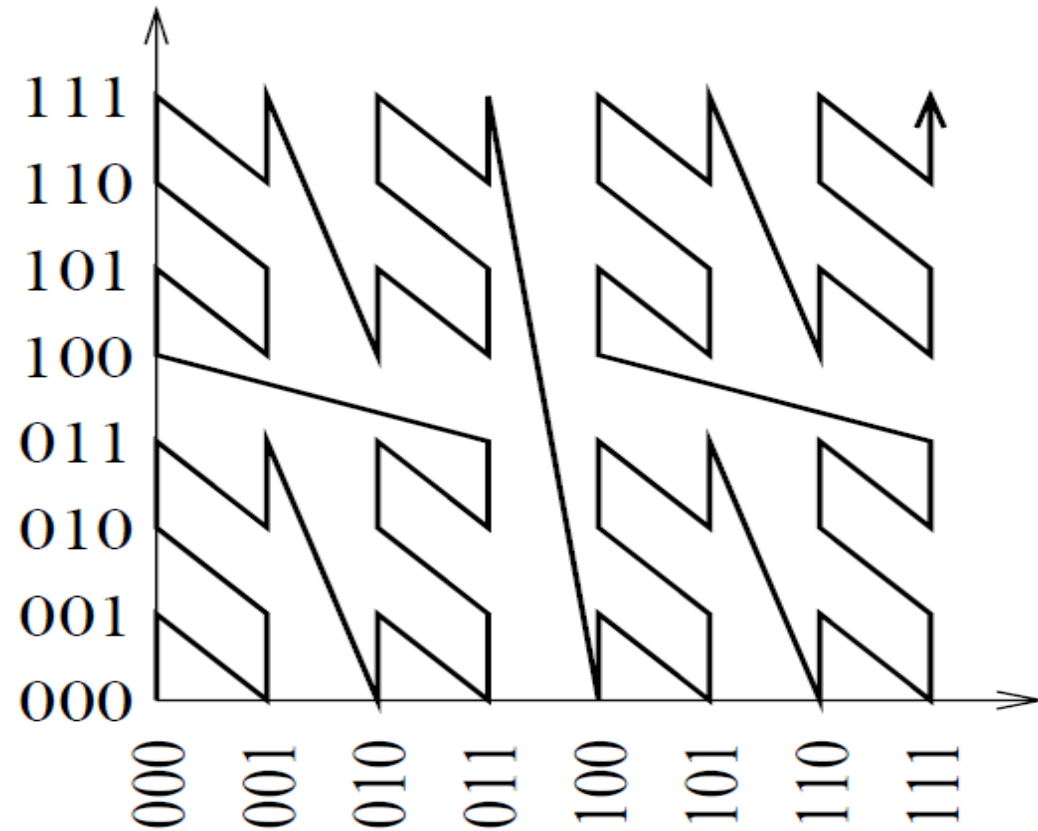
Z-ordering with 2 bits

Indexing - *space-filling curves*

- Z-ordering curve
 - the curve visits all the points in a quadrant before moving on to the next quadrant; all points in a quadrant are stored together
- indexing
 - B+ tree
 - search key: Z-value
 - store the Z-value of the point (along with the point)
 - I / D / search point
 - compute Z-value
 - I / D / search into / from / the B+ tree
 - unlike when using traditional B+ tree-based indexing, points are clustered together by spatial proximity in the X-Y space
 - spatial queries in the X-Y space become linear range queries
 - efficient evaluation on the B+ tree organized by Z-values

Indexing - *space-filling curves*

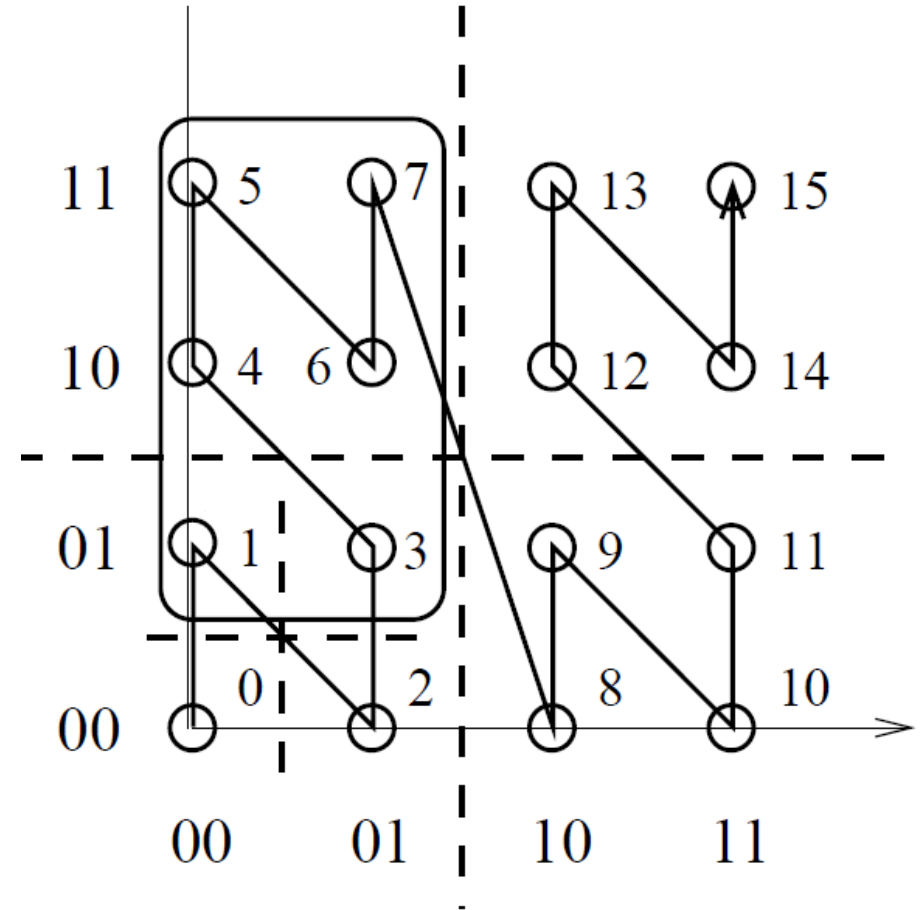
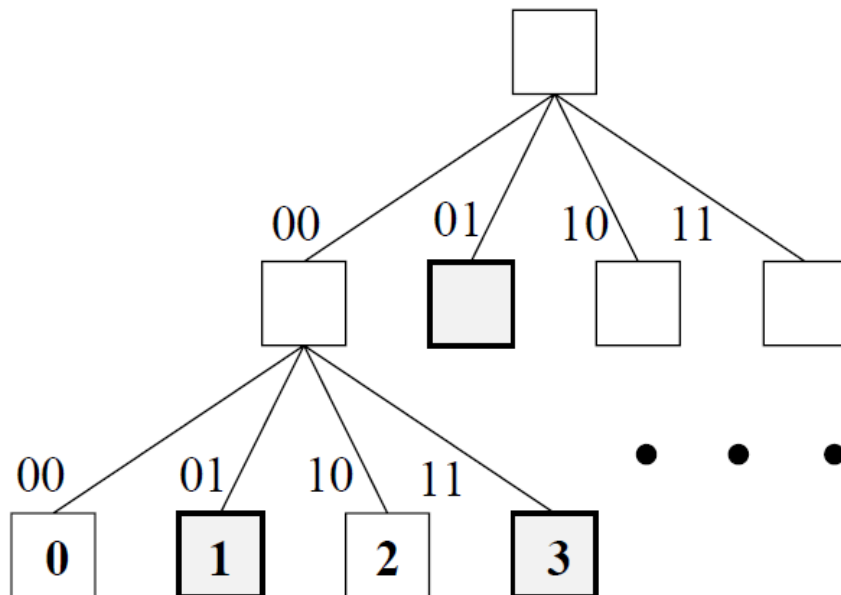
- Z-ordering curve when $k = 3$



Z-ordering with 3 bits

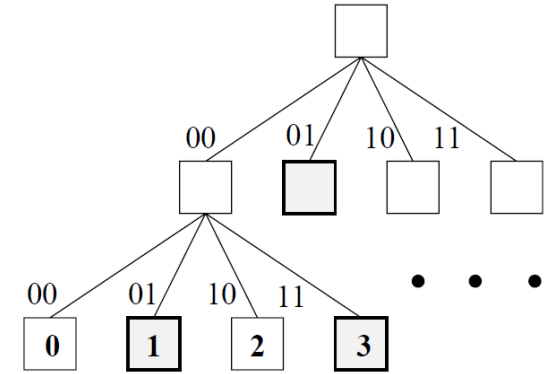
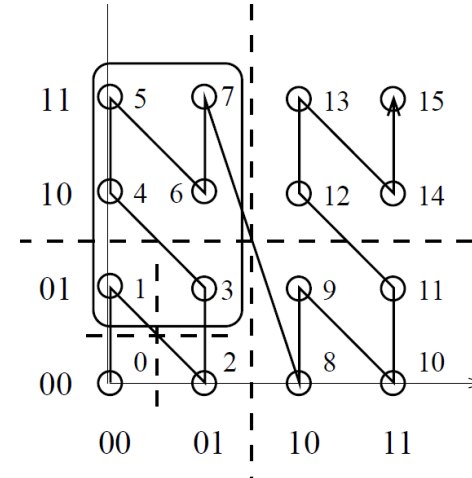
Region Quad Trees

- region data
- Z-ordering recursively decomposes space into quadrants and subquadrants
- the structure of a Region Quad tree directly corresponds to the recursive decomposition of the data space
- each node in the tree corresponds to a square region in the data space



Region Quad Trees

- root – entire data space
- internal node – 4 children
- rectangle object R stored by the DBMS
 - all points in the 01 quadrant of the root and the points with Z-values 1 and 3
- store 3 records: $\langle 0001, R \rangle$, $\langle 0011, R \rangle$, $\langle 01, R \rangle$
 - records clustered and indexed by the 1st field (the Z-value) in a B+ tree
- use B+ trees to implement Region Quad trees
- generalization - k dimensions $\Rightarrow$ at every node, the space is partitioned into 2^k subregions



References

- [Ra02] RAMAKRISHNAN, R., GEHRKE, J., Database Management Systems (3rd Edition), McGraw-Hill, 2002
- [Da03] DATE, C.J., An Introduction to Database Systems (8th Edition), Addison-Wesley, 2003
- [Ga09] GARCIA-MOLINA, H., ULLMAN, J., WIDOM, J., Database Systems: The Complete Book (2nd Edition), Pearson Education, 2009
- [Ra02S] RAMAKRISHNAN, R., GEHRKE, J., Database Management Systems, Slides for the 3rd Edition,
<http://pages.cs.wisc.edu/~dbbook/openAccess/thirdEdition/slides/slides3ed.html>
- [Si19] SILBERSCHATZ, A., KORTH, H., SUDARSHAN, S., Database System Concepts (7th Edition), McGraw-Hill, 2019
- [Si19S] SILBERSCHATZ, A., KORTH, H., SUDARSHAN, S., Database System Concepts, Slides for the 7th Edition, <http://codex.cs.yale.edu/avi/db-book/>
- [Ul11] ULLMAN, J., WIDOM, J., A First Course in Database Systems,
<http://infolab.stanford.edu/~ullman/fcdb.html>

Data Exploration with SQL using Machine Learning Techniques *

* **not** among the exam topics

Julien Cumin, Jean-Marc Petit, Vasile-Marian Scuturici, Sabina Surdu:
Data Exploration with SQL using Machine Learning Techniques.
EDBT 2017: 96-107

Julien Cumin, Jean-Marc Petit, Fabien Rouge, Vasile-Marian Scuturici, Christian
Surace, Sabina Surdu:
Requêtes discriminantes pour l'exploration des données. EGC 2016: 195-206